



Dry-AIREC Users Manual

Release 1.6.0

Tokyo Robotics Inc.

Contents:

Chapter 1 Introduction	1
1.1 About this manual . 1.2 About this robot .	1
	1
Chapter 2 Safety Precautions	3
2.1 Explanation of safety symbols . . . 2.2 Robot safety (IMPORTANT) . . .	3
2.3 Prohibited acts 2.4 Disclaimer	4
	4
	5
Chapter 3. Hardware Overview	7
3.1 Robot specifications . . . 3.2 Overall hardware	7
configuration . . . 3.2.1 Robot body	8
	8
3.2.2 Battery switch .	8
3.2.3 Battery power lamp (green) . 3.2.4 Power supply voltage drop warning	8
lamp (red) .	9
3.2.5 PC start switch . . . 3.2.6	9
Emergency stop switch . . . 3.2.7	9
Depth camera (RealSense™ SR300) . 3.2.8 Fisheye	10
camera (Insta180) . 3.2.9 Stereo	10
camera (See3CAM CU55) . 3.2.10 Speaker .	10
	11
3.2.11 Microphone . . .	11
3.2.12 Charger 3.3 Configuration . . .	11
of Electrical Circuit System . . .	11
3.4 Interface ...	13
3.4.1 Mobile Cart Expansion Interface . .	13
3.4.1.1 USB 3.1 port .	13
3.4.1.2 LAN Ports	13
3.4.1.3 5Vdc/12Vdc OUT . . .	13
3.4.2 Rear expansion interface .	14
3.4.2.1 USB 3.1 port .	16
3.4.2.2 HDMI port .	16
3.4.2.3 LAN port 3.4.2.4 Boot PC switch . . .	16
	16

Contents:

3.4.2.5 Right arm (left arm) tool connector connection terminal . 3.4.2.6 Insulated DC 24V	16
output terminal . 3.4.2.7 External emergency stop switch	17
connection terminal . 3.4.2.8 General-purpose output terminal .	18
	18
3.5 LAN Device Configuration . .	21
 Chapter 4. Controller Overview	 23
4.1 Robot controller .	23
4.1.1 Controller 1 (C1) . . 4.1.2 Controller 2 (C2) . . 4.2	23
User PC .	24
	24
 Chapter 5 Software Overview	 25
5.1 ROS	25
5.2 Overall system configuration 5.3 Overview of	25
control implementation 5.3.1 Control loop in the joint	27
controller 5.3.2 Control loop in the control PC (real-time part) 5.3.3 Applications in the robot	27
controller PC and user PC (non-real-time part)	28
	28
5.4 Robot controller PC software .	28
5.4.1 Software for Controller 1 (C1) 5.4.2 Software for Controller 2 (C2) 5.5 Software	28
for User PC	29
	29
 Chapter 6. Setup	 31
6.1 Robot Setup . . 6.1.1 Install the robot	31
	31
6.2 Robot controller setup .	33
6.2.1 How to charge the battery 6.2.2 Precautions for charging	33
6.2.3 About the charger display 6.2.4 Precautions for the	35
battery 6.2.5 Powering on the robot 6.2.6 Powering on the	35
controller PC 6.3 Setting up the user PC	36
	36
	37
	37
6.3.1 Recommended PC environment .	37
6.3.2 How to install ROS .	38
6.3.3 Creating a ROS workspace . . . 6.3.4 Installing provided ROS packages .	39
	39
6.3.4.1 Installing deb binaries and source code .	39
6.3.4.2 Add source command to bashrc . 6.3.4.3 Uninstall deb binary .	40
	41
6.3.4.4 Uninstalling ROS Package Source Code ...	41
6.4 Network setup .	41
6.4.1 Connect the PC and robot controller to the same LAN . 6.4.1.1 Connect the controller to a wired LAN .	41
	42

6.4.1.2 Connect to the controller via wireless LAN . Check the ssh connection . Set up the	42
6.4.2 ssh communication environment .	42
6.4.3	43
6.4.4 Verifying ROS communication operation using the ROS MASTER ...	43
6.4.5 Setting ROS communication environment variables .	44
6.4.6 Notes on network settings .	45
6.5 Setting up time synchronization between PCs .	45
6.5.1 About Chrony ... 6.5.2 About the time synchronization	45
method .	45
6.5.3 How to install and configure Chrony .	46
6.5.4 Checking Chrony operation .	46
Chapter 7 Running the Demo	49
7.1 Turn on the power . 7.2 Start the	49
robot controller . 7.3 Start the robot controller core program . 7.4 On the user	49
PC , specify controller 1 as the ROS MASTER . 7.5 Check the robot control mode and servo status . 7.6 Check	50
the operation of the position control mode (joint_trajectory_controller) .	50
	51
	52
7.6.1 Position command using the command box in Torobo State Viewer . 7.6.2 Position command using	53
rq_t_torobo_joint_trajectory_controller . 7.6.3 Position command using rviz + moveit .	53
	54
7.7 Check the operation of gravity compensation mode (gravity_compensation_controller) . 7.8 Check the operation of joint impedance	56
control mode (joint_impedance_controller) . 7.9 Return the robot to the home position and stop the operation . 7.10 Shut down the robot controller . 7.11	57
Turn off the power .	58
	59
	59
Chapter 8 Running the Simulator 8.1 How to Switch Between	61
Real Device Operation and Simulator Operation . 8.2 How to Run the Simulator (Gazebo) ... 8.3 Checking	61
Operation on the Simulator (Gazebo) .	62
	65
8.3.1 Check the operation of position control mode (joint_trajectory_controller) ... 8.3.2 Check the operation of joint impedance control	65
mode (joint_impedance_controller) ... 8.3.3 Return the robot's posture to the home position and stop the operation ... 8.4 How to exit the simulator (Gazebo) ...	66
	66
	66
Chapter 9. Tutorials	67
9.1 Check the URDF of the robot model ...	67
9.2 Use GUI software to move the robot .	69
9.2.1 Torobo State Viewer .	69
9.2.2 rq_t_torobo_joint_trajectory_controller .	72
9.2.3 RViz (Trajectory planning with MoveIt!) .	74
9.3 Using ROS topics , services, and actions to move the robot from the terminal . 9.3.1 Joint position command (using Ros Publish) 9.3.2 Joint position command	76
(using Ros Actionlib)	76
	77

Contents:

9.3.3 Servo ON/OFF (using Ros Service) . .	79
9.4 Use Python to move the robot ...	80
9.4.1 Controlling joint position using ROS Publish (one-way communication) .	80
9.4.2 Controlling joint position using ROS Actionlib (bidirectional communication) .	82
9.4.3 Servo ON/OFF using ROS Service . .	86
9.4.4 Acquiring and switching the controller status using the Controller Manager . 9.4.5 Position command using Teaching Points .	88
	92
9.4.6 High-frequency command of joint positions using ROS Publisher .	96
9.4.7 Position Commanding with MoveIt! !	99
9.4.8 Calculating kinematics with MoveIt! 9.4.9 Opening and closing the hand	102
9.4.10 Visualizing the output of the pressure	106
sensor with RViz	111
9.4.11 Controlling the cart speed using ROS Publish ...	114
Chapter 10. How to Use the Controller 10.1 Overview	117
	117
10.1.1 About the network connection of PCs . . 10.1.2 About PCs that need to be started 10.2 How	117
to remotely connect to each controller from a user PC . .	118
	118
10.2.1 How to connect via SSH .	118
10.2.2 How to connect via RDP (Remote Desktop) .	119
10.3 How to set up a ROS Master on one PC and build a distributed system . . . 10.3.1 How to make Controller 1 (C1) the ROS Master .	120
	120
10.3.1.1 Setting up Controller 1 (C1) ... 10.3.1.2 Setting up Controller 2 (C2) ...	120
10.3.1.3 Setting up the user PC ... 10.3.2 How to make Controller 2 (C2) the ROS	121
Master .	121
	121
10.3.2.1 Setting up Controller 2 (C2) . . . 10.3.2.2 Setting up User PC . . .	121
	122
Chapter 11. Using the Camera 11.1 Overview .	123
11.2 Hardware	123
Overview .	123
11.2.1 RealSense SR300 Specifications .	123
11.2.2 OMNI shot mini specifications .	124
11.2.3 See3CAM_CU55 Specifications . 11.3 Software	125
Overview . 11.3.1 Overall System Configuration .	126
	126
11.3.2 image_pipeline . 11.3.3 realsense	127
image_pipeline . . . 11.4 Operation method .	127
	128
11.4.1 Preparing the execution environment (settings for controllers C1/C2 and user PC) . . 11.4.1.1 When moving the robot arm, mobile	128
cart, or camera . 11.4.1.2 When moving only the camera .	128
	129
11.4.2 Camera operation method . .	130
11.4.2.1 How to operate RealSense .	130

11.4.2.2 How to operate OMNI shot mini . 11.4.2.3 How to operate see3CAM_CU55	134
11.4.2.4 How to adjust the camera parameters for each camera . 11.4.2.5 How to	135
use the camera in the simulator	138
.	138
Chapter 12 : Using Microphones and Speakers	139
12.1 How audio devices work in Linux	139
12.1.1 ALSA (Audio Driver)	139
12.1.2 PulseAudio (Audio Service) 12.2 How to Use Audio Devices	139
.	140
12.2.1 Executing the PulseAudio enable script 12.2.2 Explanation of the sample program that	140
records with a microphone and plays on a speaker 12.2.3 Explanation of the sample program that creates audio data and plays on a speaker	142
.	142
Chapter 13 Software Specifications 13.1 Custom ROS	143
message specifications	143
13.1.1 ROS Topic yyyyyyyyy 13.1.1.1 torobo_sensor_msgs/ToroboState	143
13.1.1.2 torobo_control_msgs/CollisionInfoRequest . 13.1.1.3 torobo_control_msgs/	143
CollisionInfoResponse . 13.1.1.4 torobo_control_msgs/LoadInertia . 13.1.1.5 gpio_msgs/GpioStates	145
13.1.1.6 cartesian_trajectory_msgs/CartesianTrajectoryPoint . 13.1.1.7 cartesian_trajectory_msgs/	146
CartesianTrajectoryError . 13.1.1.8 cartesian_trajectory_msgs/CartesianTrajectory	146
13.1.1.9 cartesian_trajectory_msgs/	146
CartesianTrajectoryControllerState . . 13.1.1.10 cartesian_trajectory_msgs/CartesianState . . 13.1.1.11	147
cartesian_trajectory_msgs/CartesianImpedance . 13.1.1.12 cartesian_trajectory_msgs/CartesianImpedanceControllerState ;	147
13.1.1.13 cartesian_trajectory_msgs/CartesianImpedanceTrajectory	147
.	148
.	149
.	149
.	149
.	150
13.1.2 Message Specification for ROS Services	151
13.1.2.1 torobo_std_msgs/SetString.srv . 13.1.2.2 torobo_std_msgs/	151
SetStringArray.srv . 13.1.2.3 torobo_control_msgs/CheckCollision.srv . . 13.1.2.4	151
torobo_control_msgs/GetCollisionInfo.srv . 13.1.2.5 torobo_control_msgs/SetLoad.srv . 13.1.2.6	151
torobo_control_msgs/UnsetLoad.srv . 13.1.2.7 torobo_control_msgs/ComputeZmp.srv . . . 13.1.2.8	152
torobo_control_msgs/EvaluateZmpStability.srv	152
.	153
.	153
.	154
13.2 Torobo Easy Command Specifications	155
13.2.1 Torobo State Viewer commands	156
13.2.2 Example of control using commands	157
13.3 Diagnostic Status Specifications	157
13.3.1 Notes on Diag Status	158
13.3.2 Diag Status List . 13.4 Specifications of Safety	158
Monitoring Function	162

Contents:

13.4.1 About Safety State . 13.4.2 How to check the current	162
Safety State .	162
13.4.2.1 How to check from the terminal . .	162
13.4.2.2 How to check using Torobo State Viewer ...	163
13.4.3 How to return to Healthy from Emergency Stop or Protective Stop . 13.4.3.1 How to return by terminal operation .	163
	163
13.4.3.2 How to restore using buttons on Torobo State Viewer .	163
13.4.4 About SafetyMonitor ... 13.4.5 How to configure	163
controllers.yaml ... 13.4.5.1 Controllers.yaml location directory ... 13.4.5.2 How	164
to write controllers.yaml ... 13.4.5.3 About writing in controllers.yaml ... 13.4.6 Common	164
matters for each SafetyMonitor ...	164
	165
	165
13.4.6.1 SafetyMonitor Namespace .	165
13.4.6.2 SafetyMonitor parameters and dynamic_reconfigure . 13.4.6.3 Temporarily disabling the safety monitoring state . .	165
13.4.7 torobo_safety_controller . 13.4.8 torobo_safety_monitors/SafetyUnitMonitor .	165
	166
	166
13.4.8.1 Emergency Stop requirements .	166
13.4.8.2 Protective Stop requirement conditions . 13.4.8.3 Return	166
conditions .	166
13.4.8.4 Action Interface	166
13.4.8.5 Subscribed Topics . . .	167
13.4.8.6 Published Topics .	167
13.4.8.7 Services . .	167
13.4.8.8 Parameters	167
13.4.8.9 Reconfigurable Parameters	167
13.4.9 torobo_safety_monitors/SelfCollisionMonitor .	167
13.4.9.1 Emergency Stop requirements .	167
13.4.9.2 Protective Stop requirement . 13.4.9.3 Recovery condition .	168
	168
13.4.9.4 Action Interface	168
13.4.9.5 Subscribed Topics . . .	168
13.4.9.6 Published Topics .	168
13.4.9.7 Services . .	168
13.4.9.8 Parameters	168
13.4.9.9 Reconfigurable Parameters 13.4.10 torobo_safety_monitors/	168
StopTopicMonitor .	168
13.4.10.1 Emergency Stop requirements .	169
13.4.10.2 Protective Stop requirement conditions . 13.4.10.3 Return	169
conditions .	169
13.4.10.4 Action Interface	169
13.4.10.5 Subscribed Topics . . .	169
13.4.10.6 Published Topics .	169
13.4.10.7 Services . .	169

13.4.10.8 Parameters	169
13.4.10.9 Reconfigurable Parameters 13.4.11 torobo_safety_monitors/	170
JointErrorMonitor	170
13.4.11.1 Emergency Stop requirements	170
13.4.11.2 Protective Stop request condition 13.4.11.3 Return condition	170
	170
13.4.11.4 Action Interface	170
13.4.11.5 Subscribed Topics	170
13.4.11.6 Published Topics	170
13.4.11.7 Services	170
13.4.11.8 Parameters	171
13.4.11.9 Reconfigurable Parameters	171
13.4.12 torobo_safety_monitors/ControlToleranceMonitor	171
13.4.12.1 Emergency Stop requirements	172
13.4.12.2 Protective Stop request conditions 13.4.12.3 Return	172
conditions	172
13.4.12.4 Action Interface	172
13.4.12.5 Subscribed Topics	172
13.4.12.6 Published Topics	172
13.4.12.7 Services	172
13.4.12.8 Parameters	172
13.4.12.9 Reconfigurable Parameters 13.4.13 torobo_safety_monitors/	173
JointTrajectoryCommandMonitor	174
13.4.13.1 Emergency Stop requirements	174
13.4.13.2 Protective Stop request conditions ... 13.4.13.3 Return	174
conditions ...	174
13.4.13.4 Action Interface	174
13.4.13.5 Subscribed Topics	174
13.4.13.6 Published Topics	175
13.4.13.7 Services	175
13.4.13.8 Parameters	175
13.4.13.9 Reconfigurable Parameters 13.4.14 torobo_safety_monitors/	175
ZmpStabilityMonitor	176
13.4.14.1 Emergency Stop requirements	176
13.4.14.2 Protective Stop request conditions ... 13.4.14.3 Return	176
conditions ...	176
13.4.14.4 Action Interface	176
13.4.14.5 Subscribed Topics	176
13.4.14.6 Published Topics	176
13.4.14.7 Services	176
13.4.14.8 Parameters	177
13.4.14.9 Reconfigurable Parameters	177
13.4.15 blank	177
13.5 Specifications of the Self-Collision Detection Function ... 13.5.1	177
How to Use the Self-Collision Detection Function ...	178

Contents:

13.6 Specifications of Center of Gravity, ZMP Position Calculation, and Stability Evaluation Functions ...	178
13.6.1 How to check the robot's center of gravity . 13.6.2 How to check the	178
robot's ZMP position .	178
13.6.2.1 How to check the current ZMP position and support polygon shape using RViz ...	178
13.6.2.2 How to set the threshold that determines the marker color that indicates the ZMP position . 13.6.3 How to calculate	181
the ZMP position relative to the command joint trajectory . 13.6.4 How to evaluate the stability of the	181
ZMP position relative to the command joint trajectory .	181
13.6.5 How to check the ZMP position of the orbital points planned by MoveIt!	181
13.6.6 blank	183
13.7 Specifications of your own ROS Controller ... 13.7.1 About your	183
own ROS Controller ... 13.7.2 How to configure controllers.yaml ... 13.7.2.1	183
Controllers.yaml location directory ... 13.7.2.2 How to write controllers.yaml ...	183
13.7.2.3 Controllers that must be written in controllers.yaml ... 13.7.3 Common matters for	183
each Controller ...	183
	185
	185
13.7.3.1 Controller Namespace .	185
13.7.3.2 Controller parameters and dynamic_reconfigure ... 13.7.4 torobo_state_controller/ToroboStateController .	186
	186
13.7.4.1 Action Interface	186
13.7.4.2 Subscribed Topics ...	186
13.7.4.3 Published Topics .	186
13.7.4.4 Services ...	187
13.7.4.5 Parameters	187
13.7.4.6 Reconfigurable Parameters 13.7.5 torobo_controllers/	188
SafetyController .	188
13.7.5.1 Action Interface	188
13.7.5.2 Subscribed Topics ...	188
13.7.5.3 Published Topics .	188
13.7.5.4 Services ...	189
13.7.5.5 Parameters	189
13.7.5.6 Reconfigurable Parameters	189
13.7.6 effort_controllers/ToroboJointTrajectoryController ...	189
13.7.6.1 Action Interface	189
13.7.6.2 Subscribed Topics ...	190
13.7.6.3 Published Topics .	190
13.7.6.4 Services ...	190
13.7.6.5 Parameters	190
13.7.6.6 Reconfigurable Parameters	191
13.7.7 effort_controllers/ToroboJointImpedanceController ...	191
13.7.7.1 Action Interface	191
13.7.7.2 Subscribed Topics ...	191
13.7.7.3 Published Topics .	191
13.7.7.4 Services ...	192
13.7.7.5 Parameters	192

13.7.7.6 Reconfigurable Parameters	193
13.7.8 effort_controllers/GravityCompensationController . .	193
13.7.8.1 Action Interface	193
13.7.8.2 Subscribed Topics . . .	194
13.7.8.3 Published Topics .	194
13.7.8.4 Services . .	194
13.7.8.5 Parameters	194
13.7.8.6 Reconfigurable Parameters 13.7.9 effort_controllers/	194
ToroboVelocityController .	194
13.7.9.1 Action Interface	194
13.7.9.2 Subscribed Topics . . .	194
13.7.9.3 Published Topics .	195
13.7.9.4 Services . .	195
13.7.9.5 Parameters	195
13.7.9.6 Reconfigurable Parameters 13.7.10 torobo_controllers/	195
ToroboOnlineJointTrajectoryController . . .	195
13.7.10.1 Action Interface	196
13.7.10.2 Subscribed Topics . . .	196
13.7.10.3 Published Topics .	196
13.7.10.4 Services . .	196
13.7.10.5 Parameters	196
13.7.10.6 Reconfigurable Parameters 13.7.11 torobo_controllers/	197
ToroboOnlineJointImpedanceController . . .	197
13.7.11.1 Action Interface	197
13.7.11.2 Subscribed Topics . . .	197
13.7.11.3 Published Topics .	198
13.7.11.4 Services . .	198
13.7.11.5 Parameters	198
13.7.11.6 Reconfigurable Parameters	199
13.7.12 torobo_controllers/CartesianImpedanceController . . .	199
13.7.12.1 Action Interface	200
13.7.12.2 Subscribed Topics . . .	200
13.7.12.3 Published Topics .	200
13.7.12.4 Services . .	201
13.7.12.5 Parameters	201
13.7.12.6 Reconfigurable Parameters	205
13.7.13 torobo_controllers/OnlineCartesianImpedanceController .	205
13.7.13.1 Action Interface	205
13.7.13.2 Subscribed Topics . . .	205
13.7.13.3 Published Topics .	206
13.7.13.4 Services . .	206
13.7.13.5 Parameters	206
13.7.13.6 Reconfigurable Parameters	206
13.7.14 torobo_controllers/CartesianAdmittanceController . .	207
13.7.14.1 Action Interface	207

Contents:

13.7.14.2 Subscribed Topics . . .	208
13.7.14.3 Published Topics .	208
13.7.14.4 Services . .	208
13.7.14.5 Parameters	208
13.7.14.6 Reconfigurable Parameters	209
13.7.15 torobo_controllers/OnlineCartesianAdmittanceController .	209
13.7.15.1 Action Interface	209
13.7.15.2 Subscribed Topics . . .	209
13.7.15.3 Published Topics .	210
13.7.15.4 Services . .	210
13.7.15.5 Parameters	210
13.7.15.6 Reconfigurable Parameters 13.7.16 omni_drive_controller/	210
OmniDriveController .	211
13.7.16.1 Action Interface	211
13.7.16.2 Subscribed Topics . . .	211
13.7.16.3 Published Topics .	211
13.7.16.4 Services . .	212
13.7.16.5 Parameters	212
13.7.16.6 Reconfigurable Parameters	213
13.7.17 torobo_controllers/ToroboHandJointTrajectoryController .	213
13.7.17.1 Action Interface	213
13.7.17.2 Subscribed Topics . . .	213
13.7.17.3 Published Topics .	213
13.7.17.4 Services . .	214
13.7.17.5 Parameters	214
13.7.17.6 Reconfigurable Parameters	215
13.7.18 gpio_controllers/GpioStateController . . .	215
13.7.18.1 Action Interface	216
13.7.18.2 Subscribed Topics . . .	216
13.7.18.3 Published Topics .	216
13.7.18.4 Services . .	216
13.7.18.5 Parameters	216
13.7.18.6 Reconfigurable Parameters	216
13.7.19 gpio_controllers/GpioController .	216
13.7.19.1 Action Interface	217
13.7.19.2 Subscribed Topics . . .	217
13.7.19.3 Published Topics .	217
13.7.19.4 Services . .	217
13.7.19.5 Parameters	217
13.7.19.6 Reconfigurable Parameters	218
13.7.20 blank	218
13.8 Specifications of the load setting function . 13.8.1	218
How to set the load on the hand , etc. 13.8.2 How to check the	218
current load setting .	218
13.9 TeachingPointManager Specification .	219

13.9.1 How to start TeachingPointManager . 13.9.2 Specifications of Joint Angle TP and Cartesian TP	219
13.9.2.1 Specifications of Joint Angle TP	219
13.9.2.2 Specifications of Cartesian TP	219
13.9.3 teaching_point_utils Functions ...	220
13.9.4 Format of the yml file for recording TP .	222
13.10 ToroboHandTeachingMotion Specifications ... 13.10.1 How to Use ...	223
13.10.1.1 Starting method .	223
13.10.1.2 GUI section configuration 13.10.1.3 Adding a new action	224
13.10.1.4 Updating action content 13.10.1.5 Deleting an action	226
13.10.1.6 Issuing an action message 13.10.2 About the YAML file for describing standard actions	227
13.10.2.1 Loading a YAML file with standard actions from the GUI . 13.10.2.2 Loading a YAML file with standard actions from Launch etc. as rosparam .	230
13.10.2.3 Directory where sample YAML files describing standard actions are placed . 13.10.2.4 How to write a YAML file for describing standard actions .	232
13.10.3 Hand routine action commands (using Ros Actionlib) .	233
13.11 Torobo Moveit Commander Specifications .	234
13.11.1 About Torobo Moveit Commander .	234
13.11.2 Sample scripts .	235
13.11.3 List of ToroboMoveitCommander methods .	235
13.12 FlexBE Specifications .	242
13.12.1 Basic Use of FlexBE ...	242
13.12.2 How to start FlexBE . . 13.12.3 About unique states . .	242
13.12.3.1 torobo_general_states . 13.12.3.2 torobo_geometric_states . 13.12.3.3 torobo_hand_states . . 13.12.3.4 torobo_manipulation_states . . .	243
13.12.4 About sample behavior ...	245
13.12.4.1 torobo_general_behavior . 13.12.4.2 torobo_geometric_behaviors . . . 13.12.4.3 torobo_hand_behaviors .	246
13.12.4.4 torobo_manipulation_behaviors .	246
13.12.5 Points to note when creating a State .	248
Chapter 14 Conservation and Management	249
14.1 How to update the Ubuntu environment . 14.2 How to update the provided ROS packages .	249
14.2.1 How to update Controller 1 (C1) . 14.2.2 How to update Controller 2 (C2) .	250
14.2.3 How to update User PCs .	251

Contents:

14.2.4 How to back up source code and check the differences . . . 14.2.5 How to check the change log of deb binaries 252

. 253

14.3 Managing and deleting logs from the PC in the robot controller 14.4 Checking the free disk space on the 253

PC in the robot controller 253

Chapter 15. Troubleshooting 255

15.1 An error is displayed immediately after launching the robot . 15.2 Communication with the robot cannot 255

be performed . 15.3 The core program (bringup_real.launch) does not 256

start up properly on the actual robot . 15.4 An error "cannot open shared object file" appears and the core program does not start up properly 256

. 256

15.5 How to recover when an Over Position error occurs . 15.6 How to recover when an 257

Emergency Stop is issued due to self-interference detection . 15.6.1 Release the brake on the joint and manually backdrive it . 15.6.2 Temporarily 258

disable the self-interference monitoring state and servo-drive the joint 258

. 259

15.7 When I start a ROS package, Rviz or Gazebo frequently encounters an error during startup 260

15.8 Changes made to URDF are not reflected . 15.9 Stop issuing static_tf between map 261

and odom frames . 15.10 Joints move on their own in torque control mode even though no external force is being 261

applied . 15.11 About obtaining torque sensor offset values 262

. 262

15.12 How to return to normal operation when a Protective Stop is issued by the ZMP monitoring anti-tip function 263

15.12.1 Disable ZMP stability monitoring and servo drive joints 263

15.13 The message " Battery error" appears in the Error Device of Robot Monitor . 15.14 If the problem persists . 15.14.1 If the problem persists 265

. 265

. 265

Chapter 16 Appendix 16.1 267

Overview of ROS 267

16.1.1 How ROS works 267

16.1.2 Controlling a robot using ROS Actionlib (two-way communication) 268

16.1.3 How ROS -based distributed systems work . 16.2 Setting up the built-in Wi-Fi 269

router (wireless adapter) 270

16.2.1 How to connect to the setting screen of GL-AR750S-Ext 270

16.2.2 How to change the network mode of GL-AR750S-Ext . 16.3 How to change the IP address and network connection 271

settings of the controller PC 271

16.3.1 Precautions when changing network settings . 16.3.2 Preparation for changing 271

network settings 272

16.3.2.1 Changing the settings on the controller PC 16.3.2.2 Changing the settings via ssh remote 272

connection 272

16.3.3 Rewriting Netplan's configuration 272

16.3.4 Reflecting the Netplan settings . 16.4 About the interference 273

mesh of the eyebolt for hanging the robot 274

16.4.1 How to change the robot's interference mesh to one without eyebolts 275

Chapter 17 Copyright 279

Chapter 18 Registered Trademarks 281

xii

Chapter 19 Revision History	283
19.1 Release 1.6.0 (2023-03-07)	283
19.2 Release 1.5.0 (2022-11-30)	283
19.3 Release 1.4.0 (2022-10-31)	283
19.4 Release 1.3.1 (2022-03-21)	284
19.5 Release 1.3.0 (2022-03-17)	284
19.6 Release 1.2.1 (2022-01-07)	284
19.7 Release 1.2.0 (2021-10-14)	285
19.8 Release 1.1.0 (2021-09-17)	285
19.9 Release 1.0.0 (2021-08-31)	285

1 - Introduction

1.1 About this manual

This manual describes the specifications of the robot and how users can use the robot.
Please read the manual carefully and pay close attention to the precautions before operating the robot.

1>About the two-robot

This robot , Dry-AIREC , is composed of a dual arm equipped with hands, a torso (waist/neck), and an omnidirectional cart. All joints in the arms and waist are equipped with torque sensors, which allows not only position and speed control but also torque control. Pressure sensors are built into each part of the body, and the head is equipped with various cameras, speakers, and microphones, making it possible to configure applications that allow the robot to recognize the external environment and generate movements.



2 - Safety Considerations

2.1 Explanation of safety markings

This manual uses the following symbols to alert you to safety precautions.

Mark	Meaning
	Danger: This indicates that improper handling may result in a dangerous situation, which may lead to a serious accident, including death or serious injury.
	Danger: Improper handling may result in electrical hazards such as electric shock, resulting in death or serious injury. This is used when a serious accident could occur, such as serious injury.
	Warning: Improper handling may result in a dangerous situation that may result in death or serious injury. Indicates cases that could lead to serious accidents.
	Warning: This indicates that improper handling may result in dangerous electrical situations, such as electric shock, which may lead to serious accidents, including death or serious injury.
	Warning: This table shows that the surface is hot and touching it may result in burns or other injuries.
	Caution: Indicates that improper handling may result in injury or accidents. *Even in this case, it may lead to a serious accident depending on the situation, so please be sure to follow the instructions.
	Note: Instructions required for operation and cautions to be noted when handling.

2. Safety precautions

When operating a robot, please be sure to read and observe these precautions. Symbol marks are also affixed to the product in accordance with the same standards.

2.2 Robot Safety (Important)

To reduce risk, this robot and robot controller are equipped with limit monitoring mechanisms for current, speed, position, torque, etc., as well as self-interference detection functions. However, these functions have not been certified for safety, and do not guarantee safe operation. Therefore, when operating the robot, please make sure that there are no people or objects within the robot's range of motion before proceeding.

If you intend to carry out collaborative work that involves physical contact between humans and robots, please test thoroughly in advance and carry out the work in a safe environment at your own risk.

	Danger: The robot may go out of control due to unexpected software or hardware malfunctions, unintended usage, etc. When performing collaborative work that involves physical contact between humans and robots, be sure to keep the emergency stop button ready to be pressed instantly at any time.
---	--

2.3 Prohibited Matters

Do not disassemble the robot body or the robot controller. If you do, you will be responsible for any damages, whether caused by a fee or free of charge. Regardless, it will not be covered by repair.

	Warning: Disassembling the robot body or robot controller may pose a risk of electric shock or damage to the robot. Please refrain from disassembling the product as this may cause it to break down.
---	--

Do not connect or disconnect electrical cables connected to the robot body, gripper, hand, robot controller, power supply, shunt regulator, etc., or insert or remove connectors when your hands are wet or the power is on. Doing so may cause damage to the robot body, gripper, hand, or robot controller.

Breakdowns due to the above reasons will not be covered by free repairs.

	Warning: If you cut off electrical cables connected to the robot body, gripper, hand, robot controller, etc., or insert/remove connectors while your hands are wet or the power is on, there is a risk of electric shock. power supply is turned off.
---	--

2.4 Disclaimer

Except as provided by law, we shall not be liable for any malfunction or damage to this product or any damage to the user (including incidental damages arising from the use or inability to use this product, such as loss of business profits or business interruption) caused by any of the following reasons:

- If the user uses this product in a manner other than that specified in this manual.
- If the user uses this product in an environment other than the operating environment specified by our company.
- If the user uses this product in violation of any of the prohibited acts set out in this manual.
- When caused by force majeure such as fire, earthquake, flood, or lightning.

Please note that the contents of our software (both binary and source code) and manuals may be changed without prior notice.

3 - Hardware Overview

This section provides an overview of this robot and the hardware required to operate it.

3.1 Robot specifications

Robot product name Dry-AIREC	Robot model
number TRBD-A Total: 36 axes	Left arm: 7
Degrees of Freedom	axes Right arm: 7 axes Left hand: 6 axes Right hand: 6 axes Torso: 3 axes Head: 3 axes Cart: 4 axes Approx .
Total	166 cm
length Total weight	Approx. 150
camera	kg Depth camera (Intel RealSense SR300): 1 unit Stereo camera (e-con Systems See3CAM CU55): 2 units Fisheye camera (ELECOM OMNI shot mini) : 1 unit Touch sensor Pressure sensor Battery nominal power
Pressure	DC51.8V / 26.0Ah(1346Wh)
General-purpose digital input/output force	Input: 7ch Output: 7ch
General-purpose analog input/output	Input: 8ch Output: 2ch
Power supply for connecting external devices	DC12V•2A / DC5V•3A

* For detailed specifications of the robot (basic joint specifications, maximum payload, physical parameters, DH parameters, etc.) , please contact the manufacturer.
Please refer to the product DataSheet .

3. Hardware Overview

3.2 Overall Hardware Configuration

The overall hardware configuration is shown in the diagram below. For details on how to use it, please refer to the Setup chapter.

... image:: /_resources/images/hardware_overview/hardware_overall_configuration_TRBD.png

scale 40%

align center

3.2.1 Robot body

It consists of a dual-arm with hands on both hands, a torso (waist/neck), and an omnidirectional trolley.

All joints in the arms and waist are equipped with torque sensors, which allows not only position and speed control but also torque control. Pressure sensors are built into each part of the body, and various cameras, speakers, and microphones are installed in the head, so it is possible to configure applications that allow the robot to recognize the external environment and generate movements.

The torso contains two controllers: Controller 1 , which controls the robot's posture, and Controller 2, which performs calculations such as image processing. It is equipped with a PC , and the mobile cart is also equipped with a battery, allowing it to operate in a standalone mode.

3.2.2 Battery Switch

This is the switch that supplies battery power to the entire system. When the robot is not in use, turn off the PC . Make sure to turn it off after shutting down.

Note 	If the battery switch is on even after shutting down the PC and shutting down the system, the robot's internal circuitry will continue to consume power. Be sure to turn the battery switch off after use . If the battery becomes over-discharged, it may not be able to be restored by normal charging.
--	---

3.2.3 Battery power light (green)

This light comes on when the battery switch is turned ON and battery power is being supplied to the system.

3.2.4 Power supply voltage drop warning lamp (red)

When the battery level is below 10 % and charging is required, the light will flash with an alert sound. When this light starts flashing, charge the battery immediately. Please note that if you continue to consume the battery without charging it in this state, the battery output will be automatically shut down to protect against over-discharge.

3.2.5 PC Start Switch

This is the startup switch for controller 1 and controller 2 inside the robot . For detailed instructions on how to start the internal PC , see the setup section below. The green LED on the left and the blue LED on the right indicate the startup status of controller 1 and controller 2 , respectively . If you press the startup switch while each controller is running, the PC is set to shut down.



3.2.6 Emergency Stop Switch

When the emergency stop switch is pressed, it cuts off the power supply to the robot's drive system. If the robot behaves unintentionally or if the robot's operation poses even the slightest risk of danger, do not hesitate to press the emergency stop switch to cut off the power supply. To return the switch to its original position, turn it clockwise to release the latch.

3. Hardware Overview



When the **Danger** Robot is operating, the operator must always be ready to press the emergency stop switch to cut off the power supply to the robot if the robot behaves unexpectedly.

stomach.

3.2.7 Depth Camera (**RealSense™ SR300**)

It is equipped with a depth camera (RealSense™ SR300) . It is connected to Controller 2 as a USB device .

3.2.8 Fisheye camera (**Insta180**)

It has a fisheye camera attached to it. It is connected to controller 2 as a USB device .

3.2.9 Stereo Camera (**See3CAM CU55**)

The head is equipped with two wide-angle cameras (See3CAM CU55) that can be used as a stereo camera.

The ESXi 5.0 and ESXi 5.0 servers are connected to controller 2 as virtual interfaces .

3.2.10 Speakers

A speaker (FOSTER 628421) is built into the underside of the head . It is connected to Controller 2 as a USB speaker.

3.2.11 Microphone

There are two microphones on the sides of the head, one on each side. They are connected to Controller 2 as USB microphones.

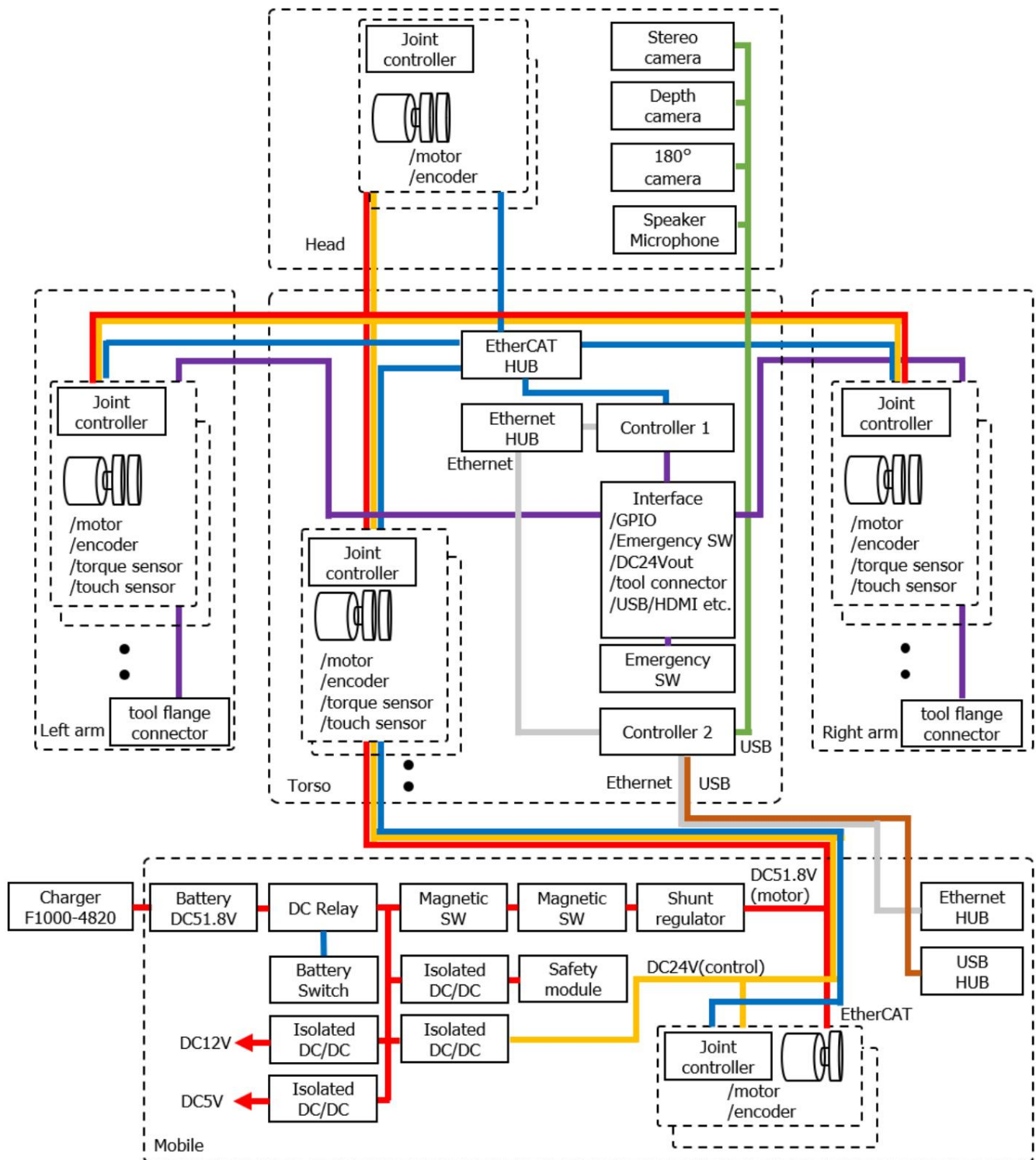
3.2.12 Charger

This is a dedicated charger for the robot. For details on how to charge the robot, please refer to the section below on how to charge the robot.

3.3 Configuration of Electric Circuit System

The overall configuration of the electrical circuit system is shown in the diagram below.

3. Hardware Overview

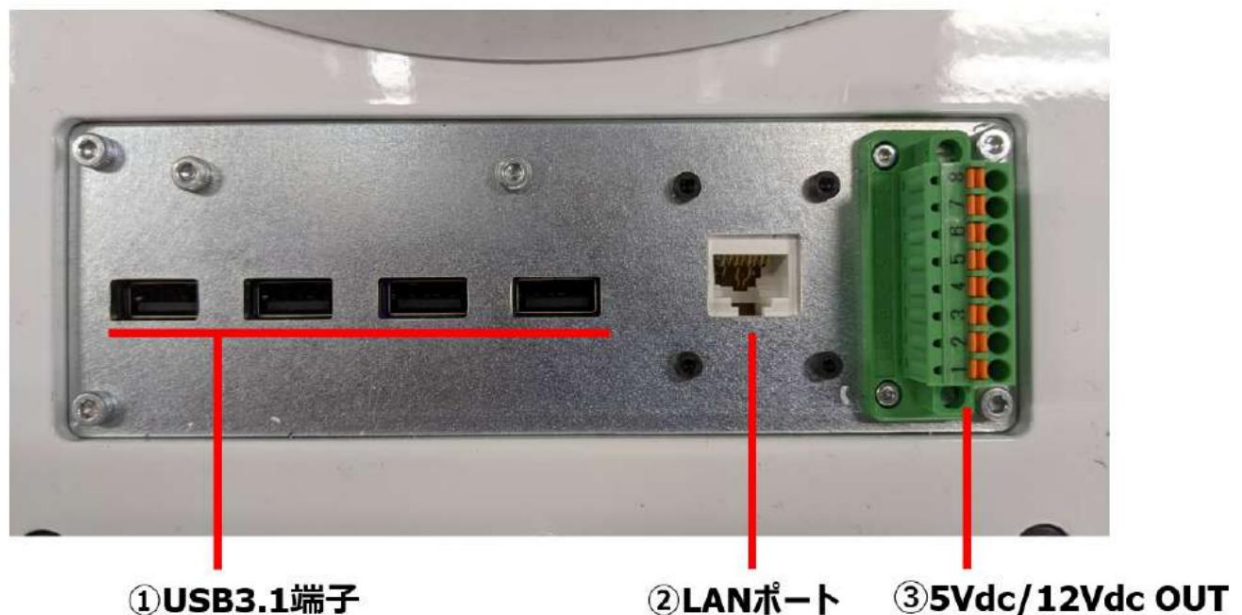


3.4 Interface

Here, we will introduce the power supply and GPIO for connecting external devices mounted on the back of the mobile cart and the robot. This section describes the service.

3.4.1 Mobile Cart Expansion Interface

The top panel of the mobile cart has an expansion USB port, a LAN port, and a power supply for external devices, as shown in the figure below. Each port has a dust cap, so please remove it when using the port.



3.4.1.1 USB 3.1 Port

USB 3.1 port for connecting external devices . Connected to controller 2 .

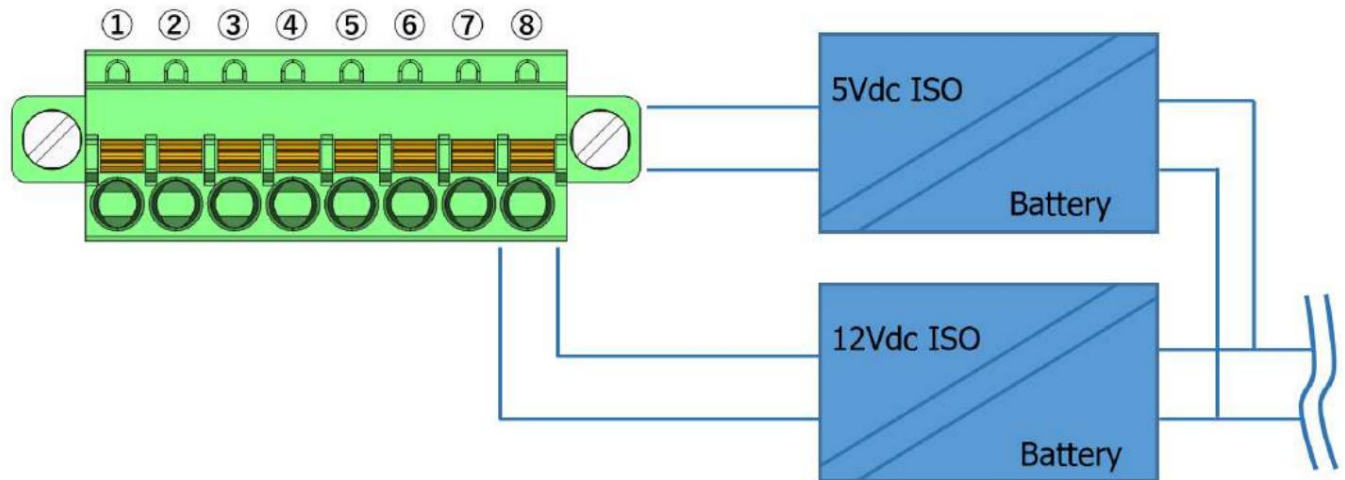
3.4.1.2 LAN Ports

This is a LAN port that can be connected to external sensors, etc. It is connected to controller 2 .

3.4.1.3 5Vdc/12Vdc OUT

It supplies DC5V and DC12V power isolated from the power supply inside the robot . It can be used as a power supply for additional sensors, etc. Please pay attention to the polarity and voltage. The terminal is a push-in type. When using a solid-wire cable or a cable with a crimped ferrule terminal, insert it until it hits the terminal block. In the case of a twisted wire, insert it while pressing the orange release pin with a flat-head screwdriver. When removing the cable, press the release pin with a flat-head screwdriver and pull it out.

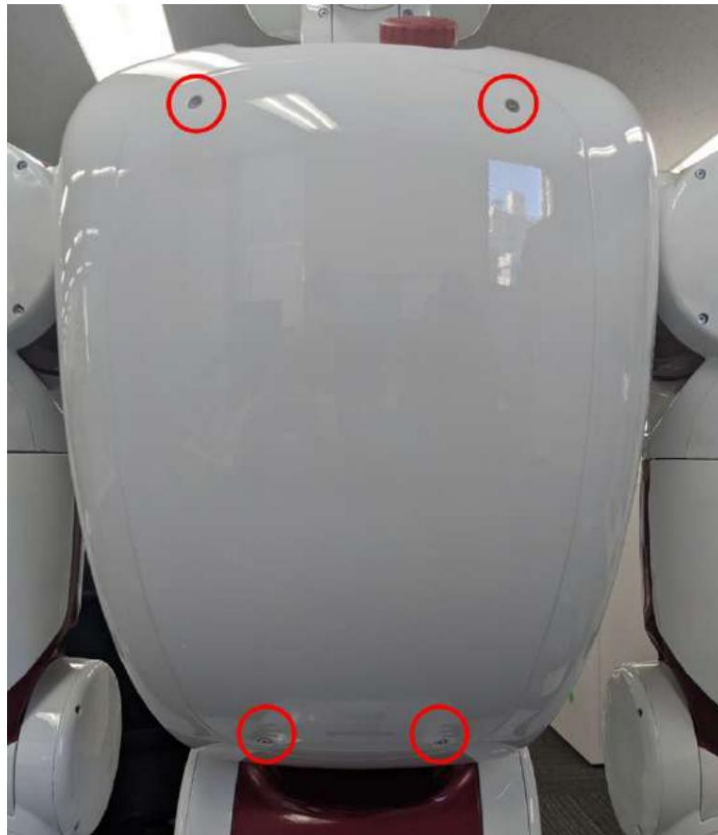
3. Hardware Overview



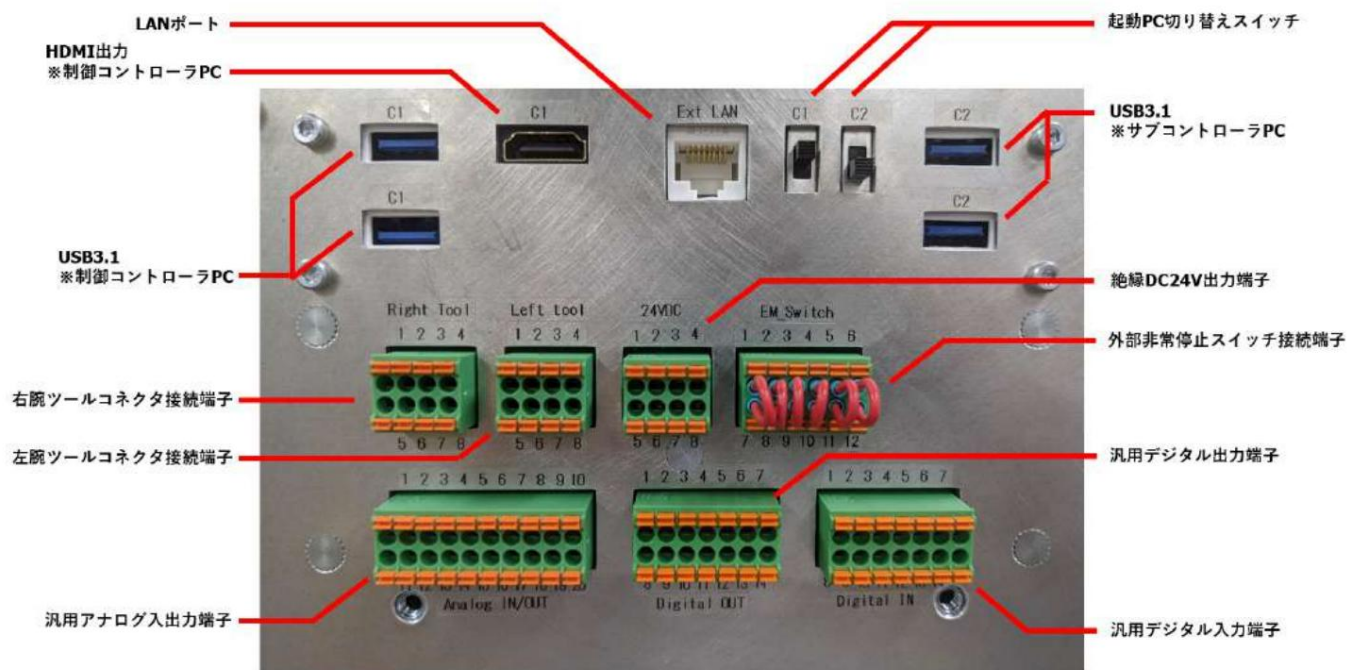
Pin No.	Voltage	remarks
1	DC5V	1, 3 total maximum 3A
2	GND1	DC5V GND
3	DC5V	1, 3 total maximum 3A
4	GND1	DC5V GND
5	DC12V	5, 7 total maximum 2A
6	GND2	DC12V GND
7	DC12V	5, 7 total maximum 2A
8	GND2	DC12V GND

3.4.2 Rear expansion interface

On the back of the robot, there are expansion interfaces such as USB terminals and general-purpose IOs connected to each PC . The interface can be accessed by removing the four bolts circled in red in the diagram below with a wrench .



The schematic diagram of the rear expansion interface is shown below.



3. Hardware Overview

3.4.2.1 USB 3.1 Port

USB 3.1 ports for connecting external devices . The left port is connected to controller 1, and the right port is connected to controller 2.
can be done.

3.4.2.2 HDMI port

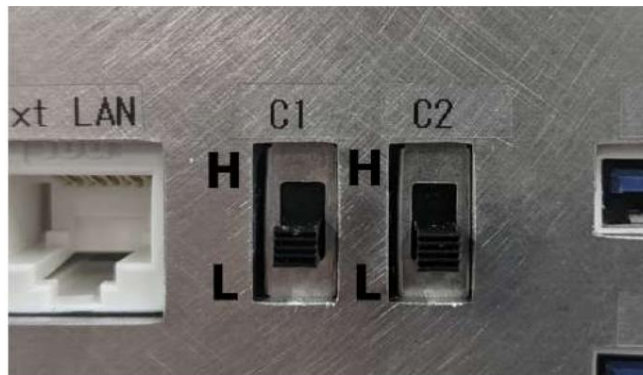
This is the HDMI output of controller 1 .

3.4.2.3 LAN Ports

This is a LAN port that is connected to the robot's internal network . It is used to connect to an external network.

3.4.2.4 Boot PC Switch

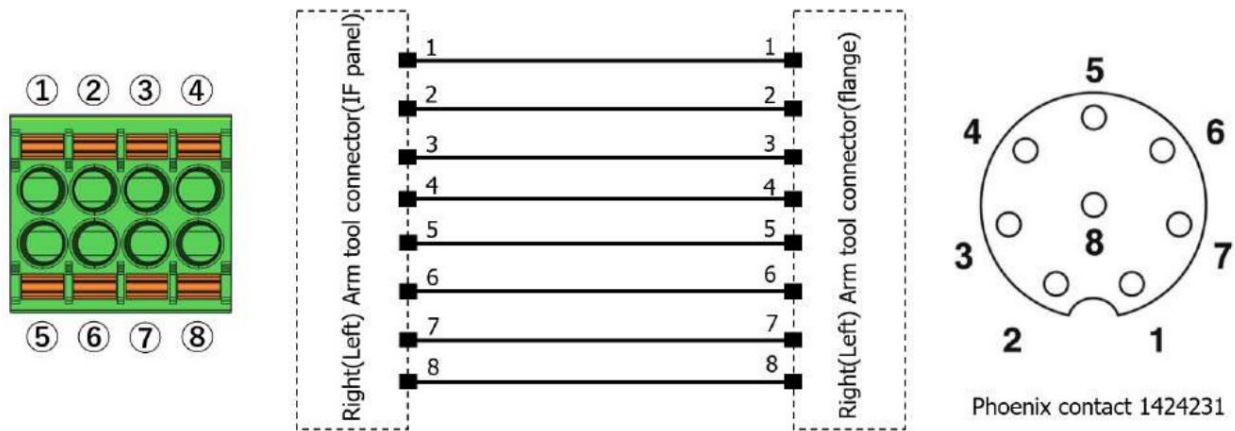
This changes the PC to be started in conjunction with the PC startup switch. One PC startup switch can be used to switch between Controller 1 and Controller 2. The two controllers can be started simultaneously or selectively. By setting the start PC changeover switch shown below ,
Change the PC that is linked to the startup switch .



C1	C2	Status Operation	
H	H	State 1: Both controller 1 and controller 2 are up and running	
H	L	State 2: Only controller 1 is running	
L	H	State 3: Only controller 2 is up	
L	L	Undefined Not used	

3.4.2.5 Right (Left) Arm Tool Connector Connection Terminal

It is connected to the right arm (left arm) end tool connector via the wiring inside the arm . A current of 1A can be passed per wire .
Also, 1-2, 3-4 , 5-6, and 7-8 are cross cables with paired wires, allowing differential signals to pass through.
It is also possible.



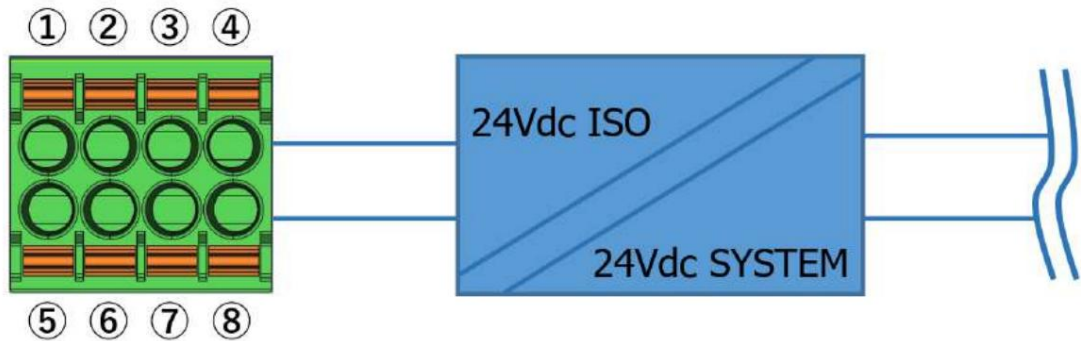
Please use Phoenix Contact's M8 shell connector "1404182" as the mating connector for the tool flange connector.

Please use.

3.4.2.6 Insulated 24V DC output terminal

It supplies DC24V power isolated from the robot system . It can be used as a power source for isolated digital input/output, which will be described later.

Masu.

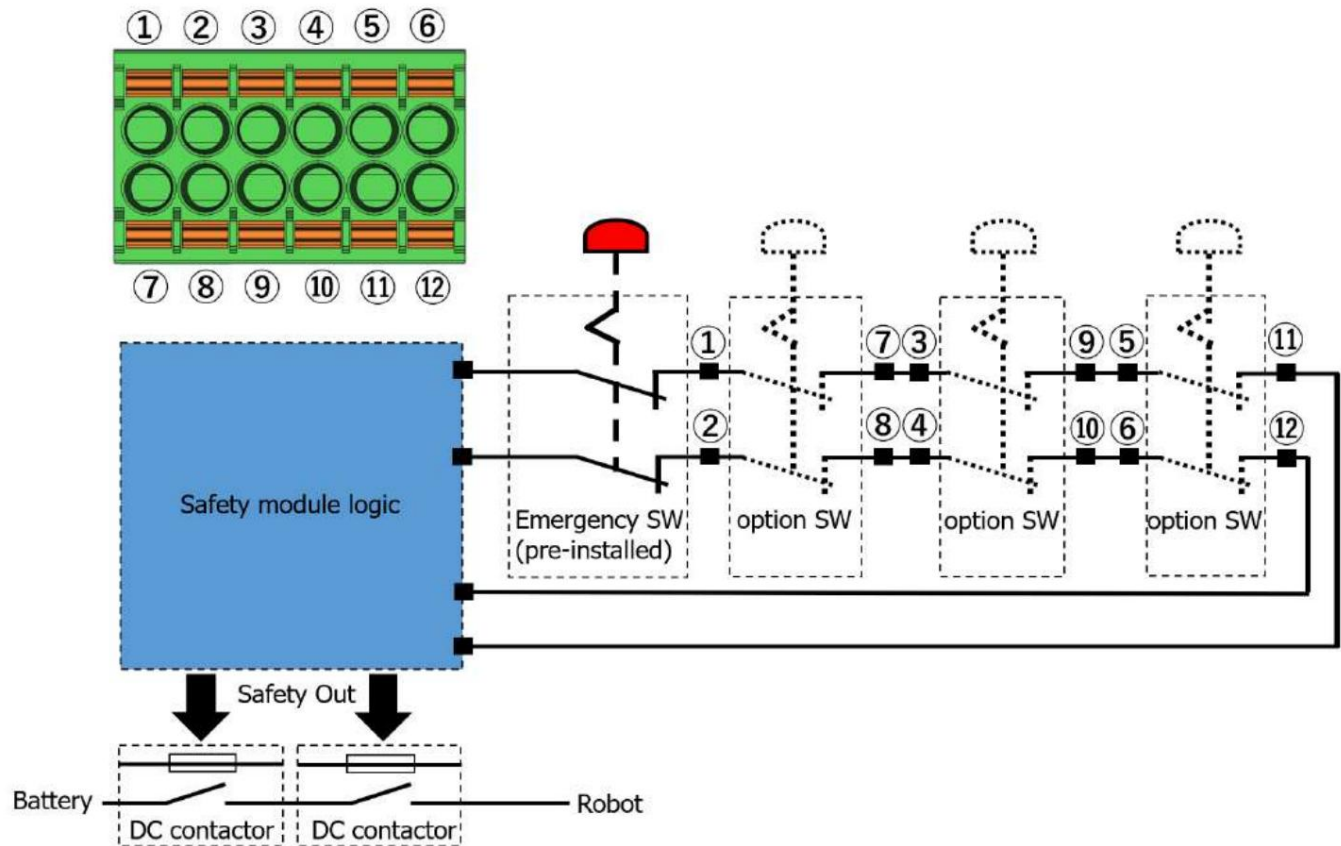


Pin No.	Voltage	remarks
1y4	DC24V	1 to 4 total maximum 2A
5y8	GND	

3. Hardware Overview

3.4.2.7 External emergency stop switch connection terminal

In addition to the emergency stop switch that is installed on the back of the robot as standard, you can connect up to three emergency stop switches. When connecting additional emergency stop switches, remove the jumper cable that is connected at the time of shipment. Also, use an emergency stop switch with duplicated b contacts as shown in the figure below.

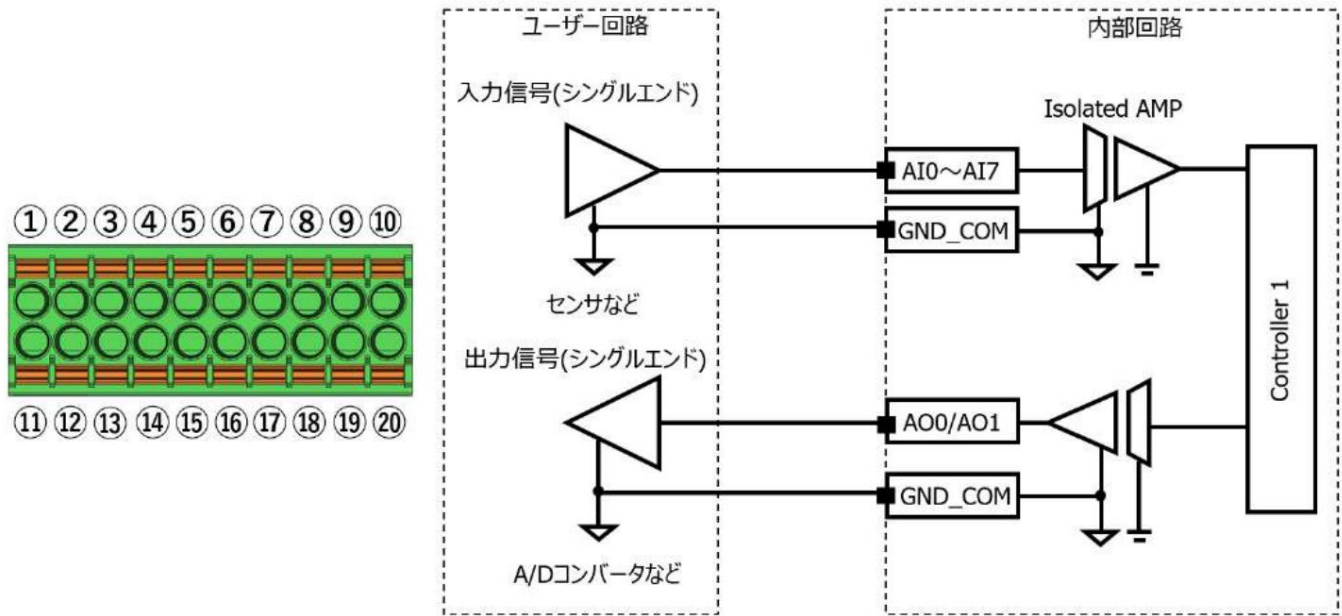


If no additional external emergency stop switch is connected, short 1-7, 2-8, 3-9, 4-10, 5-11, and 6-12.

3.4.2.8 General-purpose output terminals

General-purpose analog input/output terminals

Isolated analog output for connecting external single-ended analog output sensors or devices that require analog voltage output. Log I/O terminal. Can be accessed from controller 1.



Pin No.	name	function
1	AI0	Analog Input 0
2	AI1	Analog Input 1
3	AI2	Analog Input 2
4	AI3	Analog Input 3
5	AI4	4 analog inputs
6	AI5	Analog Input 5
7	AI6	6 analog inputs
8	AI7	Analog Input 7
9	AO0	Analog Output 0
10	AO1	Analog Output 1
11~20	GND_COM Common	Ground

Analog Input	
Input voltage	± 10V± 10V
range Sampling rate	10kS/s max ±
A/D converter resolution	11bits
Input impedance	1M Ohm

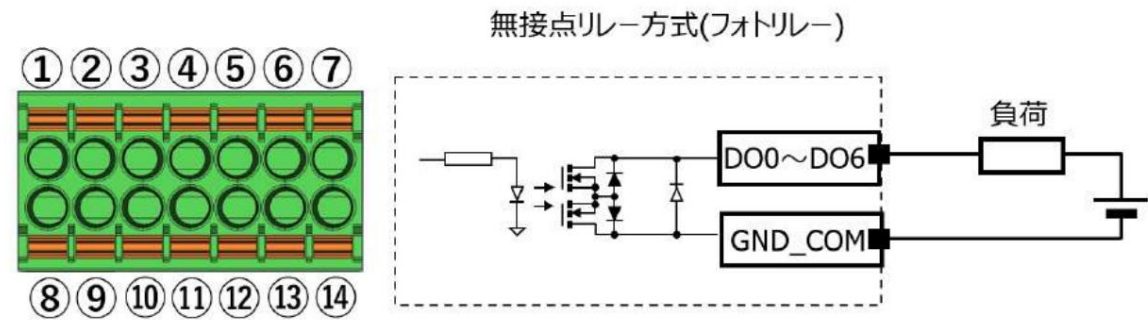
Analog Output	
Output Voltage Range	0V~5V
D/A converter resolution	12bits
Load drive capability	15mA typ.

*The sampling rate varies depending on the number of analog input channels used. For example, when using 4 channels ,
The sampling rate of each channel is 10k/4 = 2.5kS/s .

3. Hardware Overview

General-purpose digital output terminal

You can connect loads such as LEDs and relays. The output is a non-contact relay (photorelay) type. Connection is a sink type. It can be accessed from controller 1 .

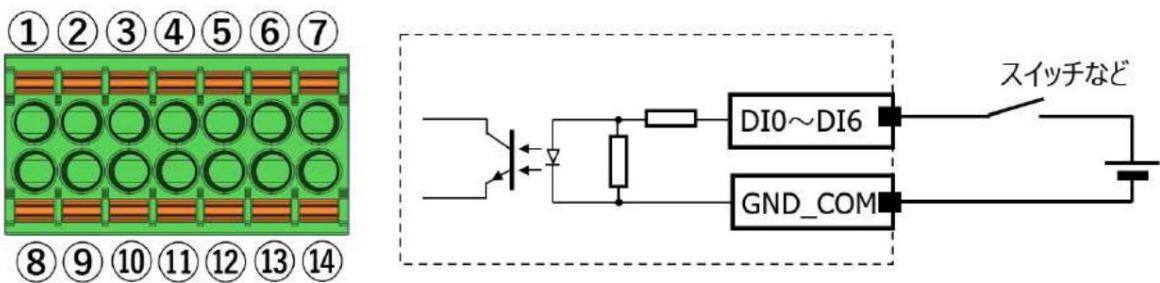


Pin No.	name	function
1	DO0	Digital Output 0
2	DO1	Digital Output 1
3	DO2	Digital Output 2
4	DO3	Digital Output 3
5	DO4	Digital Output 4
6	DO5	Digital Output 5
7	DO6	Digital Output 6
8~14	GND_COM Common	Ground

Output method	Non-contact relay (photorelay)
Rated voltage	Maximum DC26.4V
Output load current	Max 400mA

General-purpose digital input terminal

Used to input digital signals such as connecting a switch. The connection is a sink type .
Seth can do it.

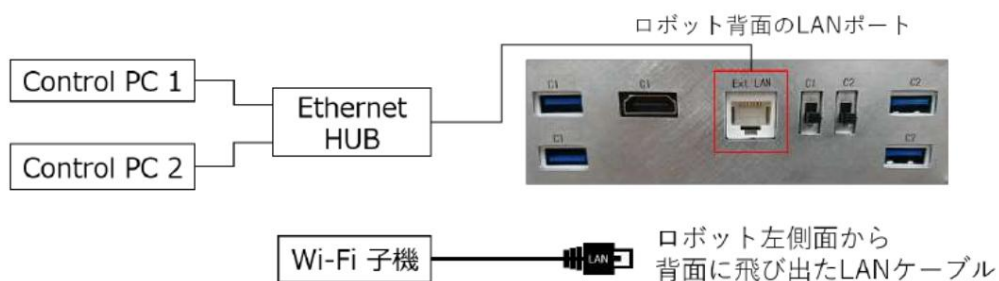


Pin No.	name	function
1	DI0	Digital Input 0
2	DI1	Digital Input 1
3	DI2	Digital Input 2
4	DI3	Digital Input 3
5	DI4	Digital Input 4
6	DI5	Digital Input 5
7	DI6	Digital Input 6
8~14	GND_COM Common	Ground

Rated input	DC5V~DC24V
voltage Allowable input	DC4.5V~26.4V
voltage range Input	3.3k Ohm
impedance	1.8mA typ.
Input current Off-	DC1.0V or less
state voltage On-state voltage	DC4.0V or higher

3.5 LAN Device Configuration

The diagram below shows the LAN device configuration used for communication between the controller and the user PC .



Controller 1 and Controller 2 are connected to the same Ethernet HUB , which is connected to the back of the robot. It is connected to the LAN port. Connect this LAN port to a router, etc., and the user's PC will also be on the same network. By connecting it, communication with the controller becomes possible. In addition, the robot itself is equipped with a Wi-Fi slave unit (manufactured by GL.iNet). The Wi-Fi adapter and the LAN cable connected to it come out from the left side of the back of the robot . By connecting this LAN cable to the LAN port on the back of the robot, controllers 1 and 2 can be connected to Wi-Fi . For information on how to set up Wi-Fi, see Setting up the [Built-in Wi-Fi Router \(Wireless Adapter\)](#) Please also refer to:

4 - Controller Overview

4.1 Robot controller

This robot has a controller 1 (hereafter referred to as C1 or control PC) built into the robot body and a control PC . It is controlled by two PCs, C1 and C2 (hereafter referred to as C2). C1 is installed on the front of the fuselage and C2 is installed on the rear of the fuselage. It has been done.

C1 is the PC that controls the robot body , and controls the motors of each axis built into the robot via EtherCAT. It communicates with the driver board, which transmits command values and receives sensor values. C2 acquires images from the head camera and This is a PC for upper control that is assumed to use these devices for image processing and to execute user applications . The various cameras, speakers, microphones, and other devices installed in the device are connected to the C2 . It can be accessed from C2 .

C1 and C2 are connected to the robot's internal network via a LAN cable, and the user can access the robot from C2. Then, the robot control core program on C1 is started, and the user application on C2 is started. It is also possible to remotely access C1 and C2 by connecting a user PC to the same LAN . For more information about network configuration, see Setting up the network.

4.1.1 Controller 1 (C1)

YOU	Ubuntu 20.04.4 LTS
Linux kernel	5.10.89 - Xenomai 3.2.1
ROS	ROS Noetic Ninjemys
CPU	Intel Core i5-8500
RAM	8GB
SSD	250GB
communication	Communication with each axis driver: EtherCAT Communication with C2, user PC , etc.: Ethernet

4. Controller Overview

4.1.2 Controller 2 (C2)

YOU	Ubuntu 20.04.4 LTS
ROS	ROS Noetic Ninjemys
CPU	Intel Core i7-8700
RAM	16GB
SSD	250GB
communication	Head cameras: USB Speaker: USB Microphone: USB Communication with C1, user PC , etc.: Ethernet

4.2 User PC

The user's PC sends operation commands to the robot controller via GUI or user-created programs, and it is used to operate the simulator. You can connect a display and keyboard to the robot controller.

Since it is possible to operate the actual robot with a PC, it is not necessary to have a user PC, but it will improve the convenience of development. It is recommended that you prepare it for the purpose.

You will need to prepare your own user PC.

For setup instructions, please refer to [Setting up your user PC](#).

	Note: ROS may become unstable in a virtual PC environment. Please use the PC environment.
---	--

The PC specifications required to run the ROS simulator are listed in the table below. Note that the following specifications are the minimum required. This requires a certain environment, and we cannot guarantee smooth operation or stability. If possible, we recommend using a computer with higher specs. We recommend that you bring your own PC.

CPU	Intel i5 or higher processor
Memory	8GB or more
Disk capacity	40GB or more
Graphics card	A dedicated graphics card, not a graphics chip built into the CPU . In particular, NVIDIA graphics cards are recommended. This is mainly recommended for stable operation of numerical calculations and 3D rendering of Gazebo/Rviz. For the recommended PC specifications for Gazebo, please refer to the following website. Please refer to http://gazebosim.org/tutorials?tut=guided_b1#Systemrequirements
YOU	Ubuntu 20.04.4 LTS
ROS Version	ROS Noetic Ninjemys

5 - Software Overview

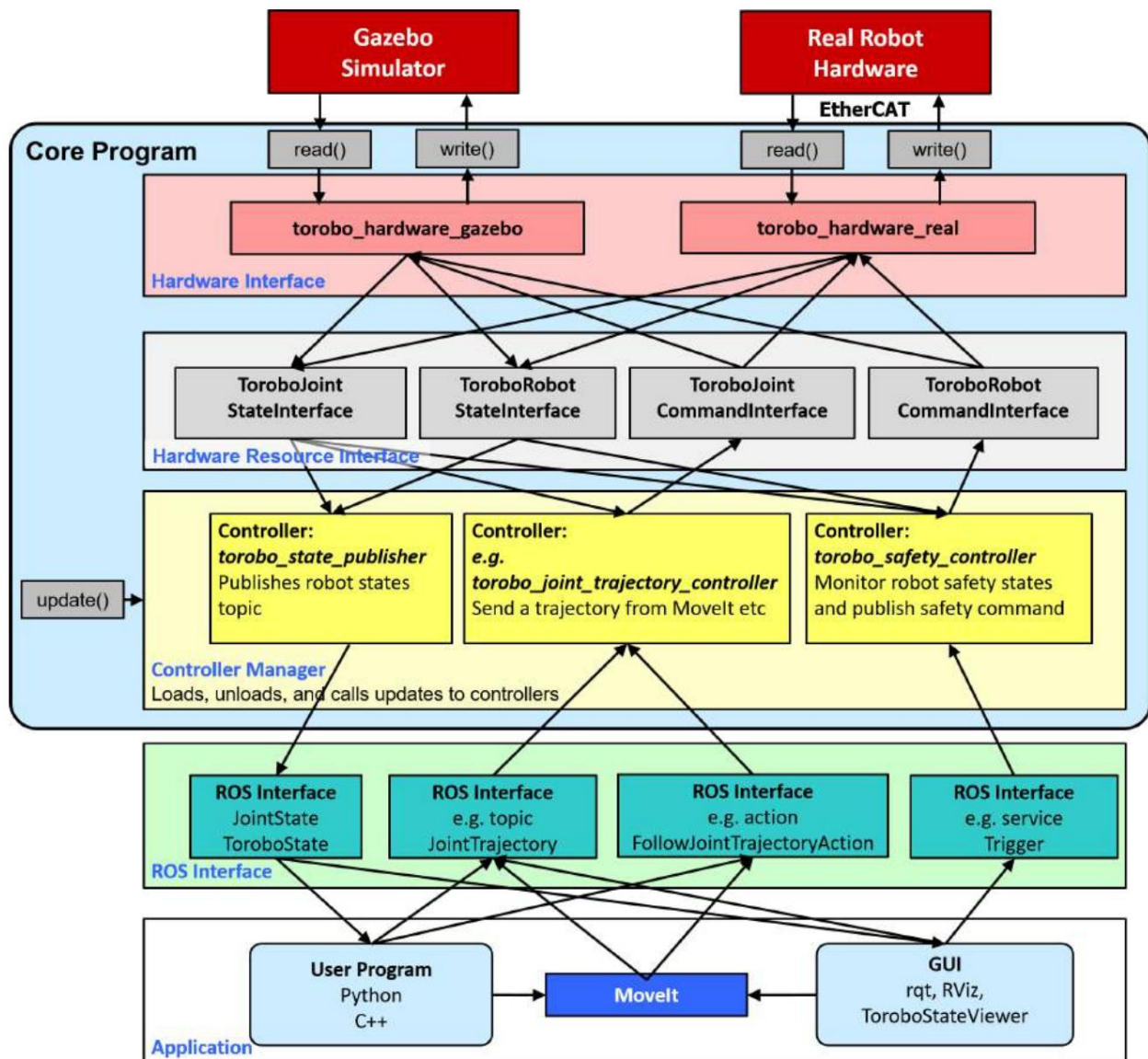
This section provides an overview of the software for this robot.

5.1 ROS

The software for this robot, Dry-AIREC , is implemented based on ROS , which is widely used around the world as middleware for robots . For a basic explanation of ROS, please refer to [the ROS overview](#) . Even if you have no knowledge of ROS, you can perform basic operations of this robot by following the contents of this manual. However, if you are thinking of creating your own applications or performing more advanced robot control, knowledge of ROS is essential. [If you do not have much experience using ROS, please refer to http://wiki.ros.org/](#) We recommend that you study the following separately by referring to the following:

5.2 Overall System Configuration

The overall system configuration is shown in the diagram below.



The core program shown in the figure is the core of the system. When running a real robot, the core program "toroboHardware_real" runs on the control PC in the robot controller, and when running a Gazebo simulation, the core program "toroboHardware_gazebo_plugin" runs on the user's PC to operate the robot.

The core program is implemented in accordance with `ros_control`, a standard ROS real-time control framework, and it is possible to use a common control program implementation and ROS Interface (Topic, Service, Action) whether you operate the robot using a real device or a simulator (e.g. Gazebo). (For more information on `ros_control`, see http://wiki.ros.org/ros_control (see

The differences between the real machine and the simulator are absorbed by a function called "hardware_interface". The process of sending the joint command values output by the controller to the actual joints and the process of obtaining the current joint state are different between the real machine and the simulator. Therefore, a "hardware_interface" is prepared for each of the real machine and the simulator, and the processes for issuing commands to the joints and obtaining the current state are implemented here. When switching between the real machine and the simulator, you can operate the robot with the same program by switching this "hardware_interface".

In the core program, the following three processes, read, update, and write, are executed periodically to update the robot's control is realized.

1. **read** : Read from the robot hardware (or the robot in the simulator) via the hardware_interface

Obtain information such as current position, speed, and torque from

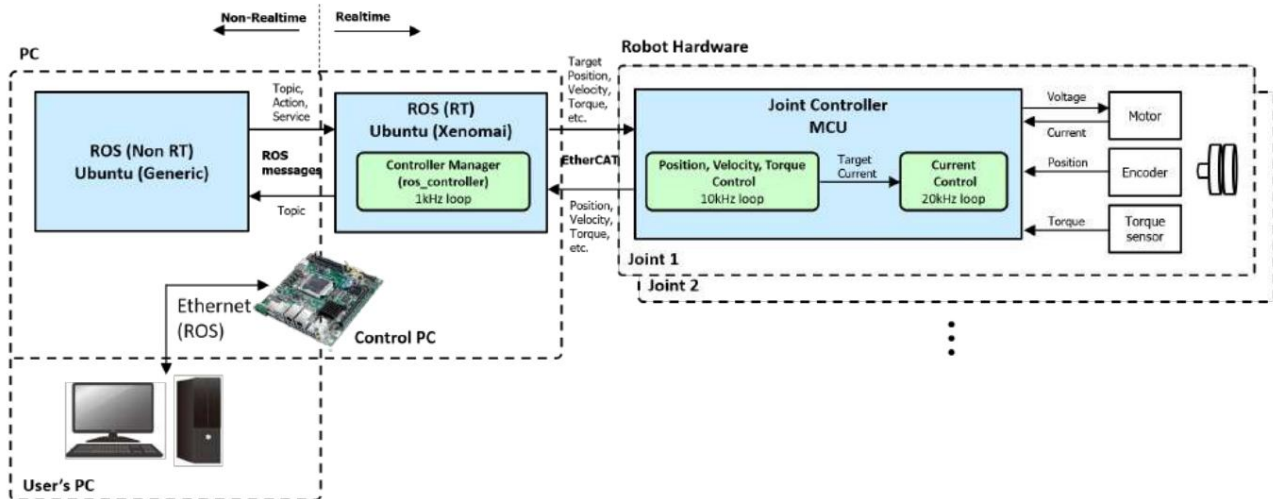
2. **update** : The Controller (hereafter referred to as ROS Controller or `ros_controller`) updates each joint of the robot.
Calculates commands such as position, speed, and torque
3. **write** : Position, velocity, torque, etc. to the robot hardware (or the robot on the simulator)
Set the directive

Each "`ros_controller`" is implemented in the form of a plug-in, and the loading and unloading of these, as well as the switching of the "`ros_controller`" that operates them, are realized by a function called the "Controller Manager" . By switching this "`ros_controller`" , it is possible to change the control mode from joint position control mode (eg `torobo_joint_trajectory_controller`) to joint impedance control mode (eg `torobo_joint_impedance_controller`) while the robot is moving.

From the user PC or robot controller PC , the user can subscribe to ROS topics corresponding to each "`ros_controller`" via the core program's ROS Interface to obtain the current robot status, and can send control commands to the robot by publishing ROS topics , services, and actions .

5.3 Overview of control implementation method

This robot achieves full-body motor control through close cooperation between the control software on the control PC and the joint controller built into the robot. The following diagram shows an overview of the control implementation method, including the connections and roles of the control-related software and hardware, and the cycles of each control loop.



5.3.1 Control loop in the joint controller

The robot has a built-in joint controller for controlling each joint. These measure sensor information such as the current joint position and torque from the encoder and torque sensor built into the joint, and send it to the control PC via EtherCAT. The motor is then controlled according to the target position, speed, and torque commands sent from the control PC . All of this is done within a periodic loop with real-time characteristics, and each cycle is completed by communicating with the control PC.

EtherCAT communication is 1 [kHz] , position, speed and torque control is 10 [kHz], and current control (vector control) is 20 [kHz] .

5. Software **Overview**

5.3.2 Control loop in the control **PC** (real-time part)

The control PC performs a loop operation in which it receives the current state of each robot joint from the joint controller via EtherCAT , calculates control commands based on the current state, and then sends the control commands to the joint controller. This loop is executed at a cycle of 1 [kHz] within the core program , and has the same real-time characteristics as the joint controller.

Normally , ROS or an OS that runs on a normal Linux kernel such as Ubuntu does not have real-time capabilities, so it is not possible to realize such a control loop. However, the control PC for this robot uses Xenomai (<https://xenomai.org>) , a hard real-time extension of Linux . This functionality is achieved by combining the ROS real - time control framework "ros_control" .

5.3.3 Applications in the robot controller PC and user **PC** (non-real-time part)

Software that is more advanced than "ros_control", such as motion planning by MoveIt! or applications that combine advanced recognition to realize robot operation , usually requires a long calculation time and is not deterministic, so it does not have real-time properties. In addition, to guarantee real-time properties, it is necessary to run the software on a real-time OS , and to implement it, you need knowledge of real-time OSes and various real-time processes, which is not easy.

However, ros_control also has the ability to act as a buffer between the non-real-time and real-time parts, and the various ros_controllers implemented in this robot are designed to utilize this function. Therefore, by sending robot operation commands generated by a user-created upper-level application to various ros_controllers via ROS Topics and Actions , it is possible to realize the robot's operation without compromising the real-time nature of ros_control and below.

5.4 Robot Controller **PC** Software

This robot is equipped with two PCs (C1, C2) as robot controllers. For details on the controller hardware , refer to [Robot Controller](#) . The necessary software for each controller PC is
The software is already set up at the factory.

5.4.1 Controller 1 (**C1**) Software

A core program that controls all the joints of the robot is installed in Controller 1 (C1) . When operating the actual robot, the core program must be started on C1 . When the core program is running, it is possible to monitor the current state of the robot and send control commands from C2 or a user PC connected to the same LAN .

5.4.2 Controller 2 (C2) Software

Controller 2 (C2) has installed programs for acquiring data from the various cameras, speakers, microphones, and other devices mounted on the head, and for issuing operational commands. In addition, by executing user-created programs on C2, it is possible to realize user applications, such as processing images acquired from a camera and issuing operational commands to the robot.

5.5 Software on User PC

Users can use the ROS operation tool 'Rqt' and visualization tool 'Rviz' on a PC to give motion commands to the robot. Users can also use CLI, Python scripts, and C++ programs to give motion commands to the robot directly using ROS messages. By using the trajectory planning framework MoveIt!, it is also possible to execute trajectory planning that avoids obstacles in the environment.

When operating the robot with a simulator (e.g. Gazebo), start the core program on the user PC. (It is also possible to start the simulator on the control PC, but the control PC does not have sufficient specifications to run the simulator. When running the simulator, we recommend that you basically use the user PC.)

By running the simulator on the user's PC, it is possible to perform offline simulations using only the user's PC, without using the actual robot or control PC. By checking the operation of the control program created by the user in advance using offline simulation, it is possible to smoothly move on to verification on the actual robot.

6 - Setup

6.1 Robot setup

Caution: Please 	be sure to observe the following points when installing the robot: 1. Make sure there are no walls or objects within the range of the robot's movement. 2. Ensure that the base is tall enough to accommodate the range of the robot's movement . 3. Furthermore, ensure that the base is heavy and wide enough to prevent it from falling over even when the robot experiences inertia.
---	---

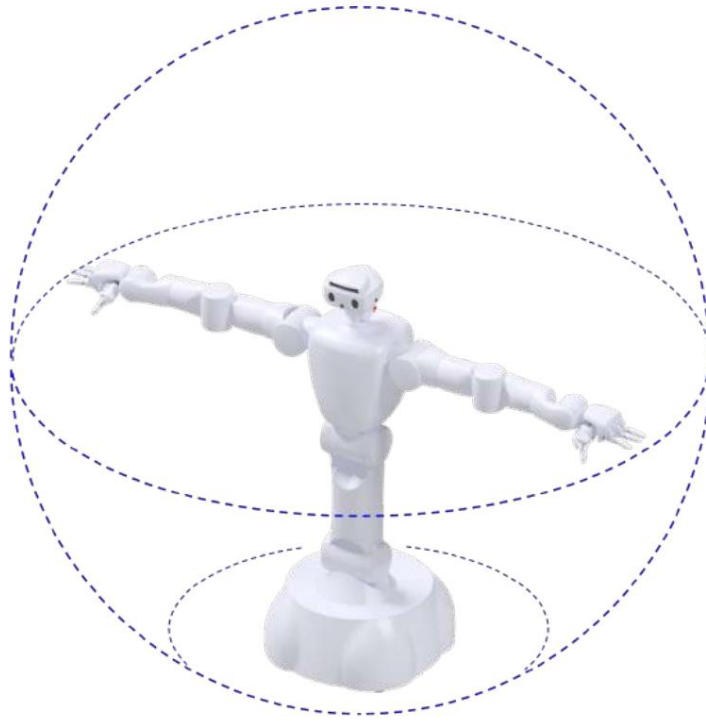
6.1.1 Install the robot

An example of how to install the robot is shown in the figure below. Install the robot in the standard position (home position) as shown in the figure.

6. Setup



The robot's range of motion is shown in the figure below.



6.2 Robot Controller Setup

	Please be sure to observe the following points when installing Danger . 1. Do not touch the robot, robot controller, connected power supply, cables, or wiring with wet hands. 2. Always turn off all power before changing wiring or cable connections or connecting or disconnecting them. This should be done in a clean, dark place.
---	--

6.2.1 How to charge the battery

This section explains how to charge the battery built into the robot.

1. Make sure the power switch on the charger is turned off.

6. Setup



2. Insert the charger's robot connector into the robot's charging port so that the keys align, then press the connector flange. Be careful not to let it slip out.



3. Insert the charger's AC plug into an AC 100V outlet. The charger can provide up to **1500W** of power. Since it consumes electricity, be sure to connect the charger to the outlet separately.

4. Turn on the charger's power switch. While charging, the charging lamp will flash as shown below. When charging is complete, the charging completion lamp will light up.

5. When charging is complete, turn off the charger and remove the charging cable from the robot. When recharging, turn the charger's power switch back on. If you start charging when the battery level is below 10%, which is when the battery alert sounds, it will take approximately one and a half hours to fully charge. *The charging time may vary depending on the battery usage. It will become





Note: If the charging cable is left plugged into the robot, the charger's detection circuitry will be activated.

This will consume a small amount of battery power, so if you do not use the robot for a long period of time, If this occurs, please disconnect the charging cable from the robot.

6.2.2 Charging Considerations

It is possible to charge the robot or robot controller while it is running, but in this case the maximum capacity Charging may not be fully charged and may stop at around 90%. Turn off all power to the robot before charging. This allows the battery to be charged to near its maximum capacity.

Please also see ["Battery error" message displayed in Error Device of Robot Monitor](#) .

6.2.3 Charger markings

Detailed charger indication is as shown in the table below.

	meaning
Display Charging lamp	The battery is charging.
flashes Charging	The battery has charged successfully.
completion lamp is on Alarm	There is no battery connection.
lamp flashes quickly Alarm	Charging time over
lamp flashes slowly	Battery abnormality
Alarm lamp is on Battery remaining display	The remaining battery level is displayed in four levels.

6. Setup

6.2.4 Battery Precautions

 Caution:	Please be sure to observe the following points regarding batteries. 1. Always charge with the dedicated charger. 2. Insert the charger into a separate outlet that can handle a 1500W load. Sharing the outlet with other devices may cause overheating or fire. 3. Do not use the device in hot environments such as under the blazing sun. 4. If the power supply voltage drop alarm sounds, charge the device immediately. If you continue to use the device in this state, the battery output will stop due to over-discharge protection. If you leave the device in this state, the battery will become deeply discharged and will not be able to be restored by charging it. 5. If you do not plan to use the device for a long period of time, fully charge it once every three months. This can cause the cell to expand due to self-discharge and dark current. 6. Be careful not to block the charger fan vent. 7. Do not touch the charging connector or charger with wet hands.
--	--

6.2.5 Power on the robot

Flip the battery switch up to turn on the robot. When the robot is turned on, the battery power lamp will turn green.



6.2.6 Power on the controller PC

Press the PC startup switch once. Both controller 1 and controller 2 PCs will start up. To power on only one of the PCs, refer to the section on the startup PC changeover switch on the rear expansion interface described later.



6.3 Setting up the user PC

If you want to operate the robot using a user PC or operate the robot in a simulator environment, you need to set up the user PC. If you want to operate the actual robot using only a PC with a built-in robot controller, you can skip this procedure.

Note: An internet connection is required to set up the user PC.



So please prepare in advance and read the following steps.

6.3.1 Recommended PC environment

Note: ROS may become unstable in a virtual PC environment.



Please use the PC environment.

6. Setup

CPU	Intel i7 or higher processor
Memory	8GB or more
Disk capacity	40GB or more
Graphics card	This must be a dedicated graphics card, not a graphics chip built into the CPU . NVIDIA graphics cards are especially recommended. Graphics cards are mainly recommended for stable operation of numerical calculations and 3D rendering in Gazebo/Rviz . For the recommended PC specifications for Gazebo, please refer to the following website: http://gazebosim.org/tutorials?tut=guided_b1#Systemrequirements
YOU	Ubuntu 20.04.4 LTS (The English version is recommended as problems may occur with the Japanese version)
ROS Version	ROS Noetic Ninjemys

6.3.2 ROS installation method

If you have not already installed Ubuntu 20.04.4 LTS , install Ubuntu 20.04.4 LTS on your PC . Next, if you have not installed ROS on your PC , install it using the following procedure.

On the user's PC , enter "[Ctrl+Alt+t]" on the keyboard. This will launch the terminal.

Install the libraries necessary to use programming languages such as C++ and Python on Ubuntu .

```
$ sudo apt update -y $ sudo apt
install -y build-essential python3-dev python3-pip curl
```

Next, install ROS.

```
$ sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list' $ curl -k https://raw.githubusercontent.com/ros/rosdistro/master/ros.key | sudo apt-key add - $
sudo apt update $ sudo apt
install -y ros-noetic-desktop-full python3-rosdep $ sudo rosdep init $ rosdep update $ echo "source /opt/ros/noetic/setup.bash" >>
~/.bashrc $ sudo apt
install -y python3-rosinstall $ sudo apt install -y python3-catkin-tools $ sudo apt install -y ros-noetic-moveit $ sudo apt install -y ros-noetic-moveit-visual-tools $
sudo apt install -y ros-noetic-gazebo-ros-control $ sudo apt install -y ros-noetic-ros-controllers $ source ~/.bashrc $ echo "source `catkin locate --shell-verbs`" >> ~/.bashrc $ source ~/.bashrc
```

6.3.3 Creating a ROS workspace

Once the ROS installation is complete, create and initialize a workspace for ROS.

```
$ mkdir -p ~/catkin_ws/src $ cd ~/catkin_ws  
$ catkin init
```

6.3.4 Installing provided ROS packages

By installing the deb binary and source code provided with the robot on the user's PC ,
You can create an operating environment for Dry-AIREC .

Before performing the following procedure, update the software installed in Ubuntu by performing the following operations on the user PC .
Please update it to the latest version.

```
$ sudo apt update $ sudo  
apt upgrade
```

6.3.4.1 Installing deb binaries and source code

Copy "install_torobo_ros.sh" and the included "resources" directory in the torobo_ros folder of the delivered software to the user's PC . Start the terminal, move to the destination directory, and then execute the installation script with the following operations. Administrator privileges are required during execution, so if you are asked to enter a password, enter the password of a user with administrator privileges.

```
$ sh install_torobo_ros.sh ~/catkin_ws
```

The above assumes that the workspace for ROS is in "~/catkin_ws" , so if you have created the workspace in a different location, please read accordingly. When you run the above script, the deb binary is installed .
Install the installer and source code.



Note: Be sure to unzip the delivered software zip file on a Linux PC . If you unzip the file on a Windows PC , the file execution permissions will be changed and the ROS program will not run properly.

Next, change the workspace build setting to Release .

```
$ catkin config -DCMAKE_BUILD_TYPE=Release
```

After performing the above operations, run the "catkin config" command. If the following is displayed, the settings have been configured correctly.

6. Setup

(Omitted)

Additional CMake Args : `-DCMAKE_BUILD_TYPE=Release`

If you change the build setting to "Release" using the above settings , the compiler will optimize the source code in the workspace when it is built. This will speed up program execution, so we recommend building with the "Release" setting in most cases. However, this can cause problems when debugging, so in that case, change the build setting to "Debug" (or "RelWithDebInfo") using the following command.

```
$ catkin config -DCMAKE_BUILD_TYPE=Debug
```

After that, we will use rosdep to resolve dependencies on external ROS packages. When you run the following command, external ROS packages that the source code in the workspace depends on will be automatically listed, and will be installed sequentially using the apt install command. (For more information about rosdep , see <http://wiki.ros.org/rosdep> .) (see

```
$ cd ~/catkin_ws $ rosdep
update $ rosdep install --
from-paths src --ignore-src -r -y
```

Finally, build the source code you have placed.

```
$ catkin build $ catkin
source
```

If no errors are displayed, the installation is complete. From now on, run "catkin build" every time you rewrite a program or file in src , and also run "catkin source" if the package configuration has changed. The "catkin source " command has the same effect as running "source ~/catkin_ws/devel/setup.bash" .

6.3.4.2 Add source command to **bashrc**

It is a hassle to type the commands "catkin source" and "source ~/catkin_ws/devel/setup.bash" every time , so We recommend that you add the following example setting to the last line of "~/.bashrc" on the user's PC .

*The 'catkin source' command will not work unless it is executed under a ROS workspace directory such as 'catkin_ws'. Therefore, we recommend that you use the 'source' command to add commands to bashrc as in the following example .

List 6.1 Example of adding to ~/.bashrc

```
1 source ~/catkin_ws/devel/setup.bash
```

In addition, when ROS communication is performed between multiple PCs , such as when issuing commands from a user PC to a real robot , We also recommend that you add the settings explained in [ROS communication environment variable settings](#) to your bashrc .

6.3.4.3 deb binary uninstall

If you want to uninstall all the deb binaries installed using the above procedure , please do the following:

```
$ sh /opt/tokyorobotics/torobo_ros_debs/uninstall_debs.sh
```

6.3.4.4 Uninstalling ROS package source code

If you want to uninstall the ROS package source code installed using the procedure above , delete the "torobo_ros" directory under catkin_ws/src and clean catkin_ws as follows. *Please make sure to check carefully before proceeding so as not to accidentally delete source code created by users.

```
$ cd ~/catkin_ws $ rm -rf  
src/torobo_ros $ catkin clean -y
```

6.4 Network Setup

In order to use the user's PC to communicate with the PC with the built-in robot controller to obtain the robot status and send commands, the user's PC and the PC with the built-in robot controller must be placed in the same LAN beforehand. If you are only using the PC with the built-in robot controller to operate the actual robot, you do not need to perform this procedure, so please skip it.

6.4.1 Connect the PC and robot controller to the same LAN

Controller 1, Controller 2 and the user PC are connected to the same LAN via a switching hub or similar. By doing so, it is possible to communicate using ROS.

The configuration of network devices is described in [LAN Device Configuration](#) . Please refer to it as necessary.

6. Setup

6.4.1.1 Connecting to the controller via wired LAN

The LAN port for Ethernet connection is attached to the top of the back of the robot , and the LAN port to which the user's PC is connected. Please connect the switching hub or router in the computer with a LAN cable.

6.4.1.2 Connecting to the controller via wireless LAN

Please refer to the setting up of the [built-in Wi-Fi router \(wireless adapter\)](#) and set up the Wi-Fi router built into the robot. The LAN cable protruding from the left side of the back of the robot is connected to the Wi-Fi router. Connect this cable to the LAN port for Ethernet connection attached to the top of the back of the robot.

Alternatively, you can attach your own wireless device to the back and connect it to the LAN port on the back.

Connection is possible.

Even though they are connected to a wireless LAN , controller 1 and controller 2 are connected to a wired LAN via an Ethernet HUB. This ensures stability and speed of communication between the two.

 Note	The stability of Wi-Fi communication is greatly affected by the surrounding environment and the use of other wireless devices. If you require a stable communication, we recommend using a wired LAN connection.
--	---

6.4.2 Checking SSH Connection

Start the control PC and the user PC . On the user PC , enter "Ctrl+Alt+t" to start the terminal and enter the following:

Please check if you can connect to the control PC via ssh using the command .

```
$ ssh torobo@[Control PC's IP Address]
```

If "avahi-daemon" is installed and enabled on the user PC , it is possible to make an SSH connection by specifying "[hostname].local" as shown below, instead of specifying the IP address of the control PC as shown above (avahi-daemon is installed by default on the desktop version of Ubuntu 20.04).

```
$ ssh torobo@[Control PC's Hostname].local
```

By using the host name, you can connect to the SSH server without changing the settings even if the IP address is reassigned and changed. This is usually the recommended method since it is possible to do so.

The host name of the robot controller PC is indicated on the back of the robot or the robot controller .

It is printed on the label, so please check it.

If the user PC does not have the avahi-daemon installed, the display and keyboard can be accessed directly from the control PC . Connect the board etc. and check the IP address.

6.4.3 Setting up the ssh communication environment

Since it is a hassle to enter the IP address and host name every time you connect via SSH , write the following settings in the "~/.ssh/config" file on the user's PC . If the "~/.ssh/config" file does not exist, create it in advance using the touch command or similar.

```
$ touch ~/.ssh/config
```

Listing 6.2 Example of ~/.ssh/config when specifying IP addresses

```
1 Host robot
2     User torobo
3     HostName [Control PC's IP Address].local ForwardAgent yes
4
```

Listing 6.3 Example of ~/.ssh/config with hostname

```
1 Host robot
2     User torobo
3     HostName [Control PC's Hostname].local ForwardAgent yes
4
```

If you do this, you will be able to connect via SSH just by typing ssh robot .

6.4.4 Verifying ROS communication operation using the ROS MASTER

In the above, communication between the user PC and control PC was performed using ssh , but when operating the robot, communication between the user PC and control PC is performed using the distributed system mechanism based on ROS (for information on building a distributed system with ROS, please also refer to [the distributed system mechanism based on ROS](#)).

In this case, the control PC must act as the "ROS MASTER" and send commands over the network, so you must set the environment variables "ROS_MASTER_URI" to indicate its location, and "ROS_IP" or "ROS_HOSTNAME" to indicate your own IP address.

When opening a new terminal on a user's PC , set the environment variables with the following command. If the host name of the user's PC is unknown, you can check it by executing the hostname or echo \$HOSTNAME command in the terminal.

```
$ export ROS_MASTER_URI=http://[Control PC's IP Address]:11311 $ export ROS_IP=[User PC's IP Address]

or

$ export ROS_HOSTNAME=[User PC's Hostname].local
```

Also, you can write "[Control PC's Hostname].local" as the 'ROS_MASTER_URI' part without any problem. No problem.

```
$ export ROS_MASTER_URI=http://[Control PC's Hostname].local:11311
```

Try running the rostopic list command from your PC in this state . You will get the following error message: The error should be printed.

6. Setup

```
$ rostopic list ERROR: Unable
to communicate with master!
```

Next, open another terminal and execute the `roscore` command to start "ROS MASTER" on the control PC . The following assumes that the above settings are written in "`~/ssh/config`", and shows an example of remotely logging in to the control PC with the `ssh` command and executing the `roscore` command.

```
$ ssh robot $
roscore
```

After that, run the same command again in the same terminal where you ran `rostopic list` .

If the topics "`/rosout`" and "`/rosout_agg`" are displayed as shown above, the operation was successful.

```
$ rostopic list /rosout /
rosout_agg
```

In this way, by setting '`ROS_MASTER_URI`' and '`ROS_IP`' or '`ROS_HOSTNAME`' , It is possible to share "ROS_MASTER" between multiple PCs and communicate via ROS .

6.4.5 ROS communication environment variable settings

It is a hassle to input the settings of "`ROS_MASTER_URI`", "`ROS_IP`", and "`ROS_HOSTNAME`" every time, so We recommend that you add the following example setting to the last line of "`~/.bashrc`" on the user's PC .

List 6.4 Example 1 of adding to `~/.bashrc`

```
1 export ROS_MASTER_URI=http://[Control PC's Hostname].local:11311 2 export ROS_HOSTNAME=$(hostname).local
```

However, if you write "`export ROS_MASTER_URI`" in "`~/.bashrc`" as in this example , the control PC will always be set as "ROS_MASTER" , which can be inconvenient when running a simulation on a single user PC . Therefore, in such cases, instead of writing "`export ROS_MASTER_URI`" directly, execute "`export ROS_MASTER_URI`" using 'alias' as in the example below .

List 6.5 Example 2 of adding to `~/.bashrc`

```
1 alias robot="export ROS_MASTER_URI=http://[Control PC's Hostname].local:11311" 2 export ROS_HOSTNAME=$(hostname).local
```

In this case, if you want to set the control PC to "ROS_MASTER", enter the following command on each terminal of the user PC . Run the command.

```
$ robot
```

This is the same as executing `export ROS_MASTER_URI=http://[Control PC's Hostname].local:11311` . You can use any method to set it, so use the method you prefer.

The environment variables for sharing the "ROS_MASTER" explained above are already set on the control PC . Therefore, there is no need to change the settings. Please make the above settings only on the user's PC .

6.4.6 Network configuration notes

In the following chapters, for consistency, the SSH connection and ROS_MASTER_URI settings are written in the following format.

```
$ ssh torobo@[Control PC's IP Address]
```

```
$ export ROS_MASTER_URI=http://[Control PC's IP Address]:11311
```

These parts can be replaced with the format explained in this chapter and still work in the same way, so please use them as necessary. Please read as appropriate.

6.5 Setting up time synchronization between PCs

When using a user PC to communicate with a PC with a built-in robot controller via ROS to obtain the robot status or send commands, if there is a large difference in the time settings between the PCs, the commands may not be executed correctly or an error may occur when using the ROS standard coordinate transformation management system `tf` .

Therefore, we recommend that you follow the instructions below to set up time synchronization between the user PC and the robot controller PC in advance.

6.5.1 About Chrony

Chrony is an implementation of NTP client and server for Linux , and can be used with Ubuntu. Normally, it obtains the accurate current time from an external public time distribution server connected to an atomic clock via the Internet, and adjusts the PC's time.

6.5.2 Policy for time synchronization method

When using ROS to control a robot, the robot is often not connected to the Internet.

As long as it is synchronized with the time on the robot control PC , it does not necessarily have to be set to the actual exact time.

This robot is set to have Controller 1 as the NTP server when shipped , so the user's PC Set it as an NTP client to synchronize the time with Controller 1 , which is the NTP server.

6. Setup

6.5.3 How to install and configure Chrony

First, install Chrony with the following command:

```
$ sudo apt install -y chrony
```

Next, open the Chrony configuration file, "/etc/chrony/chrony.conf" , using an editor.

Administrator privileges are required to edit the file. The "pool" on lines 17 to 20 in the file specifies the NTP server to synchronize with. However, since we want to synchronize only with Controller 1 this time, add "#" to the beginning of these lines and change the setting. Comment it out.

```
17 # pool ntp.ubuntu.com 18 # pool iburst maxsources 4
0.ubuntu.pool.ntp.org iburst maxsources 1
19 # pool 1.ubuntu.pool.ntp.org iburst maxsources 1
20 # pool 2.ubuntu.pool.ntp.org iburst maxsources 2
```

Next, rewrite lines 46 and after as follows. The following settings specify Controller 1 as the synchronization destination.
can be done.

```
46 makestep 0.1 10
47 server [Control PC's IP Address] iburst prefer
OR
47 server [Control PC's Hostname].local iburst prefer
```

Then, run the following command to update Chrony:

```
sudo systemctl restart chrony.service
```

6.5.4 Checking Chrony operation

First, log in to Controller 1 , which is the server side, via ssh or similar.

```
$ ssh torobo@[Control PC's IP Address]
```

After logging in, run sudo chronyc -a clients . The following command is executed for the client user:
If the PC 's IP address and other information are displayed, the setup is successful.

```
$ sudo chronyc -a clients
Hostname                               NTP Drop Int IntL Last          Cmd Drop Int Last
=====
192.168.1.67                           13           0     6           9           0           0
torobo@torobo-user-PC ^ee^82^b0 ~/catkin_ws ^ee^82^b0 chronyc -a clients
Hostname                               NTP Drop Int IntL Last          Cmd Drop Int Last
=====
```

Next, run chronyc sources on the client PC .

If the IP address etc. is displayed, everything is normal.

```
$ chronyc sources
210 Number of sources = 1
MS Name/IP address          Stratum Poll Reach LastRx Last sample
=====
^* 192.168.1.42              2         6 377          54 +1295us[+2334us] +/- 56ms
```

Next, run `chronyc tracking` . If the synchronization information is displayed as shown below, the synchronization is successful. Masu.

```
$ chronyc tracking
Reference ID      : C0A8012A (192.168.1.42)
Stratum          : 3
Ref time (UTC)   : Tue Oct 05 03:52:52 2021
System time      : 0.000088710 seconds fast of NTP time
Last offset      : +0.000984302 seconds
RMS offset       : 0.001475697 seconds
Frequency : 19.171 ppm fast
Residual freq : +2.271 ppm
Skew : 12.974 ppm
Root delay Root : 0.104585208 seconds
dispersion : 0.005577440 seconds
Update interval : 65.0 seconds
Leap status    : Normal
```

Finally, to check whether the time is actually synchronized, run `watch` on the server and client side.

`-n 0.1 Run timedatectl`. This is a command that displays the current time every 0.1 seconds.

Compare the local time displayed on the consoles , and if they show almost the same time, then everything is normal.

After checking, press `Ctrl+C` to end the command execution.

7 - Run the demo

We will explain an example of actually moving a robot using the environment we have set up.

7.1 Turn on the power

Lift the battery switch at the bottom of the back to turn on the battery power. Since only controller 1 is used in this demo, set only C1 of the "Start PC Switch" to H (up) and press the PC start switch. If both the battery power lamp and controller 1 of the PC start switch are lit green, the system has started up normally.

7.2 Start the robot controller

Log in to the started controller 1 .

- When using a user **PC**

Start a terminal on the user PC ([Ctrl+Alt+t]) and connect to Controller 1 via SSH . Execute the following command to connect via SSH.

```
$ ssh torobo@[Control PC's IP Address]
```

You will be asked for your login password, so please enter the password "torobo".

- If you do not use a user **PC**

Connect a monitor and keyboard directly to controller 1 and log in with the user "torobo" and password "torobo" . Then, start up a terminal ([Ctrl+Alt+t]) .

7. Run the **demo**

7.3 Start the robot controller core program

Now it's time to start ROS.

Make sure the emergency stop button is released.

In the terminal on controller 1 , run the following command to start the robot controller core program. Note that the environment variables required to run ROS are already written in `~/bashrc`, so there is no need to set the environment variables every time you start the program.

```
$ roslaunch torobo_bringup bringup_real.launch
```

If the message "You can start planning now!" appears in green on the terminal , the program has been successfully started. It sends commands to the core program running on Controller 1 and to the ROS MASTER to control the robot.



Note: If an error message appears in the ROS log on the terminal , enter [Ctrl+C] to force quit ROS .

Then, make sure that you are running the terminal on Controller 1 and that the emergency stop switch is released.

7.4 On the user **PC** , specify Controller **1** as the **ROS MASTER**

If you do not use a user PC , this step is unnecessary. Please skip it.

Unless otherwise specified, subsequent GUI operations and command transmissions are performed on the user PC and the terminal launched on the user PC . In this case, it is necessary to send commands over the network with Controller 1 as the ROS MASTER, so it is necessary to set the environment variable `ROS_MASTER_URI` that indicates its location . When opening a new terminal on the user PC , set `ROS_MASTER_URI` with the following command (for information on building a distributed system with ROS , please also refer to [How a Distributed System Works with ROS](#)).

```
$ export ROS_MASTER_URI=http://[Control PC's IP Address]:11311
```



Note

The `ROS_MASTER` setting must be performed on each terminal that communicates with the controller via ROS on the user PC side.

7.5 Check the robot control mode and servo status

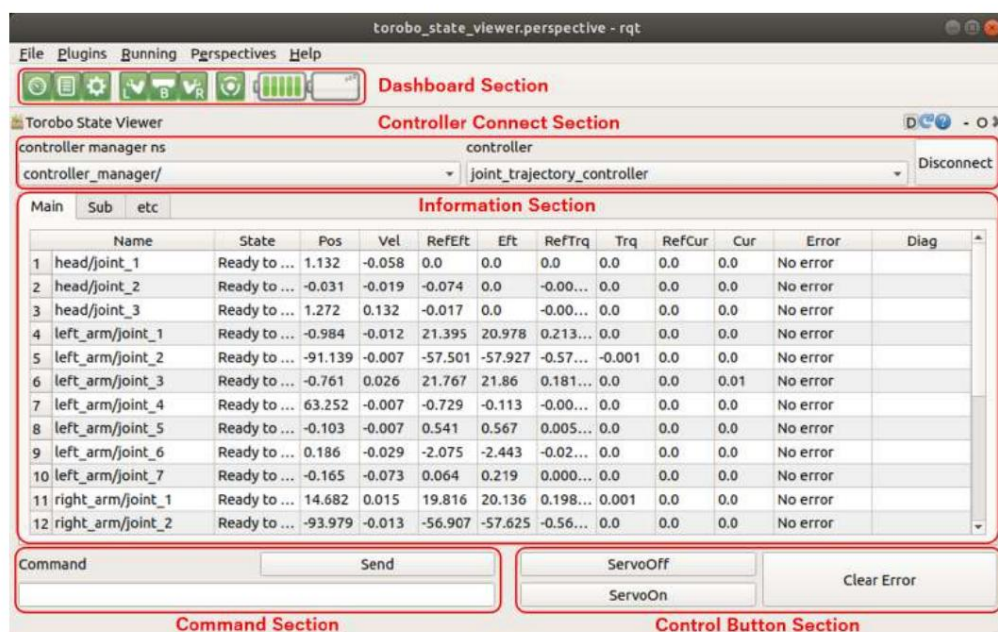
Now, let's check the robot's control mode and servo status using the started Controller 1 and ROS . To check the robot's control mode and servo status, use Torobo State Viewer . In a newly opened terminal, start it with the following command.

```
$ roslaunch torobo_dashboard torobo_state_viewer.launch
```

If startup on the user PC fails, check the setting of the environment variable ROS_MASTER_URI (specify controller 1 as the ROS MASTER on the user PC) .

The displayed Torobo State Viewer is composed of four main elements, each with the following functions.

- Dashboard Section: Display and release the robot safety monitoring function status
- Controller Connect Section: View and switch controllers
- Information Section: Shows the status of each joint
- Command Section: Servo ON/OFF and operation command input
- Control Button Section: Servo ON/OFF and clear error display



First, check the Information Section in the center . The Information Section displays the status of each joint on the robot, and if each value is constantly updated, it means that Controller 1 and the robot are communicating correctly (for details on each column, see [Torobo State Viewer](#)).

- If an error is displayed in the **Error** column for each joint

Immediately after starting the core program, an error may be displayed in the Error column for each joint. Click "Clear Error" from the Control Button Section and verify that the error disappears.

In addition, the robot is designed to stop in an emergency if an error occurs as a safety measure. If the Safety State Display on the far right of the Dashboard Section is displayed in red, click the Safety

7. Run the **demo**

Reset using the State Reset Button (for details on the Dashboard Section display and Safety State , refer to [the Safety Monitoring Function Specifications](#)).



When the State column for each axis shows "Ready to switch on ," the motor is ready to move.
Oh, in this state the joint servos are not yet active.

This completes the startup of the robot controller.



Note: For robots equipped with a safety module, power supply to the robot drive system is cut off immediately after the main power is turned on. With the emergency stop button released, power supply to the robot drive system can be resumed by issuing a manual reset command to the safety module. To issue a manual reset, press the Safety State Reset Button from the gear mark in the Dashboard Section . For details about the safety observation function and the safety module, refer to [the specifications of the safety observation function](#) .

7.6 Operation check of **position control mode (joint_trajectory_controller)**

First, let's check the operation of the position control mode.

Select `joint_trajectory_controller` from the controller tab in the Controller Connect Section of Torobo State Viewer .
Click "Disconnect" ÷ "Connect" twice to switch to position control mode.

Next, press the "ServoOn" button in the Control Button Section to turn on the servos for all joints of the robot . This will release the brakes on the robot and change the State column from Ready to switch on to Operation enabled .



When you turn on the **Danger** servo , the robot will start moving. To turn on the servo,
Be sure to stay away from the robot's moving area and be able to press the emergency stop switch instantly while working.

Now, let's send position control commands and actually move the robot. Position control commands can be sent in the following three ways. In any case, when you have finished moving the robot, press the "Servo Off" button to turn off the joint servos.



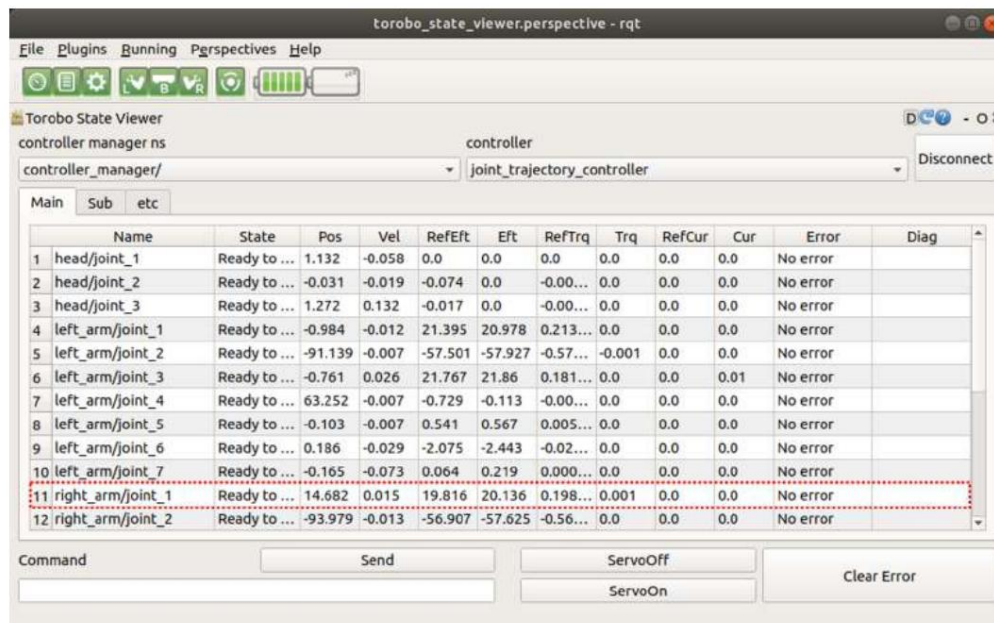
Caution: Always be prepared to press the emergency stop button immediately at any time.

If the robot moves unexpectedly, immediately press the emergency stop button to turn off the servos. The servos will turn off and the brakes will be applied to stop the robot.

7.6.1 Position command using the command box of Torobo State Viewer

The command box of Torobo State Viewer has a position command `tpts`. Here, the right arm We will explain an example of moving the `joint_1` axis.

Verify that `right_arm/joint_1` is displayed in the 11th row of the Name column, enter "`tpts 11 30 10`" in the command box, and press the Enter key or the "Send" button. If the robot's J1 joint moves to 30[deg], the operation is successful.



The arguments to the `tpts` command are given in the following order: "line number of target axis, target position [deg], movement time [s]". In other words, the above command inputs a position control command that says "move the right arm J1 joint on line 11 to a position of 30 [deg] in 10 seconds."

7.6.2 Position command using `rqt_torobo_joint_trajectory_controller`

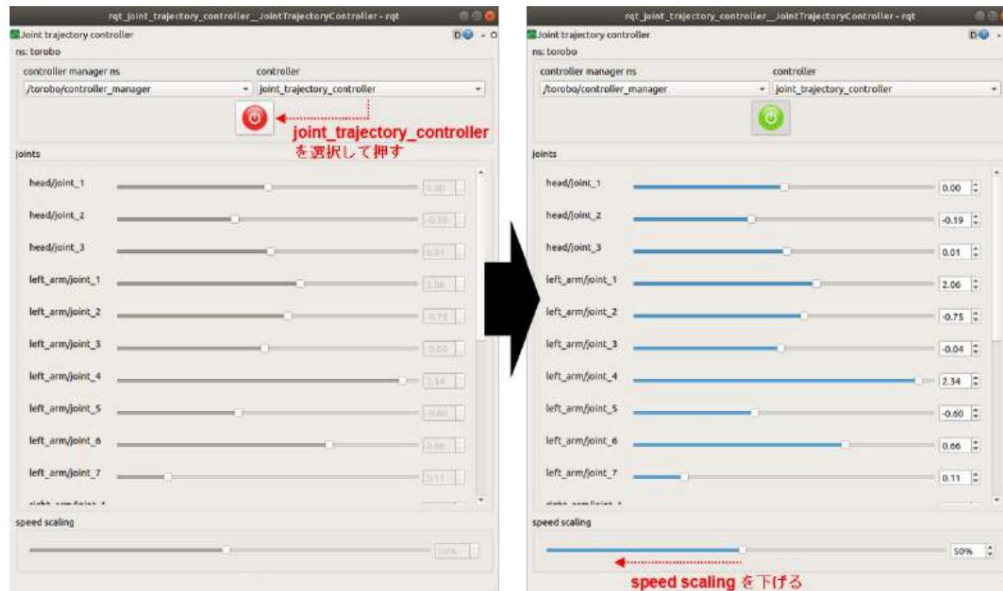
It is also possible to use sliders to control joint angles instead of CLI or command operations. The gui and `rqt_torobo_joint_trajectory_controller` can be started with the following command.

```
$ roslaunch rqt_torobo_joint_trajectory_controller rqt_torobo_joint_trajectory_
yccontroller.launch
```

When `rqt_torobo_joint_trajectory_controller` starts, select `/torobo/controller_manager/` from controller manager ns, select `joint_trajectory_controller` from controller, and press the red button.

7. Run the **demo**

Once connected, the button will turn green and the slider will become movable.



In this state, set the bottom speed scaling to 20% and slowly move the sliders on each axis.

If the joint angles move in accordance with the rider's position, then it's a success.

Note 	We recommend a speed scaling of 20% or less. In the <code>rqt_torobo_joint_trajectory_controller</code>, a trajectory to the target position is generated every time the slider position changes, so if the speed scaling is large, sudden acceleration and deceleration will occur frequently, and the movement of the robot joints may become oscillatory.
--	---

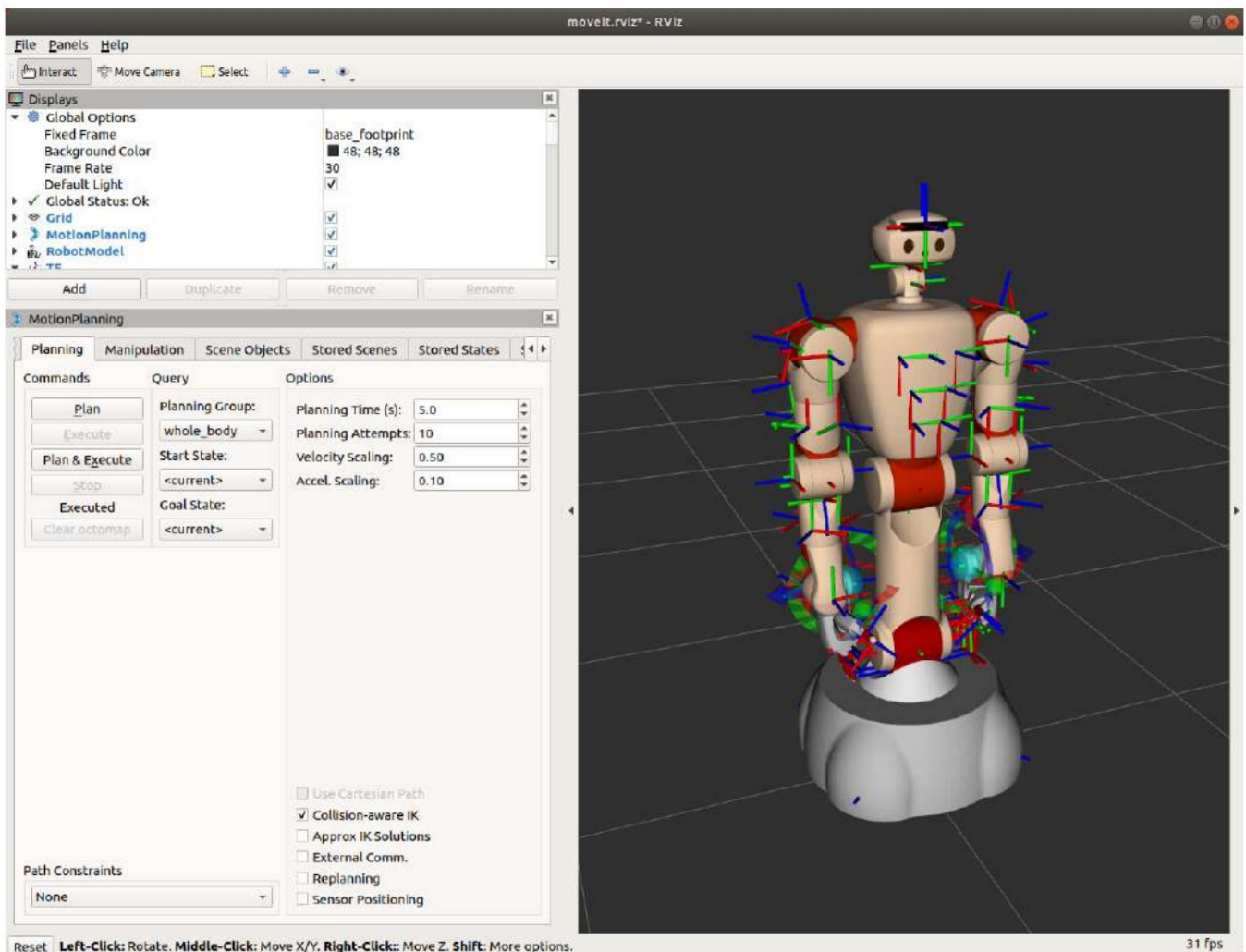
7.6.3 Position command using rviz + moveit

You can also check the robot's movement in the simulator before sending a position command.

Open the simulator rviz .

```
$ roslaunch torobo_bringup rviz.launch
```

At the start of the rviz screen, the Displays and Motion Planning screens are displayed on the left , and the simulator screen is displayed on the right half. If Motion Planning is not open at startup, check the Motion Planning check box in Displays to open it.



When the Motion Planning screen opens, go to the Planning tab, select the joint group to send commands from the Planning Group in Query, set Start State to <current> , and Goal State to <current> . At the same time, set Velocity Scaling in Options to 0.01 .

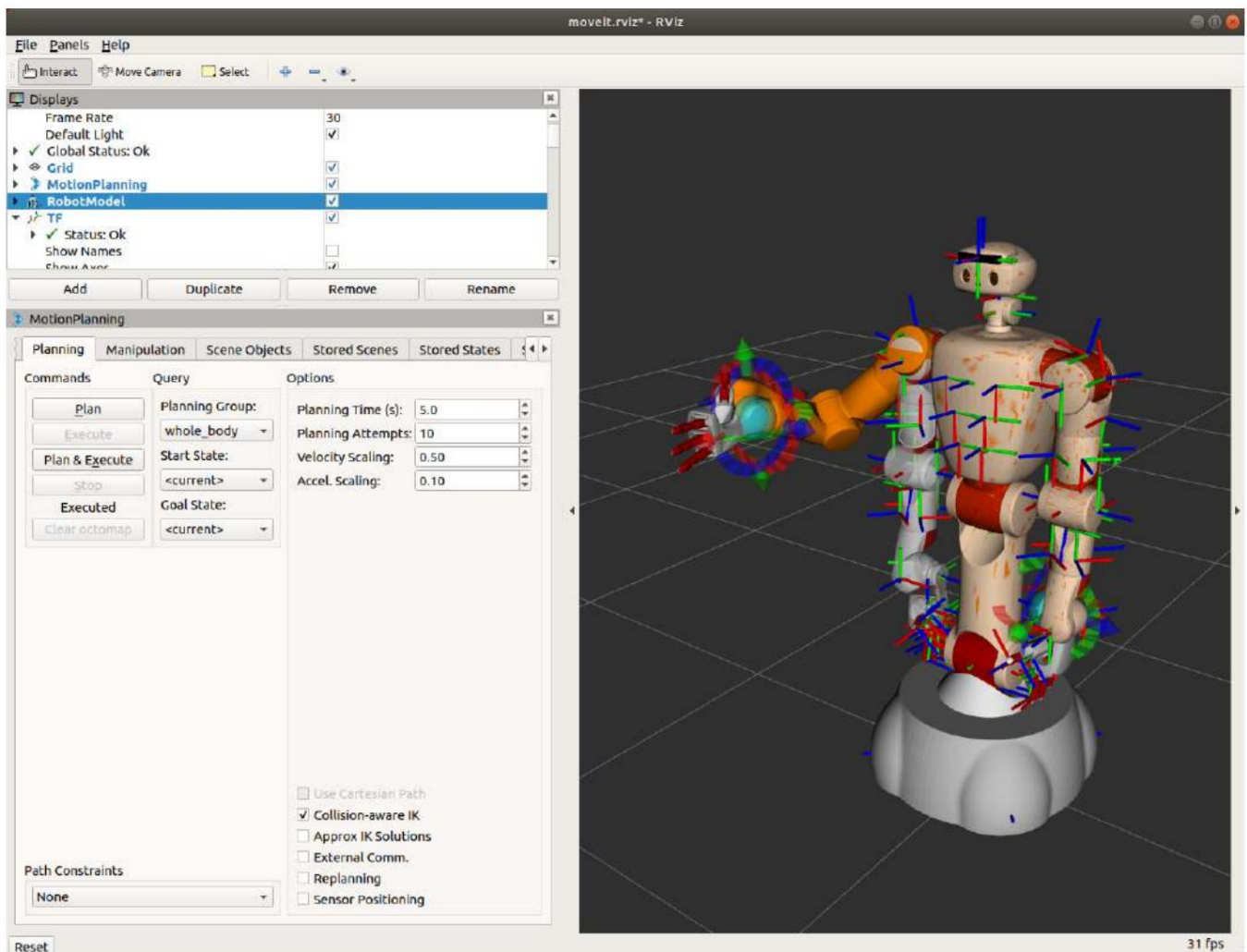
Danger 	Be sure to set Velocity Scaling to a small enough value, otherwise it may run too fast and be dangerous.
--	--

Next, for the robot model in the simulator screen, move the joint group selected in Planning Group , which is displayed in orange, to the target position. Movement is performed using the arrow and circle displayed at the center of the robot model's hand. Dragging the arrow moves the hand in parallel, and dragging the circle rotates the hand. You can also move the hand in various directions spatially by dragging the light blue ball displayed at the hand.

The simulator's viewpoint can be moved by dragging anywhere except the fingertips, and left-clicking rotates the object. Rotating the mouse wheel and right-clicking will zoom in and out, while pressing the mouse wheel will move parallel.

As an example, the screen shown below appears when you select whole_body in the Planning Group.

7. Run the demo



Once you have moved the robot model to the target position and orientation on the simulator screen, send a command to the actual machine to take the same position and orientation. Press "Plan" from Commands, check that there are no problems with the movement trajectory, and then press "Execute" to send the command to the actual machine. If the actual machine takes on the same orientation as the robot model in the simulator, it is a success.

7.7 Operation check of **gravity compensation mode (gravity_compensation_controller)**

Next, let's check how the weight compensation mode works.

In the self-weight compensation mode, each joint outputs a torque equivalent to the robot's own weight. This allows you to When you want to change your posture while doing other things, you can move it easily by applying a small amount of force.

As with the position control mode, select `gravity_compensation_controller` from the controller menu in the Torobo State Viewer to switch to weight compensation mode, press the "ServoOn" button, and confirm that the State column has changed from Ready to switch on to Operation enabled .

Now, let's actually touch the robot and move its joints.

If you can change your position or posture, you've succeeded.

Once the robot has finished moving, press the "Servo Off" button to turn off the joint servos.



When the **Danger** servo is turned ON , the brakes on the joints are released and the joints become movable.

When turning the servo ON , be sure to leave the robot's movable area. Also, be sure to be able to press the emergency stop switch instantly while working.

7.8 Joint impedance control mode (joint_impedance_controller)

Check the work

Next, let's check the operation of the joint impedance control mode.

In the joint impedance control mode, the spring, mass and damper coefficients are set by torque control around each joint axis. This allows the robot to move in response to external forces.

As with the position control mode and weight compensation mode , switch to joint impedance control mode by selecting joint_impedance_controller from controller in Torobo State Viewer , press the "ServoOn" button, and confirm that the value in the State column has changed from Ready to switch on to Operation enabled .



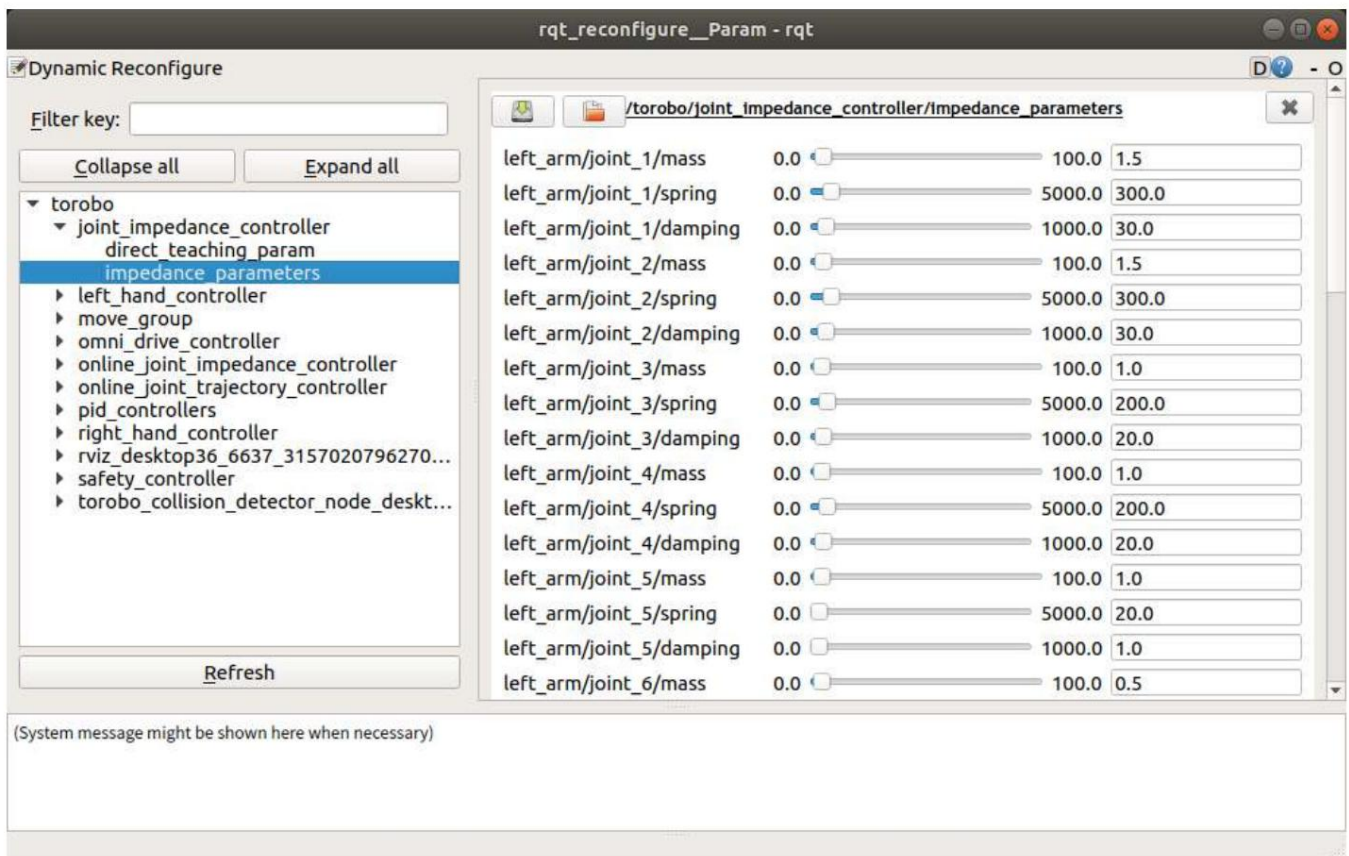
When the **Danger** servo is turned ON , the brakes on the joints are released and the joints become movable.

When turning the servo ON , be sure to leave the robot's movable area. Also, be sure to be able to press the emergency stop switch instantly while working.

Now, try moving the robot by hand. If it is functioning properly, when you move the robot by hand, you will be able to move each joint smoothly while receiving a weak reaction force that returns it to its original position due to the springiness. You can also change the spring, mass, and damper coefficients from the GUI . Start a separate terminal and start rqt_reconfigure with the following command.

```
$ rosrn rqt_reconfigure rqt_reconfigure
```

When rqt_reconfigure starts, expand the torobo tab on the left and select impedance_parameters from joint_impedance_controller . You can see a list of the coefficients of the joint impedance controller , and you can change them by moving the slider or entering values.

7. Run the **demo**

As an example, let's set the spring of right_arm/joint_1 to 900 and the damping to 90. Try moving it by hand again.

When you do this, you should feel as if the movement of your right arm around the J1 axis has become stiffer.

Furthermore, since the joint_impedance_controller also supports the tpts command, try changing the robot's posture. Enter "tpts 11 30 10" in the Command Section of the Torobo State Viewer and press the Enter key or the "Send" button. If you can confirm that the arm balances with gravity even when raised and moves in accordance with the external force, then the command is successful (for information on how to use the tpts command, see also [Position Commands Using the Command Box in the Torobo State Viewer](#)).

Once you have confirmed that the operation is correct, press the "Servo Off" button to turn off the joint servos.

7.9 Return the robot to the home position and stop the operation

When you're done, return the robot to its home position. To return it to its home position, press the

We will use Torobo State Viewer and rviz (for how to use Torobo State Viewer and rviz , please also refer to [Checking the operation of position control mode \(joint_trajectory_controller\)](#) and [Position command using rviz + moveit](#)).

First, open the Torobo State Viewer , select the joint trajectory controller from the controller column , change it to position control mode, and press the "Servo On" button to turn the servo on .

Next, open the Planning tab from Motion Planning in rviz , select whole_body from the Planning Group, set Start State to <current> , and Goal State to home_position .

After making sure that there are no people or objects in the robot's range of motion and that Velocity Scaling is set to 0.01 or less, Press "Plan & Execute" to move the robot to its home position.

Once the movement is complete, turn off the servo using the "Servo Off" button in Torobo State Viewer .



Caution: Always be prepared to press the emergency stop button immediately at any time. If the robot moves unexpectedly, immediately press the emergency stop button to turn off the servos. The servos will turn off and the brakes will be applied to stop the robot.

7.10 Shut down the robot controller

Finally, stop ROS and shut down controller 1 .

Press the emergency stop switch to turn off the power supply, and press [Ctrl+C] in the terminal of Controller 1 where the robot core program was started to shut down the ROS Core . When the ROS Core has finished, "shutting down processing monitor complete" is displayed, and the terminal returns to a command waiting state.

After confirming that ROS Core has terminated, enter the following command in the same terminal to shut down Controller 1 .

```
$ sudo shutdown now
```

Note



Be sure to shut down ROS Core and Controller 1 before turning off the power, as this may result in data corruption.

7.11 Turn off the power

Verify that the power switch LED is off to confirm that shutdown of controller 1 is complete, then restart the robot controller. Turn off the controller .

That's all for the demo operation.

8 - Run the simulator

This section explains how to start and operate Gazebo , a dynamics simulator .

8.1 Mechanism for switching between real machine operation and simulator operation

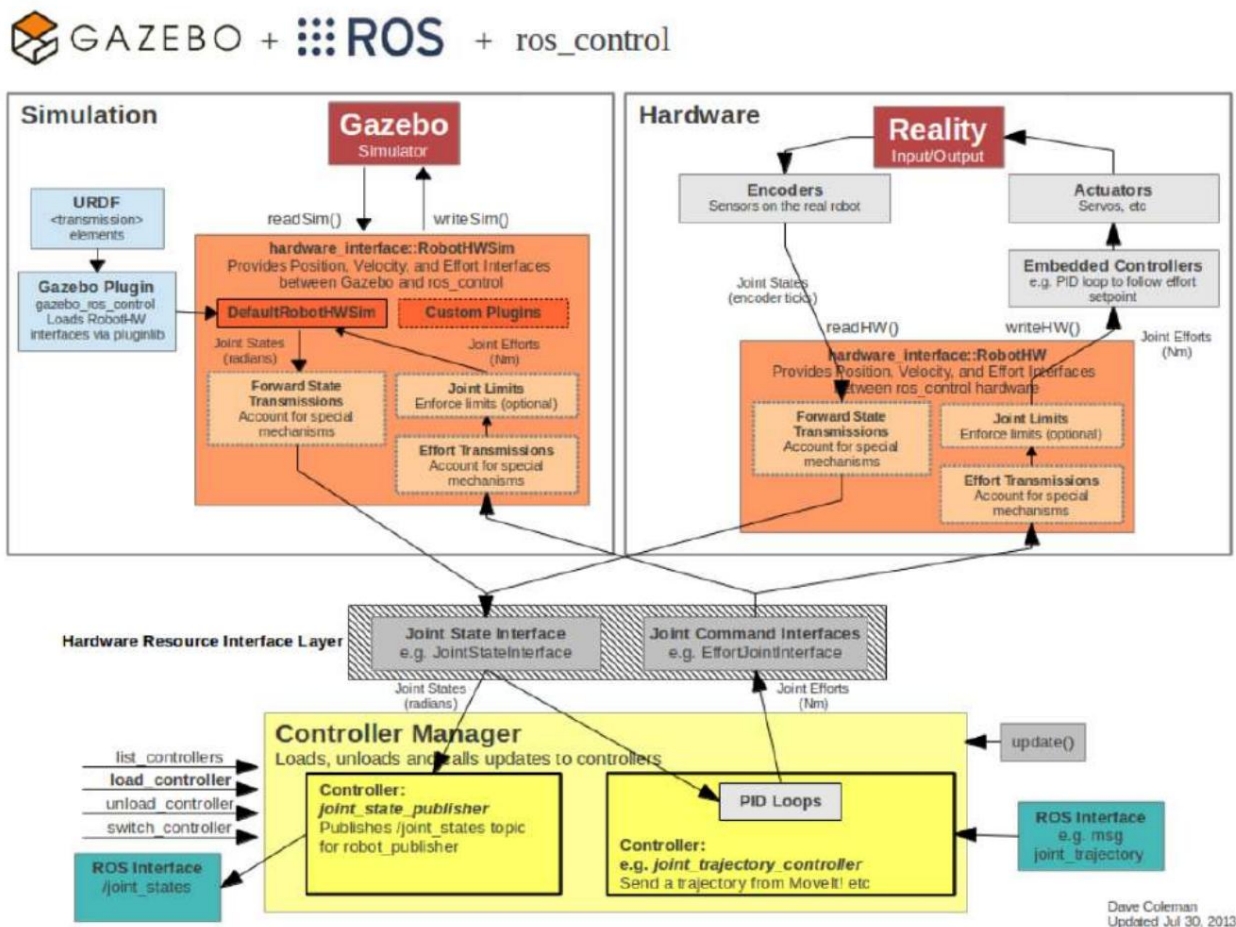


Figure 8.1 Data flow of `ros_control` and Gazebo (Source: http://gazebosim.org/tutorials/?tut=ros_control)

In ROS , you can run a real device and a simulator (Gazebo) with the same ROS program. This is achieved by using the `ros_control` and `hardware_interface` mechanisms shown in the figure above.

By using `ros_control` , you can build a controller that has a common interface between the real machine and the simulator. In other words, when acquiring the current joint state or issuing commands to the joints, the controller can use the same interface as the real machine.

8. Run the **simulator**

There is no need to separate the implementation in the simulator.

The differences between the real machine and the simulator (Gazebo) are absorbed by the `hardware_interface`. The process of sending the joint command values output by the controller to the actual joints and the process of obtaining the current joint state are different between the real machine and the simulator. Therefore, we prepare a `hardware_interface` for each of the real machine and the simulator, and implement the process of issuing commands to the joints and obtaining the current state here.

When switching between the actual device and the simulator, you can use the same ROS program by switching this `hardware_interface`. (Controller) enables you to operate the robot.

For information on how the `ros_control` part of this robot software works, please also refer to [the overall system configuration](#).

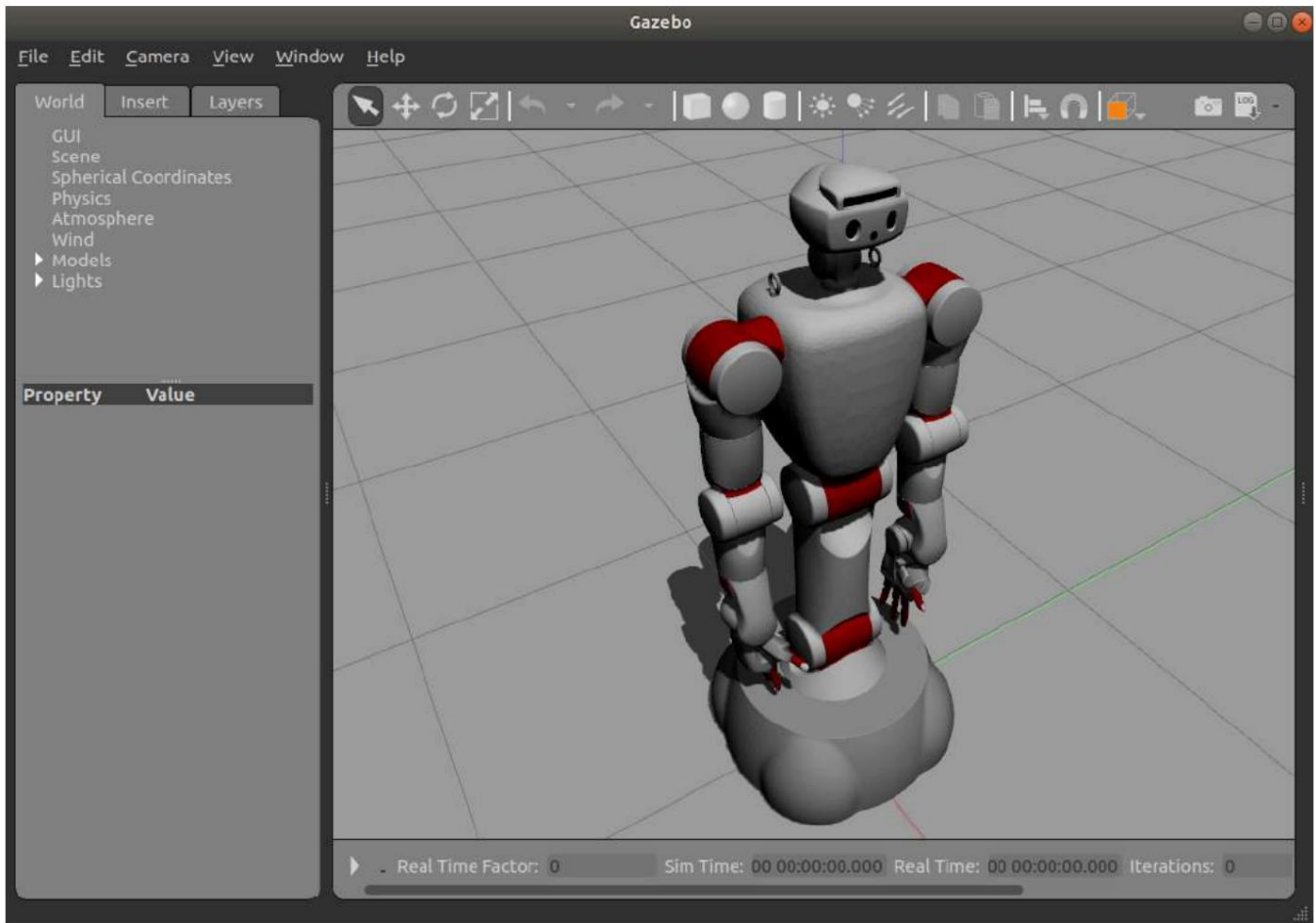
8.2 How to run the simulator (Gazebo)

For Gazebo, a PC environment with a graphics card (dedicated card type, not built-in to the CPU) is recommended. Since the control PC in the robot controller does not have a graphics card, we recommend that you start Gazebo on the user's PC.

On the user's PC, enter "[Ctrl+Alt+t]" from the keyboard. This will launch the terminal. Next, Please start Gazebo with the following command.

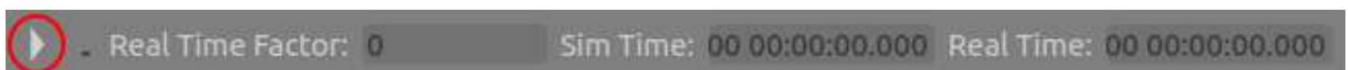
```
$ cd catkin_ws $ catkin
source $ roslaunch
torobo_bringup bringup_sim.launch
```

This will open the Gazebo window.



When you start Gazebo for the first time, it will take some time to start Gazebo. Gazebo downloads data from the web. This is to load it.

Then click the right arrow icon in the bottom left of the GUI to start the simulation.



If there is no ROS_ERROR in the terminal and the value of "Sim Time" at the bottom of the GUI changes over time, then it is correct. It's always running.



Note: If you get an error in the ROS log,

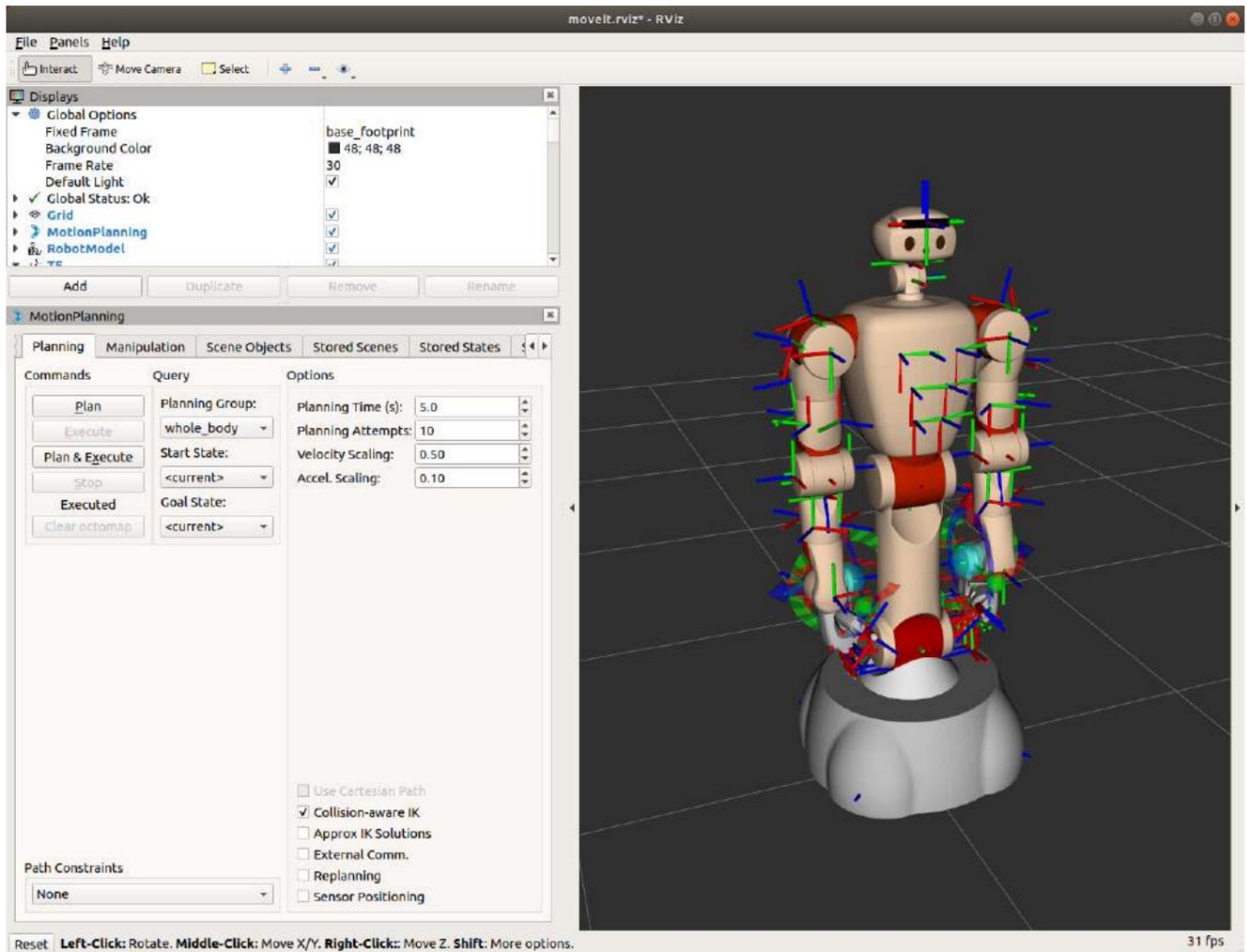
Please enter [Ctrl+C] to force quit ROS and then start ROS again. If the specifications of your PC are insufficient, RViz or Gazebo may fail to start at a certain frequency. This phenomenon is thought to be caused by the implementation of RViz or Gazebo, so please contact the ROS or Gazebo developer forum.

Next, start RViz, which is a GUI that displays the robot's status and issues commands using MoveIt!. Enter "[Ctrl+Alt+t]" on your board to open the terminal, and then run the following command to start RViz.

```
$ cd catkin_ws $ catkin
source $ roslaunch
torobo_bringup rviz.launch
```

8. Run the **simulator**

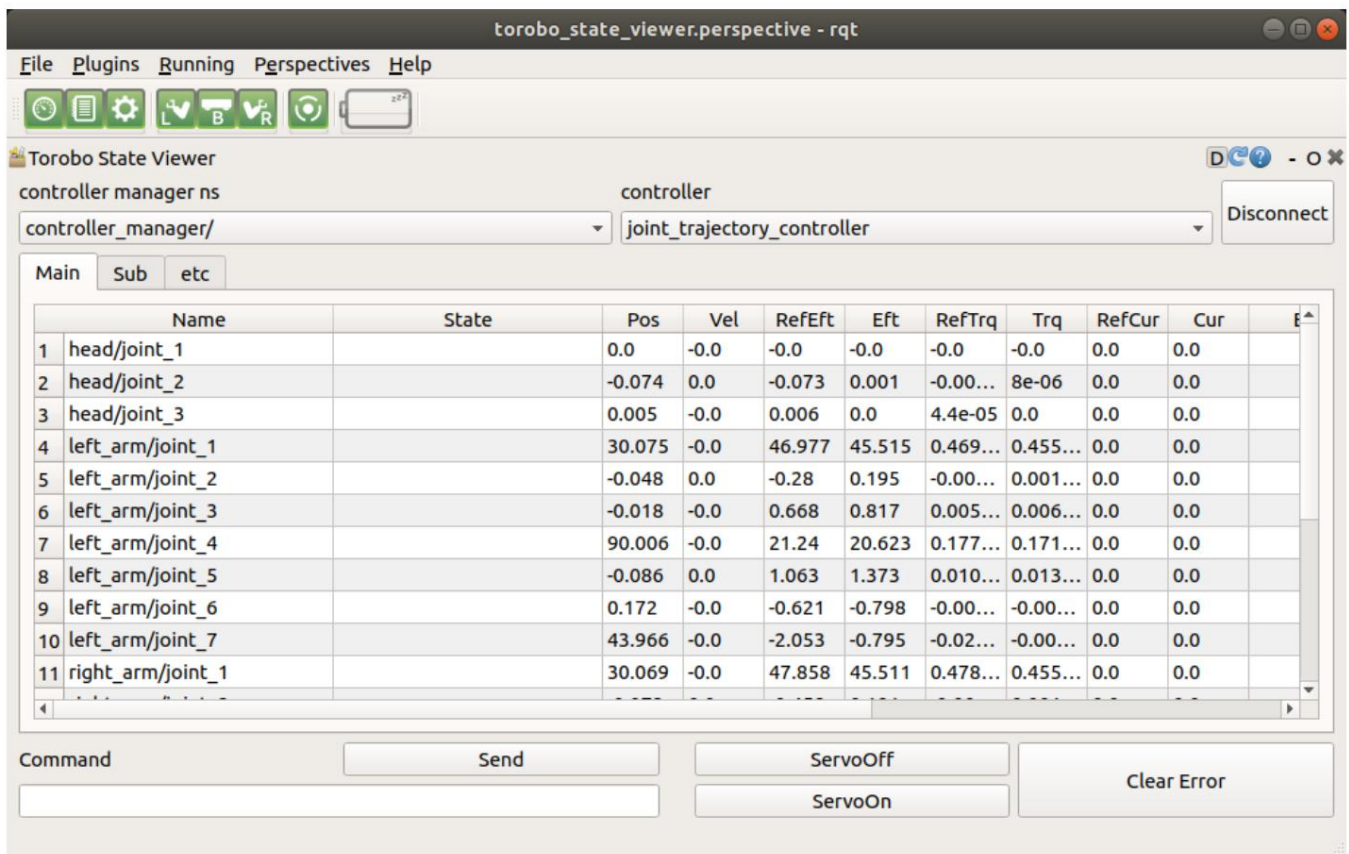
You should then see the following screen:



Finally, start Torobo State Viewer , which is a GUI for displaying and operating the robot's status. Enter "[Ctrl+Alt+t]" on the keyboard to open the terminal, and then run the following command to start Torobo State Viewer .

```
$ cd catkin_ws $
catkin source $
roslaunch torobo_dashboard torobo_state_viewer.launch
```

Then the following GUI will be displayed:



8.3 Operation check on simulator (Gazebo)

On the simulator, it is possible to operate using the position control mode (`joint_trajectory_controller`) and joint impedance control mode (`joint_impedance_controller`). The operation method is basically the same as the actual machine. However, unlike the actual machine, there is no need to turn the servo on/off.

8.3.1 Operation check of position control mode (`joint_trajectory_controller`)

You can check the operation in the same way as [for the position control mode \(`joint\_trajectory\_controller`\)](#). However, please omit the operation related to servo ON/OFF as it is not necessary.

8. Run the **simulator**

8.3.2 Operation check of **joint impedance control mode (joint_impedance_controller)**

You can check the operation in the same way as [for joint impedance control mode \(joint_impedance_controller\)](#) . However, please omit the operation related to servo ON/OFF as it is not necessary.

8.3.3 Return the robot to the home position and stop the operation

You can check the operation [by returning the robot to the home position and stopping it](#) in the same way. However, Please omit operations related to servo ON/OFF as they are not necessary.

8.4 How to exit the simulator (Gazebo)

You can exit Gazebo by entering [Ctrl+c] in the terminal where you started it .

Note 	Gazebo may take a long time to shut down. This phenomenon is thought to be caused by the Gazebo implementation, so please contact the ROS or Gazebo developer forums.
---	---

9 - Tutorial

This section explains how to operate and program the robot.

9.1 Check the URDF of the robot model

In ROS , robot models are described in an XML format file called Unified Robot Description Format (URDF). This file is read and used by various programs on ROS .

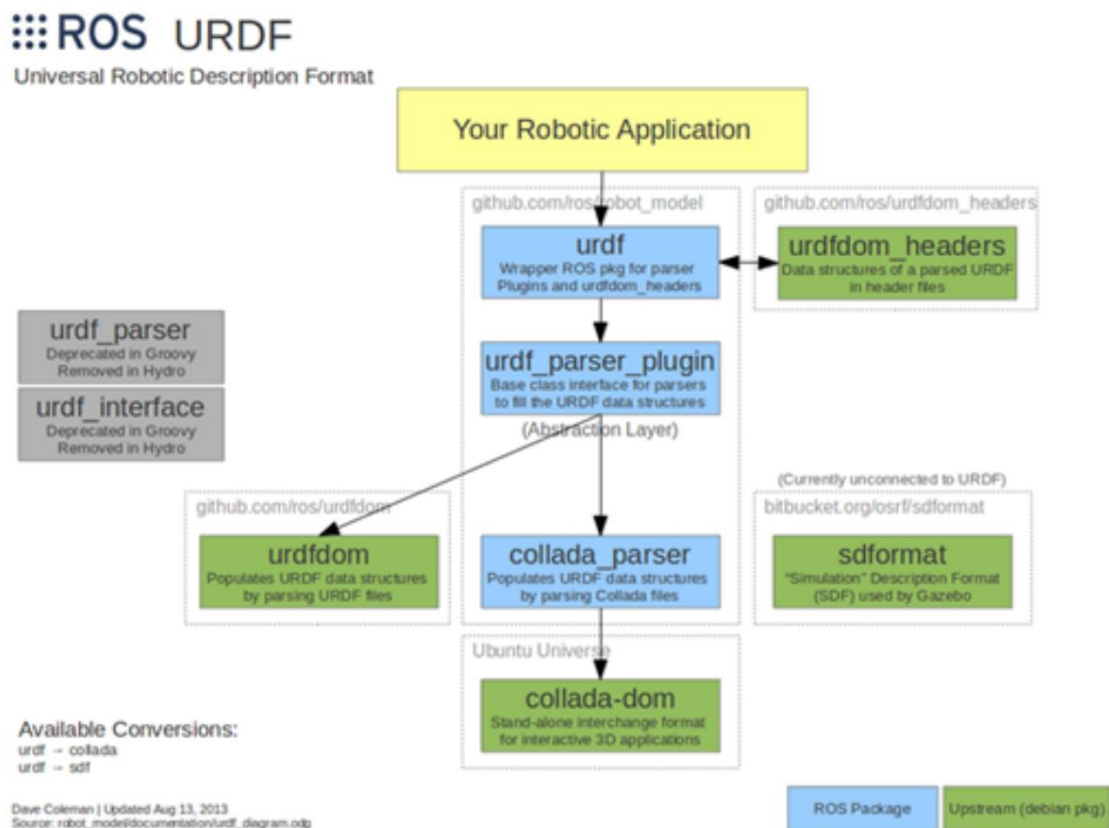


Figure 9.1 Components and relationships for URDF in ROS (Source: <https://wiki.ros.org/urdf>)

URDF describes structural information such as the shape of each link and the arrangement of joints, as well as the types of sensors built into the robot.

9. Tutorial

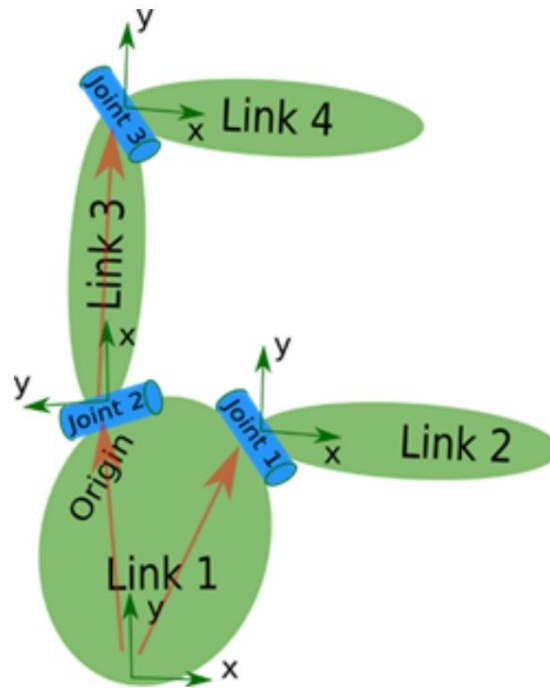
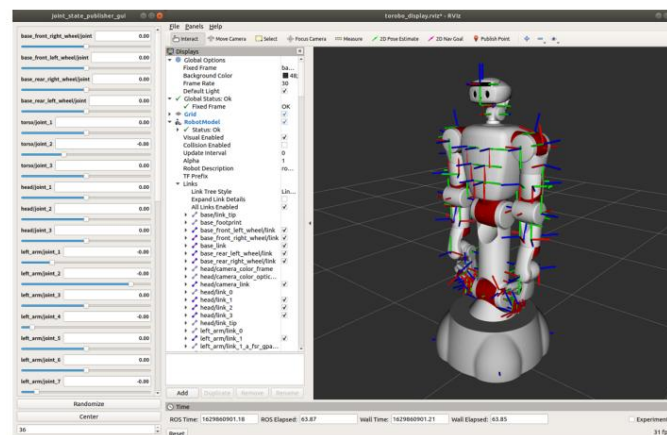


Figure 9.2 Example of a link tree structure described in URDF (Source: <http://wiki.ros.org/en/urdf/Tutorials/Create%20your%20own%20urdf%20file>)

By executing the following command from the terminal, you can check the model on Rviz , the standard ROS visualization tool .

```
$ roslaunch torobo_description torobo_display.launch
```



You can operate the model by manipulating the sliders on the joint_state_publisher_gui displayed with RViz .

9.2 Using the GUI software to move the robot

There are four types of GUIs available for controlling the robot .

We will explain how to use it.

- Torobo State Viewer
- rqt_torobo_joint_trajectory_controller
- RViz (Trajectory planning with MoveIt!)
- torobo_hand_teaching_motion

9.2.1 Torobo State Viewer

Torobo State Viewer mainly plays the following roles:

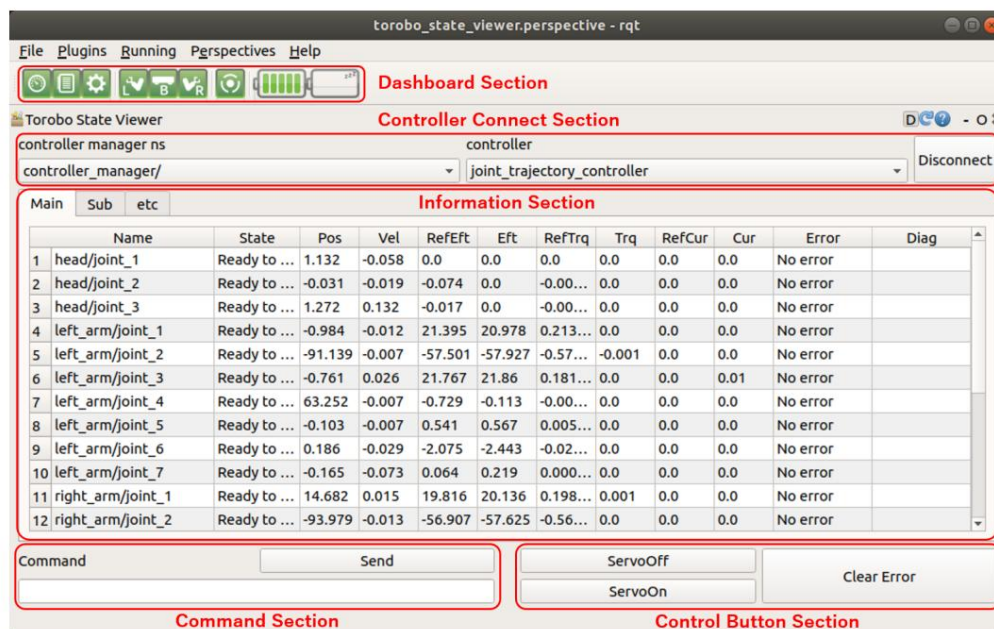
- Displays the status of each joint
- Controller switching
- Display and switch Safety State
- Command buttons

To start Torobo State Viewer , execute the following command:

```
$ roslaunch torobo_dashboard torobo_state_viewer.launch
```

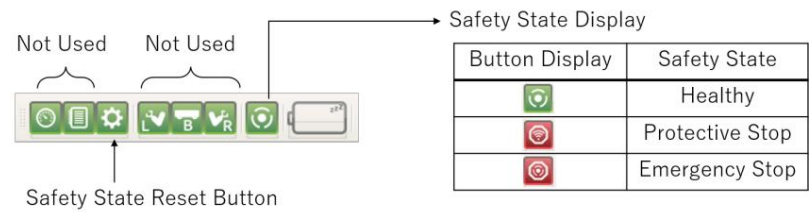
You should see the following screen. However, this may vary depending on the robot you are using and the controller you are running.

The contents displayed in the Information Section may differ depending on the product.



As shown in red in the image above, Torobo State Viewer consists of five sections.

Dashboard Section Display and switch the robot safety state



The robot is designed to make an emergency stop for safety reasons if an abnormality is detected. This stopped state is called the Safety State . There are three safety states. For details about the safety state , refer to [the specifications of the safety monitoring function](#) .

State Name	No
Healthy	abnormalities in specifications. Servo can be turned on . Control is possible by command.
Emergency Stop (e_stop)	Emergency stop state. Servo ON is not possible. The servo will be turned OFF automatically .
Protective Stop (p_stop)	Protective stop state. Servo ON is possible. Position control is performed to maintain the current joint position regardless of the control mode. No action is taken even if a command is sent.

The Safety State Display in the Dashboard Section shows the current Safety State : Protective Stop,
To release the Emergency Stop and return to Healthy mode , press the Safety State Reset Button and select "Reset" from the options that appear.

Controller Connect SectionSwitching Controllers

controller manager ns	Switch the namespace of controller_manager . Usually, you can leave it as the default. Select the Controller to connect
controller	to from the pull-down menu.
Disconnect/Connect button	Turns the connection to the Controller OFF/ON . When connected to a Controller, the start process of the connected Controller will be executed automatically.

Information SectionStatus display of each joint of the robot

Displays the current status of each joint. Details of the information displayed are as follows:

Name	Joint ID and joint name. When sending a command in the Command Section, specify the target joint. Please use this joint ID .
State	Servo ON/OFF status
Pos	Current position of the joint [deg]. Calculated from the encoder on the output shaft of the reducer.
Vel	Current joint speed [deg/s]. Calculated from the encoder on the output shaft of the reducer.
RefEft	Target torque of joint [Nm]
Yes	Current torque value [Nm] . Measured value of the torque sensor.
RefTrq	Motor target torque [Nm]
Street	Current torque of the motor [Nm]
RefCur	Target current value for joint [A]
Sowing	Current value of the joint [A]
Error	Summary of Error Conditions
Diag	Detailed information about the error condition

Command Section: Issue simple commands to the robot

Simple commands such as servo ON/ OFF for each joint , brake release, and position commands for joints can be issued.

You can enter a command in the Command field and press the Enter key or the Send button .

Please refer to [Torobo State Viewer Commands](#) for supported commands .

Control Button Section Buttons for frequently used control commands are provided.

ServoOff	Turns off the servos of all joints controlled by the Controller (same as the "s all 0" command). righteous).
ServoOn	Turns on the servos of all joints controlled by the Controller (same as the "s all 1" command). righteous).
Clear Error	Resets the errors of all joints controlled by the Controller ("reset all" command) (Synonymous with do).

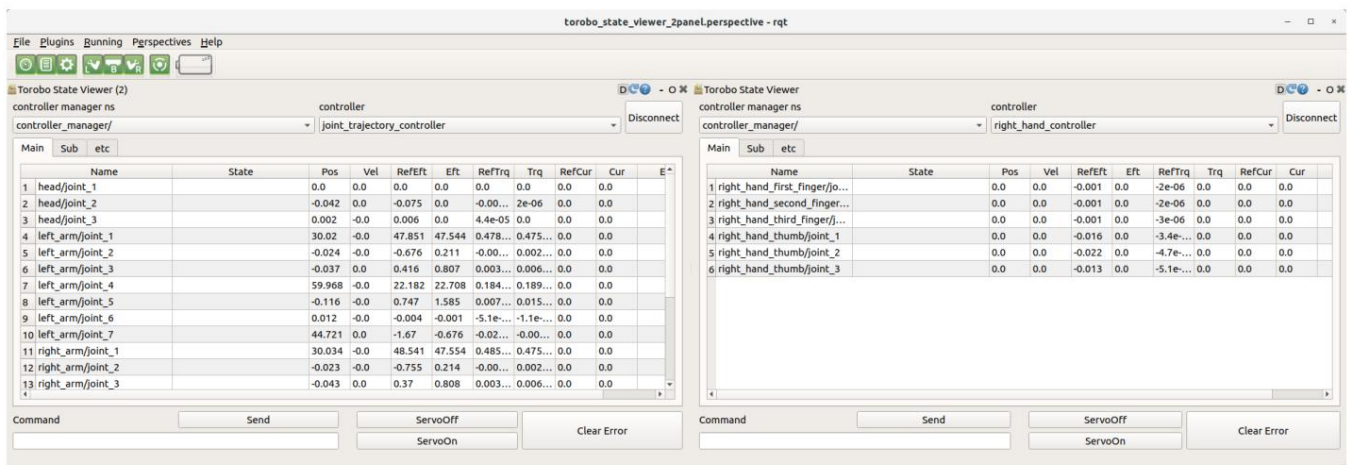
It is possible to run multiple Torobo State Viewers at the same time. To run two at the same time, execute the following command.

Please do.

```
$ roslaunch torobo_dashboard torobo_state_viewer_dual.launch
```

This will open a window containing two Torobo State Viewers , as shown below.

9. Tutorial



You can also open four at the same time with the following command:

```
$ roslaunch torobo_dashboard torobo_state_viewer_quad.launch
```

9.2.2 rqt_torobo_joint_trajectory_controller

By using `rqt_torobo_joint_trajectory_controller`, you can control the position of the robot joint by moving the slider on the GUI. You can send commands.

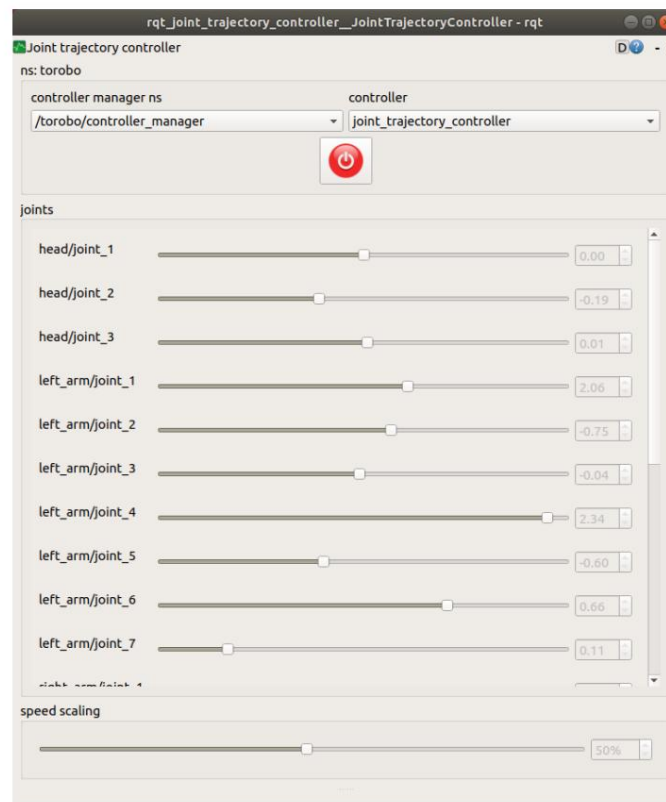
* There is a package called "rqt_joint_trajectory_controller" as a standard ROS package, but "rqt_torobo_joint_trajectory_controller" is our own package based on this one, with improvements such as smoother trajectory generation and integration with safety monitoring functions. Please use this package in normal cases.

As preparation, start the core program on the control PC or start the simulation and switch to position control mode (`joint_trajectory_controller`). If you are using a real machine, make sure there are no interfering objects nearby and turn the servo ON.

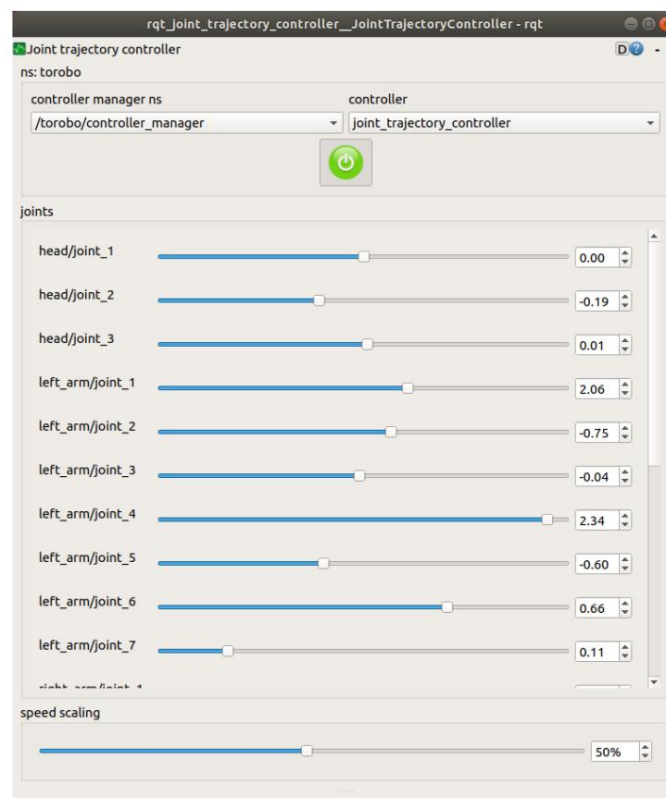
Next, start `rqt_torobo_joint_trajectory_controller`. To start it, run the following command.

```
$ roslaunch rqt_torobo_joint_trajectory_controller rqt_torobo_joint_trajectory_
  ycontroller.launch
```

The following screen will be displayed. If the controller manager ns and controller fields are blank, select `/torobo/` controller_manager and `joint_trajectory_controller` from the pull-down menus, respectively.



At this stage, the sliders for each joint are grayed out, and dragging them will not send position commands to the joints. If you click the red power mark at the top here, the mark will turn green as shown below. In this state, dragging the sliders for each joint will send position commands to the joints.



9. Tutorial

**Note**

We recommend a speed scaling of 20% or less. In the `rqt_torobo_joint_trajectory_controller`, a trajectory to the target position is generated every time the slider position changes, so if the speed scaling is large, sudden acceleration and deceleration will occur frequently, and the movement of the robot joints may become oscillatory.

9.2.3 RViz (Trajectory planning with MoveIt!)

You can use the trajectory planning library MoveIt! from RViz. It supports complex tasks such as obstacle avoidance and object grasping as standard, and can greatly simplify the complex flow of trajectory planning and execution that has traditionally plagued developers. It can be used on both real machines and simulators.

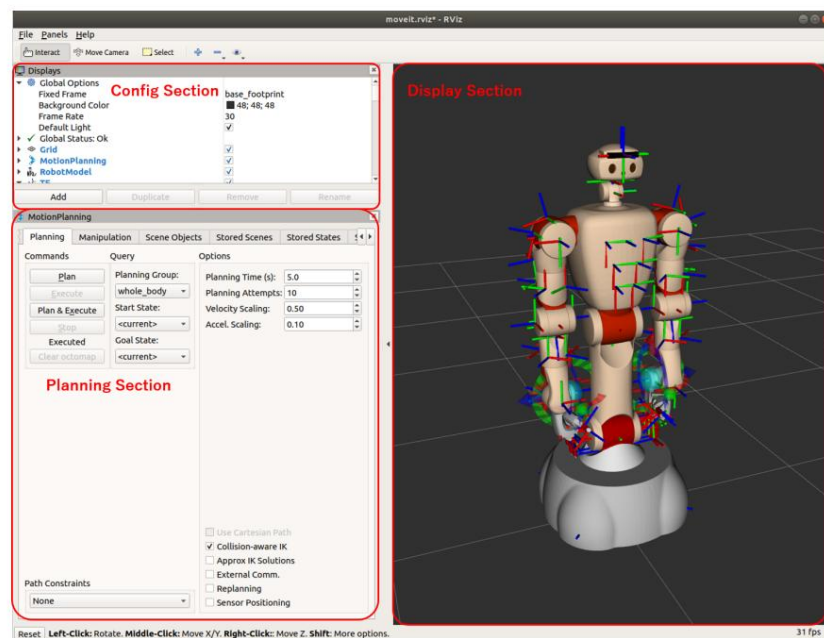
In preparation, start the core program on the control PC or start the simulation and set it to position control mode.

Please switch to (joint_trajectory_controller).

To start RViz, run the following command:

```
$ roslaunch torobo_bringup rviz.launch
```

If the startup is successful, you should see a screen like this:



RViz consists of three main sections:

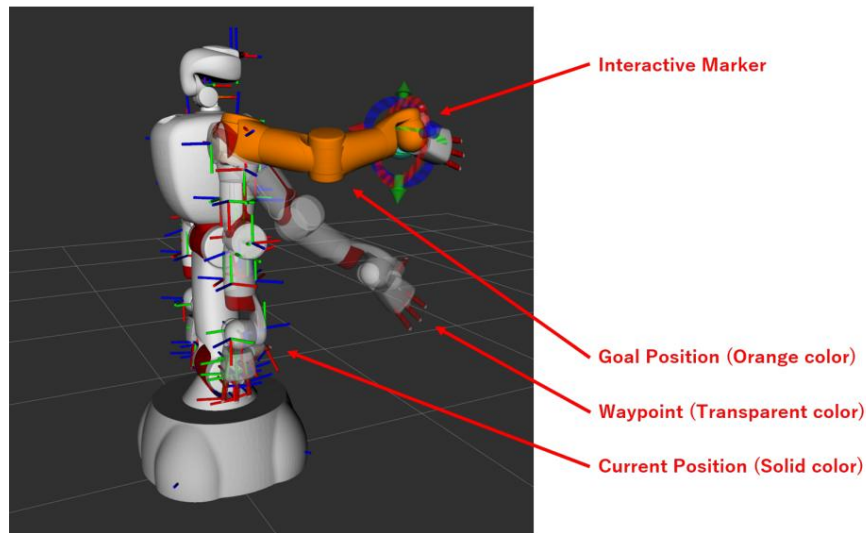
Config Section Select the information to display in RViz. You can switch the information to display on RViz by checking the checkbox. If you want to perform trajectory planning using MoveIt!, check "Motion Planning" and Please make sure the Planning Section is visible.

Display Section: Display the 3D model of the robot and specify the target posture. Joint position information obtained from the robot. Based on this, the current robot pose is displayed as a 3D model.

You can also specify the target posture of the trajectory you plan with MoveIt!. A light blue ball and red, green, and blue circles and arrows are displayed at the end of the arm. These are called interactive markers, and you can position the end of the robot arm.

When you operate the interactive marker, the orange arm moves.

This orange arm represents the target position when MoveIt! plans its trajectory.



Planning Section This section issues commands for orbit planning.

Query → Planning Group	Switches the group of joints to be trajectory planned.
Query → Start State	Set the starting position of the orbit. Choose from random , random , add-valid rent, same as goal, or home_position .
Query → Select Goal State	Set the target position of the orbit. Choose from , random , add-random valid rent, same as start, or home_position .
Commands → Plan	Plan the trajectory based on the start and target positions set in the query . Pressing this button alone will not send any command to the robot. The planned trajectory will be displayed in the Display Section . Be sure to check that a complete orbital has been generated. Sai.
Commands → Execute	The robot is commanded to follow the trajectory planned in Plan . Position control mode (joint_trajectory_controller) and servo ON If you have one, the actual device will work, and the simulator will work as well.
Commands -> Plan and Execute Plan and execute the trajectory at the same time.	This can be dangerous when an irregular orbit is generated. It is not recommended to use .
Slider → Waypoints	The waypoints of the planned trajectory are displayed in the Robot Display and Operation section. You can check the value by moving this slider bar. The waypoints are displayed semi-transparently. If the bar is not visible, click Panels -> Click on MotionPlanning - Slider .
Velocity/Acceleration Scaling	Set the maximum value of the speed or acceleration with Scaling . It is 0.0 to 1.0 . It is configurable. The value to be scaled is specified in the URDF Joint tag. This is the Max Velocity / Acceleration described in .
Goal Tolerance	Specifies the tolerance margin for the target goal.

9.3 Use ROS topic , service, and action from the terminal to run the robot Move

Here we will explain how to operate the robot by executing commands from the terminal.

9.3.1 Joint position command (using Ros Publish)

The position control mode (joint_trajectory_controller) and joint impedance control mode (joint_impedance_controller) provide topics for moving joints . Users can move joints by publishing messages to these topics .

Control Mode	Topic /	Message type
joint_trajectory_controller	torobo/joint_trajectory_controller/ command	trajectory_msgs/JointTrajectory
joint_impedance_controller	/torobo/joint_impedance_controller/ command	trajectory_msgs/JointTrajectory

In the following example, we will use the position control mode as an example to move the joint to the target angle by publishing a command from the terminal. We'll show you how.

First, change the robot's control mode to position control mode (joint_trajectory_controller). Select "joint_trajectory_controller" from the "controller" pull-down menu in Torobo State Viewer and press "Disconnect". The button will change to "Connect", so press it again. If the button returns to "Disconnect", the control mode has been successfully switched. Also, if you are operating the actual machine, make sure the servo is ON .

Next, we give the robot a command for the target joint position. The command is given in the /torobo/joint_trajectory_controller/ command. This is done by publishing trajectory_msgs/JointTrajectory to the pick. For example ,

- Set the target position of the second joint of the left arm (left_arm/joint_2) to -1.5rad
- Transition time is 5.0 seconds

To do this, run the following command from the terminal:

```
$ rostopic pub /torobo/joint_trajectory_controller/command trajectory_msgs/JointTrajectory "header:
  seq: 0
  stamp:
    secs: 0
    nsecs: 0
  frame_id: "
joint_names: -
'left_arm/joint_2' points: -
positions:
[-1.5] velocities: [0]

accelerations: [0]"
```

(Continued on next page)

(Continued from previous page)

```
effort: [0]
time_from_start: {secs: 5, nsecs: 0}"
```



Depending on the posture of **the Danger** Robot, it may interfere with the surrounding environment. Make sure there are no objects in the vicinity that may interfere with it, and be ready to press the emergency stop button immediately before issuing a command.



Note: If you copy and paste the above command into the terminal, the format will be corrupted and it will not be displayed correctly.

In some cases, you may not be able to enter the command correctly. In that case, type `rostopic pub /torobo/joint_trajectory_controller/command  $\ddot{y}$` ($\ddot{y}$: half-width space) in the terminal , then press the Tab key several times. The command after that will be automatically completed.

9.3.2 Joint position command (using Ros Actionlib)

In the position control mode (`joint_trajectory_controller`) and joint impedance control mode (`joint_impedance_controller`) , the ROS standard `control_msgs` (http://wiki.ros.org/control_msgs) is used as a Service by Actionlib to move the joints. We provide an Actionlib Service that uses the ActionLib API (a service that handles the joints of a robot). Users can move joints by calling this service.

Control Mode	ActionService $\ddot{y}$	action type of control_msg
<code>joint_trajectory_controller</code>	<code>/torobo/joint_trajectory_controller/ follow_joint_trajectory /</code>	<code>FollowJointTrajectory.action</code>
<code>joint_impedance_controller</code>	<code>torobo/joint_impedance_controller/ follow_joint_trajectory</code>	<code>FollowJointTrajectory.action</code>

Below, we will use the position control mode as an example to explain how to move a joint to a target angle using Ros Actionlib from the terminal . First, set the robot's control mode to position control mode (`joint_trajecroy_controller`). If you are unsure of the procedure , please refer to the description of [joint position command \(using Ros Publish \)](#) . Also, if you are using a real machine, make sure the servo is ON .

When using Actionlib commands, you can obtain information about whether or not a joint has reached the target position as a result of executing a trajectory. The results of the `FollowJointTrajectory.action` of the `joint_trajectory_controller` used here can be confirmed in the `/torobo/joint_trajectory_controller/follow_joint_trajectory/result` topic. Therefore, let's prepare a terminal to display the results of Actionlib. Enter "[Ctrl+Alt+t]" on the keyboard to launch the terminal and execute the following command.

```
$ cd catkin_ws $ catkin
source $ rostopic echo /
torobo/joint_trajectory_controller/follow_joint_trajectory/result
```

9. Tutorial

Next, use Ros Actionlib to give commands. If you are giving commands from a terminal, just publish the goal trajectory to `/torobo/joint_trajectory_controller/follow_joint_trajectory/goal`. For example,

- Set the target position of the second joint of the left arm (`left_arm/joint_2`) to `-1.5rad`
- Transition time is 5.0 seconds

To do this, run the following command from the terminal:

```
$ rostopic pub /torobo/joint_trajectory_controller/follow_joint_trajectory/goal
ÿcontrol_msgs/FollowJointTrajectoryActionGoal "header:
  seq: 0
  stamp:
    secs: 0
    nsecs: 0
  frame_id: ""
goal_id: stamp:
  secs: 0
  nsecs: 0
  id: ""
goal:
  trajectory:
    header:
      seq: 0
      stamp:
        secs: 0
        nsecs: 0
      frame_id: ""
    joint_names: - 'left_arm/
    joint_2' points: - positions: [-1.5]
    velocities:
    [0] accelerations: [0] effort: [0]

    time_from_start: {secs: 5, nsecs: 0}
path_tolerance: - {name: "",
position: 0.0, velocity: 0.0, acceleration: 0.0} goal_tolerance: - {name: "", position: 0.0, velocity: 0.0, acceleration: 0.0}
goal_time_tolerance: {secs:
0, nsecs: 0}"
```

Then the robot will start moving.



Depending on the posture of the **Danger** Robot, it may interfere with the surrounding environment. Make sure there are no objects in the vicinity that may interfere with it, and be ready to press the emergency stop button immediately before issuing a command.



Note: Even if you copy and paste the above command into the terminal, the format may be distorted and it may not be possible to issue the command correctly. In that case, type `rostopic pub /torobo/joint_trajectory_controller/follow_joint_trajectory/goal " (ö: half-width space) in the terminal, then press the Tab key several times. The command after that will be automatically completed.`

When the operation is complete, the following should be displayed in the result display terminal. If `error_code` is 0, it indicates that the target position was reached successfully. If any other number is output, the operation may have failed. For the meaning of the numbers, refer to the definition file of `control_msgs/FollowJointTrajectory.action` (http://docs.ros.org/en/api/control_msgs/html/action/FollowJointTrajectory.html).

```
header:
  seq: 5
  stamp:
    secs: 10554
    nsecs: 124000000
  frame_id: ""
status:
  goal_id:
    stamp:
      secs: 10549
      nsecs: 74000000
    id: "/gazebo-6-10549.74000000"
    status: 3
    text: ""
result:
  error_code: 0
  error_string: ""
```

9.3.3 Servo ON/OFF (using Ros Service)

We provide a ROS service that uses our own message type, `torobo_std_msgs`, as a ROS service for turning the robot's servos on and off. For more information about our own message type, please refer to the specifications of [our own ROS message](#).

torobo_std_msgs service	explanation	Service Name
SetStringArray	Turns the servo of the specified joint (multiple joints can be specified)	/torobo/robot_hw/servo_on
SetStringArray	ON . Turns the servo of the specified joint (multiple joints can be specified) OFF .	/torobo/robot_hw/servo_off

For example, to turn on the servo for the 5th joint of the left arm , run the following command from the terminal.

```
$ rosservice call /torobo/robot_hw/servo_on "data: - 'left_arm/joint_5'"
```

9. Tutorial

To turn the servo off , execute the following command from the terminal.

```
$ rosservice call /torobo/robot_hw/servo_off "data:
- 'left_arm/joint_5'"
```

9.4 Using Python to move the robot

9.4.1 Controlling joint position using ROS Publish (one-way communication)

Position control mode (`joint_trajectory_controller`) and joint impedance control mode (`joint_impedance_controller`)

We have prepared a topic for moving the joints . Users can publish messages to this topic .

You can move the joints with . The difference with Actionlib is that it has lighter communication processing and is suitable for high-frequency operations.

However, unlike Actionlib, it is not possible to wait until the operation is completed, so the script that publishes the message

You need to control the timing of publishing on the script side .

In position control mode (`joint_trajectory_controller`), joints are moved by `JointTrajectory.msg` in `trajectory_msgs`.

Here is a sample script (python) for this. This script is stored in the `torobo_demo` package.

This code works on both the real device and the simulator (Gazebo) . You can also write the same code in C++.

It is possible to do so.

```
1 # /usr/bin/env python
2 #-*- coding: utf-8 -*-
3
4 import rospy
5 import actionlib
6 import numpy as np
7 from trajectory_msgs.msg import JointTrajectoryPoint, JointTrajectory
8
9
10 def main():
11
12     TOPIC_NAME = '/torobo/joint_trajectory_controller/command'
13
14     # Initializes a rospy node.
15     rospy.init_node('publish_joint_trajectory_node')
16
17     # Create a publisher.
18     publisher = rospy.Publisher(TOPIC_NAME, JointTrajectory, queue_size=1)
19
20     # Wait until the publisher gets ready.
21     while publisher.get_num_connections() == 0:
22         rospy.sleep(1)
23
24     # Publish a message.
25     publish_joint_trajectory(
26         publisher = publisher,
27         joint_names = ['left_arm/joint_2'],
28         positions = np.deg2rad([-22.5]),
29         time_from_start = 5.0
30     )
```

(Continued on next page)

(Continued from previous page)

```

31
32     # Wait 5 seconds
33     rospy.sleep(5)
34
35     # Publish a message.
36     publish_joint_trajectory(
37         publisher = publisher,
38         joint_names = ['left_arm/joint_2', 'right_arm/joint_1'],
39         positions = np.deg2rad([-45.0, 45.0]),
40         time_from_start = 5.0
41     )
42
43     # Wait 5 seconds
44     rospy.sleep(5)
45
46     rospy.loginfo("finished.")
47
48
49 def publish_joint_trajectory(publisher, joint_names, positions, time_from_start):
50     —
51
52     Function for publishing message to move joints
53
54     Parameters
55     —
56     publisher : rospy.Publisher
57         publisher
58     joint_names : list of string
59         list of joint names
60     positions : list of float
61         list of joint's goal positions(radian)
62     time_from_start : float
63         transition time from start
64
65     Returns
66     —
67     None
68
69     Throws
70     —
71     None
72     —
73
74     # Creates a message.
75     trajectory = JointTrajectory()
76     trajectory.header.stamp = rospy.Time.now()
77     trajectory.joint_names = joint_names
78     point = JointTrajectoryPoint()
79     point.positions = positions
80     point.velocities = [0.0] * len(joint_names)
81     point.accelerations = [0.0] * len(joint_names)
82     point.effort = [0.0] * len(joint_names)
83     point.time_from_start = rospy.Duration(time_from_start)
84     trajectory.points.append(point)

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

85     # Publish the message.
86     publisher.publish(trajjectory)
87
88
89 if __name__ == '__main__':
90     main()

```

In the above script, we wait 5 seconds between publishing messages . Unlike Actionlib, we are not notified that the arm has actually moved to the target position within 5 seconds .

The specific steps to move the arm using the above script are described below. First, create the core program of the actual machine. Or start Gazebo .

Next, switch the joint_trajectory_controller to the running state using Torobo State Viewer etc., and turn on the servos for all axes . Next, open another terminal and execute the script with the following command. Check that the arm moves.

```
$ rosrn torobo_demo publish_joint_trajectory.py
```



Depending on the posture of the **Danger** Robot, it may interfere with the surrounding environment. Make sure there are no objects interfering with the surroundings, and give the command while you are ready to press the emergency stop button immediately. please.

9.4.2 Controlling joint position using ROS Actionlib (bidirectional communication)

In the position control mode (joint_trajectory_controller) and joint impedance control mode (joint_impedance_controller) , the ROS standard `control_msgs` (http://wiki.ros.org/control_msgs) is used as a Service by Actionlib to move the robot joints. We provide an Actionlib Service that uses the . You can move a robot by implementing Actionlib Client .

Here is a sample script (python) that moves the arm using FollowJointTrajectory.action in control_msgs . This script is stored in the torobo_demo package. This code works on both the real machine and the simulator (Gazebo) . It is also possible to write similar code in C++.

```

1 # /usr/bin/env python 2 # -*- coding:
utf-8 -*-
3
4 import rospy 5 import
actionlib 6 import numpy as
np 7 from actionlib_msgs.msg
import GoalStatus 8 from trajectory_msgs.msg import
JointTrajectoryPoint 9 from control_msgs.msg import FollowJointTrajectoryAction,
FollowJointTrajectoryGoal,
yFollowJointTrajectoryResult 10 from
control_msgs.msg import JointTolerance

```

(Continued on next page)

(Continued from previous page)

```

12 ACTION_STATUS_DICT = {
13     GoalStatus.PENDING : 'PENDING',
14     GoalStatus.ACTIVE : 'ACTIVE',
15     GoalStatus.PREEMPTED : 'PREEMPTED',
16     GoalStatus.SUCCEEDED : 'SUCCEEDED',
17     GoalStatus.ABORTED : 'ABORTED',
18     GoalStatus.REJECTED : 'REJECTED',
19     GoalStatus.RECALLING : 'RECALLING',
20     GoalStatus.RECALLED : 'RECALLED',
21     GoalStatus.LOST : 'LOST',
22 }
23
24 ACTION_RESULT_DICT = {
25     FollowJointTrajectoryResult.SUCCESSFUL: 'SUCCESSFUL',
26     FollowJointTrajectoryResult.INVALID_GOAL: 'INVALID_GOAL',
27     FollowJointTrajectoryResult.INVALID_JOINTS: 'INVALID_JOINTS',
28     FollowJointTrajectoryResult.OLD_HEADER_TIMESTAMP: 'OLD_HEADER_TIMESTAMP',
29     FollowJointTrajectoryResult.PATH_TOLERANCE_VIOLATED: 'PATH_TOLERANCE_VIOLATED',
30     FollowJointTrajectoryResult.GOAL_TOLERANCE_VIOLATED: 'GOAL_TOLERANCE_VIOLATED',
31 }
32
33 def main():
34
35     ACTION_SERVICE_NAME = '/torobo/joint_trajectory_controller/follow_joint_
    ytrajectory'
36
37     try:
38         # Initializes a rospy node.
39         rospy.init_node('toroboarm_actionlib_follow_joint_trajectory_node')
40
41         # Creates the SimpleActionClient.
42         action_client = actionlib.SimpleActionClient(ACTION_SERVICE_NAME,
    yFollowJointTrajectoryAction)
43
44         # Waits until the action server has started up.
45         action_client.wait_for_server()
46
47         # Executes an action.
48         follow_joint_trajectory(
49             action_client = action_client,
50             joint_names = ['left_arm/joint_2'],
51             positions = np.deg2rad([-22.5]),
52             time_from_start = 5.0
53         )
54
55         # Executes an action.
56         follow_joint_trajectory(
57             action_client = action_client,
58             joint_names = ['left_arm/joint_2', 'right_arm/joint_1'],
59             positions = np.deg2rad([-45.0, 45.0]),
60             time_from_start = 5.0
61         )
62
63     except rospy.ROSInterruptException:

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

64         rospy.logerr('ROSInterruptException occurred')
65
66     finally:
67         pass
68
69     rospy.loginfo("finished.")
70
71 def create_joint_tolerance(name, position=-1, velocity=-1, acceleration=-1):
72     """
73     Function for create control_msgs/JointTolerance message
74
75     Parameters
76     ---
77     name : string
78         joint name to specify tolerance
79     position : float
80         position tolerance of the specified joint (in radians or meters)
81         (If the default value of -1 is set, the tolerance setting is ignored)
82     velocity : float
83         position tolerance of the specified joint (in rad/s or m/s)
84         (If the default value of -1 is set, the tolerance setting is ignored)
85     acceleration : float
86         position tolerance of the specified joint (in rad/s^2 or m/s^2)
87         (If the default value of -1 is set, the tolerance setting is ignored)
88
89     Returns
90     ---
91     joint_tolerance : control_msgs/JointTolerance
92         created JointTolerance message
93
94     Throws
95     ---
96     None
97     """
98
99     return JointTolerance(name, position, velocity, acceleration)
100
101 def follow_joint_trajectory(action_client, joint_names, positions, time_from_start):
102     """
103     Function for action to move joints
104
105     Parameters
106     ---
107     action_client : actionlib.SimpleActionClient
108         SimpleActionClient
109     joint_names : list
110         list of joint names
111     positions : list
112         list of joint's goal positions(radian)
113     time_from_start : float
114         transition time from start
115
116     Returns
117     ---

```

(Continued on next page)

(Continued from previous page)

```

118     None
119
120     Throws
121     -
122     e : rospy.ROSInterruptException
123         action fail
124     -
125
126     # Creates a goal.
127     goal = FollowJointTrajectoryGoal()
128     goal.goal_time_tolerance = rospy.Time(1.0)
129     goal.path_tolerance = [create_joint_tolerance(name=name, position=0.1) for name
    yin joint_names]
130     goal.goal_tolerance = [create_joint_tolerance(name=name, position=0.01) for name
    yin joint_names]
131     goal.trajectory.header.stamp = rospy.Time.now()
132     goal.trajectory.joint_names = joint_names
133     point = JointTrajectoryPoint()
134     point.positions = positions
135     point.velocities = [0.0 for i in range(len(joint_names))]
136     point.accelerations = [0.0 for i in range(len(joint_names))]
137     point.effort = [0.0 for i in range(len(joint_names))]
138     point.time_from_start = rospy.Duration(time_from_start)
139     goal.trajectory.points.append(point)
140
141     # Sends the goal.
142     action_client.send_goal(goal)
143
144     # Waits for the server.
145     finished_before_timeout = action_client.wait_for_result(timeout=rospy.
    yDuration(time_from_start + 10.0))
146
147     # Log state and result.
148     state = action_client.get_state()
149     result = action_client.get_result()
150     rospy.loginfo('[goal status] : %s' % ACTION_STATUS_DICT[state])
151     rospy.loginfo('[action result]: %s' % ACTION_RESULT_DICT[result.error_code])
152
153     # Check timeout
154     if not finished_before_timeout:
155         e = rospy.ROSInterruptException('action timeout')
156         rospy.logerr(e)
157         raise e
158     # Check state
159     if state != actionlib.GoalStatus.SUCCEEDED:
160         e = rospy.ROSInterruptException('action fail')
161         rospy.logerr(e)
162         raise e
163
164
165 if __name__ == '__main__':
166     main()

```

In the above script, if the action fails , a ROSInterruptException exception is thrown from the follow_joint_trajectory function.

9. Tutorial

is thrown and the processing stops.

The specific steps to move the arm using the above script are described below. First, create the core program of the actual machine. Or start Gazebo .

Next, switch the joint_trajectory_controller to the running state using Torobo State Viewer etc., and turn on the servos for all axes . Next, open another terminal and execute the script with the following command. Check that the arm moves.

```
$ roslaunch torobo_demo actionlib_follow_joint_trajectory.py
```

	Depending on the posture of the Danger Robot, it may interfere with the surrounding environment. Make sure there are no objects interfering with the surroundings, and give the command while you are ready to press the emergency stop button immediately. please.
---	---

9.4.3 Servo ON/OFF using ROS Service

We provide a ROS service that uses our own message type, torobo_std_msgs, to turn the robot's servo on and off. By implementing a Service Client to call the service , you can turn the servo on and off . For information about the unique message type, please refer to [the specifications of the unique ROS message](#) .

A sample script that calls a service to switch a servo (On/Off) and uses the response result to determine whether the switch was successful. The script is posted below.

```
1 # /usr/bin/env python 2 # -*- coding:
utf-8 -*-
3
4 import rospy 5 from
torobo_std_msgs.srv import SetStringArray
6
7 def main():
8     rospy.init_node('servo_on_off_node')
9
10    joint_names = ['left_arm/joint_5', 'right_arm/joint_4']
11
12    try:
13
14        # Set servo on
15        set_servo_on(joint_names)
16
17        # wait a while
18        rospy.sleep(3.0)
19
20        # Set servo off
21        set_servo_off(joint_names)
22
23    except rospy.ROSInterruptException:
```

(Continued on next page)

(Continued from previous page)

```

24         rospy.logerr('ROSErrorException occurred')
25
26     finally:
27         pass
28
29     rospy.loginfo("finished.")
30
31
32 def set_servo_on(joint_names):
33     """
34     Set servo on
35
36     Parameters
37     -----
38     joint_names : list
39         list of target joints' names (string)
40     """
41
42     # Call service
43     service_name = '/torobo/robot_hw/servo_on'
44     rospy.wait_for_service(service_name)
45     service = rospy.ServiceProxy(service_name, SetStringArray)
46     rospy.loginfo('call set_servo_on service')
47     response = service(joint_names)
48
49     # Check whether send is succeeded
50     if not response.success:
51         e = rospy.ROSErrorException('set_servo_on service is failed.')
52         rospy.logerr(e)
53         raise e
54
55     rospy.loginfo('set_servo_on service is succeeded.')
56
57 def set_servo_off(joint_names):
58     """
59     Set servo off
60
61     Parameters
62     -----
63     joint_names : list
64         list of target joints' names (string)
65     """
66
67     # Call service
68     service_name = '/torobo/robot_hw/servo_off'
69     rospy.wait_for_service(service_name)
70     service = rospy.ServiceProxy(service_name, SetStringArray)
71     rospy.loginfo('call set_servo_off service')
72     response = service(joint_names)
73
74     # Check whether send is succeeded
75     if not response.success:
76         e = rospy.ROSErrorException('set_servo_off service is failed.')
77         rospy.logerr(e)

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

78         raise and
79
80         rospy.loginfo('set_servo_off service is succeeded.')
81
82 if __name__ == '__main__':
83     main()

```

The specific steps to move the arm using the above script are described below. This script can only be run correctly on a real device. It always works. First, start the core program on the actual machine.

Next, open another terminal and execute the script with the following command .

The operation is successful if you can confirm that the servos of both axes turn ON and then turn OFF after 3 seconds.

```
$ rosrn torobo_demo servo_on_off.py
```

9.4.4 Acquiring and switching the **Controller** status using Controller Manager

Each control mode, such as position control mode and weight compensation mode, is operated by our proprietary ROS Controller . To start or switch between these Controllers , use the [controller_manager](#) Uses the Controller Manager package. By using the ROS Service provided by , you can obtain and switch the controller status.

A sample script that uses the Controller Manager to get the status of all Controllers and start/stop Controllers . This script is stored in the torobo_demo package. It works on both real devices and simulators (Gazebo) . You can also write similar code in C++.

```

1 #!/usr/bin/env python
2 #-*- coding: utf-8 -*-
3
4 import rospy
5 from controller_manager_msgs.srv import ListControllers
6 from controller_manager_msgs.srv import SwitchController, SwitchControllerRequest
7
8 def main():
9     rospy.init_node("switch_controller_node")
10
11     try:
12
13         # get controller states from controller_manager
14         controller_states = get_controller_states()
15
16         # display controller states
17         for state in controller_states:
18             rospy.loginfo('---- [%s] ----' % (state.name))
19             rospy.loginfo('- state: %s' % (state.state))
20             rospy.loginfo('- type : %s' % (state.type))
21             for claimed_resource in state.claimed_resources:
22                 rospy.loginfo('- hardware_interface: %s' % (claimed_resource.
23 hardware_interface))
24                 rospy.loginfo(' - resources:')

```

(Continued on next page)

(Continued from previous page)

```

24         for resource in claimed_resource.resources:
25             rospy.loginfo('          - %s' % resource)
26
27         # stop joint_trajectory_controller
28         rospy.loginfo('---- stop joint_trajectory_controller ----')
29         switch_controller(start_controllers=[],
30                           stop_controllers=['joint_trajectory_controller'])
31         rospy.sleep(2.0)
32
33         # start joint_trajectory_controller
34         rospy.loginfo('---- start joint_trajectory_controller ----')
35         switch_controller(start_controllers=['joint_trajectory_controller'],
36                           stop_controllers=[])
37         rospy.sleep(2.0)
38
39     except rospy.ROSInterruptException:
40         rospy.logerr('ROSInterruptException occurred')
41
42     finally:
43         pass
44
45     rospy.loginfo("finished.")
46
47 def get_controller_states(timeout=3.0):
48     """
49     Call list controllers service and return controller states
50
51     Parameters
52     ~~~~~
53     timeout : float
54         the timeout in seconds. Zero for infinite
55
56     Returns
57     ~~~~~
58     controller_states : controller_manager_msgs/ControllerState
59         controller states loaded in controller_manager
60
61     Throws
62     ~~~~~
63     e : rospy.ROSInterruptException
64         switch controllers failed
65
66     """
67     service_name = '/torobo/controller_manager/list_controllers'
68     try:
69         rospy.loginfo('call list_controllers service')
70         rospy.wait_for_service(service_name, timeout=timeout)
71         service = rospy.ServiceProxy(service_name, ListControllers)
72         response = service()
73     except rospy.ROSError as e:
74         e = rospy.ROSInterruptException('list_controllers service is failed.')
75         rospy.logerr(e)
76         raise
77

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

78     rospy.loginfo('list_controllers service is succeeded.')
79
80     return response.controller
81
82 def switch_controller(start_controllers=[],
83                       stop_controllers=[],
84                       timeout=3.0):
85     """
86     Call switch controllers service
87
88     Parameters
89     ---
90     start_controllers : list of string
91         the list of controller names to start
92     stop_controllers : list of string
93         the list of controller names to stop, and
94     timeout : float
95         the timeout in seconds. Zero for infinite
96
97     Returns
98     ---
99     response : bool
100         result of switch_controller service call
101
102     Throws
103     ---
104     e : rospy.ROSInterruptException
105         switch controllers failed
106     """
107
108     service_name = '/torobo/controller_manager/switch_controller'
109     try:
110         rospy.loginfo('call switch_controller service')
111         rospy.wait_for_service(service_name, timeout=timeout)
112         service = rospy.ServiceProxy(service_name, SwitchController)
113
114         request = SwitchControllerRequest()
115         request.start_controllers = start_controllers
116         request.stop_controllers = stop_controllers
117         request.strictness = SwitchControllerRequest.STRICT
118         request.start_asap = True
119         request.timeout = timeout
120
121         response = service(request)
122
123         # Check whether send is succeeded
124         rospy.loginfo('switch controller result: [%s], start_controllers: [%s], stop_
125 ycontrollers: [%s]'
126
127                       % (str(response.ok), ' '.join(start_controllers), ' '.join(stop_
128 ycontrollers)))
129
130     except rospy.ROSException as e:
131         e = rospy.ROSInterruptException('switch_controller service is failed.')
132         rospy.logerr(e)

```

(Continued on next page)

(Continued from previous page)

```

130         raise and
131
132     if not response.ok:
133         e = rospy.ROSInterruptException('switch_controller service is failed.')
134         rospy.logerr(e)
135         raise and
136
137     rospy.loginfo('switch_controller service is succeeded.')
138
139     return response.ok
140
141 if __name__ == '__main__':
142     main()

```

In the above script, we first use the ROS Service `controller_manager/list_controllers` provided by the Controller Manager to get the status of all Controllers.

The message type is `controller_manager_msgs/ListControllers.srv`. The state of all controllers is in `controller_manager_msgs/ControllerState.msg`. It is retrieved as an array of.

Next, `controller_manager/switch_controller`, which are ROS Services provided by Controller Manager.

The ROS Service message type is `controller_manager_msgs/SwitchController.srv`. The above script stops and restarts the `joint_trajectory_controller`.

However, you can switch controllers by specifying the controller for both `start_controller` and `stop_controller`.

It is also possible to do so.

The specific steps to run the above script are described below. First, run the core program or Gazebo. Please start the program.

Next, open another terminal and execute the script with the following command.

After the above status is displayed, if "switch_controller service is succeeded." is displayed, the operation was successful.

```
$ rosrn torobo_demo demo_controller_manager.py
```

Danger

Depending on the controller's switching destination, the joints may move suddenly.

For example, when an external force is applied to a joint, the `joint_trajectory_controller` (position control mode)

When you switch to `gravity_compensation_controller` (self-weight compensation mode), the robot will be able to withstand external forces.

The joints may move in response to the switch.

Please perform the switching operation.

Note

Depending on the controller you switch to, the servo may automatically turn off.

This is because the servos of joints that are not assigned to a controller are automatically turned off.

For example, if you use the `joint_trajecotry_controller` with the servo on,

When the script is executed, the `joint_trajectory_controller` is stopped.

When the controller assigned to each joint is no longer present, the servos will automatically turn off.

9. Tutorial

9.4.5 Position command by teaching point

Based on the teaching points (TP) , which are robot posture information prepared in advance , it is possible to give motion commands to the arm. Yes, you can. This function is useful when performing routine operations.

Teaching points are managed by the TeachingPointManager described in [the TeachingPointManager specifications](#) . Use config/tp_sample.yaml as the sample file .

The sample script below performs the following steps:

1. Load the teaching points listed in the sample file .
2. Read the joint name and target position corresponding to the teaching point name (tp_name) from teaching_point_manager
3. Send the retrieved joint name and target position via the action client

```

1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 import rospy
5 import rospkg
6 import actionlib
7 from actionlib_msgs.msg import GoalStatus
8 from trajectory_msgs.msg import JointTrajectoryPoint
9 from control_msgs.msg import FollowJointTrajectoryAction, FollowJointTrajectoryGoal,
   FollowJointTrajectoryResult
10 from control_msgs.msg import JointTolerance
11 import torobo_teaching_point_manager.teaching_point_utils as tp_utils
12
13 ACTION_STATUS_DICT = {
14     GoalStatus.PENDING : 'PENDING',
15     GoalStatus.ACTIVE : 'ACTIVE',
16     GoalStatus.PREEMPTED : 'PREEMPTED',
17     GoalStatus.SUCCEEDED : 'SUCCEEDED',
18     GoalStatus.ABORTED : 'ABORTED',
19     GoalStatus.REJECTED : 'REJECTED',
20     GoalStatus.RECALLING : 'RECALLING',
21     GoalStatus.RECALLED : 'RECALLED',
22     GoalStatus.LOST : 'LOST',
23 }
24
25 ACTION_RESULT_DICT = {
26     FollowJointTrajectoryResult.SUCCESSFUL: 'SUCCESSFUL',
27     FollowJointTrajectoryResult.INVALID_GOAL: 'INVALID_GOAL',
28     FollowJointTrajectoryResult.INVALID_JOINTS: 'INVALID_JOINTS',
29     FollowJointTrajectoryResult.OLD_HEADER_TIMESTAMP: 'OLD_HEADER_TIMESTAMP',
30     FollowJointTrajectoryResult.PATH_TOLERANCE_VIOLATED: 'PATH_TOLERANCE_VIOLATED',
31     FollowJointTrajectoryResult.GOAL_TOLERANCE_VIOLATED: 'GOAL_TOLERANCE_VIOLATED',
32 }
33
34 def main():
35
36     CONTROLLER_NAME = 'joint_trajectory_controller'
37     ACTION_SERVICE_NAME = CONTROLLER_NAME + '/follow_joint_trajectory'

```

(Continued on next page)

(Continued from previous page)

```

38
39     try:
40         # Initializes a rospy node.
41         rospy.init_node('torobo_move_to_teaching_point_node')
42
43         # get joint_names from rosparam
44         joint_names = rospy.get_param(CONTROLLER_NAME + '/joints')
45
46         # Load a yaml file containing teaching points
47         file_path = rospkg.RosPack().get_path('torobo_demo') + '/config/tp_sample.
48     yyaml'
49         tp_utils.load_teaching_point_from_yaml(file_path, overwrite=True)
50
51         # Creates the SimpleActionClient.
52         action_client = actionlib.SimpleActionClient(ACTION_SERVICE_NAME,
53     yFollowJointTrajectoryAction)
54
55         # Waits until the action server has started up.
56         action_client.wait_for_server()
57
58         # Move to the teaching point in turn.
59         move_to_tp('home_tp', 8, action_client, joint_names)
60         move_to_tp('both_hands_up', 8, action_client, joint_names)
61         move_to_tp('look_at_left_hand', 4, action_client, joint_names)
62         move_to_tp('look_at_right_hand', 4, action_client, joint_names)
63         move_to_tp('home_tp', 8, action_client, joint_names)
64
65     except rospy.ROSInterruptException:
66         rospy.logerr('ROSInterruptException occurred')
67
68     finally:
69         pass
70
71     rospy.loginfo("finished.")
72
73 def move_to_tp(tp_name, duration, action_client, joint_names):
74     _
75     Function for move to the joint's positions given by the teaching point
76
77     Parameters
78     _
79     tp_name : string
80         name of teaching point corresponding to joint's goal positions(radian)
81     duration : float
82         transition time to the teaching point
83     action_client : actionlib.SimpleActionClient
84         SimpleActionClient
85     joint_names : string[]
86         list of joint names belonging to the controller
87
88     Returns
89     _
90     None

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

Throws

—

None

```
rospy.loginfo('---- move to TP[%s] ----' % tp_name)
```

```
# Get target joint's positions
```

```
tp = tp_utils.get_teaching_point(tp_name)
```

```
tp_joint_names = []
```

```
tp_positions = []
```

```
for name, position in zip(tp.name, tp.position):
```

```
    if name in joint_names:
```

```
        tp_joint_names.append(name)
```

```
        tp_positions.append(position)
```

```
if len(tp_joint_names) == 0:
```

```
    rospy.logwarn('skip tp[%s]' % tp_name)
```

```
    return
```

```
# Executes an action.
```

```
follow_joint_trajectory(
```

```
    action_client=action_client,
```

```
    joint_names=tp_joint_names,
```

```
    positions=tp_positions,
```

```
    time_from_start=duration
```

```
)
```

```
120 def create_joint_tolerance(name, position=-1, velocity=-1, acceleration=-1):
```

```
    ---
```

```
Function for create control_msgs/JointTolerance message
```

```
Parameters
```

```
—
```

```
name : string
```

```
    joint name to specify tolerance
```

```
position : float
```

```
    position tolerance of the specified joint (in radians or meters)
```

```
    (If the default value of -1 is set, the tolerance setting is ignored)
```

```
velocity : float
```

```
    position tolerance of the specified joint (in rad/s or m/s)
```

```
    (If the default value of -1 is set, the tolerance setting is ignored)
```

```
acceleration : float
```

```
    position tolerance of the specified joint (in rad/s^2 or m/s^2)
```

```
    (If the default value of -1 is set, the tolerance setting is ignored)
```

```
Returns
```

```
—
```

```
joint_tolerance : control_msgs/JointTolerance
```

```
    created JointTolerance message
```

```
Throws
```

(Continued on next page)

(Continued from previous page)

```

144     -
145     None
146     -
147
148     return JointTolerance(name, position, velocity, acceleration)
149
150 def follow_joint_trajectory(action_client, joint_names, positions, time_from_start):
151     -
152     Function for action to move joints
153
154     Parameters
155     -
156     action_client : actionlib.SimpleActionClient
157         SimpleActionClient
158     joint_names : list
159         list of joint names
160     positions : list
161         list of joint's goal positions(radian)
162     time_from_start : float
163         transition time from start
164
165     Returns
166     -
167     None
168
169     Throws
170     -
171     e : rospy.ROSInterruptException
172         action fail
173     -
174
175     # Creates a goal.
176     goal = FollowJointTrajectoryGoal()
177     goal.goal_time_tolerance = rospy.Time(1.0)
178     goal.path_tolerance = [create_joint_tolerance(name=name, position=0.1) for name
    ÿin joint_names]
179     goal.goal_time_tolerance = [create_joint_tolerance(name=name, position=0.01) for name
    ÿin joint_names]
180     goal.trajectory.header.stamp = rospy.Time.now()
181     goal.trajectory.joint_names = joint_names
182     point = JointTrajectoryPoint()
183     point.positions = positions
184     point.velocities = [0.0 for i in range(len(joint_names))]
185     point.accelerations = [0.0 for i in range(len(joint_names))]
186     point.effort = [0.0 for i in range(len(joint_names))]
187     point.time_from_start = rospy.Duration(time_from_start)
188     goal.trajectory.points.append(point)
189
190     # Sends the goal.
191     action_client.send_goal(goal)
192
193     # Waits for the server.
194     finished_before_timeout = action_client.wait_for_result(timeout=rospy.
    ÿDuration(time_from_start + 10.0))

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

195
196 # Log state and result.
197 state = action_client.get_state()
198 result = action_client.get_result()
199 rospy.loginfo('[goal status] : %s' % ACTION_STATUS_DICT[state])
200 rospy.loginfo('[action result]: %s' % ACTION_RESULT_DICT[result.error_code])
201
202 # Check timeout
203 if not finished_before_timeout:
204     e = rospy.ROSInterruptException('action timeout')
205     rospy.logerr(e)
206     raise e
207
208 # Check state
209 if state != actionlib.GoalStatus.SUCCEEDED:
210     e = rospy.ROSInterruptException('action fail')
211     rospy.logerr(e)
212     raise e
213
214 if __name__ == '__main__':
215     main()

```

The specific steps to move the arm using the above script are described below. First, create the core program of the actual machine. Or start Gazebo .

Next, switch the joint_trajectory_controller to the running state using the Torobo State Viewer , etc., and Turn on the servo . Next, open another terminal and execute the script with the following command.

For each teaching point in tp_sample.yaml , if you can confirm that the robot posture changes in the order specified by move_to_tp,

The operation was successful.

```
$ roslaunch torobo_demo demo_move_to_teaching_point.launch
```



Depending on the posture of the **Danger** Robot, it may interfere with the surrounding environment.

Make sure there is nothing blocking your view and that you are ready to press the emergency stop button immediately. please.

9.4.6 High-frequency command of joint positions using ROS Publisher

The controllers named "online" such as online joint trajectory controller and online joint impedance controller

The controller determines the target position, velocity, and acceleration in the shortest time under constraints such as the maximum velocity and maximum acceleration of each joint.

Therefore, the movement of the position control mode (joint_trajectory_controller) is

The joint trajectory controller and joint impedance control mode (joint_impedance_controller) used in the operation check

Unlike the joint impedance controller used in the operation check , there is no argument related to the moving time, and it is continuous and high frequency. It is suitable for tasks that require rapid response to commands.

The online joint trajectory controller sends commands at 30Hz so that the right_arm/joint_2 moves back and forth between -30° and 0°.

A sample script is provided below.

```

1 #! /usr/bin/env python
2 #-*- coding: utf-8 -*-
3
4 import rospy
5 import actionlib
6 import numpy as np
7 from trajectory_msgs.msg import JointTrajectoryPoint, JointTrajectory
8
9
10 def main():
11
12     TOPIC_NAME = '/torobo/online_joint_trajectory_controller/command'
13     # TOPIC_NAME = '/torobo/online_joint_impedance_controller/command'
14
15     # Initializes a rospy node.
16     rospy.init_node('torobo_publish_online_joint_command_node')
17
18     # Create a publisher.
19     publisher = rospy.Publisher(TOPIC_NAME, JointTrajectory, queue_size=1)
20
21     # Wait until the publisher gets ready.
22     while publisher.get_num_connections() == 0:
23         rospy.sleep(1)
24
25     # Create JointTrajectory message
26     trajectory = create_joint_trajectory_base()
27     trajectory.joint_names = ['right_arm/joint_2']
28
29     command_rate = 30 # Hz
30     T = 6 # Sec
31     dt = 1. / command_rate
32     r = rospy.Rate(command_rate)
33     try:
34         for i in range(500):
35             # Update command position
36             pos = np.deg2rad(-30) * (1 - np.cos(2. * np.pi * i * dt / T)) / 2
37             trajectory.points[0].positions = [pos]
38             trajectory.header.stamp = rospy.Time.now()
39
40             # Publish a message.
41             publisher.publish(trajectory)
42             r.sleep()
43
44     except KeyboardInterrupt as e:
45         rospy.loginfo("Shutdown request received.")
46         rospy.logerr(e)
47         import traceback
48         traceback.print_exc()
49     except IOError as e:
50         rospy.logerr("IOError occurred.")
51         rospy.logerr(e)
52         import traceback
53         traceback.print_exc()

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

54
55     rospy.loginfo("finished.")
56
57 def create_joint_trajectory_base():
58     """
59     Function to create an empty JointTrajectory message.
60
61     Parameters
62     —
63     None
64
65     Returns
66     —
67     JointTrajectory
68
69     Throws
70     —
71     None
72     """
73
74     # Creates a message.
75     trajectory = JointTrajectory()
76     trajectory.header.stamp = rospy.Time.now()
77     point = JointTrajectoryPoint()
78     point.time_from_start = rospy.Time(0)
79     trajectory.points.append(point)
80
81     return trajectory
82
83 if __name__ == '__main__':
84     main()

```

The specific steps to move the arm using the above script are described below. First, create the core program of the actual machine. Or start Gazebo .

Next, switch the online_joint_trajectory_controller to the running state using Torobo State Viewer , etc. Turn on the servos for all axes . Next, open another terminal and execute the script with the following command.

The operation is successful if you can confirm that the fifth axis of the right arm moves back and forth between $\pm 30^\circ$.

```
$ roslaunch torobo_demo demo_online_joint_trajectory_command.py
```



Depending on the posture of the **Danger** Robot, it may interfere with the surrounding environment.

Make sure there is nothing blocking your view and that you are ready to press the emergency stop button immediately. please.

9.4.7 Position command using MoveIt!

The trajectory plan by MoveIt! used in [position command using rviz + moveit](#) can also be used from a script. is.

First, when using MoveIt! from a script, there are two ways to plan a trajectory:

1. How to plan a trajectory given a start and end point
2. How to plan a trajectory by connecting multiple waypoints

The sample script below performs the following steps:

1. First, move the robot to the StartPosition (joint 4 is at 90°, the others are at 0°) (method 1)
2. Next, create waypoints
3. Use MoveIt! to plan a trajectory to the first waypoint and move the arm (method 1)
4. Use MoveIt! to plan a trajectory connecting the second , third, and fourth waypoints and move the arm (method 2)

```

1 #!/usr/bin/env python
2 #-*- coding: utf-8 -*-
3 import six
4 import rospy
5 import rosparam
6 import numpy as np
7 import copy
8 import tf
9 import moveit_commander
10 from geometry_msgs.msg import Pose
11
12 # parameters
13 VEL_SCALING = 0.1
14 ACC_SCALING = 0.1
15
16 def main():
17     rospy.init_node("torobo_moveit_commander_node", anonymous=True)
18
19     # Get move_group_commander.
20     arm = get_move_group_commander('left_arm')
21
22     # (1) Moving to start position by "go" method.
23     rospy.loginfo("Moving to start position.")
24     start_position = {name:0.0 if 'joint_4' not in name else 1.57 for name in arm.
    yget_active_joints()}
25
26     arm.set_start_state_to_current_state()
27     arm.set_joint_value_target(start_position) arm.go() rospy.loginfo(" -->
28
29     + str(arm.get_current_pose().pose))
30
31     # (2) create waypoints
32     start_pos = arm.get_current_pose().pose.position
33     start_rpy = np.degrees(arm.get_current_rpy())

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

34
35     model = rosparam.get_param('/torobo/robot_config/model')
36     rospy.loginfo("model name '%s'" % model)
37     if ('dry-airec' in model) or ('torobo' in model):
38         wp1 = create_pose(0.45, 0.40, 0.9, np.radians(180.0), np.radians(0.0), np.
39         y radians(0.0))
40         wp2 = create_pose(0.45, 0.40, 1.1, np.radians(180.0), np.radians(0.0), np.
41         y radians(0.0))
42         wp3 = create_pose(0.45, 0.20, 1.1, np.radians(180.0), np.radians(0.0), np.
43         y radians(0.0))
44         wp4 = create_pose(0.45, 0.20, 0.9, np.radians(180.0), np.radians(0.0), np.
45         y radians(0.0))
46         elif 'toala' in pattern:
47             wp1 = create_pose(0.50, 0.50, 0.9, np.radians(-90.0), np.radians(0.0), np.
48             y radians(-90.0))
49             wp2 = create_pose(0.50, 0.50, 1.1, np.radians(-90.0), np.radians(0.0), np.
50             y radians(-90.0))
51             wp3 = create_pose(0.50, 0.30, 1.1, np.radians(-90.0), np.radians(0.0), np.
52             y radians(-90.0))
53             wp4 = create_pose(0.50, 0.30, 0.9, np.radians(-90.0), np.radians(0.0), np.
54             y radians(-90.0))
55             elif 'prayers' in model:
56                 wp1 = create_pose(0.45, 0.40, 0.8, np.radians(180.0), np.radians(0.0), np.
57                 y radians(0.0))
58                 wp2 = create_pose(0.45, 0.40, 1.0, np.radians(180.0), np.radians(0.0), np.
59                 y radians(0.0))
60                 wp3 = create_pose(0.45, 0.20, 1.0, np.radians(180.0), np.radians(0.0), np.
61                 y radians(0.0))
62                 wp4 = create_pose(0.45, 0.20, 0.8, np.radians(180.0), np.radians(0.0), np.
63                 y radians(0.0))
64             else:
65                 rospy.logerr("model name '%s' is not defined!" % model)
66                 return
67
68     # (3) Moving to wp1 by "plan and execute" method.
69     rospy.loginfo("Moving to wp1 by 'plan and execute' method.")
70     arm.set_start_state_to_current_state() # set start
71     arm.set_pose_target(wp1) plan = # set target
72     plan_trajectory(arm)
73
74     arm.execute(plan, wait=True) rospy.loginfo(" --> # execute
75     + str(arm.get_current_pose().pose))
76
77     # (4) Moving to wp2,wp3,wp4 "compute_cartesian_path" method.
78     rospy.loginfo("Moving to wp2,wp3,wp4 by 'cartesian path' method.")
79     waypoints = [wp2, wp3, wp4]
80     arm.set_start_state_to_current_state() # set start
81     (plan, fraction) = arm.compute_cartesian_path(waypoints, eef_step=0.001, jump_
82     y threshold=20.0)
83     if not np.isclose(fraction, 1.0):
84         rospy.logerr("Failed to plan. computed path fraction is [%f]. fraction must
85         y be 1.0. Aborted." % fraction)
86         return
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

(Continued on next page)

(Continued from previous page)

```

74     # retime trajectory to apply velocity and acceleration scaling factor
75     plan = arm.retime_trajectory(
76         ref_state_in=arm.get_current_state(), traj_in=plan,
77         velocity_scaling_factor=VEL_SCALING,
78         acceleration_scaling_factor=ACC_SCALING,
79         algorithm='time_optimal_trajectory_generation')
80
81     if plan is None:
82         rospy.logerr('Failed to retime trajectory. Aborted.')
83         return
84
85     arm.execute(plan, wait=True)
86     rospy.loginfo(" --> " + str(arm.get_current_pose().pose))
87
88 def get_move_group_commander(name):
89     """
90     Function for getting moveit's move_group commander
91
92     Parameters
93     ---
94     name : string
95         move_group's name
96
97     Returns
98     ---
99     commander : MoveGroupCommander
100         moveit's move_group commander
101     """
102
103     # Initializing move_group_commander.
104     # robot = moveit_commander.RobotCommander(robot_description=rospy.get_
105     ynamespace() + "robot_description" , ns=rospy.get_namespace())
106     robot = moveit_commander.RobotCommander()
107     commander = moveit_commander.MoveGroupCommander(name, robot_description=rospy.
108     yget_namespace() + "robot_description", ns=rospy.get_namespace())
109
110     # Settings.
111     commander.set_planner_id('RRTConnectkConfigDefault')
112     commander.set_max_velocity_scaling_factor(VEL_SCALING)
113     commander.set_max_acceleration_scaling_factor(ACC_SCALING)
114     commander.set_goal_orientation_tolerance(0.001)
115     commander.set_goal_position_tolerance(0.001)
116     commander.set_goal_joint_tolerance(0.001)
117     commander.set_goal_tolerance(0.001)
118     commander.set_num_planning_attempts(50)
119     commander.set_planning_time(2.0)
120     commander.set_pose_reference_frame('base_link')
121
122     return commander
123
124 def create_pose(x, y, z, roll, pitch, yaw):
125     q = tf.transformations.quaternion_from_euler(roll, pitch, yaw)
126     pose = Pose()
127     pose.position.x = x
128     pose.position.y = y

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```
126     pose.position.z = z
127     pose.orientation.x = q[0]
128     pose.orientation.y = q[1]
129     pose.orientation.z = q[2]
130     pose.orientation.w = q[3]
131     return pose
132
133 def plan_trajectory(move_group_commander):
134     # For Python2 (Melodic)
135     if six.PY2:
136         plan = move_group_commander.plan()
137     # For Python3 (Noetic)
138     else:
139         (success, plan, planning_time, error_code) = move_group_commander.plan()
140
141     return plan
142
143 if __name__ == '__main__':
144     main()
145
```

The specific steps to move the arm using the above script are described below. First, create the core program of the actual machine. Or start Gazebo .

Next, switch the joint_trajectory_controller to the running state using the Torobo State Viewer , etc., and Turn on the servo . Next, open another terminal and execute the script with the following command.

After each joint of the arm moves to ZeroPosition , the left hand moves to the positions and postures represented by wp1, wp2, wp3, and wp4 in order. If you can confirm that this is the case, the operation was successful.

```
$ roslaunch torobo_demo demo_moveit_commander.launch
```



Depending on the posture of the **Danger** Robot, it may interfere with the surrounding environment.

Make sure there is nothing blocking your view and that you are ready to press the emergency stop button immediately.

please

9.4.8 Calculating Kinematics with MoveIt!

You can use MoveIt! to solve forward and inverse kinematics. In forward kinematics, the orthogonal The position and orientation of the robot in the coordinate system (x, y, z, roll, pitch, yaw) are obtained, and inverse kinematics is used to calculate the position of the robot in the Cartesian coordinate system. By giving the position and orientation (x, y, z, roll, pitch, yaw) , the joint values can be obtained . You can get the calculation results by inquiring the Service (table below) .

Method	Service name	Service type
kinematics	compute_fk	moveit_msgs/GetPositionFK
Inverse kinematics	compute_ik	moveit_msgs/GetPositionIK

The script for obtaining the calculation results of forward kinematics and inverse kinematics is provided below.

```

1 #! /usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 import rospy
5 import tf
6 import numpy as np
7 import moveit_commander
8 from std_msgs.msg import Header
9 from geometry_msgs.msg import Pose, PoseStamped, TransformStamped
10 from moveit_msgs.srv import GetPositionFK, GetPositionIK
11 from moveit_msgs.msg import RobotState, PositionIKRequest
12
13
14 def main():
15
16     torobo = Torobo(movegroup="left_arm", tooltip="left_arm/link_tip")
17     res = torobo.compute_fk(joint_angles=np.radians([50.0, 0.0, 50.0, 50.0, 50.0, 50.
18     0, 50.0]))
19     res = torobo.compute_ik(x=res[0], y=res[1], z=res[2], roll=res[3], pitch=res[4],
20     yyaw=res[5])
21
22
23 class Torobo:
24
25     def __init__(self, movegroup, tooltip):
26         # constructor of this class
27
28         # Parameters
29         # movegroup : string
30         # Specify move group's name(string) you want to operate. Selectable move
31         # ygroup is written in moveit_config's srdf file.
32         # tooltip : string
33         # Specify tooltip's name(string) you want to set position.
34
35         rospy.init_node("torobo_moveit_kinematics_node", anonymous=True)
36         rospy.Rate(1)
37         rospy.sleep(1)
38
39         # load MoveGroupCommander by string('left_arm') registerd in 'torobo.srdf'
40         self._robot = moveit_commander.MoveGroupCommander(movegroup, robot_
41         ydescription=rospy.get_namespace() + "robot_description", ns=rospy.get_namespace())
42         self._robot.set_pose_reference_frame('base_link')
43         self._active_joints = self._robot.get_active_joints()
44         self._tooltip = tooltip
45
46         # load fk/ik service
47         rospy.loginfo("loading fk")
48         rospy.wait_for_service('compute_fk')
49         rospy.wait_for_service('compute_ik')
50
51     try:

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

50         self._moveit_fk = rospy.ServiceProxy('compute_fk', GetPositionFK)
51         self._moveit_ik = rospy.ServiceProxy('compute_ik', GetPositionIK)
52     except rospy.ServiceException as e:
53         rospy.logerror("Service call failed: %s"%e)
54
55
56     def compute_fk(self, joint_angles):
57         _
58
59         Function for computing forward kinematics
60
61         Parameters
62         _
63         joint_angles : list
64             list of joint's angles [rad]
65
66         Returns
67         _
68         x, y, z, roll, pitch, yaw : float
69             result of compute_fk (position(meter) and pose(radian))
70         _
71
72         rospy.loginfo("---- compute fk ----")
73
74         # check input array length
75         if len(self._active_joints) != len(joint_angles):
76             rospy.logwarn("joint_angles doesn't matches active_joints")
77             num_diff = len(self._active_joints) - len(joint_angles)
78             if num_diff > 0:
79                 for _ in range(num_diff):
80                     joint_angles = np.append(joint_angles, 0.0)
81
82         # print input
83         rospy.loginfo("[input]")
84         for i in range(len(self._active_joints)):
85             rospy.loginfo("          {0} : {1:.2f}".format(self._active_joints[i], np.
86               degrees(joint_angles[i])))
87
88         # create messages
89         header = Header(0, rospy.Time.now(), "base_link")
90         rs = RobotState()
91         rs.joint_state.name = self._active_joints
92         rs.joint_state.position = joint_angles
93
94         # get forward kinematics solution
95         fk_result = self._moveit_fk(header, [self._tooltip], rs)
96         assert len(fk_result.pose_stamped) > 0, "fk has no solution"
97         pose = fk_result.pose_stamped[0].pose
98         euler = tf.transformations.euler_from_quaternion(
99             (pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.
100              orientation.w))
101
102         # print result
103         rospy.loginfo("[output]")

```

(Continued on next page)

(Continued from previous page)

```

102     rospy.loginfo("          x          : {:.2f}".format(pose.position.x))
103     rospy.loginfo("          and          : {:.2f}".format(pose.position.y))
104     rospy.loginfo("          we          : {:.2f}".format(pose.position.z))
105     rospy.loginfo("          roll : {:.2f}".format(np.degrees(euler[0])))
106     rospy.loginfo("          pitch : {:.2f}".format(np.degrees(euler[1])))
107     rospy.loginfo("          yaw : {:.2f}".format(np.degrees(euler[2])))
108
109     return pose.position.x, pose.position.y, pose.position.z, euler[0], euler[1],
euler[2]
110
111
112     def compute_ik(self, x, y, z, roll, pitch, yaw):
113         _
114         Function for computing inverse kinematics
115
116         Parameters
117         _
118         x, y, z, roll, pitch, yaw : float
119             tooltip's position(meter) and pose(radian)
120
121         Returns
122         _
123         joint_state : JointState
124             result of compute_ik (list of joint angles)
125         _
126
127         rospy.loginfo("---- compute ik ----")
128
129         # print input
130         rospy.loginfo("[input]")
131         rospy.loginfo("          x          : {:.2f}".format(x))
132         rospy.loginfo("          and          : {:.2f}".format(y))
133         rospy.loginfo("          we          : {:.2f}".format(z))
134         rospy.loginfo("          roll : {:.2f}".format(np.degrees(roll)))
135         rospy.loginfo("          pitch : {:.2f}".format(np.degrees(pitch)))
136         rospy.loginfo("          yaw : {:.2f}".format(np.degrees(yaw)))
137
138         quaternion = tf.transformations.quaternion_from_euler(roll, pitch, yaw)
139
140         # create request
141         req = PositionIKRequest()
142         req.timeout = rospy.Duration(5.0)
143         req.ik_link_name = self._tooltip
144         req.pose_stamped.header.frame_id = "base_link"
145         req.pose_stamped.pose.position.x = x
146         req.pose_stamped.pose.position.y = y
147         req.pose_stamped.pose.position.z = z
148         req.pose_stamped.pose.orientation.x = quaternion[0]
149         req.pose_stamped.pose.orientation.y = quaternion[1]
150         req.pose_stamped.pose.orientation.z = quaternion[2]
151         req.pose_stamped.pose.orientation.w = quaternion[3]
152         req.robot_state.joint_state.name = self._active_joints
153         req.robot_state.joint_state.position = self._robot.get_current_joint_
values() # current values as seeds

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

154         req.group_name = self._robot.get_name()
155         req.avoid_collisions = False
156
157         # get i result
158         ik_result = self._moveit_ik(req).solution
159
160         # print result
161         rospy.loginfo("[output]")
162         for i in range(len(ik_result.joint_state.name)):
163             joint_name = ik_result.joint_state.name[i]
164             if joint_name in self._active_joints:
165                 rospy.loginfo("    {0} : {1:.2f}".format(joint_name, np.degrees(ik_
166 yresult.joint_state.position[i])))
167
168         return ik_result.joint_state
169
170 if __name__ == '__main__':
171     main()
172

```

Next, open another terminal and execute the script with the following commands. Forward kinematics and inverse kinematics, respectively
If you can confirm that the correct answer [output] is displayed for the [input] above , then you have succeeded.

```
$ roslaunch torobo_demo demo_moveit_kinematics.launch
```

9.4.9 Opening and closing the hand

The joint position control using ROS Publish (one-way communication) and the joint position control using ROS Actionlib (two-way communication)
The robot uses ROS Publish and ROS Actionlib to move the knuckles of the hand , as well as control the arm of the robot (two-way communication).
You can.

Below is a sample script that uses publisher and action_client in order to command the opening and closing of the left hand.
Masu.

```

1 # /usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4
5 import rospy
6 import actionlib
7 import numpy as np
8 from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
9 from control_msgs.msg import FollowJointTrajectoryAction, FollowJointTrajectoryGoal
10
11 def main():
12
13     CONTROLLER_NAME = 'left_hand_controller'
14     JOINTS_PARAM_NAME = '/torobo/' + CONTROLLER_NAME + '/joints'
15     TOPIC_NAME = '/torobo/' + CONTROLLER_NAME + '/command'

```

(Continued on next page)

(Continued from previous page)

```

16 ACTION_SERVICE_NAME = '/torobo/' + CONTROLLER_NAME + '/hand_cmd'
17
18 try:
19     # Initializes a rospy node.
20     rospy.init_node('torobo_publish_hand_command_node')
21
22     joint_names = rospy.get_param(JOINTS_PARAM_NAME)
23     rospy.loginfo("joint size is '%d'" % len(joint_names))
24     if len(joint_names) == 3:
25         joint_names = [
26             'left_hand/joint_1',
27             'left_hand/joint_2',
28             'left_hand/joint_3',
29         ]
30         positions1 = np.deg2rad([85.0, 85.0, 0.0])
31         positions2 = np.deg2rad([45.0, 0.0, 85.0])
32     elif len(joint_names) == 6:
33         joint_names = [
34             'left_hand_thumb/joint_1',
35             'left_hand_thumb/joint_2',
36             'left_hand_thumb/joint_3',
37             'left_hand_first_finger/joint_1',
38             'left_hand_second_finger/joint_1',
39             'left_hand_third_finger/joint_1'
40         ]
41         positions1 = np.deg2rad([0.0, 0.0, 0.0, 85.0, 85.0, 85.0])
42         positions2 = np.deg2rad([40.0, 0.0, 40.0, 0.0, 0.0, 85.0])
43     else:
44         rospy.logerr("invalid joint size!")
45         return
46
47     zero_positions = [0.] * len(joint_names)
48     max_efforts = [1.] * len(joint_names)
49
50     # Create a publisher.
51     publisher = rospy.Publisher(TOPIC_NAME, JointTrajectory, queue_size=1)
52
53     # Creates the SimpleActionClient.
54     action_client = actionlib.SimpleActionClient(ACTION_SERVICE_NAME,
55 yFollowJointTrajectoryAction)
56
57     # Waits until the action server has started up.
58     action_client.wait_for_server()
59
60     # Wait until the publisher gets ready.
61     while publisher.get_num_connections() == 0:
62         rospy.sleep(1)
63
64     rospy.loginfo('move to zero position')
65     follow_joint_trajectory(
66         action_client=action_client,
67         joint_names=joint_names,
68         positions = zero_positions,
69         max_efforts = max_efforts

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

69         )
70
71         # publish joint trajectory
72         rospy.loginfo('publish hand joint trajectory')
73         publish_joint_trajectory(
74             publisher=publisher,
75             joint_names=joint_names,
76             positions = positions1,
77             max_efforts = max_efforts
78         )
79
80         # wait for movement
81         rospy.sleep(3)
82
83         # execute follow joint trajectory action
84         rospy.loginfo('execute hand follow joint trajectory action')
85         follow_joint_trajectory(
86             action_client=action_client,
87             joint_names=joint_names,
88             positions = positions2,
89             max_efforts = max_efforts
90         )
91
92         rospy.loginfo('move to zero position')
93         follow_joint_trajectory(
94             action_client=action_client,
95             joint_names=joint_names,
96             positions = zero_positions,
97             max_efforts = max_efforts
98         )
99
100     except rospy.ROSInterruptException:
101         rospy.logerr("ROSInterruptException occurred")
102
103     finally:
104         pass
105
106     rospy.loginfo("finished.")
107
108
109 def publish_joint_trajectory(publisher, joint_names, positions, max_efforts):
110     """
111     Function for publishing message to move the hand
112     Note that position's unit is "radians" and max_effort's unit is "Nm".
113
114     Parameters
115     """
116     publisher : rospy.Publisher
117         publisher
118     joint_names : list of string
119         list of joint names
120     positions : list of float
121         list of target finger positions.
122     max_effort : list of float

```

(Continued on next page)

(Continued from previous page)

```

123         list of finger's max efforts.
124
125     Returns
126     ---
127     None
128
129     Throws
130     ---
131     None
132     ---
133
134     # Creates a message.
135     trajectory = JointTrajectory()
136     trajectory.header.stamp = rospy.Time.now()
137     trajectory.joint_names = joint_names
138     point = JointTrajectoryPoint()
139     point.positions = positions
140     point.velocities = [0.0] * len(joint_names)
141     point.accelerations = [0.0] * len(joint_names)
142     point.effort = max_efforts
143     point.time_from_start = rospy.Duration(0) # ignored in hand_controller
144     trajectory.points.append(point)
145
146     # Publish the message.
147     publisher.publish(trajectory)
148
149
150 def follow_joint_trajectory(action_client, joint_names, positions, max_efforts):
151     ---
152     Function for action to move joints
153
154     Parameters
155     ---
156     action_client : actionlib.SimpleActionClient
157                     SimpleActionClient
158     joint_names : list of string
159                  list of joint names
160     positions : list of float
161                list of target finger positions.
162     max_effort : list of float
163                list of finger's max efforts.
164
165     Returns
166     ---
167     None
168
169     Throws
170     ---
171     e : rospy.ROSInterruptException
172         action fail
173     ---
174
175     # Creates a goal.
176     goal = FollowJointTrajectoryGoal()

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

177     goal.goal_time_tolerance = rospy.Time(1.0)
178     goal.trajectory.header.stamp = rospy.Time.now()
179     goal.trajectory.joint_names = joint_names
180     point = JointTrajectoryPoint()
181     point.positions = positions
182     point.velocities = [0.0] * len(joint_names)
183     point.accelerations = [0.0] * len(joint_names)
184     point.effort = max_efforts
185     point.time_from_start = rospy.Duration(0) # ignored in hand_controller
186     goal.trajectory.points.append(point)
187
188     # Sends the goal.
189     action_client.send_goal(goal)
190
191     # Waits for the server.
192     finished_before_timeout = action_client.wait_for_result(timeout=rospy.
193     yDuration(10.0))
194
195     # Log state and result.
196     state = action_client.get_state()
197     result = action_client.get_result()
198     rospy.loginfo('[state ]: ' + str(state))
199     rospy.loginfo('[result]: ' + str(result))
200
201     # Check timeout
202     if not finished_before_timeout:
203         e = rospy.ROSInterruptException('action timeout')
204         rospy.logerr(e)
205         raise e
206
207     # Check state
208     if state != actionlib.GoalStatus.SUCCEEDED:
209         e = rospy.ROSInterruptException('action fail')
210         rospy.logerr(e)
211         raise e
212
213 if __name__ == '__main__':
214     main()

```

The specific steps to move the hand using the above script are described below. First, Or start Gazebo .

Next, switch the left_hand_controller to the running state using the Torobo State Viewer , etc., and control all the axis servos. Please turn it ON . Before executing the script, please note that if any of the fingers are already closed or if the hand and If the body and base are too close together, your fingers may interfere with each other or with the body or base.

Therefore, the fingers are opened by position commands using the above-mentioned rqt_torobo_joint_trajectory_controller , and the arm, torso, and base are connected.

The distance between the two devices should also be kept apart in advance.

Next, run the script with the command below. The joints of the left hand and fingers specified in joint_names will be executed in order. The operation is successful if you can confirm that it moves to the corresponding positions .

```
$ rosrn torobo_demo demo_hand_command.py
```

	Danger: Depending on the hand/arm position, fingers may interfere with each other or with the torso/base, resulting in self-interference. The detection may cause the operation to stop or undue strain to your fingers. If an operation that applies an excessive load occurs, immediately press the emergency stop button. stomach.
---	---

9.4.10 Visualize the pressure sensor output value with RViz

The skin of the robot is equipped with a pressure sensor (FSR, <https://akizukidenshi.com/download/ds/interlinkelec/fsr.pdf>). The pressure sensor value ranges from 0 to 4095, and the greater the pressure, the higher the value.

The pressure sensor value is our own message type for the gpio_state_controller/gpio_state Topic . The `analog_io_port_names` field contains the names of the pressure sensors . The name of the link it is attached to is stored, and the `analog_io_data` field contains the sensor value.

A sample for subscribing to the pressure sensor value (gpio_state_controller/gpio_state Topic) and visualizing it on RViz. This script is stored in the torobo_demo package.

This code will only run on a real device. It is also possible to write similar code in C++.

```
1 # /usr/bin/env python
2 #-*- coding: utf-8 -*-
3
4 import rospy
5 from visualization_msgs.msg import MarkerArray, Marker
6 from geometry_msgs.msg import Pose, Point, Vector3, Quaternion
7 from gpio_msgs.msg import GpioStates
8 import numpy as np
9 import tf
10 from tf2_msgs.msg import TFMessage
11
12 def euler_to_quaternion(euler, degrees=False):
13     """Convert Euler Angles to Quaternion
14
15     euler: geometry_msgs/Vector3
16     quaternion: geometry_msgs/Quaternion
17
18     angles = [euler.x, euler.y, euler.z]
19     angles = np.radians(angles) if degrees else angles
20     q = tf.transformations.quaternion_from_euler(*angles)
21     return Quaternion(x=q[0], y=q[1], z=q[2], w=q[3])
22
23 class GpioVisualizer():
24
25     def __init__(self):
26         GPIO_STATE_TOPIC_NAME = "/torobo/gpio_state_controller/gpio_states"
27         GPIO_MARKERS_TOPIC_NAME = "/gpio_markers"
28
29         self._rate = 10
```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

30         self._last_pub_time = rospy.Time.now()
31         self._tf_static_list = []
32         self._publish_first = True
33         self._gpio_sub = rospy.Subscriber(GPIO_STATE_TOPIC_NAME, GpioStates, self._
gpio_state_callback)
34         self._pub = rospy.Publisher(GPIO_MARKERS_TOPIC_NAME, MarkerArray, queue_
ysize=1)
35         self._tf_sub = rospy.Subscriber("/tf_static", TFMessage, self._tf_callback)
36
37     def _tf_callback(self, msg):
38         for transform in msg.transforms:
39             self._tf_static_list.append(transform.child_frame_id)
40             rospy.loginfo('tf_static subscribed.')
41
42     def _gpio_state_callback(self, msg):
43         t = rospy.Time.now()
44         if (t - self._last_pub_time).to_sec() < (1.0 / self._rate):
45             return
46         self._last_pub_time = t
47
48         # publish analog data to visualize
49         # Only publish port_name that exists in tf_static
50         port_names = []
51         analog_data = []
52         for port_name, data in zip(msg.analog_io_port_names, msg.analog_io_data):
53             if port_name + '/link' in self._tf_static_list:
54                 port_names.append(port_name)
55                 analog_data.append(data)
56         self._publish(port_names, analog_data)
57
58     def _create_marker(self, frame_id, time, marker_id, scale):
59         checkmark = Checkmark()
60         marker.header.frame_id = frame_id
61         marker.header.stamp = time
62         marker.ns = 'basic_shapes'
63         marker.id = marker_id
64
65         marker.type = Marker.ARROW
66         marker.action = Marker.ADD
67
68         marker.bag = Bag(
69             Point(0, 0, 0),
70             euler_to_quaternion(Vector3(0, -90, 0), degrees=True))
71
72         marker.scale.x = scale
73         marker.scale.y = 0.005
74         marker.scale.z = 0.005
75
76         marker.color.r = 1.0
77         marker.color.g = 0.0
78         marker.color.b = 0.0
79         marker.color.a = 0.5
80
81         marker.lifetime = rospy.Duration()

```

(Continued on next page)

(Continued from previous page)

```

82
83     return marker
84
85     def _create_marker_array(self, port_names, port_values):
86         ma = MarkerArray()
87         time = rospy.Time.now()
88         marker_id = 0
89         for name, value in zip(port_names, port_values):
90             frame_id = name + '/link'
91             scale = np.max([value / 4095.0 / 2.0, 0.005])
92             ma.markers.append(self._create_marker(frame_id, time, marker_id, scale))
93             marker_id += 1
94         return ma
95
96     def _publish(self, port_names, port_values):
97         self._pub.publish(self._create_marker_array(port_names, port_values))
98         if self._publish_first:
99             rospy.loginfo('/gpio_markers published.')
100             self._publish_first = False
101
102     def run(self):
103         rospy.spin()
104
105 def main():
106     try:
107         rospy.init_node("gpio_visualizer_node")
108         visualizer = GpioVisualizer()
109         visualizer.run()
110     except:
111         pass
112
113     rospy.loginfo('finished.')
114
115
116 if __name__ == "__main__":
117     main()

```

Visualization on RViz is in `visualization_msgs/Marker`. The arrow is displayed by publishing a message of type `visualization_msgs/Marker`.

The specific steps to execute the above script are described below. First, start the core program of the actual device. Please move.

Next, open another terminal and run the script with the following command:

```
$ rosrn torobo_demo gpio_visualizer.py
```

Next, start RViz to check the pressure sensor value. Open another terminal and execute the following command. Please go.

```
$ roslaunch torobo_bringup rviz.launch
```

When Rviz starts, press the "Add" button at the bottom left to open the Create visualization menu. Select the `/gpio_markers` topic from the tab and press the "OK" button.

9. Tutorial

An arrow should be drawn from the position with a length proportional to the sensor value.



Note: Due to the nature of the sensor, the sensor value of a pressure sensor attached to a curved surface may have a large offset.

There are cases where the set is on. In this case, even if it is not touched, the Please note that a larger value is output.

9.4.11 Controlling the cart speed using ROS Publish

The cart is controlled by [omni_drive_controller /OmniDriveController](#) , which provides a topic for issuing speed commands to the cart .

The user can make the cart move by publishing a message to this topic .

The wheels of the dolly are omni-wheels, allowing it to move in all directions .

[try_msgs/Twist](#) By publishing according to the type , it is possible to run in any direction.

[omni_drive_controller/OmniDriveController](#) is used to run a cart. Sample script (python) is available.

This script is stored in the `torobo_demo` package. This code can also be used to simulate the actual machine.

It also works with Gazebo . You can also write similar code in C++.

```

1 #! /usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 import rospy
5 from geometry_msgs.msg import Twist
6
7 def main():
8
9     try:
10         # Initializes a rospy node.
11         rospy.init_node('torobo_publish_move_base_twist_node')
12
13         controller_list = rospy.get_param('/torobo/controller_list')
14         if 'omni_drive_controller' in controller_list:
15             TOPIC_NAME = '/torobo/omni_drive_controller/cmd_vel'
16         elif 'base_controller' in controller_list:
17             TOPIC_NAME = '/torobo/base_controller/cmd_vel'
18         else:
19             rospy.logerr("move_base controller is not found in controller list!")
20             return
21
22         # Create a publisher.
23         publisher = rospy.Publisher(TOPIC_NAME, Twist, queue_size=1)
24
25         # Wait until the publisher gets ready.
26         while publisher.get_num_connections() == 0:
27             rospy.sleep(1)
28
29         # publish twist
30         rospy.loginfo('publish rotate+ twist ')

```

(Continued on next page)

(Continued from previous page)

```

31     publish_twist(
32         publisher=publisher,
33         v_x = 0.0, v_y = 0.0, omega_z = 0.2,
34         duration = 3.0
35     )
36
37     # publish twist
38     rospy.loginfo('publish rotate- twist ')
39     publish_twist(
40         publisher=publisher,
41         v_x = 0.0, v_y = 0.0, omega_z = -0.2,
42         duration = 3.0
43     )
44
45     except rospy.ROSInterruptException:
46         rospy.logerr("ROSInterruptException occurred")
47
48     finally:
49         pass
50
51     # publish twist
52     rospy.loginfo('publish move_base twist')
53     publish_twist(
54         publisher=publisher,
55         v_x = 0.0, v_y = 0.0, omega_z = 0.0,
56         duration=0.5
57     )
58
59     rospy.loginfo("finished.")
60
61
62 def publish_twist(publisher, v_x, v_y, omega_z, duration):
63     """
64     Function for publishing message to move the move_base
65
66     Parameters
67     —
68     publisher : rospy.Publisher
69         publisher
70     v_x : float
71         x-axis linear velocity (m/s)
72     v_y : float
73         y-axis linear velocity (m/s)
74     omega_z : float
75         z-axis(yaw) angular velocity (rad/s)
76     duration : float
77         publish time in seconds
78
79     Returns
80     —
81     None
82
83     Throws
84     —

```

(Continued on next page)

9. Tutorial

(Continued from previous page)

```

85  None
86  --
87
88  # Creates a message.
89  twist = Twist()
90  twist.linear.x = v_x
91  twist.linear.y = v_y
92  twist.linear.z = 0
93  twist.angular.x = 0
94  twist.angular.y = 0
95  twist.angular.z = omega_z
96
97  # Publish the message.
98  rospy.loginfo('publish twist v_x: %f, v_y: %f, omega_z: %f' % (v_x, v_y, omega_
99  yz))
100  command_rate = 10
101  rate = rospy.Rate(command_rate)
102  loop_count = int(duration * command_rate)
103  for _ in range(loop_count):
104      publisher.publish(twist)
105      rate.sleep()
106 if __name__ == '__main__':
107     main()
108

```

In the above script, the speed command is published at 10Hz .

If the `omni_drive_controller/OmniDriveController` does not receive a command for a certain period of time (0.5 seconds by default), the This is because the specification is to stop the line.

The specific steps to run the cart using the above script are described below. First, the core program of the actual machine, Or start Gazebo .

Next, make sure that the `base_controller` (or `omni_drive_controller`) is in the running state using Torobo State Viewer or similar. Switch it to ON and turn on all axis servos. Then execute the above script.

Please check if there is any interference in the surrounding environment. Open another terminal and run the following command: Sai.

```
$ rosrun torobo_demo publish_move_base_twist.py
```

	Danger The cart will suddenly accelerate depending on the command to run. There must be no objects interfering with the surrounding area. Please check the above and make sure you are ready to press the emergency stop button immediately before issuing the command.
---	--

10 - How to use the controller

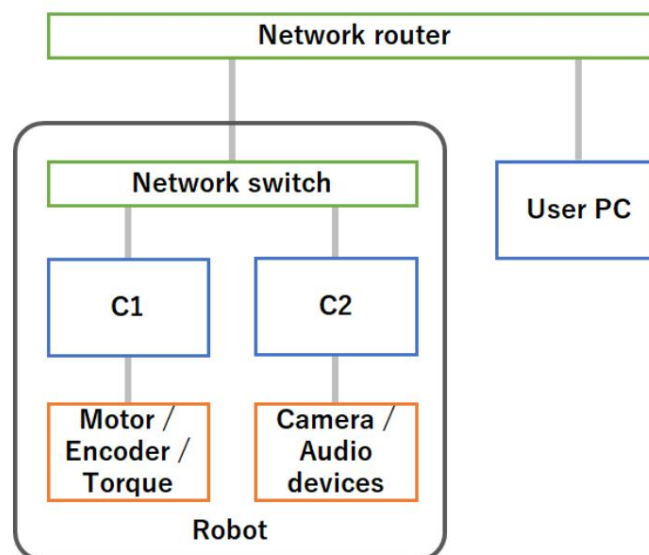
10.1 Overview

10.1.1 PC network connections

This robot has two built-in control PCs, Controller 1 (C1) and Controller 2 (C2). These are connected via Ethernet through a switch hub and can communicate with each other. For details, see [Robot Controller](#).

In addition, by connecting a user's PC to this switch hub, it will be possible to communicate with the user's PC.

The network connections are shown in the diagram below.



10. How to use the **controller****10.1.2 PCs that need to be started**

Depending on the purpose of using the robot, there are "PCs that must be started" and "PCs that do not must be started ." Please refer to the table below.

Purpose for using robot		Launch of PC		
arms, mobile base	cameras, audios	C1	C2	User PC
✓		●	○	○
	✓	○	●	○
✓	✓	●	●	○

● : Required
○ : Optional

For example, if you are only moving a robot arm or a mobile cart (without using a camera), the minimum PC configuration would be a . It is OK to start only Controller 1 (C1) .

On the other hand, if you are only using the robot's camera (without moving the arm or the mobile cart), the minimum PC configuration is a computer. You only need to start up the Troller 2 (C2) .

However, rather than connecting a display, mouse and keyboard directly to Controller 1 (C1) or Controller 2 (C2) , it is recommended that you log in to C1 or C2 via a remote connection from the user's PC , as this will enable you to operate the robot remotely.

This chapter explains the following items:

1. How to remotely connect to each controller from a user PC
2. How to set up a ROS Master on one PC and build a distributed system

10.2 How to remotely connect to each controller from a user PC

There are two ways to connect remotely: "SSH connection" and "RDP (Remote Desktop) connection". In either case, please log in with "User: torobo , Password: torobo" for authentication .

10.2.1 How to connect via SSH

It is possible to log in to C1 and C2 from a user's PC via SSH connection.

```
$ ssh torobo@[C1's IP Address]
```

```
$ ssh torobo@[C2's IP Address]
```

You can also connect by specifying "[target PC 's hostname].local" instead of the IP address.

For more information about how to connect via SSH , see Checking the [SSH connection](#) .

10.2.2 How to connect via RDP (Remote Desktop)

If you are using a Windows 10 PC , connect via RDP using a tool called "Remote Desktop Connection."

If you are using an Ubuntu PC, you can connect via RDP using a tool called Remmina.

Note 	For Ubuntu , you need to install Remmina in the terminal, using the following command: <pre>\$ sudo apt install remmina</pre> After starting Remmina , press the + button to open the Remote Desktop settings screen, select "RDP" as the protocol, select "True color (32bpp)" as the color depth, and enter the address into the server field. Press the Connect button and the login screen will be displayed. Set the Session to Xorg , and enter "torobo" as the username and password to log in.
Note 	After logging in locally to C1/C2 (logging in by directly connecting a display or keyboard) , attempting to log in via RDP connection will result in an error and will fail. The reverse is also true; if you try to log in locally after logging in via an RDB connection, it will fail. This is because the virtual screen numbers for C1/C2 are different for "local login" and "RDP connection login" . If you want to switch between these login methods, make an SSH connection and use the following command to close the previous GUI session before connecting. <pre>\$ sudo pkill gnome-session-b</pre>

10. How to use the **controller**

Note 	If you are using Windows 10 , you need to configure the following mDNS settings to connect to C1/C2 using a host name . Follow the steps below to configure mDNS . 1. From the Control Panel, go to "Windows Firewall Settings" 2. Open the "Advanced Settings" screen . 3. Right-click on "Inbound Rules" on the left tab and select "New Rule". do. 4. The wizard will open, so set the following: • "Port" – "UDP" – "Specific local port=5353" • "Allow connections to" – For all "Domain", "Private" and "Public" Check • "name" – Anything is fine. For example, mDNS 5. After completing the above settings, restart the OS.
--	--

10.3 How to set up a ROS Master on one PC and build a distributed system

If you use two or more PCs among C1, C2, and user PC , you need to build a distributed system using ROS. In that case, set one PC as "ROS Master" and set other PCs to refer to that ROS Master .

For more information on how to build a distributed system in ROS , see [How a distributed system works with ROS](#) .

10.3.1 How to make Controller 1 (C1) ROS Master

If you are using a robot arm or mobile cart, we recommend that you use C1 as the ROS Master .

10.3.1.1 Controller 1 (C1) Configuration

First , start roscore on Controller 1 (C1) . Then, C1 will become the ROS Master .

```
$ roscore
```

After this, we recommend that you start the core program with the following command on C1. For details, see [Overview](#) . Please refer to.

```
$ roslaunch torobo_bringup bringup_real.launch
```

10.3.1.2 Controller 2 (C2) Configuration

* If you do not use Controller 2 (C2) , this procedure is not necessary.

Next, set the environment variables on Controller 2 (C2) . Open "~/.rosrc" on Controller 2 (C2) with an editor,
Set the value of "USE_C1_AS_ROS_MASTER" to ON . Then, use the following command to reflect the changes in ~/.rosrc.

```
$ source ~/.rosrc
```

From now on, all ROS nodes started on C2 will refer to the ROS Master on C1 .

For example, you can start torobo_state_viewer with the following command . (First, start the core program on C1.
You need to move it.)

```
$ roslaunch torobo_dashboard torobo_state_viewer.launch
```

10.3.1.3 User PC Settings

* If you do not use a user PC , this procedure is not necessary.

Next, set the environment variables for the user. Open a terminal and set the environment variables with the following command.

```
$ export ROS_MASTER_URI=http://[C1's IP Address]:11311 $ export  
ROS_IP=[User PC's IP Address]
```

After this, all ROS nodes started on the user PC will refer to the ROS Master on C1 .

For example, you can start torobo_state_viewer with the following command . (First, start the core program on C1.
You need to move it.)

```
$ roslaunch torobo_dashboard torobo_state_viewer.launch
```

10.3.2 How to make Controller 2 (C2) ROS Master

If you are not using the robot arm or mobile cart, but are only using the camera and audio devices, you do not need to start C1.
In this case, it is recommended to set up and operate Controller 2 (C2) as the ROS Master .

10.3.2.1 Controller 2 (C2) Configuration

Set the environment variables on Controller 2 (C2) . Open "~/.rosrc" on Controller 2 (C2) with an editor,
Set the value of "USE_C1_AS_ROS_MASTER" to OFF . After that, use the following command to reflect the changes in ~/.rosrc.

```
$ source ~/.rosrc
```

Next, start roscore on Controller 2 (C2) . Then, C2 will become the ROS Master .

10. How to use the **controller**

```
$ roscore
```

After this, we recommend that you start the camera video distribution node with the following command on the C2. Details Please refer to the [overview](#) .

```
$ roslaunch torobo_cameras cc_image_publisher.launch
```

10.3.2.2 User **PC** Settings

* If you do not use a user PC , this procedure is not necessary.

Next, set the environment variables for the user. Open a terminal and set the environment variables with the following command.

```
$ export ROS_MASTER_URI=http://[C2's IP Address]:11311 $ export ROS_IP=[User  
PC's IP Address]
```

After this, all ROS nodes started on the user PC will refer to the ROS Master on the C2 .

For example, you can start rqt_image_view with the following command . (First start torobo_cameras on C2.
(You must have

```
$ rosrn rqt_image_view rqt_image_view
```

11 - How to use the camera

11.1 Overview

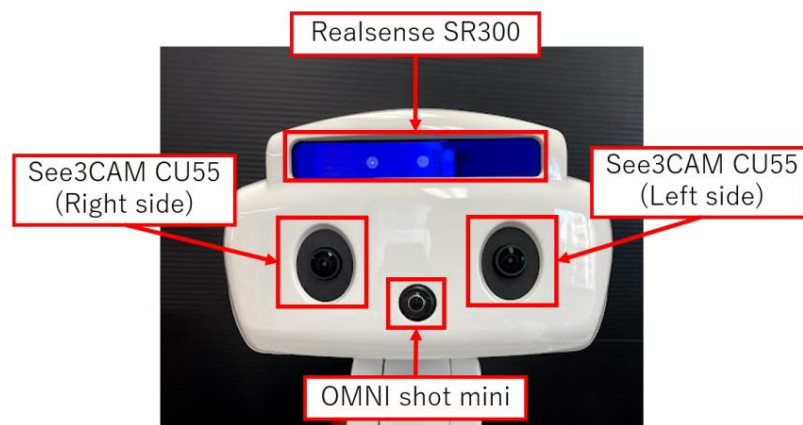
This robot is equipped with one Intel RealSense SR300 head camera and one ELECOM OMNI shot

It is equipped with one "See3CAM_CU55" manufactured by e-con Systems and two " See3CAM_CU55 " manufactured by e -con Systems .

The RealSense SR300 captures RGB and infrared (IR) images, and also generates depth data. Intel has released drivers and packages for using RealSense devices with ROS . The Omni shot mini is a camera that captures RGB images with a 360-degree angle of view .

Two See3CAM_CU55 cameras are installed, allowing you to capture stereo images.

It is also possible to combine them and use them as a single camera .



In addition, the torobo_cameras package provides utility functions for each camera.

This manual explains how to use the cameras using the torobo_cameras package.

11.2 Hardware Overview

11.2.1 RealSense SR300 Specifications

The main specifications of Intel 's RealSense SR300 are listed below.

For the latest specifications, please refer to the Datasheet published by Intel . <https://www.intel.com/content/dam/support/us/en/documents/emerging-technologies/intel-realsense-technology/realsense-sr300-datasheet1-0.pdf>

11. How to use the camera



Product Name	RealSense SR300 (Intel)
Sensors	Color sensor (RGB), Infrared (IR)
Laser	Class 1 Laser compliant
Output Stream	Depth (~640x480(VGA), ~60FPS, 16bit) Color (~1920x1080(FHD), ~30FPS (when ~1280x720(VGA), ~60FPS), 8bit)
Operating range	0.2 ~ 1.5m
Angle of view	[Color] H: 68, V: 41.5, D: 75.2
Interface	USB3.0
size	110mm x 12.5mm x 3.75mm
Power consumption	1.8 W
Calibration Factory calibrated	

11.2.2 OMNI shot mini specifications

The main specifications of the ELECOM OMNI shot mini are listed below.



For the latest specifications, please refer to the ELECOM website : <https://www.elecom.co.jp/products/OCAM-VRU01BK.html>

Product Name	OMNI shot mini OCAM-VRU01BK (manufactured by ELECOM)
Sensors	Color sensor (RGB)
Output Stream	(MJPG, ~3008x1504, ~30FPS, 8bit) (H264, ~2560x1280, ~30FPS, 8bit)
Angle of View	Horizontal angle of view : 210 degrees Vertical angle of view : 210 degrees
Interface	USB2.0
size	40.5mm x 40.0mm x 46.5mm

11.2.3 See3CAM_CU55 Specifications

The main specifications of See3CAM_CU55 from e-con Systems are listed below.

11. How to use the camera



For the latest specifications, please refer to the Datasheet published by e-con Systems : <https://www.e-consystems.com/jp/5mp-low-noise-usb-camera-jp.asp#documents>

Product	See3CAM_CU55 (e-con Systems)
Name	Color sensor (RGB)
Sensor Output Stream	(UYVY, ~2592x1944, ~24FPS, 8bit) (MJPG, ~2592x1944, 60FPS, 8bit)
Angle of View	FOV: 140.5 degrees
Interface	USB3.1 Gen1
Size Power	30mm x 30mm x 30.2mm
Consumption	Max. 2.185 W

11.3 Software Overview

11.3.1 Overall system configuration

ROS has a mechanism called `image_pipeline` for handling camera images .
Users can obtain images in the format they want without being aware of differences between camera vendors.
You need to be thoroughly familiar with the mechanism that goes from the driver to the image pipeline .

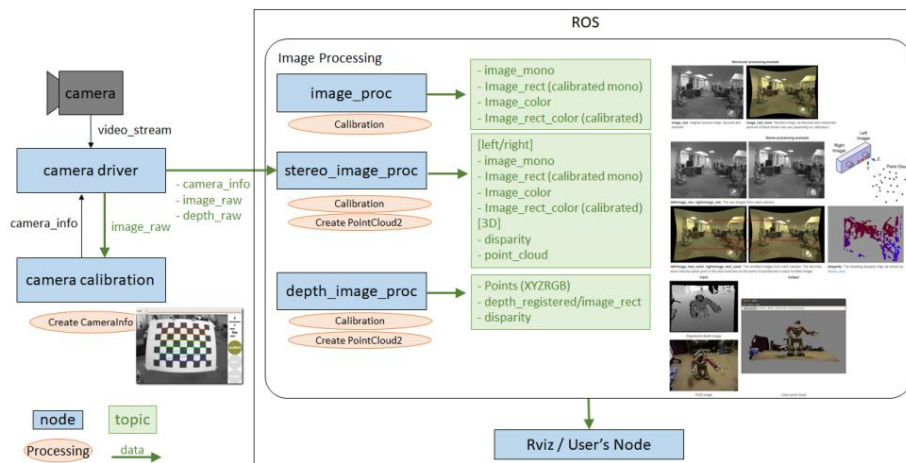
On the other hand, RealSense uses `realsense_ros` to provide users with the same interface as `image_pipeline` .
To use the `realsense_ros` package, you need to install the `librealsense2` driver .

For this robot, the `librealsense2` driver is installed on controller 2 (C2) .
The `image_pipeline` package also supports ROS2 .

11.3.2 image_pipeline

This is an image processing architecture for handling camera images in ROS .
It's Kucha (see below).

image pipeline (http://wiki.ros.org/image_pipeline)



The camera driver publishes the calibrated camera_info , color image, and depth image to the ROS topic . Then, image_proc , stereo_image_proc, and depth_image_proc , which are standard packages in ROS as part of the image_pipeline, perform image processing such as calibration and 3D reconstruction, and publish the resulting images and 3D point clouds .

Users can subscribe to these topics and obtain the image data they want.

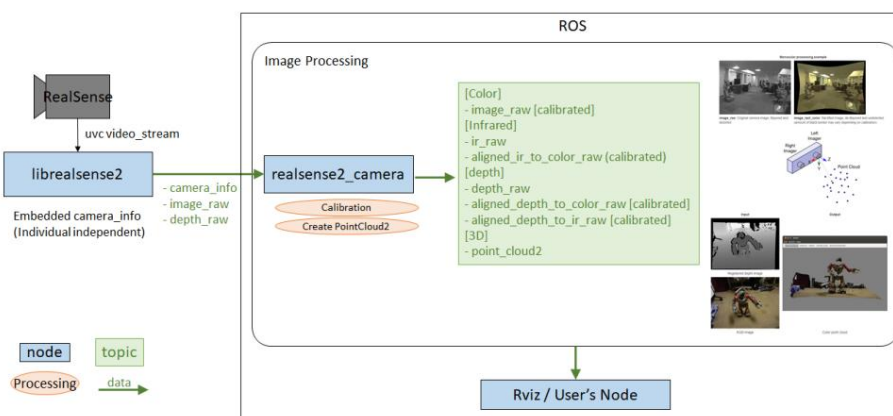
As with OMNI shot mini and See3CAM , images are acquired on ROS using the image_pipeline in the same way as above .

11.3.3 realsense image_pipeline

In the image_pipeline provided as standard in ROS , the user must prepare the camera_info and code to publish image data to ROS .

On the other hand, the realsense_ros package does not require users to do this.
It processes various images to be published and publishes the same topics as image_pipeline .

realsense image pipeline



11. How to use the camera

The RealSense SR300 is designed to output color images that have been calibrated on the device side. Therefore, there is no need for users to perform calibration for Color images.

In addition, the `realsense_ros` package uses the depth image to publish an `aligned_depth_to_color` image transformed into the coordinate system of the color sensor. The `pointcloud2` point cloud is generated from the color image and the `aligned_depth_to_color` image. In other words, everything is handled in the coordinate system of the color sensor.

Note 	In the <code>realsense_ros</code> package, PointCloud2 generation is realized using the <code>rgbd_launch</code> package. The <code>rgbd_launch</code> package calls the <code>image_depth_proc</code> package of the ROS <code>image_pipeline</code> to generate PointCloud2. (At that time, the color image and the <code>aligned_depth_to_color</code> image are used.) Therefore, the <code>realsense_ros</code> package depends on the <code>rgbd_launch</code> package.
--	---

11.4 Operation Method

This section explains how to operate the device.

First, we will explain how to set up the controllers C1/C2 and the user PC to prepare the execution environment. User PC We will use it by connecting to controllers C1/C2 via SSH.

Next, the operation method will be explained separately for each camera. Since all cameras are connected to Controller 2 (C2), the commands to start the camera in this section should be executed on Controller 2 (C2).

On the other hand, `rviz` is used to check camera images, and `dynamic_reconfigure` is used to change camera shooting settings. It is recommended that you execute these `rviz` and `dynamic_reconfigure` commands on a user PC that has already been set up, or on the C2 connected via RDP from the user PC.

11.4.1 Preparing the execution environment (settings for controller C1/C2 and user PC)

First, to prepare the execution environment, we will configure the controllers C1/C2 and the user PC.

However, the procedure is different depending on whether you are moving the robot arm, the mobile cart, or the camera, or if you are moving only the camera. Each case is explained below.

11.4.1.1 When moving a robot arm, a mobile cart, or a camera

First, turn on the robot power, then turn on controller 1 (C1) and controller 2 (C2).

Connect to Controller 1 (C1) from the user PC via SSH.

```
$ ssh torobo@[Control1 PC's IP Address]
```

After connecting to C1 via SSH, start the core program on C1.

```
$ roslaunch torobo_bringup bringup_real.launch
```

Next, connect to Controller 2 (C2) from the user PC via SSH .

```
$ ssh torobo@[Control2 PC's IP Address]
```

After connecting to C2 via SSH , the C2 is configured to refer to C1 as ROS_MASTER .

Open "~/.rosrc" in the editor. Turn on " USE_C1_AS_ROS_MASTER" in "~/.rosrc".

```
# ~/.rosrc  
USE_C1_AS_ROS_MASTER="ON"
```

Then, reflect the changes to "~/.rosrc" on the C2 .

```
$ source ~/.rosrc
```

Finally, set the user PC to refer to C1 as ROS_MASTER. Open a terminal on the user PC .

Then, set the environment variables with the following command.

```
$ export ROS_MASTER_URI=http://[C1's IP Address]:11311 $ export ROS_IP=[User PC's  
IP Address]
```

11.4.1.2 When moving only the camera

If you are only moving the camera, there is no need to use C1 .

First, turn on the power to the robot . Then turn on the power to controller 2 (C2) .

Connect to Controller 2 (C2) from the user PC via SSH .

```
$ ssh torobo@[Control2 PC's IP Address]
```

Next, set Controller 2 (C2) to ROS_MASTER . On the C2 side, open "~/.rosrc" in any editor .

Open "~/.rosrc" and turn off " USE_C1_AS_ROS_MASTER".

```
# ~/.rosrc  
USE_C1_AS_ROS_MASTER="OFF"
```

Then, reflect the changes to "~/.rosrc" on the C2 .

```
$ source ~/.rosrc
```

Finally, configure the user PC to refer to the C2 as ROS_MASTER. Open a terminal on the user PC .

Then, set the environment variables with the following command.

```
$ export ROS_MASTER_URI=http://[C2's IP Address]:11311 $ export ROS_IP=[User PC's  
IP Address]
```

11. How to use the camera

11.4.2 Camera operation method

11.4.2.1 How to operate RealSense

11.4.2.1.1 Starting RealSense

RealSense executes processes that are started at run time and processes that read the depth sensor shooting settings and color image sensor shooting settings.

These settings can be set by editing `config/"model"/rs_cap_prop.yaml` in the `torobo_camera_description` package .
It is possible to change it by

The value of the model key in `config/products/default.yaml` of the `torobo_description` package corresponds to "model" .

The properties read in this file are originally provided in the `realsense_ros` package, so please refer to the `realsense_ros` package for details. In addition, the properties in the `torobo_cameras` package that have different settings from the `realsense_ros` package are listed below.

Setting name	color / depth	
visual_prest	depth	Description Specifies a preset with settings for a specific scene . The preset numbers 1 to 10 are preset numbers, and when these numbers are set, they will not be used in conjunction with other presets. The settings for the Depth sensor are ignored. If 11 is set, the default preset number is specified and the settings for the other Depth sensors are reflected.

Connect to C2 via SSH .

```
$ ssh torobo@[Control2 PC's IP Address]
```

To obtain various images from RealSense , enter and execute the following command in the C2 terminal.

```
$ roslaunch torobo_cameras rs_image_publisher.launch
```

This command issues a tf message that indicates the position of the RealSense camera_link , and also the `realsense_ros` package .
Run `rs_rgbd.launch` in the cage to issue various images and PointCloud2 messages.

In addition, the YAML file (`config/"model"/rs_config.yaml` in the `torobo_camera_description` package) that is loaded when the above command is executed contains keys that specify the resolution and FPS of the color and depth images . If you want to run with different parameters at startup, edit the above file.

11.4.2.1.2 Checking the tf of RealSense camera_link

Once you have started RealSense , start Rviz to check that the camera_link position is correct .

How you start Rviz depends on whether you have already started the core program in C1 .

- If you have already started the core program on C1 :

Please enter the following command on the user PC that has been set up .

```
$ roslaunch torobo_bringup rviz.launch
```

- If you have not started the core program on C1 :

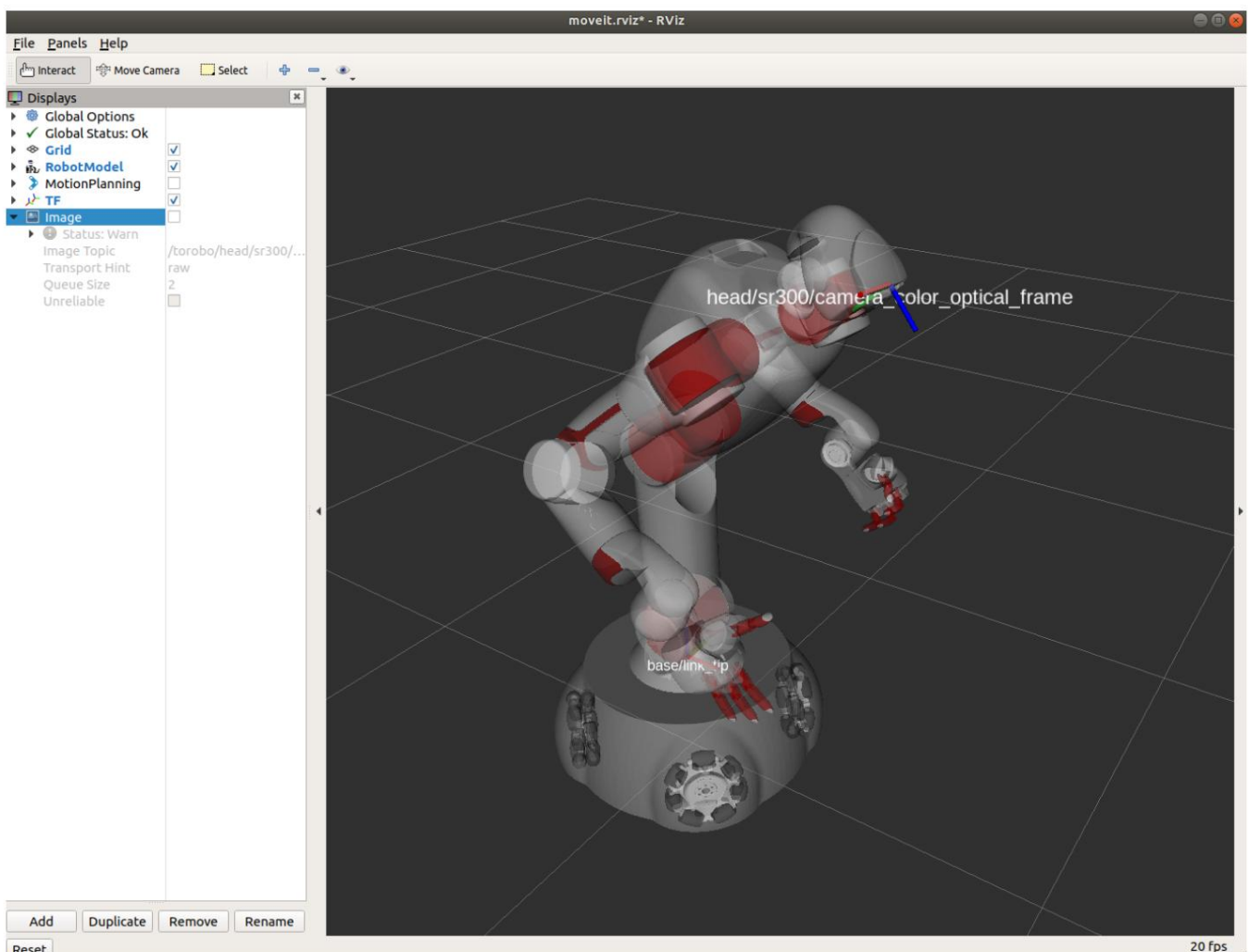
Please enter the following command on the user PC that has been set up .

```
$ roslaunch torobo_description torobo_display.launch
```

This will start Rviz.

Next, click the Add button in Rviz and select tf . Then, tf will be displayed in the 3D view . As shown below.

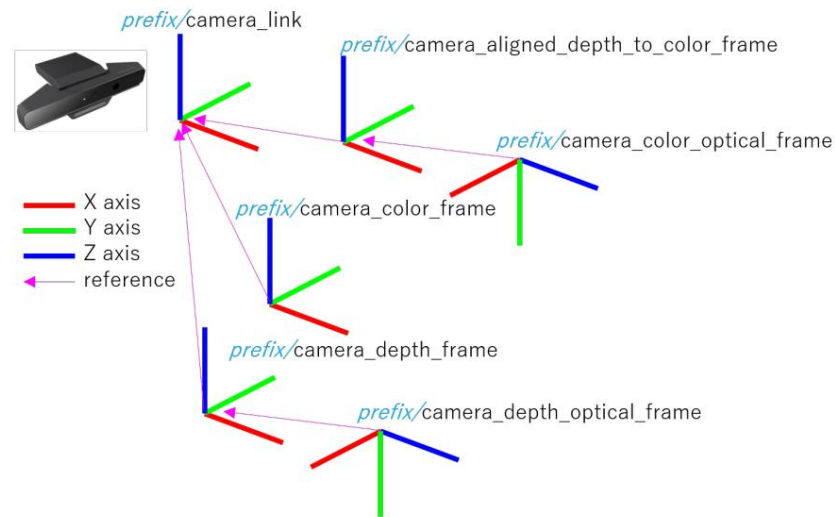
It's OK if you can confirm that the camera_link coordinates are located at the location of the robot's head camera .



11. How to use the camera

The `realsense_ros` package publishes tf so that the camera can be configured with multiple links .

The link structure of the MERA is as shown in the figure below.



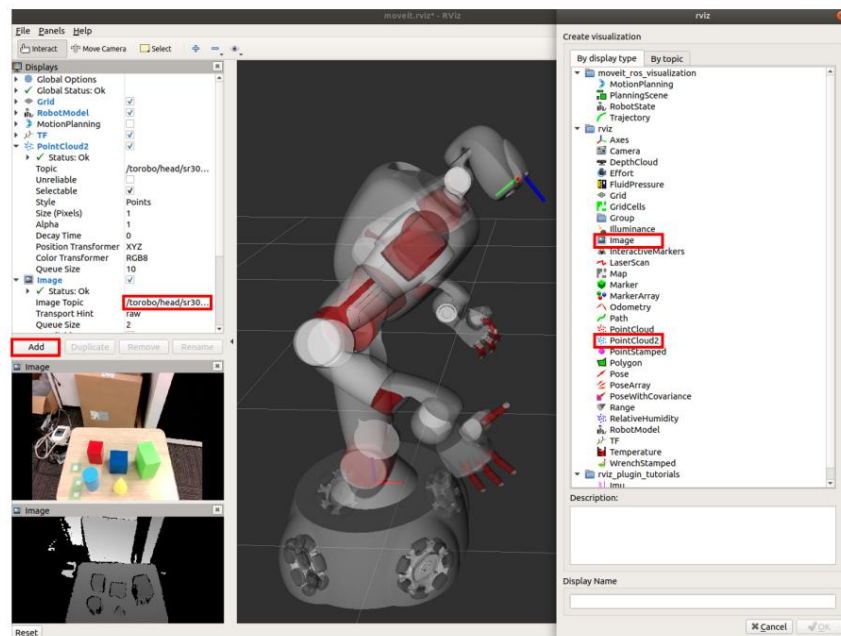
In the above image , "prefix/" will contain the string "head/sr300" to represent the head camera of this robot .

The `camera_link` is at the base position, to which `camera_depth_frame`, `camera_color_frame`, and `camera_aligned_depth_to_color_frame` are attached. In addition, an `optical_frame` (lens) is attached to each frame except `camera_color_frame`. For ease of understanding, they are all drawn at different positions in the above image, but tf is published at the same position for `camera_link` and `camera_depth_frame` . In addition, tf is published at the same position for `camera_color_frame` and `camera_aligned_depth_to_color_frame` . The z- axis of each `optical_frame` faces forward from the camera. This is to use the same coordinate system as OpenCV .

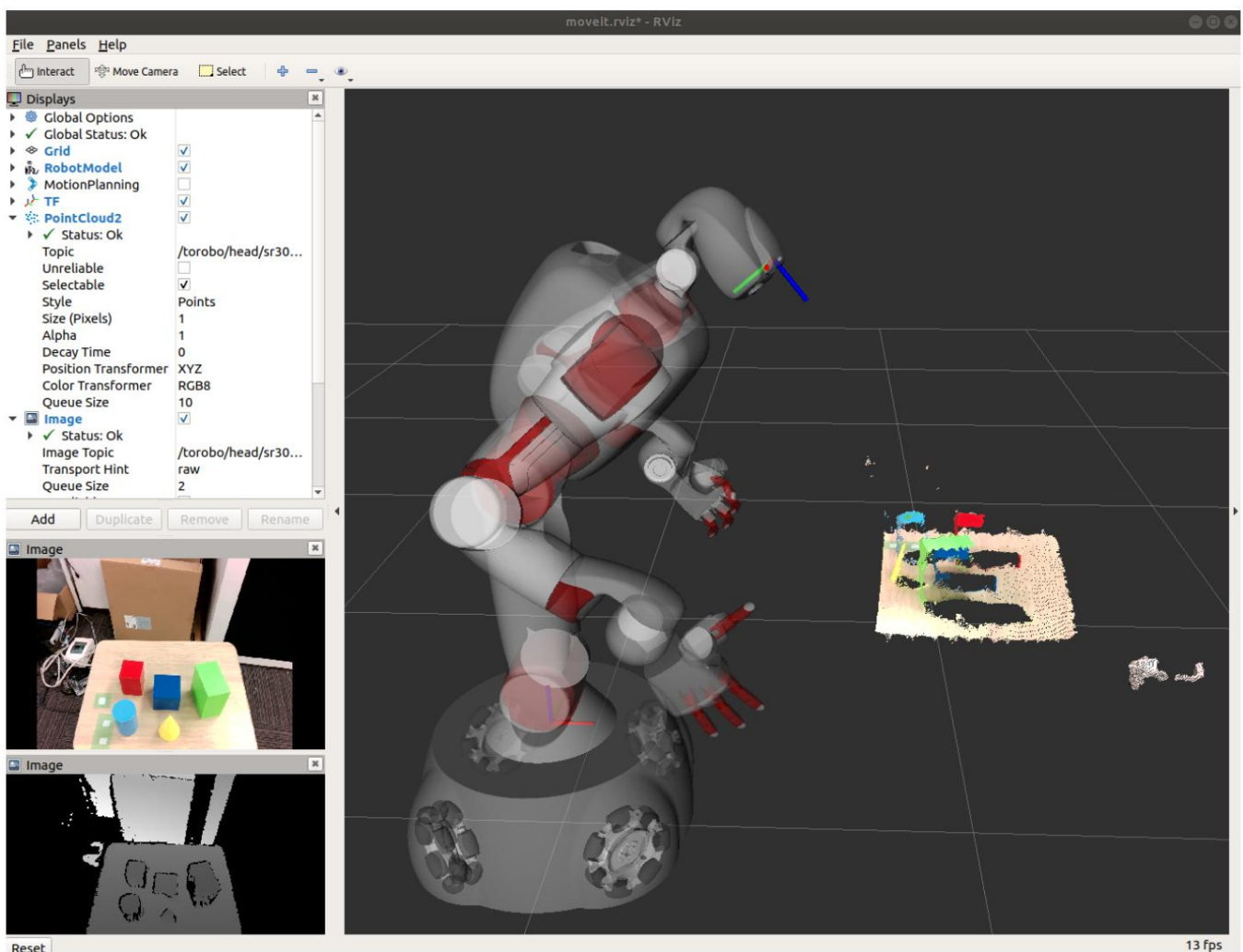
11.4.2.1.3 Checking RealSense Image and PointCloud2

Let's check the Image message and PointCloud2 message output by RealSense .

Click the Add button in Rviz and select Image . The Image window will then appear. Image Topic
Select the topic you want to display and the image will appear in the Image window.



In the Rviz GUI , open the Topic addition window by clicking "Add" at the bottom left , and select "/torobo/head/sr300/camera/depth_registered/points" from the "By Topic" tab to display the PointCloud in the 3D view .



11. **How** to use the camera

Topic ÿ /	explanation	Distortion corrected
torobo/head/sr300/ camera/color/image_raw	Color sensor image (def: 640x480 BGR24)	Yes
/torobo/head/sr300/ camera/color/ image_rect_color	Color sensor image (def: 640x480 BGR24) Same result as image_raw	Yes
/torobo/head/sr300/ camera/depth/ image_rect_raw	Depth image (def: 640x480 Z16)	No
/torobo/head/sr300/ camera/ aligned_depth_to_color/ image_raw	Depth image aligned to the color sensor coordinate system	Yes
/torobo/head/sr300/ camera/ depth_registered/points	PointCloud2 (RGBXYZ)	Yes
/tf_static	Transform from camera_link to Color sensor	

11.4.2.2 How to operate OMNI shot mini

11.4.2.2.1 Starting up **OMNI shot mini**

When starting up OMNI shot mini , the shooting settings of the Color camera are read. These settings are
 You can change it from config/"model"/cc_cap_prop.yaml in the torobo_camera_description package.
 Below is a list of the configuration values that can be set from the configuration file.

Setting name	explanation
format	Specify the image codec. "H264" is not supported. The setting value is outside the range.
width	Specifies the width of the image.
height	Specifies the height of the image.
fps	Specify fps (frames per second) .
brightness	Specifies the brightness.
contrast	Specifies the contrast value.
gamma	Specifies the gamma value.
saturation	Specifies the saturation.
gain	Specifies the gain.
hue	Specifies the hue.
auto_white_balance	Sets the automatic white balance.

Connect to C2 via SSH .

```
$ ssh torobo@[Control2 PC's IP Address]
```

To obtain various images from the OMNI shot mini , enter and execute the following command in the terminal.

```
$ roslaunch torobo_cameras cc_image_publisher.launch
```

This command publishes an Image message for the Color of the OMNI shot mini .

The see3CAM Color Image message will also be published .

Topic Name	explanation	Distortion corrected
/torobo/head/insta360/ camera/ color/ image_raw	Color sensor image (def: 2176x1088 BGR24)	No

When correcting distortion with OMNI shot mini , please prepare the camera parameters yourself.

11.4.2.3 How to operate see3CAM_CU55

11.4.2.3.1 Start see3CAM_CU55

Like the OMNI shot mini , see3CAM also reads the shooting settings of the Color camera when it starts up.

The set value can be changed in config/"model"/cc_cap_prop.yaml in the torobo_camera_description package .

The following is a list of the settings that can be set from this configuration file. There are settings for both the left and right cameras.

This allows you to set different properties for the left and right cameras.

Setting name	explanation
format	Specifies the codec for the image.
width	Specifies the width of the image.
height	Specifies the height of the image.
fps	Specify fps (frames per second) .
brightness	Specifies the brightness.
contrast	Specifies the contrast value.
gamma	Specifies the gamma value.
saturation	Specifies the saturation.
gain	Specifies the gain.
auto_white_balance	Sets the automatic white balance.
auto_exposure	Set the auto exposure.

Connect to C2 via SSH .

```
$ ssh torobo@[Control2 PC's IP Address]
```

To obtain various images from the left and right see3CAMs , enter and execute the following commands in the terminal.

This command is the same as the startup command for OMNI shot mini .

11. How to use the camera

```
$ roslaunch torobo_cameras cc_image_publisher.launch
```



Topic	explanation	Distortion corrected
name /torobo/head/ see3cam_right/camera/ color/image_raw	Color sensor image (def: 1280x960 BGR24)	No
/torobo/head see3cam_left/camera/ color/image_raw	Color sensor image (def: 1280x960 BGR24)	No

In addition, see3CAM can be used as a stereo camera configuration. Two images, one on the left and one on the right, are combined into one image.

To obtain it, run the following command after downloading the cc_image_publisher.launch mentioned above.

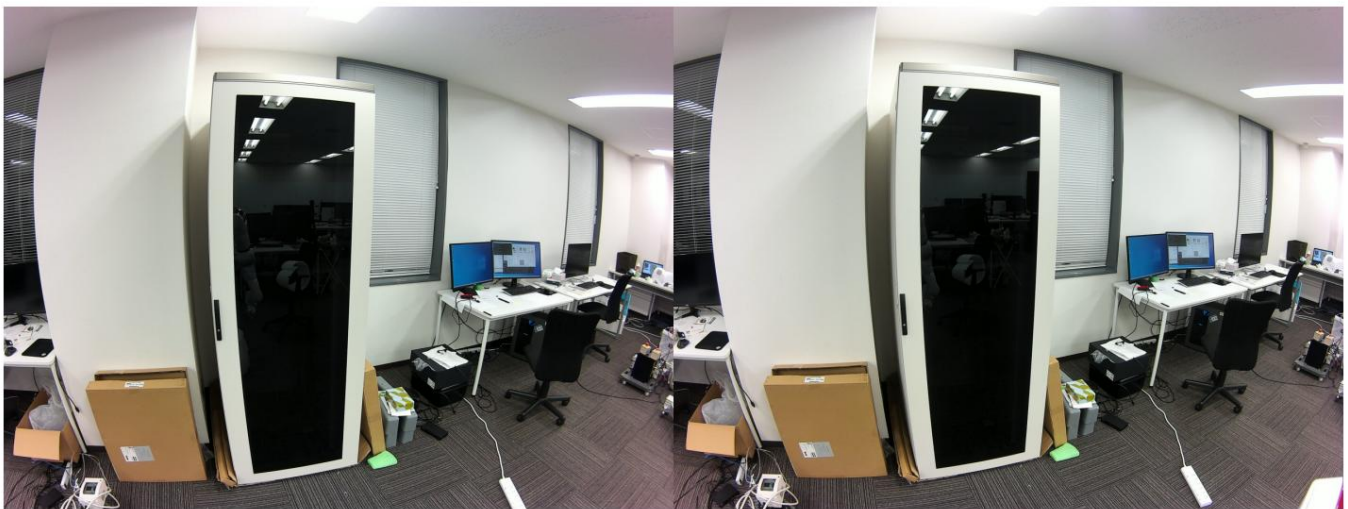
First, connect to the C2 via SSH .

```
$ ssh torobo@[Control2 PC's IP Address]
```

When capturing two images, one for each side, as in the case of an image captured with a stereo camera , the following

Execute the command.

```
$ roslaunch torobo_cameras stereo_image_publisher.launch
```



Topic Name	explanation	Distortion corrected
/torobo/head/stereo/ camera/color/image_raw	Color sensor image (def: 2560x1920 BGR24)	No

When performing distortion correction in see3CAM , please prepare the camera parameters yourself.

11. How to use the camera

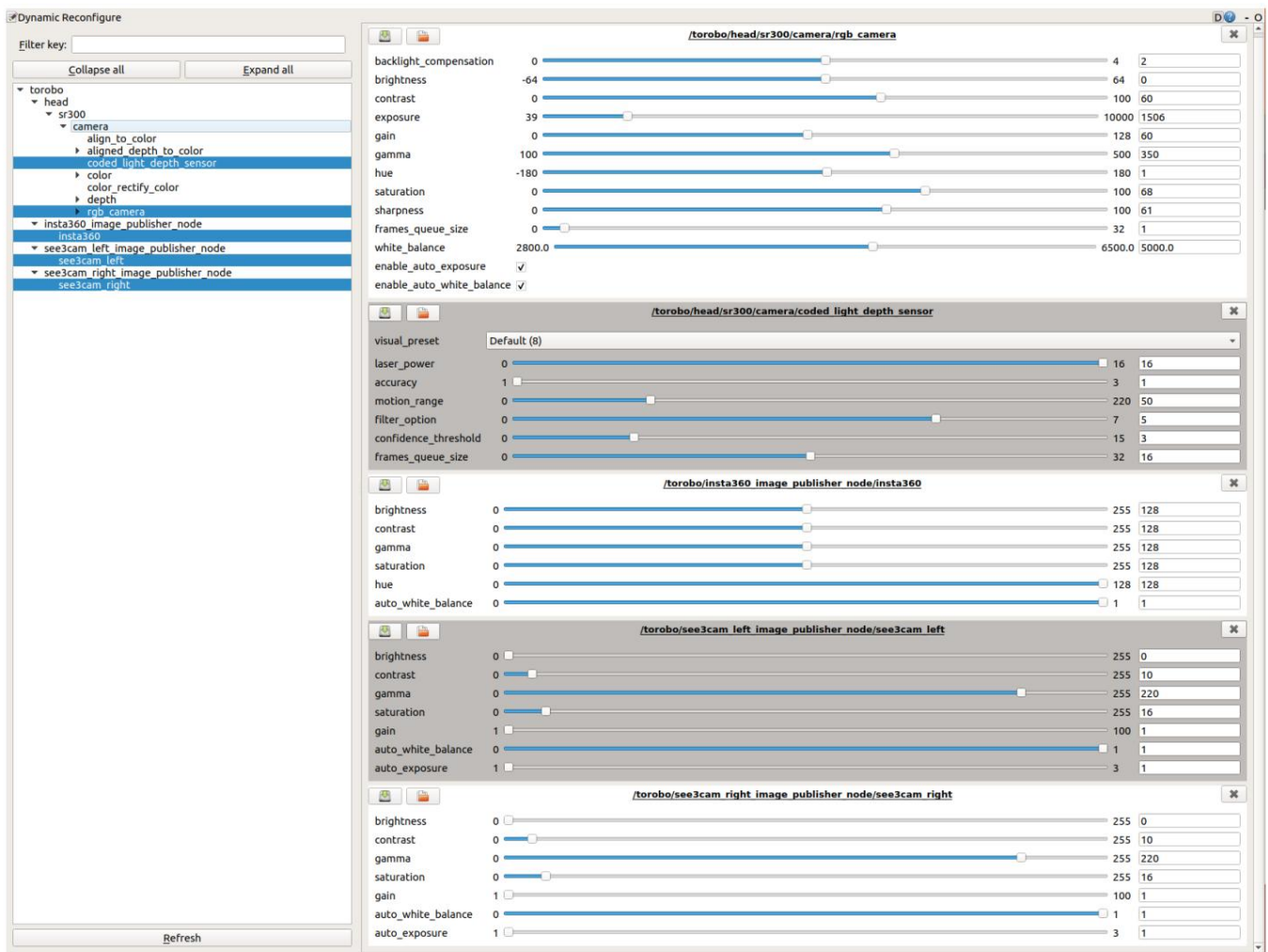
11.4.2.4 Adjusting camera parameters for each camera

The `realsense_ros` package supports Dynamic Reconfigure Params of ROS. In addition, the camera parameters set at the time of starting OMNI shot mini and see3CAM can also be changed from Dynamic Reconfigure . Dynamic Reconfigure is a function that dynamically switches parameters while ROS is running.

On a user PC that has already been set up , enter the following command to open the dynamic reconfigure screen.

```
$ rosrn rqt_reconfigure rqt_reconfigure
```

Verify that the following GUI appears:



11.4.2.5 Using the Camera in the Simulator

A camera plugin has been set in the URDF so that the camera can be used in Gazebo .

It is placed in the URDF in a position almost identical to the RGB sensor position of the camera (aiSense, OMNI shot mini, see3CAM) .

When you start this robot (simulation) with `roslaunch` , images are always published from the camera , so if you want to check the images, add the display of the Image message using the Add button in Rviz .

12 - How to use microphones and speakers

This robot is equipped with the following microphones and speakers.

- One microphone on each side of the head (total of two)
- One speaker on the underside of the head

In this chapter, we will first explain how audio devices work in Linux . Then we will actually use these microphones and Learn how to use the speaker.

12.1 How audio devices work in Linux

Linux includes a driver called "ALSA" for operating audio devices, and a driver for using multiple audio devices in combination. There is a service called "PulseAudio" that allows you to use it. We will explain each of them below.

12.1.1 ALSA (Audio Driver)

Linux provides a driver for handling audio devices called ALSA . Users can use the ALSA API You can use it to control audio devices programmatically.

However, in Linux, it is not recommended that users use the ALSA API directly, and it is recommended that they operate audio devices through a service called PulseAudio , which will be explained in the next section. Also, ALSA is not suitable for integrating and handling multiple audio devices.

12.1.2 PulseAudio (Audio Service)

Linux provides a service called PulseAudio for handling audio devices. (This service calls ALSA internally to operate the audio device.) PulseAudio has many functions, such as an echo cancellation filter.

12.2 How to use audio devices

12.2.1 Running the PulseAudio enablement script

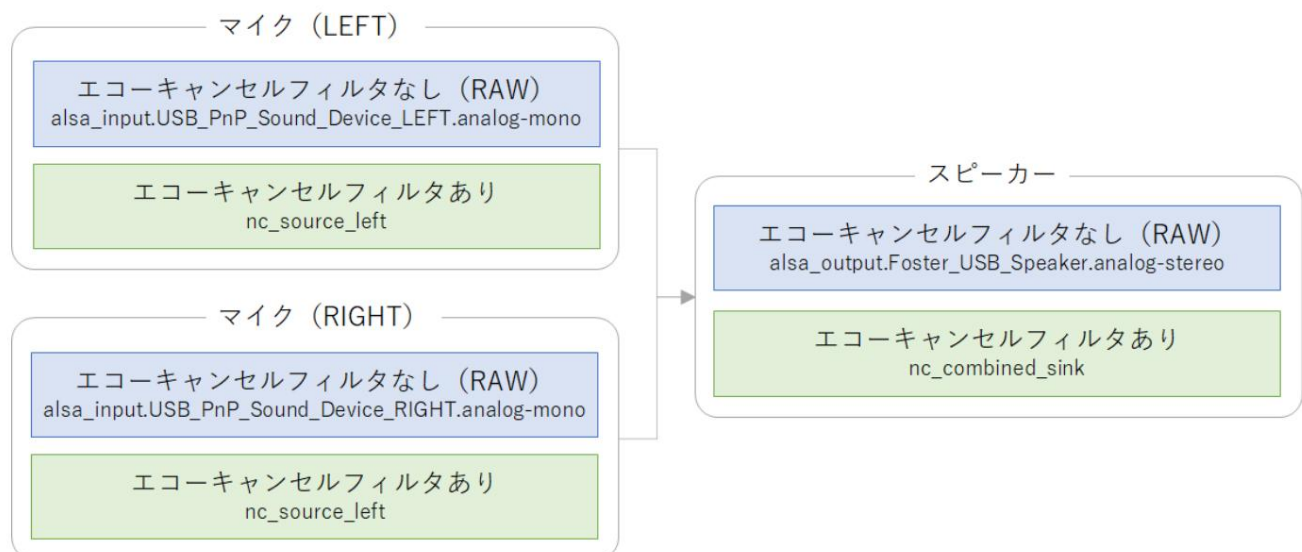
If you log in to Controller 2 (C2) via a remote connection such as SSH or RDP , the PulseAudio service will be disabled. (Note that if you log in locally to Controller 2 (C2) by connecting a display and keyboard , the PulseAudio service will be enabled.) Therefore, if you log in via one of these remote connections, you will need to enable PulseAudio .

Additionally, to use PulseAudio 's echo cancellation filter and adjust the volume of your audio device, You need to run commands for PulseAudio to configure it.

Therefore, we have prepared a script called "setup_pulseaudio.sh" on Controller 2 (C2) to perform the above steps all at once . Execute the following command from the terminal.

```
$ roslaunch torobo_audios setup_pulseaudio_node.launch
```

This will build the audio module shown below.



To see a list of configured audio devices, run the following command:

```
To check your audio input device: $
pacmd list-sources | grep -e name: -e sysfs

To check the audio output device: $
pacmd list-sinks | grep -e name: -e sysfs
```

You can change the default audio device used by user applications and change the volume of that audio device. If you want to change it, change the following line in setup_pulseaudio.sh.

```
pacmd set-default-source nc_combined_sink.monitor Specify the output device #Default audio input
pacmd set-default-
sink alsa_output.Foster_USB_Speaker.analog-stereo # Specify the default audio output device
```

(Continued on next page)

(Continued from previous page)

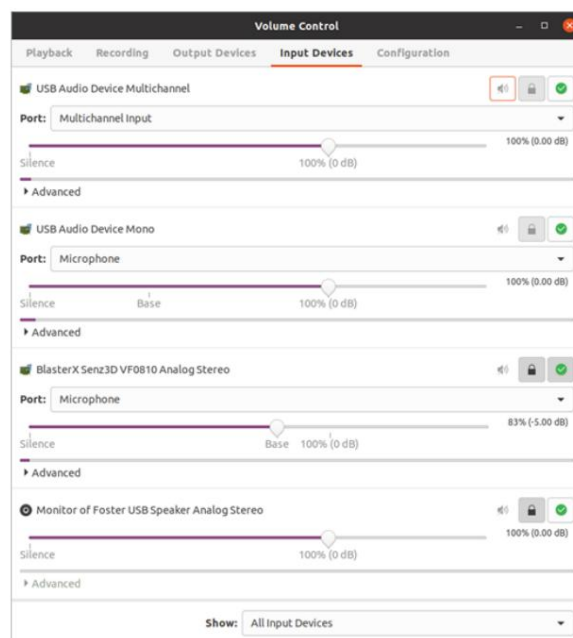
```
pacmd set-source-volume nc_source_left 0x10000 #Set the volume of the audio input device "nc_source_left "
pacmd set-source-volume nc_source_right 0x10000 #Set the volume of the audio input device "nc_source_right"
```

```
pacmd set-source-volume @DEFAULT_SOURCE@ 0x10000 #Set the volume of the default audio input device
pacmd set-sink-volume @DEFAULT_SINK@ 0x10000 #Set the volume of the default audio output device
```

You can also check the list of audio devices and adjust the volume with the GUI tool "pavucontrol".

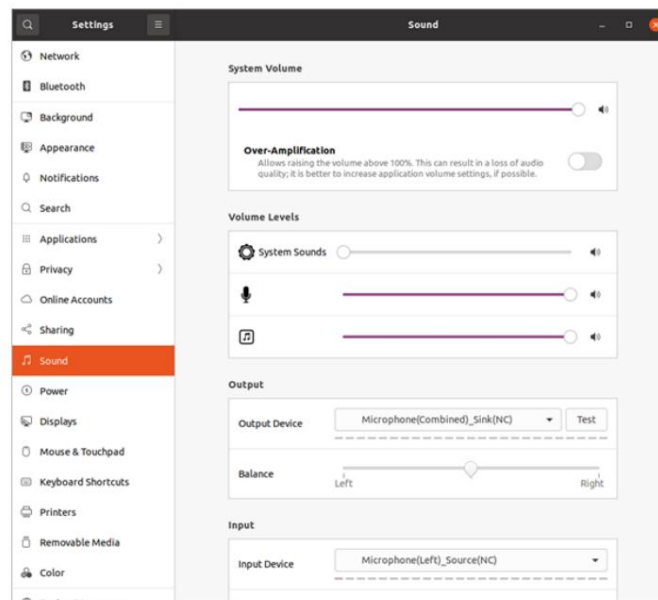
You can start the GUI tool with the command below .

```
$ pavucontrol
```



You can also adjust it in the Sound screen in the Ubuntu settings screen below . (However, pavucontrol provides more detailed adjustments.)

Adjustment is possible.)

12. How to use microphones **and** speakers

12.2.2 Explanation of sample program to record with microphone and play with speaker

By using an application called SoX (Sound eXchange) , you can record and play audio via PulseAudio. Yes, it is possible. To use SoX (Sound eXchange) , please install SoX with the following command.

```
$ sudo apt install sox
```

To record audio from the microphone, run the following command:

```
$ roslaunch torobo_audios record_sound_example_node.launch filename:=test.wav
```

To end the recording, press [Ctrl-C] . A sound file will be generated, `~/ros/test.wav`.

Next, to play the recorded audio file through the speaker, run the following command:

```
$ roslaunch torobo_audios replay_sound_example_node.launch filename:=test.wav
```

12.2.3 Explanation of sample program to create audio data and play it on speaker

The sound is generated by scipy , and the sound is played from the speaker by pygame. To use scipy and pygame , install them with the following commands .

```
$ pip3 install scipy pygame
```

Then run the following command:

```
$ roslaunch torobo_audios synthesize_sound_example_node.launch
```

13 - Software Specifications

13.1 Custom ROS message specifications

ROS provides standard ROS messages such as `std_msgs` and `sensor_msgs`, which can be used to build topics and services. There is also a mechanism for users to define their own ROS messages, and the ROS packages we provide include ROS messages that we have defined ourselves.

Here, we will explain how it is important for users to know the specifications of the ROS messages that we have defined ourselves.
Here we will explain some things that we think are useful when developing robot movements.

13.1.1 Message Specifications for ROS Topics

13.1.1.1 torobo_sensor_msgs/ToroboState

These messages are used to indicate the status of this robot.

Field Name		explanation
header	Type Header	The standard ROS message type, <code>std_msgs/Header</code> . string The robot
robot_name	name. Default is "torobo".	int32
safety_command		The safety status of the robot output by <code>safety_controller</code>. 0: No abnormality. Servo can be turned on. Control is possible by command. 1: Emergency stop. Servo ON is not possible. Servo is automatically turned OFF. can be. 2: Protection stop state. Servo ON is possible. Position control is performed to maintain the current joint position regardless of the control mode. No operation is performed even if a command is sent. 255: Status unknown
joint_names	string[]	The robot joint names. The array values of the other fields are They are arranged in the order of the joint names in <code>joint_names</code>.

Continued on next page

13. Software Specifications

Table 13.1 – continued from previous page

Field Name	Type	explanation
servo_state	int32[]	The servo status. 0: Servo OFF 1: Servo ON 2: Error 255: Status unknown
joint_control_mode	uint8[]	Joint control mode. 0: Motor torque control 1: Speed control 2: Position control 3: Link torque control 255: Control mode not set. *Please note that this is different from the upper control mode. When the control mode is joint impedance control mode, The control mode will be link torque control mode 3 .
supply_voltage	float64[]	The voltage value [V] supplied to the motor driver .
motor_driver_temperature	float64[]	Motor driver temperature [°C] . Currently not implemented, so always will output 0.0 .
current_phase_u	float64[]	Motor U -phase current value [A] .
current_phase_v	float64[]	Motor V -phase current value [A] .
current_phase_w	float64[]	Motor W -phase current value [A] .
current_axis_d	float64[]	Motor d -axis current value [A] .
current_axis_q	float64[]	Motor q -axis current value [A] .
motor_torque	float64[]	Motor output torque [Nm] . Calculated from the motor's q -axis current value We are doing it.
motor_encoder_position	int32[]	Encoder position on the motor side (input shaft side of the reducer).
link_encoder_position	int32[]	Encoder position on the link side (output shaft side of the reducer).
motor_position	float64[]	Motor side position. Currently not implemented, so 0.0 is always output. can be done.
link_position	float64[]	Link side position [rad] . Calculated from the link side encoder. can be done.
motor_velocity	float64[]	Motor speed [rad/s] .
link_velocity	float64[]	Link speed [rad/s] .
motor_acceleration	float64[]	Motor acceleration [rad/s^2] .
link_acceleration	float64[]	Acceleration on the link side [rad/s^2] .
link_effort	float64[]	Torque sensor measurement value [Nm] .
link_effort_dot	float64[]	The time derivative of the torque sensor measurement value [Nm/s] .

Continued on next page

Table 13.1 – continued from previous page

Field Name	Type	explanation
gravity_effort	float64[]	Gravitational torque applied to the joint [Nm] . Robot dynamics model It is calculated from.
dynamics_effort	float64[]	Dynamic torque applied to the joint [Nm] . It is calculated from a scientific model.
external_effort	float64[]	External torque applied to the joint [Nm] . Torque sensor value and dynamic This corresponds to the difference in mixed torque.
motor_inertia	float64[]	Motor moment of inertia [kg m^2] .
link_inertia	float64[]	Moment of inertia on the link side around the joint axis [kg m^2] .
reduction_ratio		The reduction ratio of the float64 reducer.
target_current_axis_d	float64[]	Motor d -axis current command value [A] .
target_current_axis_q	float64[]	Motor q -axis current command value [A] .
target_voltage_axis_d	float64[]	Motor d -axis voltage command value [V] .
target_voltage_axis_q	float64[]	The motor q -axis voltage command value [A] .
target_motor_torque	float64[]	Motor command torque [Nm] . From the motor command q -axis current value We are converting.
target_link_position	float64[]	Link side command position [rad] .
target_link_velocity	float64[]	Link side command speed [rad/s] .
target_link_acceleration	float64[]	Command acceleration on the link side [rad/s^2] .
target_link_effort	float64[]	Link side command torque [Nm] .

13.1.1.2 torobo_control_msgs/CollisionInfoRequest

This message is used in [torobo_control_msgs/GetCollisionInfo.srv](#) .

Field name	Type	explanation
jointState	its- sor_msgs /JointState []	This is the state of the robot that is the target of self-collision detection calculations. The robot model contains a joint name that is not included in 'jointState'. When you do this, the current position of the joint is used as is. can be done.
max_contacts	uint32	Calculates the maximum number of pairs of link pair information where self-interference is detected Specify whether to allow self-interference between many links. If this number is larger, the calculation time will be longer.

13. Software Specifications

13.1.1.3 torobo_control_msgs/CollisionInfoResponse

This message is used in [torobo_control_msgs/GetCollisionInfo.srv](#) .

Field Name		explanation
isColliding	Type bool	true if self-interference was detected , otherwise false is stored.
contancs	moveit_msgs/With-tactIn-doctor-tion []	This is the pair information of the link in which self-interference was detected.

13.1.1.4 torobo_control_msgs/LoadInertia

This message is used to indicate the physical characteristics of the load set on a robot's fingers, etc.

Field Name		explanation
header	Type Header	The standard ROS message type, std_msgs/Header .
model_name	string	The model name of the robot. Default is "torobo". Masu.
load_link_names	string[]	A list of link names for robots that can set load. The values in the field array are sorted in the order of load_link_names . I am here.
inertia	geometry_msgs/Inertias []	The physical properties of the load (mass, center of gravity, inertia matrix) vinegar.

13.1.1.5 gpio_msgs/GpioStates

This message is used to indicate the status of the general-purpose input/output terminals.

Field Name		explanation
header	Type Header	The standard ROS message type, std_msgs/Header .
digital_io_port_names	string[]	The name of the digital input/output port.
analog_io_port_names	string[]	The name of the analog input/output port.
digital_io_data	uint8[]	The value of the digital I/O port. Each element is either "0" or "1". It has two values:
analog_io_data	float64[]	Analog I/O port value. Each element is in [V] .

13.1.1.6 cartesian_trajectory_msgs/CartesianTrajectoryPoint

This message is used to indicate the status of the Cartesian coordinate system, such as its position and orientation.

13.2 cartesian_trajectory_msgs/CartesianTrajectoryPoint

Field Name	Type	explanation
time_from_start	std_msgs/duration	ROS standard message type std_msgs/Duration is.
pose	geometry_msgs/Pose	Cartesian position pose.
twist	geometry_msgs/Twist	Cartesian speed.
acceleration	geometry_msgs/Accel	Cartesian acceleration.
wrench	geometry_msgs/Wrench	Cartesian forces and torques.

13.1.1.7 cartesian_trajectory_msgs/CartesianTrajectoryError

This message is used to indicate deviations in position and orientation in the Cartesian coordinate system.

13.3 cartesian_trajectory_msgs/CartesianTrajectoryPoint

Field Name	Type	explanation
time_from_start	std_msgs/duration	ROS standard message type std_msgs/Duration is.
pose	geometry_msgs/Twist	Cartesian position deviation.
twist	geometry_msgs/Twist	Cartesian velocity deviation.
acceleration	geometry_msgs/Accel	Cartesian acceleration deviation.
wrench	geometry_msgs/Wrench	Cartesian force/torque deviation.

13.1.1.8 cartesian_trajectory_msgs/CartesianTrajectory

This message is used to indicate the position, orientation, velocity, and force trajectory relative to the TCP in the Cartesian coordinate system .

13.4 cartesian_trajectory_msgs/CartesianTrajectoryPoint

Field Name	Type	explanation
header	Header	The ROS standard message type is std_msgs/Header . vinegar.
points	cartesian_trajectory_msgs/ CartesianTrajectoryPoint[]	Cartesian position, pose, velocity, acceleration, and force points It's a column.
tcp_frame	string	Tool Center Point (TCP) which represents the state The name of the frame .
wrench_frame string		The name of the frame in which the Cartesian forces are expressed .
pose_interpolation_method int32		Specify the method for interpolating the Cartesian position and orientation points. This is the number. 0: LINE linear interpolation
wrench_interpolation_method int32		number specifying how to interpolate the sequence of Cartesian force points value. 0: STEP step transition (no interpolation) 1: RECTANGULAR linear interpolation 2: TRAPEZOIDAL Trapezoidal Interpolation 3: SPLINE Cubic spline interpolation

13.1.1.9 cartesian_trajectory_msgs/CartesianTrajectoryControllerState

A message used to indicate the tracking status of the position, orientation, velocity, and force for the TCP in the Cartesian coordinate system.

This is sage.

13.5 cartesian_trajectory_msgs/CartesianTrajectoryPoint

Field Name	Type	explanation
header	Header	The ROS standard message type is std_msgs/Header . vinegar.
tcp_frame	string	Tool Center Point (TCP) which represents the state The name of the frame .
wrench_frame	string	The name of the frame in which the Cartesian forces are expressed .
desired	cartesian_trajectory_msgs/ CartesianTrajectoryPoint	Desired Cartesian position, orientation, velocity, acceleration, and force Represents:
actual	cartesian_trajectory_msgs/ CartesianTrajectoryPoint	Actual Cartesian position, pose, velocity, acceleration, and force Represents:
error	cartesian_trajectory_msgs/ CartesianTrajectoryError	Actual Cartesian position, orientation, velocity, acceleration, and force It represents the deviation.

13.1.1.10 cartesian_trajectory_msgs/CartesianState

A message used to indicate the status of the Tool Center Point (TCP) in the Cartesian coordinate system, such as its position and orientation.

This is sage.

ÿ 13.6 cartesian_trajectory_msgs/CartesianState

Field Name	Type	explanation
header	Header	The ROS standard message type is std_msgs/Header . vinegar.
tcp_frame	string	Tool Center Point (TCP) which represents the state The name of the frame .
wrench_frame	string	The name of the frame in which the Cartesian forces are expressed .
point	cartesian_trajectory_msgs/ CartesianTrajectoryPoint	Cartesian position, orientation, velocity, acceleration, and forces.

13.1.1.11 cartesian_trajectory_msgs/CartesianImpedance

This message is used to indicate stiffness and viscosity in the Cartesian coordinate system.

ÿ 13.7 cartesian_trajectory_msgs/CartesianState

Field Name	Type	explanation
stiffness	geometry_msgs/Wrench	Cartesian stiffness.
damping	geometry_msgs/Wrench	Cartesian viscosity.

13.1.1.12 cartesian_trajectory_msgs/CartesianImpedanceControllerState

This message is used to indicate the state of impedance to the TCP in the Cartesian coordinate system .

ÿ 13.8 cartesian_trajectory_msgs/CartesianTrajectoryPoint

Field Name	Type	explanation
header	Header	The ROS standard message type is std_msgs/Header . vinegar.
impedance_frame	string	The name of the frame that expresses Cartesian stiffness and viscosity vinegar.
impedance	cartesian_trajectory_msgs/ CartesianImpedance	Represents Cartesian stiffness and viscosity.

13.1.1.13 cartesian_trajectory_msgs/CartesianImpedanceTrajectory

Used to show the position, orientation, velocity, force, and impedance trajectories relative to the TCP in the Cartesian coordinate system

This is a message.

13.9 cartesian_trajectory_msgs/CartesianTrajectoryPoint

Field Name	Type	explanation
header	Header	The ROS standard message type is std_msgs/Header . vinegar.
points	cartesian_trajectory_msgs/ CartesianTrajectoryPoint[]	Cartesian position, pose, velocity, acceleration, and force points It's a column.
impedances	cartesian_trajectory_msgs/ CartesianImpedance[]	A sequence of points representing Cartesian stiffness and viscosity.
tcp_frame	string	Tool Center Point (TCP) which represents the state The name of the frame .
wrench_frame	string	The name of the frame in which the Cartesian forces are expressed .
impedance_frame	string	The name of the frame that expresses Cartesian stiffness and viscosity vinegar.
pose_interpolation_method	int32	Specify the method for interpolating the Cartesian position and orientation points. This is the number. 0: LINE linear interpolation
wrench_interpolation_method	int32	number specifying how to interpolate the sequence of Cartesian force points value. 0: STEP step transition (no interpolation) 1: RECTANGULAR linear interpolation 2: TRAPEZOIDAL Trapezoidal Interpolation 3: SPLINE Cubic spline interpolation
impedance_interpolation_method	int32	Specifies how to interpolate the sequence of Cartesian stiffness and viscosity points. The numeric value to be set. 0: STEP step transition (no interpolation) 1: RECTANGULAR linear interpolation 2: TRAPEZOIDAL Trapezoidal Interpolation 3: SPLINE Cubic spline interpolation

13.1.2 Message Specifications for ROS Services

13.1.2.1 torobo_std_msgs/SetString.srv

A generic message that sets a string.

13.10 Request fields of torobo_std_msgs/SetString.srv

Field Name	Type	explanation
data	string	The string to set.

13.11 Response fields of torobo_std_msgs/SetStringArray.srv

Field Name	Type	explanation
success	bool	Indicates whether the execution of the service was successful.
message	string	Any messages returned as a result of the execution of the Service are stored here. vinegar.

13.1.2.2 torobo_std_msgs/SetStringArray.srv

This message is used by Services that provide functions such as turning the servo on/off for joints and clearing errors.

13.12 Request fields of torobo_std_msgs/SetStringArray.srv

Field Name	Type	explanation
data	string[]	A list of strings to set. Servo ON/OFF , etc. The Service passes a list of target joint names here.

13.13 Response fields of torobo_std_msgs/SetStringArray.srv

Field Name	Type	explanation
success	bool	Indicates whether the execution of the service was successful.
message	string	If there is a message returned as a result of the execution of the service , it will be stored here. vinegar.

13.1.2.3 torobo_control_msgs/CheckCollision.srv

This message is used to detect self-collision conditions of the robot.

13.14 Request fields of torobo_control_msgs/CheckCollision.srv

Field Name	Type	explanation
jointState	its- sor_msgs /JointState []	This is the state of the robot that is the target of self-collision detection calculations. The robot model contains a joint name that is not included in 'jointState'. When you do this, the current position of the joint is used as is. can be done.

13. Software Specifications

13.15 Response fields of torobo_control_msgs/CheckCollision.srv

Field Name	Type	explanation
isColliding	bool	Type bool true if self-interference was detected , otherwise false is stored.

13.1.2.4 torobo_control_msgs/GetCollisionInfo.srv

Messages used to detect the self-collision state of the robot and calculate the link pairs where self-collision occurs.

This is Ji.

13.16 Request fields of torobo_control_msgs/GetCollisionInfo.srv

Field Name	Type	explanation
collision_info_request	torobo_control_msgs/GetCollisionInfoRequest	This is a request to detect self-interference.

13.17 Response fields of torobo_control_msgs/GetCollisionInfo.srv

Field Name	Type	explanation
collision_info_response	torobo_control_msgs/GetCollisionInfoResponse	The self-collision detection result is stored.

13.1.2.5 torobo_control_msgs/SetLoad.srv

This message is used to set additional information for the robot's minions, etc.

13.18 Request fields of torobo_control_msgs/SetLoad.srv

Field Name	Type	explanation
parent_link_name	string	The name of the parent link for which to set load information. Only link names with the name "-link_load" can be specified. is.
mass	float64	Load mass [kg] .
with	float64[3]	The center of gravity of the load [m] . Based on the origin coordinate of parent_link_name and specify the x, y, and z positions.
inertia	float64[9]	Load inertia matrix [kg m^2] . The center of gravity of the load is the origin, In a coordinate system with the same orientation as the origin coordinate of parent_link_name Ixx , Ixy, Ixz, Iyy, Iyz, Izz , or Ixx, Ixy, Ixz, Specify the nine dimensional information: Iyx, Iyy, Iyz, Izx, Izy, and Izz .

§ 13.19 Response fields of torobo_control_msgs/SetLoad.srv

Field Name	Type	explanation
success	bool	Indicates whether the execution of the service was successful.
error	string	Any messages returned as a result of the execution of the Service are stored here. vinegar.

13.1.2.6 torobo_control_msgs/UnsetLoad.srv

A message used to reset additional information set on the robot's hands, etc., and remove the load.

is.

§ 13.20 Request fields of torobo_control_msgs/UnsetLoad.srv

Field Name	Type	explanation
parent_link_name	Type string.	Name of the parent link for which to reset the load information. Only link names with the name "-link_load" in the URDF It can be specified.

§ 13.21 Response fields of torobo_control_msgs/UnsetLoad.srv

Field Name	Type	explanation
success	bool	Indicates whether the execution of the service was successful.
error	string	Any messages returned as a result of the execution of the Service are stored here. vinegar.

13.1.2.7 torobo_control_msgs/ComputeZmp.srv

This message is used to calculate the ZMP position of the robot .

§ 13.22 Request fields of torobo_control_msgs/ComputeZmp.srv

Field Name	Type	explanation
trajectory	trajec- tory_msgs /Joint- Trajec- tory	This is the command trajectory for which the ZMP position is calculated.
base_accel	geome- try_msgs /Accel []	This is the command acceleration of the robot base that is the subject of ZMP position calculation. Specify as an array of the same length as 'trajectory.points'.

§ 13.23 Response fields of torobo_control_msgs/ComputeZmp.srv

Field Name	Type	explanation
success	bool	Indicates whether the execution of the service was successful.
error_code	string	Any messages returned as a result of the execution of the Service are stored here. vinegar.
zmps	geome- try_msgs /Point []	Calculation is performed for each point and base_accel of the input trajectory . This is an array of the output ZMP positions.

13.1.2.8 torobo_control_msgs/EvaluateZmpStability.srv

This message is used to evaluate the stability of the robot's ZMP position.

§ 13.24 Request fields of torobo_control_msgs/EvaluateZmpStability.srv

Field Name	Type	explanation
trajectory	trajec- tory_msgs /Joint- Trajec- tory	This is the command trajectory for which the ZMP position is calculated.
base_accel	geome- try_msgs /Accel []	This is the command acceleration of the robot base that is the subject of ZMP position calculation. Specify as an array of the same length as 'trajectory.points'.
compute_distance	bool	Set to true to calculate the closest distance between the ZMP and the support polygon. Specify.
clearance	float64	The threshold for the closest distance between the ZMP and the support polygon [m] . ZMP position It is used to assess the stability of

13.25 Response fields of torobo_control_msgs/EvaluateZmpStability.srv

Field Name	Type	explanation
success	bool	Indicates whether the execution of the service was successful.
error_code	string	Any messages returned as a result of the execution of the Service are stored here. vinegar.
zmps	geometry_msgs/Point []	Calculation is performed for each point and base_accel of the input trajectory . This is the array of ZMP positions . If the index is the same as 'first_unstable_point_index', The calculation is performed up to the value 'trajectory.points'.
is_stable	bool	Calculate for each point and base_accel of the input trajectory . The value of the closest distance [m] between the output ZMP position and the support polygon is True if all are above the threshold set in 'clearance' If any one of them is below the threshold, false is stored.
first_unstable_point_index	bool	If "is_stable" is determined to be false , the ZMP and support polygon The closest distance between them is below the threshold set in 'clearance' The index value of the first 'trajectory.points' is stored.
distances	float64[]	Calculated for each point and base_accel of the input trajectory . The value of the closest distance [m] between the output ZMP position and the support polygon is If "is_stable" is determined to be false , ' with the same index value as 'first_unstable_point_index' Calculation will be performed up to 'trajectory.points'.

13.2 Torobo Easy Command Specifications

The control commands available in Torobo State Viewer are explained below.

Except for the above, commands are written in the format "Command $\ddot{\gamma}$ Target joint name $\ddot{\gamma}$ Command value ($\ddot{\gamma}$: half-width space)".

Please enter the joint ID for the elephant joint name . The joint ID is the ID in the Information Section of the Torobo State Viewer.

The number is listed in the Name column. For details, please refer to [Torobo State Viewer](#) .

In addition, when specifying a joint ID , using the following format makes it easier to specify multiple joints.

format	Specified joint ID
all	The control mode (joint_trajectory_controller , etc.) to which Torobo State Viewer is connected All joints controlled by
Joint ID/Joint ID/Kan Section ID/...	You can specify multiple joint IDs by separating them with a slash symbol . For example, 2/3/4 will specify joint IDs 2, 3, and 4 .
Joint ID - Joint ID If you want	to specify a range of consecutive joint IDs , you can specify the start and end points with a hyphen. For example, 2-4 will specify joint IDs 2, 3, and 4 .

13. Software Specifications

	Note: The joint ID of each joint varies depending on the control mode. Please check the joint ID carefully on the Viewer before specifying it.
---	---

Below is a list of commands.

13.2.1 Torobo State Viewer Commands

Input statement Character Line	Processing performed by the robot controller
x	Turn off the servos for all joints . x // Servo off all joints.
s	A command value of 1 turns the servo ON, and a command value of 0 turns the servo OFF . s 1 1 // Servo of joint ID 1 is switched to ON. s 6 0 // Servo of joint ID 6 is switched to OFF. s all 1 // Servo of all joints are switched to ON.
reset	Resets the error state (the error state will not be cleared unless the cause of the error is removed). reset 1 // State of joint ID 1 is reset. reset all // States of all joints are reset.
brake	Enter the command in the format "brake y joint ID y command value". A command value of 1 turns the brake on, and a command value of 0 turns the brake on. (Caution!) When the brake is turned off , the robot's joints will be in back drive mode. Please note that the joints may fall unintentionally due to their own weight. brake 1 0 // Brake of joint ID 1 is switched to OFF.
tpts or jt	The position command is given with command value 1 as the target position [deg] and command value 2 as the transition time [sec] . Please note that the joints will start moving. However, if the target position is outside the movable range, the joints will not move. plug. tpts 1 30 5 // Command joint ID 1 to move to 30 degree in 5 seconds. jt 5 -40 3 // Command joint ID 5 to move to -40 degree in 3 seconds.
in	Changes the target speed [deg/s or mm/s] in speed control to a command value. v 5 3 // Set the target velocity of joint ID 5 to 3.0 [deg/s]
cs	After resetting the error status of all joints, the system will exit from Emergency Stop and Protective Stop states. A command to return to the Healthy state is issued. However, this command is only available if the cause of the Emergency/Protective Stop state is removed. If it is not, it will not be possible to return to a healthy state. cs // Reset all joint error and clear the emergency stop state or yprotective stop state.

13.2.2 Example of control using commands

An example of control is shown below.

- How to restore a joint servo when it is turned off by some limiter

When releasing the brake, the joint may fall under its own weight. Please take great care when working on it, such as supporting it with your hands to prevent it from falling, and making sure you can press the emergency stop button instantly. For example, if a joint angle limiter is applied to joint ID 8 , use the following command.

brake 8 0	[Send] // Brake of joint ID 8 is switched to OFF. Here manually move the joints to the home position.
brake 8 1	[Send] // Brake of joint ID 8 is switched to ON.
reset 8	[Send] // State of joint 8 is reset.
s 8 1	[Send] // Servo of joint ID 8 is switched to ON.

- Position command method in position control mode or joint impedance control mode

Please change to position control mode (joint_trajectory_controller) or joint impedance mode (joint_impedance_controller) before executing the command.

s 11 1	[Send] // Servo of joint ID 11 is switched to ON.
tpts 11 90 5	[Send] // // Command joint ID 11 to move to 30 degree in 5 seconds.



The robot will start moving when **the Danger** servo is turned ON or a command value is input.

When turning on the servo or inputting command values, be sure to be outside the robot's movable area. Also, make sure you can press the emergency stop switch instantly.



Depending on the command to **the danger** joint, the robot may move suddenly. For example, if a high speed

is commanded in the speed control mode, or if a short transition time is specified in the position control mode. When issuing such commands, please be careful to start with a small command value and gradually increase it. Also, do not press the emergency stop switch instantly.

Please keep it in a position where it can be used.

13.3 Diagnostic Status Specifications

If the joint controller built into the robot detects an error, a Motor error will be displayed in the "Error" column of the Torobo State Viewer, and a Diag status (error message) will be displayed in the "Diag" column , as shown in the example image below .

Error	Diag
Motor error	IcmulnvCrc_MB4[0][2]
No error	

This section explains the diagnostic status display and its contents.

13.3.1 Notes on Diag Status

Immediately after starting the robot, a diagnostic status may be displayed on the Torobo State Viewer along with an error, but for some errors (items marked with an asterisk in the status display column of the diagnostic status list), if the error display disappears by clicking Clear Error , there will be no problem with the operation of the robot. For details, see [An error is displayed immediately after starting the robot](#) .

If a Diag status not listed in the [Diag status list](#) occurs, or if the error state cannot be cleared even after trying to recover, some kind of malfunction may have occurred in the robot. It is very dangerous to continue operating the robot in a malfunctioning state, so please stop using the robot. After that, we would appreciate it if you would contact us by referring to "[If the problem is not resolved](#) ."

13.3.2 Diag Status List

Below is a list of the diagnostic statuses. When an error occurs, refer to the corresponding solution to recover. Also, please refer to [the troubleshooting section](#) to see if there is a corresponding item.

Status Display		Solution:
CoreFault*	Description This means that an error (such as "CycExeFrameMis-match") has occurred in the core function of the joint controller .	Run Clear Error .
CycExeFrameMismatch*	This occurs when the joint controller and the control PC are not synchronized properly. This may occur when starting up the core program.	Please run Clear Error .
CtrlMngWdtOvrFlw*	This occurs when the synchronization between the joint control controller and the control PC is not good. This may occur when the core program is started. This	Please run Clear Error .
402STTO*	occurs when the servo state transition such as servo ON/OFF fails. If Clear Error is executed when an error has occurred and the cause of the error has not been eliminated (e.g. joint position limit over state) , the error clear will fail, and this also occurs in this case.	Please execute Clear Error after eliminating the cause of the error (joint position limit exceeded, emergency stop switch pressed , etc.).

Continued on next page

Table 13.28 – continued from previous page

Status Display		Solution
UnderVolt*	Description This occurs when the voltage supplied to the joint controller falls below the threshold. In a robot controller equipped with a safety relay, the error state is cleared by executing Clear Error after the safety output is output in the specified procedure .	Check whether the emergency stop switch is pressed, the safety relay is in a pressed state, and the power supply is in a turned on state.
OverVolt	This occurs when the voltage supplied to the joint controller exceeds the	Check the power supply status.
OverCur	threshold. This occurs when an instantaneous current that exceeds the allowable value flows in the motor of the target joint.	Check that the load on the joints is not too large, that the joints are not locked, that the movement command is not too fast, that vibratory movement is not occurring, etc. Check that the load on the
IntegCur	This occurs when a continuous current exceeding the allowable value flows through the motor of the target joint for a certain	joints is not too large, that the joints are not locked, that the movement command is not too fast, that vibratory
OverPwrCnsmp	period of time. This occurs when power exceeding the allowable value is supplied to the motor of the target joint.	movement is not occurring, etc.
DrvPortDis	This occurs when an instantaneous current that exceeds the allowable value flows	Check the same information as for OverCur . After
DrvNotInit*	through the motor of the target joint. This occurs when motor drive power is not being supplied, or when Clear Error has never been executed since motor drive	checking that the motor drive power is being supplied, execute Clear Error .
DrvFlt*	power was supplied. This occurs when an instantaneous current that exceeds the allowable value flows through the motor of the target joint, or when motor drive power is not being supplied, or when Clear Error has never been executed	Check out OverCur and DrvNotInit for similar content.
DrvError*	since motor drive power was supplied. This occurs when an instantaneous current that exceeds the allowable value flows through the motor of the target joint, or when motor drive power is not being supplied, or when Clear Error has never been executed	Check out OverCur and DrvNotInit for similar content. since motor drive power was supplied.

Continued on next page

Table 13.28 – continued from previous page

Status Display		Solution
OverTemp	Description This occurs when the temperature of the motor driver board for the target joint exceeds the allowable value.	Leave the robot with the servo OFF for a while , wait until the robot has cooled down sufficiently, and then resume using the robot. Also, check that the robot is not being operated in a state where the load on the joints continues for a long time, that the room temperature of the operating environment is not high, that there are no hot objects near the robot, etc. Check that the command values to the joints (transition time, speed, torque, etc.)
InOverVel	This occurs when the motor speed of the target joint exceeds the allowable value.	are not too small or large, that vibration is not occurring, etc. Check that the command values to the joints (transition time, speed, torque, etc.) are not too small or large, that vibration is not occurring, etc. Check that
InOverAcc	This occurs when the motor acceleration of the target joint exceeds the allowable value.	the joint position is within the movable range. If it is outside the movable range , refer to the recovery method when an Over Position error occurs and perform the recovery process. Check that the command values
OutOverPos	This occurs when the link side position of the target joint deviates from its range of motion.	to the joints (transition time, speed, torque, etc.) are not too small or large, that vibration is not occurring, etc. Check that the command values to the joints (transition time, speed, torque, etc.) are not too small or large, that vibration is not occurring, etc.
OutOverVel	This occurs when the link speed of the target joint exceeds the allowable value.	Check that the load on the joints is not too large, that the joints are not locked, that the movement command is not too fast, that vibration is not occurring, etc. Check that the load on the joints is not too large, that
OutOverAcc	This occurs when the link side acceleration of the target joint exceeds the allowable value.	the joints are not locked, that the movement command is not too fast, that vibration is not occurring, etc. Continued on next page
OverEft	This occurs when the torque on the link side of the target joint exceeds the allowable value.	
OverPosDev*	During position control, this occurs when the deviation between the command joint position and the current joint position exceeds the allowable value.	

Table 13.28 – continued from previous page

Status Display		Solution:
OverDeltaPos*	Description This error occurs when the difference between the joint position read by the motor-side encoder and the joint position read by the link-side encoder exceeds the allowable value. This error does not occur under normal use, but it may occur if the joint continues to move in a direction that exceeds the range of motion with the brake released (if the joint moves more than one revolution in a	Check that the joint position is within the movable range. Please refer to the recovery method when an Over Position error occurs and perform the recovery process. If the error cannot be recovered even after rotating the joint more than once, there may be a malfunction. Please contact support.
PairJointFault*	direction that exceeds the range of motion) . In the case of a differential gear-driven joint, this error occurs when an error	Please run Clear Error .
InPosSensFault*	occurs in the paired joint. This error occurs when the motor-side encoder of the target joint fails to be read.	Please run Clear Error .
OutPosSensNotUpdt* This occurs	when there is no change in the value of the link side encoder of the target joint for a certain	Please run Clear Error .
EftSensFault*	period of time. This occurs when the torque sensor of the target joint	Please run Clear Error .
sensesNotInit	cannot be read. This occurs when the encoder of the target joint	Please run Clear Error .
SensResetFailed	cannot be initialized. This occurs when the encoder of the target joint	Please run Clear Error .
SensInvCrc*	cannot be reset. In the encoder communication of the target joint, this occurs when a CRC error is detected.	Please run Clear Error .
SensWriteReadTO	This occurs when a timeout is detected in the encoder communication of the target joint.	Please run Clear Error .
SensBissAlm	This occurs when an alarm is detected in the encoder communication of the target	Please run Clear Error .
SensBissWrn	joint. This occurs when a warning is detected in the encoder communication of the target joint.	Please run Clear Error .
SensStampNotUpdt* This occurs when	there is no change in the timestamp value for a certain period of time in the encoder communication of the target joint. This	Please run Clear Error .
SensRawNotUpdt*	occurs when there is no change in the encoder value for a certain period of time in the encoder communication of the target joint.	Please run Clear Error .

13. Software **Specifications**

13.4 Safety Observation Function Specifications

The robot controller core program of this robot is equipped with a safety monitoring function to enhance safety. The safety monitoring function is managed by one of the proprietary ROS Controllers , "torobo_controllers/ToroboSafetyController", and when an abnormality is detected, it issues an "Emergency Stop" or "Protective Stop" and changes the Safety State , which will be explained later , to stop the robot.

Please note that this function, this robot, and the robot controller have not acquired any safety certification.
When using the robot, be sure to read [the safety precautions](#) .

13.4.1 About Safety State

The Safety State represents the emergency stop state of the robot and can take the following three states:

State Name	No
Healthy	abnormalities in specifications. Servo can be turned on . Control is possible by command.
Emergency Stop (e_stop) Emergency stop	Emergency stop state. Servo cannot be turned on . Servo will be turned off automatically . Emergency stop When the switch is pressed down, the power supply to the robot drive system is also cut off.
Protective Stop (p_stop) Protective stop	Protective stop state. Servo ON is possible. The current joint position is maintained regardless of the control mode. The position is controlled so that the actuator maintains its position.

13.4.2 How to check the current **Safety State**

There are two ways to check: from the terminal or on Torobo State Viewer .

13.4.2.1 Checking from the terminal

Run the following command:

```
$ rostopic echo /torobo/safety_controller/safety_command_display
```

You should see a number displayed: 0, 1, or 2. This number corresponds to the safety state :

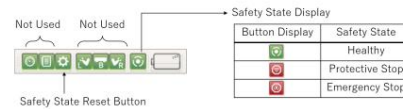
Number	Safety State
0	Healthy
1	Emergency Stop (e_stop)
2	Protective Stop (p_stop)

13.4.2.2 Checking from **Torobo State Viewer**

Start Torobo State Viewer with the following command:

```
$ roslaunch torobo_dashboard torobo_state_viewer.launch
```

At the top of the GUI , there is a section as shown in the image below. The Safety State Display indicates the current You can see the safety status .



13.4.3 How to return to **Healthy** from **Emergency Stop** or **Protective Stop**

A Ros Service is provided to clear the Emergency Stop and Protective Stop states and return to a Healthy state. However, the robot's abnormal state must be cleared before it can be restored.

There are two ways to restore: from the terminal or by pressing a button on Torobo State Viewer .

13.4.3.1 How to restore using terminal operations

To cancel the Emergency Stop (e_stop) , run the following command:

```
$ rostopic pub /torobo/safety_controller/e_stop std_msgs/Empty "{}"
```

To release Protective Stop (p_stop) , run the following command:

```
$ rostopic pub /torobo/safety_controller/p_stop std_msgs/Empty "{}"
```

13.4.3.2 How to restore using the buttons on **Torobo State Viewer**

Click the Safety State Reset Button shown in the figure on the [Torobo State Viewer](#) and select Reset. If the Safety State Display turns green, the system has been restored successfully.

13.4.4 About SafetyMonitor

"torobo_controllers/SafetyController" is designed to load and operate plugins called SafetyMonitors that monitor separate safety monitoring targets. This is the same as the relationship between controller_manager and ROS Controller described in [About Your Own ROS Controller](#) .

13. Software Specifications

13.4.5 How to configure controllers.yaml

By writing controllers.yaml , you can specify the SafetyMonitor to be loaded in torobo_controllers/SafetyController. You can.

13.4.5.1 Controllers.yaml Location Directory

The controllers.yaml is located in the following directory:

```
`~/catkin_ws/src/rod_pipe/robot_pipe/pipe_description/config/models/XXX/YYY/ ycontrollers.yaml`
```

* "XXX" and "YYY" will be replaced with text depending on the robot's configuration.

13.4.5.2 How to write controllers.yaml

In controllers.yaml , list the SafetyMonitors to be loaded into torobo_controllers/SafetyController .

The format is as follows:

```
safety_controller:
  type: torobo_controllers/SafetyController
  ...

safety_monitors:                                # SafetyMonitor is defined under the safety_monitors namespace.
  safety_monitor_name_1:                          # SafetyMonitor name. You can specify any name.

    type: "SafetyMonitorType" # SafetyMonitor type. # You can specify any of
                                the Controller types listed in this chapter .

    parameter_1: value_1      # SafetyMonitor specific parameters.
    parameter_2: value_2      # SafetyMonitor specific parameters.

  safety_monitor_name_2:
    type: "SafetyMonitorType"

    parameter_1: value_1
    parameter_2: value_2

  safety_monitor_name_3:
    ...
```

One example is the SafetyMonitor torobo_safety_monitors/JointErrorMonitor , which monitors the occurrence of joint errors.

We will introduce a setting example.

```
...
safety_monitors:                                # SafetyMonitor is defined under the safety_monitors namespace.
  ...
  joint_error_monitor:
    type: torobo_safety_monitors/JointErrorMonitor stop_type: 2 # 0:
    Disabled, 1: EmergencyStop, 2: ProtectiveStop
```


13.4.5.3 Description in controllers.yaml

The default SafetyMonitor is set with safety in mind, so you do not need to use it.
Do not delete any of the entries.

	DangerIf you delete the default SafetyMonitor setting, the safety monitor function will not function. If you delete the settings by mistake, please reinstall the source code or take other necessary steps to fix the problem.
---	--

13.4.6 Common features of each **SafetyMonitor****13.4.6.1** **SafetyMonitor** Namespace

Each SafetyMonitor may have one or more ROS interfaces , such as Topic , Service, or Action . These namespaces differ for each SafetyMonitor , so please refer to the explanation of each SafetyMonitor for details.

13.4.6.2 **SafetyMonitor** parameters and **dynamic_reconfigure**

Among the parameters that each SafetyMonitor has, the parameters listed in the "Reconfigurable Parameters" section can be reconfigured by using the `dynamic_reconfigure()` function of ROS. You can change the parameter value dynamically during operation using a function called `rqt_reconfigure` . To change the parameters that support `dynamic_reconfigure` in the GUI , use `rqt_reconfigure`. It is convenient to use `rqt_reconfigure`. `rqt_reconfigure` can be started with the following command.

```
$ rosrn rqt_reconfigure rqt_reconfigure
```

For details on each function, please refer to the reference URL .

13.4.6.3 Temporarily disabling the safety monitoring state

Some SafetyMonitors may be able to temporarily disable the safety monitoring state by using `dynamic_reconfigure` . Normally, disabling the safety monitoring state is not recommended as it reduces safety and is dangerous, but it may be necessary in some situations, such as when recovering from [an Emergency Stop being issued due to self-collision detection](#) .

If you temporarily disable the safety observation state, be sure to re-enable the safety observation state after restoring the state.

	Danger: If you temporarily disable the safety monitoring state, be sure to enable the safety monitoring state again after completing operations such as recovery. If you operate the robot with the safety monitoring state disabled, the robot may run out of control or fall over due to the lack of proper safety monitoring function.
---	---

13. Software **Specifications**

13.4.7 torobo_safety_controller

This is a controller that manages the safety monitoring function and is implemented as a ROS Controller .
For details about the interface and parameters, see [torobo_controllers/SafetyController](#) .

The torobo_safety_controller has multiple Safety Monitors built in, which are safety monitoring functions that monitor different targets. When each Safety Monitor detects an abnormality, it requests the torobo_safety_controller to issue an "Emergency Stop" or "Protective Stop" . When the torobo_safety_controller detects this, it issues the safety command, changing the Safety State and stopping the robot's operation.

After each Safety Monitor detects an abnormality, it will resume [Emergency Stop](#) and [Protective Stop](#) when the specified recovery conditions are met. You can return to [Healthy mode](#) by following the steps in [How to return to Healthy mode](#) .

Each Safety Monitor is explained below .

13.4.8 torobo_safety_monitors/SafetyUnitMonitor

This is a Safety Monitor that monitors the safety output status of the safety module mounted on the robot's power supply unit . It also has the function of issuing a manual reset command to the safety module and starting the power supply to the robot's drive system.

* Enabled only on robots equipped with a safety module

13.4.8.1 Emergency Stop Requirements

When the emergency stop switch is pressed.

13.4.8.2 Protective Stop Requirements

None

13.4.8.3 Return Conditions

The emergency stop switch is not pressed.

13.4.8.4 Action Interface

None

13.4.8.5 Subscribed Topics

None

13.4.8.6 Published Topics

None

13.4.8.7 Services

None

13.4.8.8 Parameters

safety_output_monitor_port (string, default: yy)

This parameter must be set only on robot controllers equipped with a safety module monitoring function. Users must not change the value of this parameter.

safety_manual_reset_port (string, default: yy)

This parameter must be set only on robot controllers equipped with a safety module monitoring function. Users must not change the value of this parameter.

13.4.8.9 Reconfigurable Parameters

None

13.4.9 torobo_safety_monitors/SelfCollisionMonitor

This is a Safety Monitor that monitors the self-collision state of the robot . As the robot interference model, a model that is slightly larger than the actual robot shape is used, so the robot operation can be stopped before self-collision actually occurs. For the specifications of the self-collision detection function, please refer to [the specifications of the self-collision detection function](#) .

13.4.9.1 Emergency Stop Requirements

When self-interference occurs.

13. Software **Specifications**

13.4.9.2 Protective Stop Requirements

None

13.4.9.3 Return Conditions

There is no self-interference. Regarding the recovery method, please refer to the procedure below when an [Emergency Stop](#) is issued due to a self-interference detection. Please also check [the recovery method for the device](#) .

13.4.9.4 Action Interface

None

13.4.9.5 Subscribed Topics

None

13.4.9.6 Published Topics

None

13.4.9.7 Services

None

13.4.9.8 Parameters

None

13.4.9.9 Reconfigurable Parameters

self_collision_monitor_param/enable_monitoring (bool, default: True)

Whether or not to monitor self-interference conditions.

13.4.10 torobo_safety_monitors/StopTopicMonitor

This is a Safety Monitor that subscribes to and monitors the following two topics .

- /torobo/safety_controller/e_stop
- /torobo/safety_controller/p_stop

Both message types are [std_msgs/Empty](#) is.

13.4.10.1 Emergency Stop Requirements

When subscribing to a message published to the safety_controller/e_stop Topic .

13.4.10.2 Protective Stop Requirements

When subscribing to a message published to the safety_controller/p_stop Topic .

13.4.10.3 Return Conditions

e_stop, p_stop No messages have been published to the topic for a specified period of time (default is 0.5 seconds) .

13.4.10.4 Action Interface

None

13.4.10.5 Subscribed Topics

e_stop (std_msgs/Empty)

This topic accepts commands to switch the Safety State to Emergency Stop .

p_stop (std_msgs/Empty)

This topic accepts commands to switch the Safety State to Protective Stop .

13.4.10.6 Published Topics

None

13.4.10.7 Services

None

13.4.10.8 Parameters

None

13. Software **Specifications**

13.4.10.9 Reconfigurable Parameters

None

13.4.11 torobo_safety_monitors/JointErrorMonitor

This is a Safety Monitor that monitors the error status of the joints . The requested Safety You can change the state .

13.4.11.1 Emergency Stop Requirements

When stop_type is set to 1 and an error condition is detected in any joint.

13.4.11.2 Protective Stop Requirements

When stop_type is set to 2 and an error condition is detected in any joint.

13.4.11.3 Return Conditions

All joints are not in an error state.

13.4.11.4 Action Interface

None

13.4.11.5 Subscribed Topics

None

13.4.11.6 Published Topics

None

13.4.11.7 Services

None

13.4.11.8 Parameters

stop_type (int)

Specify the behavior when the stop condition is met using one of the following three numeric values.

Number	Behavior when the stop condition is met
0	Disable judgment (if the stop condition is met) Do not issue any stop command even if stomach)
1	Issue an Emergency Stop (e_stop)
2	Issue a Protective Stop (p_stop)

13.4.11.9 Reconfigurable Parameters

monitoring_joint_error_params/stop_type_when_error_detected (Enum<int>)

Specify the behavior when the stop condition is met with one of the following three enumerated values. This parameter is

It is linked to the stop_type parameter .

Enumeration	Behavior when the stop condition is met
Disbaled(0)	Disable the judgment (if the stop condition is met) Do not issue any stop command even if stomach)
Emergen- cyStop(1)	Issue an Emergency Stop (e_stop)
ProtectiveStop(2)	Issue a Protective Stop (p_stop)

13.4.12 torobo_safety_monitors/ControlToleranceMonitor

This is a Safety Monitor that monitors the control status of joints . It is possible to monitor three types of information: position, speed, and external force. Which to monitor can be set individually using parameters.

13. Software **Specifications**

13.4.12.1 Emergency Stop Requirements

None

13.4.12.2 Protective Stop Requirements

When either the current position or current speed does not follow the target value, or when the detected external force value exceeds the threshold value

13.4.12.3 Return Conditions

If the difference between the command position and the current position, or the command speed and the current speed, is smaller than the threshold, and the detected external force value is below the threshold.

If it is rotating

13.4.12.4 Action Interface

None

13.4.12.5 Subscribed Topics

None

13.4.12.6 Published Topics

None

13.4.12.7 Services

None

13.4.12.8 Parameters

enable_position_monitoring (bool, default: True)

Whether to monitor the difference between the target joint position and the current position. If the difference is greater than "position_tolerances" ,
Issue a Protective Stop .

enable_velocity_monitoring (bool, default: True)

Whether to monitor the difference between the target joint velocity and the current velocity. If the difference is greater than "velocity_tolerances" ,
Issue a Protective Stop .

enable_effort_monitoring (bool, default: True)

Whether to monitor the detected external force value (torque measurement value - dynamics torque calculation value). If the difference is greater
than "effort_tolerances", a Protective Stop will be issued.

position_tolerances/<joint> (double)

The tolerance [rad] for the difference between the target joint position and the current position. This is used when "enable_position_monitoring" is True .

velocity_tolerances/<joint> (double)

The tolerance for the difference between the target joint velocity and the current velocity [rad/s] . Used when "enable_velocity_monitoring" is True .

effort_tolerances/<joint> (double)

This is the tolerance [Nm] for the detected external force value (torque measurement value - dynamics torque calculation value) . It is used when "enable_effort_monitoring" is True .

13.4.12.9 Reconfigurable Parameters

control_tolerance_monitor/enable_position_monitoring (bool)

Whether to monitor the difference between the target joint position and the current position. This parameter is linked to enable_position_monitoring in Parameters .

control_tolerance_monitor/enable_velocity_monitoring (bool)

Whether to monitor the difference between the target joint velocity and the current velocity. This parameter is linked to enable_velocity_monitoring in Parameters .

control_tolerance_monitor/enable_effort_monitoring (bool)

This parameter determines whether or not to monitor the detected external force value (torque measurement value - dynamics torque calculation value). This is linked to the Parameters enable_effort_monitoring .

control_tolerance_monitor/position_tolerances/<joint> (double)

This is the tolerance [rad] for the difference between the target joint position and the current position . This parameter is linked to the position_tolerances/<joint> in Parameters .

control_tolerance_monitor/velocity_tolerances/<joint> (double)

This is the tolerance [rad/s] for the difference between the target joint velocity and the current velocity . This parameter is linked to the velocity_tolerances/<joint> in Parameters .

control_tolerance_monitor/effort_tolerances/<joint> (double)

This is the tolerance [Nm] for the detected external force value (torque measurement value - dynamics torque calculation value) . This parameter is linked to effort_tolerances/<joint> in Parameters.

13.4.13 torobo_safety_monitors/JointTrajectoryCommandMonitor

This is a Safety Monitor that monitors commands to a Controller that has a JointTrajectory or FollowJointTrajectoryAction in the ROS Interface . For each JointTrajectoryPoint included in the above command , the joint position, velocity, and acceleration command The system evaluates the closest distance between the ZMP and the support polygon and whether there is self-collision, and issues a command to stop the robot from moving if it detects a risk of tipping over or self-collision. This allows the robot to stop moving without actually moving, even if a motion trajectory that would cause a transition to a dangerous state is sent by mistake.

13.4.13.1 Emergency Stop Requirements

None

13.4.13.2 Protective Stop Requirements

For a Controller that has JointTrajectory or FollowJointTrajectoryAction in its ROS Interface ,
When a command trajectory point that satisfies any of the conditions is included

1. When the posture joint position, velocity, and acceleration are realized, the closest distance between the ZMP and the support polygon is below the threshold.
2. Self-collision occurs when moving to the command joint position

13.4.13.3 Return Conditions

The JointTrajectory and FollowJointTrajectoryAction commands are issued to the target Controller for a certain period of time.
When not

13.4.13.4 Action Interface

None

13.4.13.5 Subscribed Topics

[Controller ý]/command ([trajectory_msgs/JointTrajectory](#))

This is the topic of the command trajectory subscribed to by the Controller that has the JointTrajectory to be monitored in its ROS Interface . The 'Controller name' part of the namespace contains the name of the Controller in a running state that has a controller type that includes the strings 'JointTrajectoryController' or 'JointImpedanceController' .

[Controller ý]/follow_joint_trajectory/goal ([control_msgs/FollowJointTrajectoryActionGoal](#))

This is the topic of the command trajectory subscribed to by the Controller that has the FollowJointTrajectoryAction to be monitored in the ROS Interface . The 'Controller name' part of the namespace contains the name of the Controller in the running state that has a controller type that includes the string 'JointTrajectoryController' or 'JointImpedanceController'.

13.4.13.6 Published Topics

None

13.4.13.7 Services

None

13.4.13.8 Parameters

enable_self_collision_monitoring (bool, default: True)

Whether or not to monitor self-interference conditions.

enable_zmp_stability_monitoring (bool, default: True)

Whether to monitor the stability of the robot against tipping over by comparing the closest distance between the ZMP and the support polygon with a threshold.

zmp_clearance_distance (double, default: 0.1)

The threshold for the closest distance between the ZMP and the support polygon [m] .

13.4.13.9 Reconfigurable Parameters

joint_trajectory_command_monitor_param/enable_self_collision_monitoring
(bool, default: True)

Whether or not to monitor self-collision state. If enable_self_collision_monitoring in Parameters is set to false , this parameter cannot be changed during operation.

joint_trajectory_command_monitor_param/enable_zmp_stability_monitoring (bool, default: True)

This is whether to monitor the stability of the robot against tipping over by comparing the closest distance between the ZMP and the support polygon with a threshold. If enable_zmp_stability_monitoring in Parameters is set to false , this parameter cannot be changed during operation.

joint_trajectory_command_monitor_param/zmp_clearance_distance (double)

This is the threshold value [m] for the closest distance between the ZMP and the support polygon . This parameter is linked to zmp_clearance_distance in Parameters .

13.4.14 torobo_safety_monitors/ZmpStabilityMonitor

This is a Safety Monitor that monitors the relationship between the ZMP (Zero Moment Point) position and the support polygon in the robot's current state . If the ZMP approaches the support polygon, the robot may tip over, so a stop command is issued to prevent this. The risk of tipping over can be reduced by setting the distance between the ZMP and the support polygon, which is the criterion for determining whether to issue a stop command, to a larger value. The required Safety State can be changed depending on the value of the stop_type parameter.

13.4.14.1 Emergency Stop Requirements

When the value of stop_type is set to 1 and the closest distance between the current ZMP position and the support polygon is less than or equal to the threshold.

13.4.14.2 Protective Stop Requirements

When the value of stop_type is set to 2 and the closest distance between the current ZMP position and the support polygon is less than or equal to the threshold.

13.4.14.3 Return Conditions

When the closest distance between the current ZMP position and the support polygon exceeds a threshold.

13.4.14.4 Action Interface

None

13.4.14.5 Subscribed Topics

None

13.4.14.6 Published Topics

None

13.4.14.7 Services

None

13.4.14.8 Parameters

zmp_clearance_distance (double)

The threshold for the closest distance between the ZMP and the support polygon [m] .

stop_type (int)

Specify the behavior when the stop condition is met using one of the following three numeric values.

Number	Disables behavior judgment when a stop
0	request condition is met (does not issue any stop command even if a stop condition is met)
1	Issue an Emergency Stop (e_stop)
2	Issue a Protective Stop (p_stop)

13.4.14.9 Reconfigurable Parameters

zmp_stability_monitor_param/zmp_clearance_distance (double)

This is the threshold value [m] for the closest distance between the ZMP and the support polygon . This parameter is linked to zmp_clearance_distance in Parameters .

zmp_stability_monitor_param/stop_type_when_zmp_unstable_detected (Enum<int>)

Specify the behavior when the stop condition is met with one of the following three enumerated values. This parameter is It is linked to the stop_type parameter .

Enumeration	Disables behavior judgment when a stop
Disbaled(0)	request condition is met (does not issue any stop command even if a stop condition is met)
Emergen- cyStop(1)	Issue an Emergency Stop (e_stop)
ProtectiveStop(2)	Issue a Protective Stop (p_stop)

13.4.15 blank**13.5 Specifications of the Self-Interference Detection Function**

This robot software has a function that detects self-interference of the robot by taking into account the shape and posture of the robot's links. As a robot interference model, a model that is slightly larger (thicker) than the actual robot shape is used, it is possible to detect the risk of self-interference and stop the robot's operation before it occurs in the actual robot. The above stop function is provided as standard. For details,

Please refer to [torobo_safety_monitors/SelfCollisionMonitor](#) .

13. Software **Specifications**

13.5.1 How to use the self-interference detection function

To detect self-collision, use the original ROS Service `/torobo/check_collision` or `/torobo/get_collision_info`. `/torobo/check_collision` only detects whether self-collision exists. On the other hand, `/torobo/get_collision_info` can calculate the pair information of the link where self-collision occurs in addition to the existence of self-collision, but it requires more calculation time. For the respective message [specifications](#), refer to `torobo_control_msgs/CheckCollision.srv` and `torobo_control_msgs/GetCollisionInfo.srv`.

13.6 Specifications of **center of gravity**, **ZMP** position calculation, and stability evaluation function

The control program of this robot has a function to calculate the center of gravity position of the whole body in the current state of the robot and the ZMP (Zero Moment Point) position. The ZMP position can be calculated not only for the current position but also for the state at each trajectory point of the command trajectory, and it also has a function to evaluate the positional relationship of the calculated ZMP position to the support polygon. If the ZMP position is too close to the side of the support polygon, there is a risk that the robot will fall over. In such a case, it is possible to stop the robot from executing the planned command trajectory. For safety monitoring functions aimed at preventing the robot from falling over, please also refer to `torobo_safety_monitors/ZmpStabilityMonitor` and `torobo_safety_monitors/JointTrajectoryCommandMonitor`.

13.6.1 How to check the robot's center of gravity

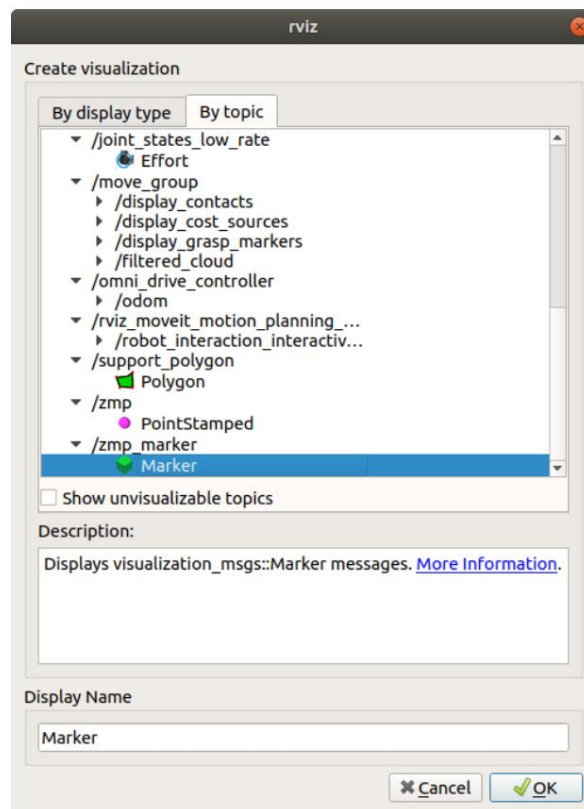
`/torobo/com` ([geometry_msgs/PointStamped](#)) Or `/torobo/com_marker` ([visualization_msgs/Marker](#)) By subscribing to the topic, you can check the center of gravity of the robot in its current state.

13.6.2 How to check the robot **ZMP** position

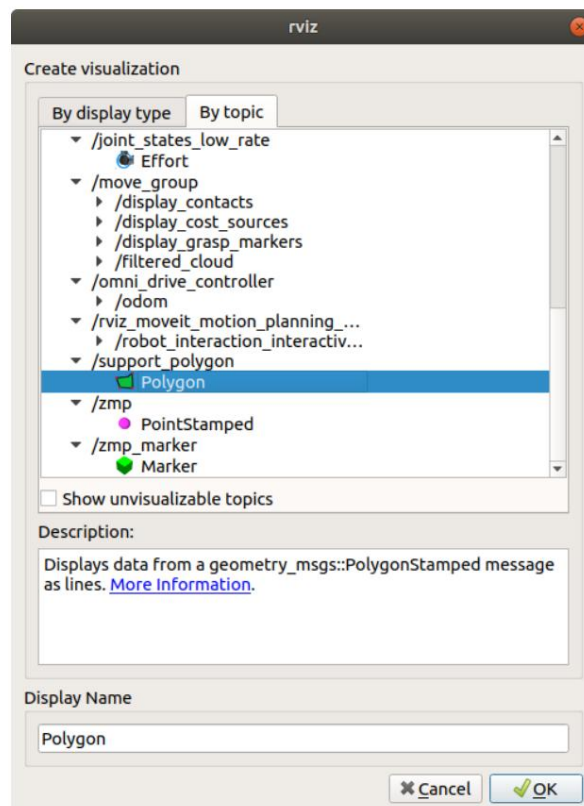
`/torobo/zmp` ([geometry_msgs/PointStamped](#)) Or `/torobo/zmp_marker` ([visualization_msgs/Marker](#)) By subscribing to the topic, you can check the ZMP position of the robot in its current state.

13.6.2.1 How to check the current **ZMP** position and support polygon shape using **RViz**

You can use RViz to check the ZMP position and support polygon shape of the robot's current posture. To check, first start RViz. Next, click Add to add a drawing target, select `"/torobo/zmp_marker/Marker"` from the ByTopic tab, and click OK.

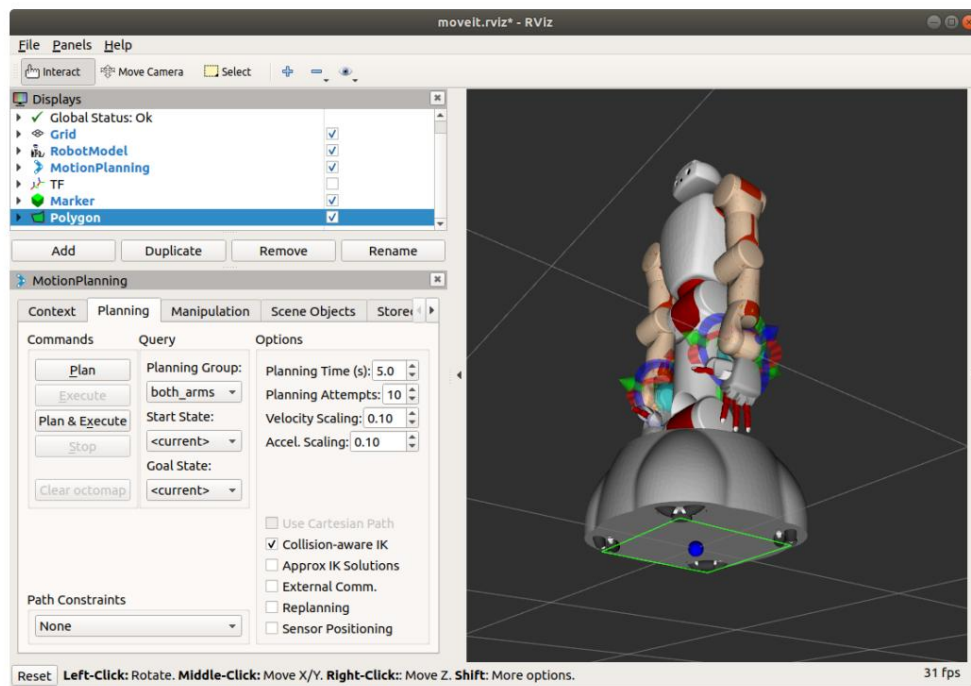


Similarly, select '/torobo/support_polygon/Polygon' from the ByTopic tab and click OK .

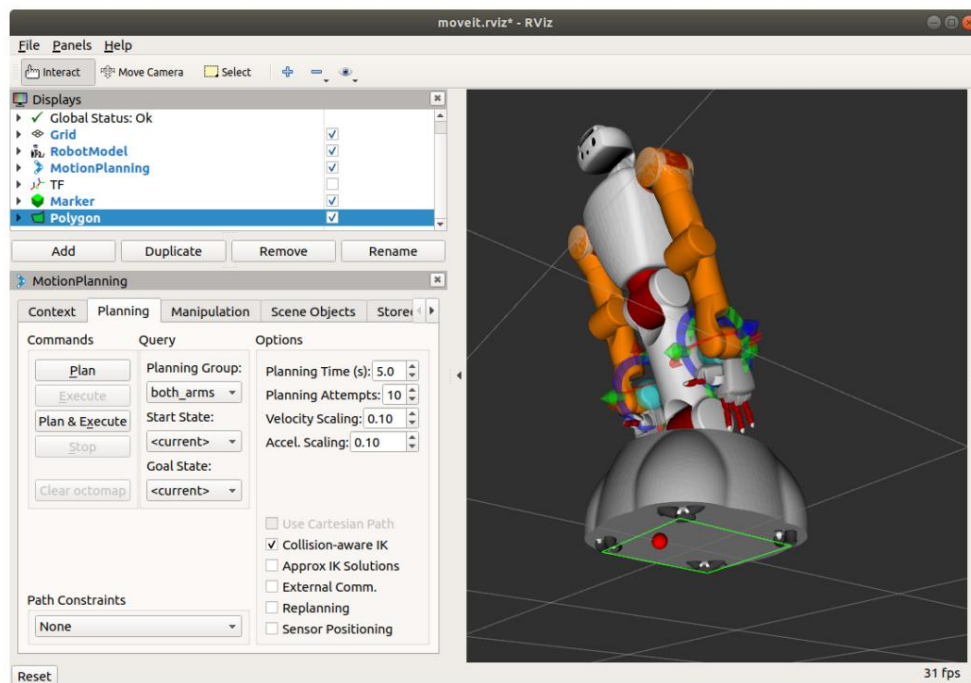


Then, the support polygon shape will be drawn in green lines on the bottom of the robot cart, and the ZMP position will be drawn in blue dots, as shown in the image below .

13. Software Specifications



If the closest distance between the ZMP position and the support polygon becomes less than the threshold, the ZMP position is displayed. The points will be drawn in red as shown in the image below.



13.6.2.2 How to set the threshold value that determines the marker color that represents the ZMP position

In the above, the threshold value for determining the color of the marker representing the ZMP position (the closest distance [m] between the ZMP and the support polygon) is as follows:

You can specify this by editing models.yaml located in the directory below .

```
`~/catkin_ws/src/rod_pipe/robot_pipe/pipe_description/config/models/XXX/YYYY/ymodels.yaml`
```

* "XXX" and "YYY" will be replaced with text depending on the robot's configuration.

The value of zmp_clearance_distance in the above models.yaml corresponds to the threshold parameter.

You can also change the threshold parameter during operation using dynamic_reconfigure . Start rqt_reconfigure , select "torobo" -> "rbdl_mobile_model" -> "rbdl_mobile_model_param" , and the value of zmp_clearance_distance that appears on the right side of the GUI corresponds to the threshold parameter.

13.6.3 Calculation method of ZMP position for command joint trajectory

To calculate the ZMP position for the command joint trajectory , use the original ROS Service /torobo/compute_zmp. For message specifications, see [torobo_control_msgs/ComputeZmp.srv](#) .

13.6.4 Method for evaluating the stability of ZMP position relative to command joint trajectory

To evaluate the stability of the ZMP position relative to the command joint trajectory , we use our proprietary ROS Service The above ROS Service uses /torobo/evaluate_zmp_stability . Please refer to [torobo_control_msgs/EvaluateZmpStability.srv](#) for message specifications . Note that the above ROS Service calculates the ZMP position when the position, speed, and acceleration of each trajectory point of the command joint trajectory ('trajectory.points') and the acceleration of the robot base ('base_accel') are achieved , and evaluates the system as stable ('is_stable' == true) if the closest distance between the ZMP and the support polygon is always above the threshold ('clearance') , and unstable ('is_stable' == false) if any one point is below the threshold .

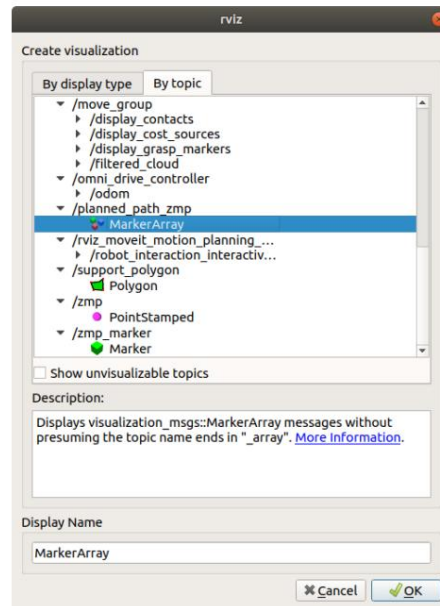
13.6.5 How to check the ZMP position of the orbital points planned by Moveit!

When planning a trajectory using the Moveit! RViz plugin from RViz , it is possible to check the ZMP position when each planned trajectory point of the robot is achieved. To check, first start RViz . Next, to have RViz draw the trajectory points planned by Moveit!, use the following message ([visualization_msgs/MarkerArray](#)) Start the node that will publish to the topic with the following command.

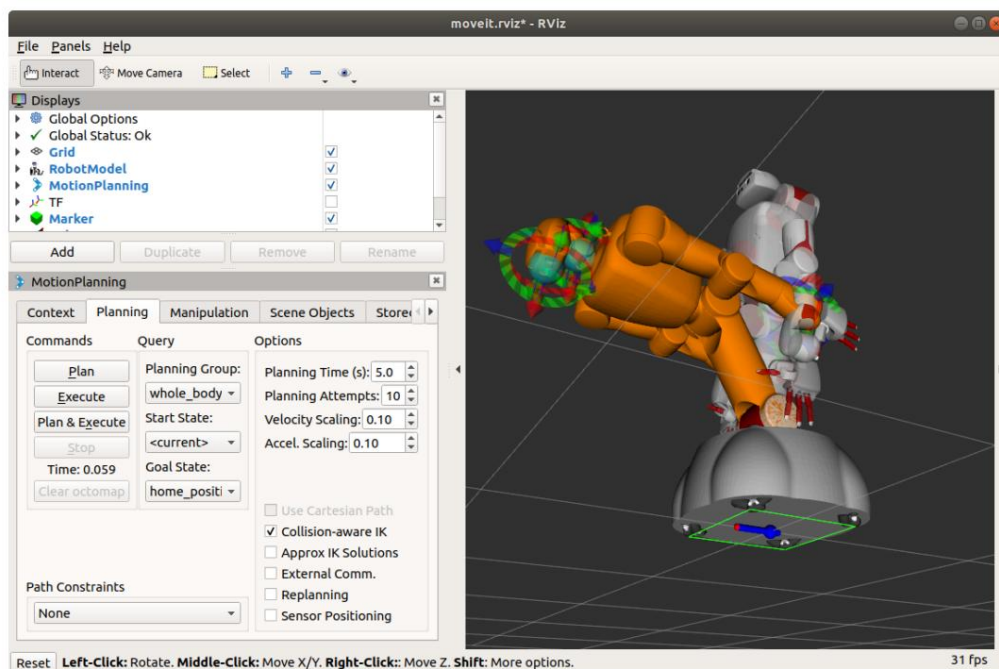
```
$ rosrn torobo_rbdl_mobile_model planned_path_zmp_visualizer __ns:=torobo
```

Then , click Add to add a drawing target to RViz , select "/torobo/planned_path_zmp/MarkerArray" from the ByTopic tab , and click OK .

13. Software Specifications



In this state, if you operate the Moveit! RViz plugin and click Plan, the ZMP positions corresponding to each trajectory point sequence will be drawn as markers, as shown in the image below. In the image below, red markers are drawn along the way, so if you execute this trajectory, the ZMP may reach an unstable position and you may fall over.



In fact, even if you click Execute to execute this trajectory by mistake, The trajectory will not be executed by the `torobo_safety_monitors/JointTrajectoryCommandMonitor` function and the robot will be in Protective Stop state.

13.6.6 blank

13.7 Specifications of Original ROS Controller

13.7.1 Custom ROS Controller

Each control mode, such as position control mode and weight compensation mode, is operated by our original ROS Controller and can be switched at any time. To switch between these controllers, use the `controller_manager`. The Torobo State Viewer uses a package. When the robot starts up, the Controllers are loaded into the Controller Manager. Users can instruct the Controller Manager to switch between controllers by operating the Torobo State Viewer or the CLI.

The Controllers that are loaded into the Controller Manager can be configured in a configuration file called `controllers.yaml`.

This section includes the following sections:

- How to configure the configuration file `controllers.yaml`
- Common features for each Controller
- Specifications of each controller

13.7.2 How to configure controllers.yaml

You can specify the Controllers that will be loaded into the Controller Manager by writing `controllers.yaml`.

13.7.2.1 Controllers.yaml Location Directory

The `controllers.yaml` is located in the following directory:

```
~/catkin_ws/src/rod_pipe/robot_pipe/pipeline_description/config/models/XXX/YYYY/ controllers.yaml
```

* "XXX" and "YYY" will be replaced with text depending on the robot's configuration.

13.7.2.2 How to write controllers.yaml

`controllers.yaml` lists the ROS Controllers to load into the Controller Manager.

The format is as follows:

```
controller_name_1:           # Controller name. You can specify any name.

type: "ControllerType" # Controller type.
                        #You can specify any Controller type listed in this chapter .
```

(Continued on next page)

(Continued from previous page)

```

default: true                                #This parameter determines whether to start the robot by default. #If omitted,
                                              false is specified.

joints: -                                  # The name of the joint controlled by the controller.
  joint_name_1 -
  joint_name_2
  ...

parameter_1: value_1                       # Controller -specific parameters.
parameter_2: value_2                       # Controller -specific parameters.

controller_name_2: type:
  "ControllerType"
default: true
joints: -
  joint_name_1 -
  joint_name_2
  ...

parameter_1: value_1
parameter_2: value_2

controller_name_3:
  ...

```

As an example, we will introduce the configuration example of `effort_controllers/ToroboJointTrajectoryController`, which is a controller in position control mode.

```

joint_trajectory_controller:
  type: "effort_controllers/ToroboJointTrajectoryController"
  default: true

  joints: -
    left_arm/joint_1 - left_arm/
    joint_2 - left_arm/joint_3

  action_ns: follow_joint_trajectory # for moveit action_type: FollowJointTrajectory
  # for moveit (==>type) action_default: true allow_partial_joints_goal: true
                                              # for moveit (==>default)

  constraints:
    goal_time: 0.5
    stopped_velocity_tolerance: 0.1 # Defaults to 0.01 left_arm/joint_1: goal:
    trajectory: 0.05
                                0.01

    left_arm/joint_2: goal:
    0.01
    trajectory: 0.05
    left_arm/joint_3: goal:
    0.01
    trajectory: 0.05

```

13.7.2.3 Controllers that must be included in controllers.yaml

The following two controllers are set by default and are essential for the robot to operate.
So please do not delete the description.

- `torobo_state_controller/ToroboStateController` This is a controller that publishes the robot's state .
- `torobo_controllers/SafetyController` This is the controller responsible for the safety monitoring function . It monitors whether the robot is operating normally, and if there is an abnormality, it switches the Safety State to Emergency Stop or Protective Stop . For details on the safety monitoring function, please also refer to [the Safety Monitoring Function Specifications](#) .

	While the Danger robot is running , <code>torobo_state_controller/ToroboStateController</code> and <code>torobo_controllers /SafetyController</code> is stopped or unloaded , the safety monitoring function and other functions may not work properly and the robot may go out of control. If you accidentally stop or unload the <code>torobo_controllers /SafetyController</code>, close all programs and start them again.
---	--

13.7.3 Common features of each Controller**13.7.3.1 Controller Namespace**

Each Controller has a ROS Interface , which can be a Topic , Service, or Action .
They are usually generated in the namespace under `"/torobo/[Controller name]/"` .

For example, `'effort_controller/ToroboJointTrajectoryController'` has the following three ROS Interfaces .

- `~/state` Topic: Publishes the state of the controller
- `~/command` Topic: Subscribe to Topic commands to the Controller
- `~/follow_joint_trajectory` Action: Subscribe to the Action command to the Controller

If you give this controller the name `"joint_trajectory_controller"` , the above three topics will be generated in the following namespaces: For Action, five topics will be generated as shown in the example below .

- `/torobo/joint_trajectory_controller/state`
- `/torobo/joint_trajectory_controller/command`
- `/torobo/joint_trajectory_controller/follow_joint_trajectory/cancel`
- `/torobo/joint_trajectory_controller/follow_joint_trajectory/feedback`
- `/torobo/joint_trajectory_controller/follow_joint_trajectory/goal`
- `/torobo/joint_trajectory_controller/follow_joint_trajectory/result`
- `/torobo/joint_trajectory_controller/follow_joint_trajectory/status`

13. Software **Specifications**

In the following explanation, with some exceptions, each ROS interface is generated in the namespace under `"/torobo/[Controller name]"/`. This is based on the assumption that

13.7.3.2 Controller parameters and `dynamic_reconfigure`

Among the parameters that each Controller has, the parameters listed in the "Reconfigurable Parameters" section are handled by ROS's `dynamic_reconfigure`. You can change the parameter value dynamically during operation using a function called `rqt_reconfigure`. To change the parameters that support `dynamic_reconfigure` in the GUI, use `rqt_reconfigure`. It is convenient to use `rqt_reconfigure`. `rqt_reconfigure` can be started with the following command.

```
$ rosrn rqt_reconfigure rqt_reconfigure
```

For details on each function, please refer to the reference URL .

13.7.4 `torobo_state_controller/ToroboStateController`

This is the controller that publishes the robot status . This controller is an exception and should not be called `"/torobo/[Controller name]"/`. Instead, a ROS interface will be generated under the `"/torobo/"` namespace .

13.7.4.1 Action Interface

None

13.7.4.2 Subscribed Topics

None

13.7.4.3 Published Topics

`joint_states` (`sensor_msgs/JointState`)

The current state of the robot's joints.

`joint_states_low_rate` (`sensor_msgs/JointState`)

The message content is the same as that published in the `"joint_states"` topic, but the publishing cycle is slower . If many nodes subscribe to a topic that is published at a high cycle , the CPU load will increase, so

If there is no problem with obtaining status at a low frequency, such as for displaying the robot status in a GUI , subscribe to this topic .

`torobo_states` (`torobo_sensor_msgs/ToroboState`)

The current state of the robot. Contains more detailed information than the `joint_states` Topic .

`torobo_states_low_rate` (`torobo_sensor_msgs/ToroboState`)

The message content is the same as that published in the "torobo_states" topic, but the publishing cycle is slower . If many nodes subscribe to a topic that is published at a high cycle , the CPU load will increase, so
If there is no problem with obtaining status at a low frequency, such as for displaying the robot status in a GUI , subscribe to this topic .

joint_gravity_torque ([sensor_msgs/JointState](#))

This message contains the gravity compensation torque value of each joint in the current robot posture instead of the current torque value of the joint in the "effort" section.

joint_dynamics_torque ([sensor_msgs/JointState](#))

In place of the current torque values of the joints, the "effort" section contains the joint torque values (dynamics torque value = gravity compensation term + centrifugal/Coriolis term + inertia term) calculated from the robot dynamics model for each joint in the current robot posture.

load_inertia ([torobo_control_msgs/LoadInertia](#))

This is the physical characteristic of the load set on the robot's hands, etc.

load_inertia_low_rate ([torobo_control_msgs/LoadInertia](#))

The message content is the same as that published to the ``load_inertia" topic, but the publication cycle is slower .

13.7.4.4 Services

None

13.7.4.5 Parameters

publish_rate (double)

Publish period [Hz] to the ``torobo_states" topic .

low_publish_rate (double, default: 20.0)

Publish period to the ``torobo_states_low_rate" topic [Hz].

ignore_joints (string[], default: [])

This is a list of the names of joints to be excluded from state management. All information for joints specified here will be set to 0 .

13. Software **Specifications**

13.7.4.6 Reconfigurable Parameters

None

13.7.5 torobo_controllers/SafetyController

This is a controller that manages the robot's safety monitoring function . If an abnormality is detected, it issues an "Emergency Stop" or "Protective Stop" and changes the Safety State.

This Controller is designed to load and operate plugins called SafetyMonitors that monitor individual safety monitoring targets, and each SafetyMonitor may have its own ROS Interface and parameters. This section describes only the ROS Interface and parameters that torobo_controllers/SafetyController itself has .

For details on the safety monitoring function and each SafetyMonitor , please also refer to [the Safety Monitoring Function Specifications](#) .

13.7.5.1 Action Interface

None

13.7.5.2 Subscribed Topics

None

13.7.5.3 Published Topics

safety_command_display ([std_msgs/Int32](#))

Current safety state . The values correspond to the following:

- 0: Healthy
- 1: Emergency Stop
- 2: Protective Stop
- 255: Unknown

/diagnostics ([diagnostic_msgs/DiagnosticArray](#))

[Diagnostics](#) is a mechanism for integrating ROS error information. This topic is published in the "/diagnostics" namespace as an exception .

13.7.5.4 Services

`clear_e_stop` (`std_srvs/Trigge`)

Requests recovery from Emergency Stop .

`clear_p_stop` (`std_srvs/Trigge`)

Requests recovery from Protective Stop .

13.7.5.5 Parameters

`publish_rate` (double)

The publication period [Hz] of `safety_command_display` .

`non_rt_thread_rate` (double)

The execution cycle [Hz] of the non-real-time thread running in the Controller .

* The detection cycle for the `self_collision_monitor` also follows this cycle, so basically do not change the setting from the default value of 100[Hz] .

13.7.5.6 Reconfigurable Parameters

None

13.7.6 `effort_controllers/ToroboJointTrajectoryController`

This is a controller for controlling the joint position of a joint trajectory. A joint trajectory must consist of one or more waypoints, and these waypoints must have a target position and a target arrival time. A waypoint may also specify a target velocity and/or target acceleration, if necessary.

The controller attempts to achieve the given target joint trajectories as far as hardware constraints allow.

13.7.6.1 Action Interface

`follow_joint_trajectory` (`control_msgs/FollowJointTrajectoryAction`)

Enter the target trajectory when using the Action Interface . For more information on how to use it, please refer to [Joint Position Command \(Using Ros Actionlib \)](#).

13.7.6.2 Subscribed Topics

`command` (`trajectory_msgs/JointTrajectory`)

Enter the target trajectory when commanding via topic communication. For detailed usage, please refer to [Joint Position Command \(Using Ros Publish \)](#).

13.7.6.3 Published Topics

`state` (`control_msgs/JointTrajectoryControllerState`)

The current state of the controller.

13.7.6.4 Services

None

13.7.6.5 Parameters

`joints` (`string[]`)

A list of the names of the joints to be controlled.

`constraints/goal_time` (`double`, default: 0.0)

The allowable error in the time of arrival at the final target position.

`constraints/stopped_velocity_tolerance` (`double`, default: 0.01)

The speed range within which the speed is considered to be zero.

`constraints/<joint>/goal` (`double`, default: 0.0)

The tolerance of the destination position relative to the final target position of the trajectory.

`constraints/<joint>/trajectory` (`double`, default: 0.0)

The tolerance of the arrival position relative to the target position on the orbit.

`allow_partial_joints_goal` (`bool`, default: False)

Whether to accept target joint trajectories for only some joints.

13.7.6.6 Reconfigurable Parameters

None

13.7.7 `effort_controllers/ToroboJointImpedanceController`

This is a controller that makes the joint follow the target impedance. The target impedance is expressed as a spring-mass-damper system, and the spring constant, inertia, and damper constant can be changed. By making the target impedance soft, the joint can move in a way that follows an external force, and by making it hard, the joint can move in a way that barely moves even when an external force is applied.

You can also specify a desired joint trajectory. Each joint is torque-controlled to follow the desired impedance connected to the desired joint trajectory. A joint trajectory must consist of one or more via points, which must have a desired position and a desired arrival time. A via point may also specify a desired velocity and/or acceleration, if necessary.

13.7.7.1 Action Interface

`follow_joint_trajectory` (`control_msgs/FollowJointTrajectoryAction`)

Enter the target trajectory when using the Action Interface . For detailed usage, please refer to Joint Position Command (Using [Ros Actionlib](#)).

13.7.7.2 Subscribed Topics

`command` (`trajectory_msgs/JointTrajectory`)

This is the target trajectory when commanding via topic communication. For detailed usage, please refer to [Joint Position Command \(Using Ros Publish \)](#).

`effort_command` (`std_msgs/Float64MultiArray`)

Target external torque of the joint [Nm] . Each joint is controlled with the target torque being the sum of the target external torque and the torque required to achieve the target impedance.

13.7.7.3 Published Topics

`state` (`control_msgs/JointTrajectoryControllerState`)

The current state of the controller.

`current_external_effort` (`std_msgs/Float64MultiArray`)

Current external torque [Nm] . External torque is the torque sensor value minus dynamic torque. Dynamic torque is calculated from the robot dynamics model.

13.7.7.4 Services

None

13.7.7.5 Parameters

joints (string[])

A list of the names of the joints to be controlled.

constraints/goal_time (double, default: 0.0)

This is the allowable error in the time of arrival at the final target position.

constraints/stopped_velocity_tolerance (double, default: 0.01)

The speed range within which the speed is considered to be zero.

constraints/<joint>/goal (double, default: 0.0)

The tolerance of the destination position relative to the final target position of the trajectory.

constraints/<joint>/trajectory (double, default: 0.0)

The tolerance of the arrival position relative to the target position on the orbit.

allow_partial_joints_goal (bool, default: False)

Whether to accept target joint trajectories for only some joints.

impedance_parameters/<joint>/mass (double)

The mass inertia of the target impedance [kg m²] . By default, this parameter is not used and the mass inertia is set to the actual inertia value around the joint axis.

impedance_parameters/<joint>/spring (double)

The spring constant of the target impedance spring [Nm/rad] .

impedance_parameters/<joint>/damping (double)

Damper constant of the target impedance damper [Nm s/rad] .

impedance_parameters/<joint>/force_gb_gain (double)

The force_fb_gain parameter is currently unused.

13.7.7.6 Reconfigurable Parameters

impedance_parameters/<joint>/mass (double)

Inertia of the mass of the target impedance [kg m²] . This parameter is linked to impedance_parameters/<joint>/mass in Parameters .

impedance_parameters/<joint>/spring (double)

The spring constant of the target impedance spring [Nm/rad] . This parameter is linked to impedance_parameters/<joint>/spring in Parameters .

impedance_parameters/<joint>/damping (double)

The damping constant of the target impedance damper [Nm s/rad] . This parameter is linked to impedance_parameters/<joint>/damping in Parameters.

13.7.8 effort_controllers/GravityCompensationController

This is a controller for self-weight compensation control. Each joint outputs gravity torque calculated from the dynamics model of the robot. The user can move the joints of the robot while directly interacting with it.

Danger! 	If the gravity torque in the dynamics model differs greatly from the actual gravity torque, the joints may move suddenly. An example of a situation where this is likely to occur is when a hand is present at the end of the arm in the model but is not actually attached. We recommend that you check that the gravity torque in the dynamics model and the torque sensor value are close while the robot is stationary before turning on the servo . The gravity torque in the dynamics model can be confirmed in the gravity_effort field of the /torobo/torobo_state Topic . The torque sensor value can be confirmed in the link_effort field of the /torobo/torobo_state Topic .
---	---

13.7.8.1 Action Interface

None

13. Software **Specifications**

13.7.8.2 Subscribed Topics

None

13.7.8.3 Published Topics

None

13.7.8.4 Services

None

13.7.8.5 Parameters

joints (string[])

A list of the names of the joints to be controlled.

13.7.8.6 Reconfigurable Parameters

None

13.7.9 **effort_controllers/ToroboVelocityController**

This is a controller that controls the speed of the joints.

13.7.9.1 Action Interface

None

13.7.9.2 Subscribed Topics

command (std_msgs/Float64MultiArray)

This is the target velocity of the joint. You need to command the target velocity of all joints controlled by the torobo_velocity_controller.

13.7.9.3 Published Topics

None

13.7.9.4 Services

None

13.7.9.5 Parameters

joints (string[])

A list of the names of the joints to be controlled.

enable_velocity_smoothing (bool)

This parameter determines whether or not to smooth the joint velocity command value so that the joint velocity command value maintains the maximum acceleration. If the joint target velocity set in command topic , etc. is used as the joint velocity command value as is, the velocity command value will be stepped and the target acceleration will become infinite. By setting this to True, the joint velocity command value can be smoothed. The specific processing is as follows:

If set to True , the velocity command value is smoothed to reach the joint target velocity with maximum acceleration. If set to False, the joint target velocity becomes the joint velocity command value as it is.

max_acceleration/<joint> (double) The maximum acceleration of a joint, used if 'enable_velocity_smoothing' is True .

13.7.9.6 Reconfigurable Parameters

smoothing_param/enable_velocity_smoothing (bool)

This parameter determines whether to smooth the joint velocity command value so that the joint velocity command value maintains the maximum acceleration. This parameter is linked to enable_velocity_smoothing in Parameters .

smoothing_param/<joint>_max_acceleration (double) The maximum acceleration of the joint, used when "enable_velocity_smoothing" is True . This parameter is linked to max_acceleration/<joint> in Parameters.

13.7.10 torobo_controllers/ToroboOnlineJointTrajectoryController

This is a controller that makes the joint follow the target position, speed, and acceleration. When the target position, speed, and acceleration are received , it controls to reach the target state in the shortest time possible while maintaining the maximum jerk, maximum acceleration, maximum speed, and range of motion of the joint . Due to this property, torobo_online_joint_trajectory_controller is suitable for high-frequency commands such as visual servoing.

The differences from joint_trajectory_controller are as follows:

13. Software **Specifications**

- The joint_trajectory_controller receives a desired trajectory consisting of multiple waypoints. Then, before executing the movement, it generates a trajectory that interpolates between the desired trajectories. It is then controlled to follow the generated trajectory. Due to this nature, if a new desired trajectory is received while the trajectory is being executed, a sudden acceleration may occur.
- The online_joint_trajectory_controller receives only the final target point. The target position, velocity, and acceleration to move to the target point are generated every control cycle. Even if the target point changes during movement, the target position, velocity, and acceleration can be updated immediately, allowing smooth tracking.

13.7.10.1 Action Interface

None

13.7.10.2 Subscribed Topics

command (trajectory_msgs/JointTrajectory)

This is the target trajectory. However, there must be only **one** waypoint. If two or more waypoints are set, an error will be output and the joint will not move.

13.7.10.3 Published Topics

state (control_msgs/JointTrajectoryControllerState)

The current state of the controller.

13.7.10.4 Services

None

13.7.10.5 Parameters

joints (string[])

A list of the names of the joints to be controlled.

allow_partial_joints_goal (bool)

Whether to accept target joint trajectories for only some joints.

filter_param/default_speed_override/<joint> (double)

This is the override of the movement. The jerk, acceleration, and speed when moving towards the target point will be the "Maximum value x Override" value. Please enter a value in the range of 0.001 - 1.0 . For safety reasons, it is recommended to set the default value to a small value.

13.7.10.6 Reconfigurable Parameters

controller_params/allow_partial_joints_goal (bool)

Whether or not to accept target joint trajectories for only some of the joints. This parameter is linked to allow_partial_joints_goal in Parameters.

override_params/<joint>_speed_override (double)

This is the override for the movement. The jerk, acceleration, and speed when moving towards the target point will be the "Maximum value x Override" value for each. Enter a value in the range of 0.001 - 1.0 . This parameter is linked to Parameters filter_param /default_speed_override/<joint> .

13.7.11 torobo_controllers/ToroboOnlineJointImpedanceController

This controller combines ToroboJointImpedanceController and ToroboOnlineJointTrajectoryController . When it receives the target position, velocity, and acceleration of the joint, it generates the target position, velocity, and acceleration for each control cycle so that the target state is reached in the shortest time while maintaining the maximum jerk, maximum acceleration, maximum speed, and range of motion of the joint.

Each joint is controlled to follow the target impedance expressed by a spring-mass-damper system. The target position (natural length position of the spring) and target speed/acceleration of the spring-mass-damper system are generated every control cycle.

By using this controller, in applications that require high-frequency joint angle commands, such as teleoperation, This allows for gentle contact with objects.

13.7.11.1 Action Interface

None

13.7.11.2 Subscribed Topics

command (trajectory_msgs/JointTrajectory)

This is the target trajectory. However, there must be only **one** waypoint. If two or more waypoints are set, an error will be output and the joint will not move.

effort_command (std_msgs/Float64MultiArray)

Target external torque of the joint [Nm] . Each joint is controlled with the target torque being the sum of the target external torque and the torque required to achieve the target impedance.

13. Software **Specifications**

13.7.11.3 Published Topics

state (`control_msgs/JointTrajectoryControllerState`)

The current state of the controller.

13.7.11.4 Services

None

13.7.11.5 Parameters

joints (`string[]`)

A list of the names of the joints to be controlled.

allow_partial_joints_goal (`bool`)

Whether to accept target joint trajectories for only some joints.

command_effort_filter_alpha (`double`, default: 0.1)

This is the coefficient of the low-pass filter applied to the torque command of each joint.

filter_param/default_speed_override/<joint> (`double`)

This is the override of the movement. The jerk, acceleration, and speed when moving towards the target point will be the "Maximum value x Override" value. Please enter a value in the range of 0.001 - 1.0 . For safety reasons, it is recommended to set the default value to a small value.

impedance_parameters/<joint>/mass (`double`)

The mass inertia of the target impedance [kg m^2] . By default, this parameter is not used and the mass inertia is set to the actual inertia value around the joint axis.

impedance_parameters/<joint>/spring (`double`)

The spring constant of the target impedance spring [Nm/rad] .

impedance_parameters/<joint>/damping (`double`)

Damper constant of the target impedance damper [Nm s/rad] .

13.7.11.6 Reconfigurable Parameters

controller_params/allow_partial_joints_goal (bool)

Whether or not to accept target joint trajectories for only some of the joints. This parameter is linked to allow_partial_joints_goal in Parameters.

controller_params/command_effort_filter_alpha (double)

The coefficient of the low-pass filter applied to the torque command of each joint. This parameter is linked to the command_effort_filter_alpha parameter in Parameters.

override_params/<joint>_speed_override (double)

This is the override for the movement. The jerk, acceleration, and speed when moving towards the target point will be the "Maximum value x Override" value for each. Enter a value in the range of 0.001 - 1.0 . This parameter is linked to Parameters filter_param /default_speed_override/<joint> .

impedance_parameters/<joint>/mass (double)

Inertia of the mass of the target impedance [kg m²] . This parameter is linked to impedance_parameters/<joint>/mass in Parameters .

impedance_parameters/<joint>/spring (double)

The spring constant of the target impedance spring [Nm/rad] . This parameter is linked to impedance_parameters/<joint>/spring in Parameters .

impedance_parameters/<joint>/damping (double)

The damping constant of the target impedance damper [Nm s/rad] . This parameter is linked to impedance_parameters/<joint>/damping in Parameters.

13.7.12 torobo_controllers/CartesianImpedanceController

This controller makes the TCP (Tool Center Point) track the target position, posture, speed, acceleration, force, and impedance in the Cartesian coordinate system. This controller only works with robots that have torque sensors mounted on the manipulator joints.

The target impedance is expressed as a spring-mass-damper system, and the spring constant and damper constant can be changed. Note that the mass cannot be changed, as the value calculated from the robot's hardware configuration is automatically used. By making the target impedance soft, the TCP can move in a way that mimics an external force, and by making it hard, the TCP can move in a way that mimics an external force . It allows you to achieve movements that are almost motionless.

It is also possible to give a target TCP trajectory. In the controller, the torque command value for each joint that follows the target impedance is calculated from the external force calculated from the measurement value of the joint torque sensor and the target TCP trajectory, and the torque of each joint is controlled according to them. The TCP trajectory must consist of one or more waypoints, and these waypoints must have a target position and a target arrival time. You may specify a target speed, acceleration, or force for the waypoints as necessary.

13. Software Specifications

Only linear link (serial link) manipulators are supported as the controlled object, and the joints included in the linear chain formed by the root_link and tip_link given as parameters of this controller are the controlled object. Note that if the number of joints included in the linear chain is 7 or more, it has redundant degrees of freedom, so null space control is possible (excluding the vicinity of singular configurations).

Please note that near singular postures, control operation is likely to become unstable, and the robot may run out of control or move at high speed. When using a Cartesian-related controller, in case the robot runs out of control, do not enter the robot's moving range as much as possible, and always keep the emergency stop button ready to be pressed instantly at any time.

	Warning: Cartesian-related controllers tend to become unstable when in the vicinity of a singular posture, which may cause the robot to run out of control or move at high speed. When using the robot, please avoid entering the robot's moving range as much as possible and always keep the emergency stop button ready to be pressed instantly at any time.
---	---

13.7.12.1 Action Interface

None

13.7.12.2 Subscribed Topics

cart_command ([cartesian_trajectory_msgs/CartesianTrajectory](#))

These are the target position, posture, speed, acceleration, and force trajectory of TCP . Note that only link names included in the linear link system to be controlled can be specified in header.frame_id .

cart_imp_command ([cartesian_trajectory_msgs/CartesianImpedanceTrajectory](#))

TCP target position, posture, speed, acceleration, force, and impedance trajectory. Note that only link names included in the linear link system to be controlled can be specified in header.frame_id .

nullspace_command ([trajectory_msgs/JointTrajectory](#))

Enter the target trajectory for null space joint control. However, there must be only one waypoint. If two or more waypoints are set, an error will be output and the joint will not move. This controller will generate a null space torque to follow the joint trajectory specified here as closely as possible.

13.7.12.3 Published Topics

state ([control_msgs/JointTrajectoryControllerState](#))

Information on the position, velocity, acceleration, and torque of each joint.

cart_state ([cartesian_trajectory_msgs/CartesianTrajectoryControllerState](#))

TCP position, orientation, velocity, acceleration, and force information.

cart_imp_state ([cartesian_trajectory_msgs/CartesianImpedanceControllerState](#))

TCP impedance information.

`cart_pose_desired` ([geometry_msgs/TwistStamped](#))

This is the target position and orientation information of the TCP , whose orientation is expressed in axis -angle representation .

`cart_pose_actual` ([geometry_msgs/TwistStamped](#))

This is the current position and attitude of the TCP , expressed in axis -angle representation .

`cart_wrench_desired` ([geometry_msgs/WrenchStamped](#))

TCP target force information.

`cart_wrench_actual` ([geometry_msgs/WrenchStamped](#))

Current target force information for TCP .

`manipulability` ([std_msgs/Float64](#))

Manipulability information of the manipulator.

`cart_manipulability` ([geometry_msgs/TwistStamped](#))

Manipulability information of the manipulator expressed for each Cartesian dimension.

13.7.12.4 Services

`reset` ([std_srvs/Trigger](#))

Requests a reset of the controller state.

`cmd_vel_display`

Requests a change to the tcp_frame .

`set_tcp_frame` ([torobo_std_msgs/SetString](#))

Requests a change to the tcp_frame .

`set_impedance_frame` ([torobo_std_msgs/SetString](#))

Requests a change to impedance_frame .

13.7.12.5 Parameters

`joints` (string[])

This is a list of the names of the joints to be controlled. Note that it is necessary to specify the names of all the joints included in the linear link system to be controlled.

`root_link` (string)

Specify the name of the root link that constitutes the linear link system to be controlled.

`tip_link` (string)

Specify the name of the link on the end-point side that constitutes the linear link system to be controlled.

tcp_frame (string)

Specify the link name to be configured as TCP .

gravity_vector (double[3])

The gravity vector used for dynamics calculations. The gravity vector is described in the same coordinate system as the link specified by root_link .

max_translational_velocity (double)

The maximum translational speed of TCP [m/s] .

max_rotational_velocity (double)

The maximum rotational angular velocity of the TCP [rad/s] .

max_translational_acceleration (double)

The maximum translational acceleration of TCP [m/s²] .

max_rotational_acceleration (double)

The maximum rotational angular acceleration of TCP [rad/s²] .

singularity_low_sensitive_lambda0 (double)

This is the scale factor for the Damped Least-Squares method . The larger this value is, the less sensitive it becomes to singular points, so excessive speed and torque commands near singular points are suppressed, but the tracking performance of TCP deteriorates.

singularity_low_sensitive_w0 (double)

This is the manipulability threshold of the Damped Least -Squares method . When the manipulability of the manipulator falls below this parameter, the Damped Least-Squares method is enabled.

impedance_param/cart_stiffness (double[6])

Cartesian stiffness. Describe in the order of x , y, z, rx, ry, rz . The values of x , y, and z are specified in [N/m], and the values of rz, ry, and rz are specified in [Nm/rad] .

impedance_param/cart_damping (double[6])

Cartesian viscosity. Describe in the order of x , y, z, rx, ry, rz . The values of x , y, and z are specified in [N s/m], and the values of rz, ry, and rz are specified in [Nm s/rad] .

impedance_param/null_stiffness (double)

Null space joint control stiffness.

impedance_param/null_damping (double)

Viscous nature of null space joint control.

joint_limits_avoidance_param/enable_joint_limits_avoidance (bool)

Whether or not to enable the joint position limit avoidance function that uses null space.

joint_limits_avoidance_param/limit_margin (double)

This is the margin for joint position limit avoidance function using null space. When the difference (absolute value) between the current position of each joint and the limit position falls below this parameter value, a joint torque is generated that moves the joint away from the limit position.

joint_limits_avoidance_param/stiffness (double)

This is the stiffness of the joint position limit avoidance function using null space. When the difference (absolute value) between the current position of each joint and the limit position falls below this parameter value, a repulsive force is generated against the joint torque.

virtual_wall_avoidance/enable_virtual_wall_avoidance (bool)

This determines whether or not to enable the virtual wall avoidance function that uses null space.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/type (string)

The type of virtual wall in the virtual wall avoidance function that uses null space. You can specify either cylinder or sphere .

virtual_wall_avoidance/virtual_walls/<virtual_wall>/frame_id (string)

This is the frame name that serves as the reference coordinate system for the virtual wall in the virtual wall avoidance function that uses null space.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/repulsive_type (int)

This is the type of repulsive force of the virtual wall in the virtual wall avoidance function that uses null space. Specifying 0 generates a repulsive force against the inside of the virtual wall, and specifying 1 generates a repulsive force against the outside of the virtual wall.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/enable_repulsive_force (bool)

This determines whether or not to enable the generation of repulsive force from virtual walls in the virtual wall avoidance function that uses null spaces.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/cylinder_origin (double[3])

The origin of the cylindrical virtual wall in the virtual wall avoidance function that uses null space. This can only be specified when the virtual wall type is specified as cylinder .

virtual_wall_avoidance/virtual_walls/<virtual_wall>/cylinder_axis (double[3])

This is a vector that represents the axis direction of the cylindrical virtual wall in the virtual wall avoidance function that uses null space. It can be written only when the type of the virtual wall is specified as cylinder .

virtual_wall_avoidance/virtual_walls/<virtual_wall>/cylinder_radius (double)

The radius of the cylindrical virtual wall in the virtual wall avoidance function that uses null space. This can only be specified when the virtual wall type is specified as cylinder .

virtual_wall_avoidance/virtual_walls/<virtual_wall>/sphere_origin (double[3])

13. Software **Specifications**

This is the origin of the spherical virtual wall in the virtual wall avoidance function that uses null space. It can only be written when the type of virtual wall is specified as sphere .

virtual_wall_avoidance/virtual_walls/<virtual_wall>/sphere_radius (double)

This is the radius of the spherical virtual wall in the virtual wall avoidance function that uses null space. This can only be specified when the type of virtual wall is specified as sphere .

virtual_wall_avoidance/virtual_walls/<virtual_wall>/min_repulsive_force (dou-

became)

This is the maximum repulsive force from a virtual wall when using the null space to avoid virtual walls.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/max_repulsive_force (double)

This is the maximum repulsive force from a virtual wall when using the null space to avoid virtual walls.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/sigmoid_func_coeff (double)

This is the coefficient of the sigmoid function used to generate the repulsive force from a virtual wall in the virtual wall avoidance function that uses null space.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/target_frames/<item>/

name (string)

This is the name of the frame that applies the repulsive force from the virtual wall in the virtual wall avoidance function that uses null space.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/target_frames/<item>/

parent_link (string)

This is the parent link name of the frame that applies the repulsive force from the virtual wall in the virtual wall avoidance function that uses null space.

virtual_wall_avoidance/virtual_walls/<virtual_wall>/target_frames/<item>/

origin (string)

This is the origin of the frame on which the repulsive force from the virtual wall acts when using the null space to avoid the virtual wall.

nullspace_online_filter/filter_param/default_speed_override/<joint> (double)

This is an override for null space joint control. The jerk, acceleration, and velocity when moving towards the target point will be the "Maximum value x Override" value. Please enter a value in the range of 0.001 - 1.0 . For safety reasons, it is recommended to set the default value to a small value.

13.7.12.6 Reconfigurable Parameters

Same as Parameters .

13.7.13 torobo_controllers/OnlineCartesianImpedanceController

This controller is almost the same as CartesianImpedanceController . For more information, please refer to the following page.

[torobo_controllers/CartesianImpedanceController](#)

The difference with CartesianImpedanceController is that when this controller receives the target position, speed, acceleration, and force of TCP , it controls to reach the target state in the shortest time while maintaining the maximum jerk, maximum acceleration, maximum speed, and range of motion. Because of this property, it is suitable for high-frequency commands such as visual servoing.

13.7.13.1 Action Interface

None

13.7.13.2 Subscribed Topics

`cart_command` ([cartesian_trajectory_msgs/CartesianTrajectory](#))

These are the target position, posture, speed, acceleration, and force trajectory of the TCP . However, there must be only one via point. If two or more via points are set, an error will be output and the joint will not move. Note that only the link names included in the linear link system to be controlled can be specified in `header.frame_id` .

`cart_imp_command` ([cartesian_trajectory_msgs/CartesianImpedanceTrajectory](#))

These are the target position, posture, speed, acceleration, force, and impedance trajectory of the TCP . However, there must be only one via point. If two or more via points are set, an error will be output and the joint will not move. Note that only link names included in the linear link system to be controlled can be specified in `header.frame_id` . Also, the function to follow the impedance command value specified in this topic has not yet been implemented.

`nullspace_command` ([trajectory_msgs/JointTrajectory](#))

Enter the target trajectory for null space joint control. However, there must be only one waypoint. If two or more waypoints are set, an error will be output and the joint will not move. This controller will generate a null space torque to follow the joint trajectory specified here as closely as possible.

13. Software **Specifications**

13.7.13.3 Published Topics

Similar to CartesianImpedanceController .

13.7.13.4 Services

Similar to CartesianImpedanceController .

13.7.13.5 Parameters

This is almost the same as CartesianImpedanceController , but the parameters that only exist in this controller are described below.

range_x (double[2])

The movable range of the TCP in the X -axis direction [m] . Enter the minimum value, then the maximum value.

range_y (double[2])

The movable range of the TCP in the Y-axis direction [m] . Enter the minimum value, then the maximum value.

range_z (double[2])

This is the movable range of the TCP in the Z -axis direction [m] . Enter the minimum value, then the maximum value.

max_translational_jerk (double)

The maximum translational jerk of TCP [m/s^3] .

max_rotational_jerk (double)

The maximum rotational jerk of TCP [rad/s^3] .

default_speed_override (double)

This is the override of the movement. The jerk, acceleration, and speed when moving towards the target point will be the "Maximum value x Override" value. Please enter a value in the range of 0.001 - 1.0 . For safety reasons, it is recommended to set the default value to a small value.

13.7.13.6 Reconfigurable Parameters

Same as Parameters .

13.7.14 torobo_controllers/CartesianAdmittanceController

This controller makes the TCP (Tool Center Point) track the target position, posture, speed, acceleration, force, and impedance in the Cartesian coordinate system. This controller only works with robots equipped with a 6- axis force sensor at the tip of the manipulator.

The target impedance is expressed as a spring-mass-damper system, and the spring constant and damper constant can be changed. By making the target impedance soft, the TCP can move in a way that mimics an external force, and by making it hard, the TCP can move almost unaffected by an external force.

It is also possible to give a target TCP trajectory. In the controller, the position, velocity, and acceleration command values for each joint that follow the target impedance are calculated from the external force calculated from the measurement values of the 6- axis force sensor at the hand and the target TCP trajectory, and the position of each joint is controlled accordingly. The TCP trajectory must consist of one or more waypoints, and these waypoints must have a target position and a time to reach the target. You may specify target velocity, acceleration, and force for the waypoints as necessary.

Only linear link (serial link) manipulators are supported as the controlled object, and the joints included in the linear chain formed by the root_link and tip_link given as parameters of this controller are the controlled object. Note that if the number of joints included in the linear chain is 7 or more, it has redundant degrees of freedom, so null space control is possible (excluding the vicinity of singular configurations).

Please note that near singular postures, control operation is likely to become unstable, and the robot may run out of control or move at high speed. When using a Cartesian-related controller, in case the robot runs out of control, do not enter the robot's moving range as much as possible, and always keep the emergency stop button ready to be pressed instantly at any time.

	Warning: Cartesian-related controllers tend to become unstable when in the vicinity of a singular posture, which may cause the robot to run out of control or move at high speed. When using the robot, please avoid entering the robot's moving range as much as possible and always keep the emergency stop button ready to be pressed instantly at any time.
---	---

13.7.14.1 Action Interface

None

13.7.14.2 Subscribed Topics

Similar to CartesianImpedanceController .

13.7.14.3 Published Topics

This is almost the same as CartesianImpedanceController , but the topics that only exist in this controller are described below.

cart_command ([cartesian_trajectory_msgs/CartesianTrajectory](#))

These are the target position, posture, speed, acceleration, and force trajectory of the TCP . However, there must be only one via point. If two or more via points are set, an error will be output and the joint will not move. Note that only the link names included in the linear link system to be controlled can be specified in header.frame_id .

cart_adm_state ([cartesian_trajectory_msgs/CartesianTrajectoryControllerState](#))

These are the position, orientation, velocity, acceleration, and force information of the virtual mass point that the TCP follows in admittance control .

external_wrench ([geometry_msgs/WrenchStamped](#))

This is the external force information acting on the TCP calculated from the measurement values of the 6- axis force sensor.

13.7.14.4 Services

Similar to CartesianImpedanceController .

13.7.14.5 Parameters

This is almost the same as CartesianImpedanceController , but the parameters that only exist in this controller are described below.

ft_sensors (string[])

This is the URDF name of the 6- axis force sensor used for control .

admittance_param/admittance_command_damping_delay (double)

The number of seconds to determine the convergence of admittance control.

admittance_param/external_wrench_detect_threshold (double[6])

This is the threshold value that sets the dead zone of the 6- axis force sensor. Write in the order of x, y, z, rx, ry, rz .

admittance_param/damping_zone_pose_error_width (double[6])

This is the dead zone width of the TCP position and orientation that determines the convergence of admittance control . Describe in the order of x, y, z, rx, ry, rz .

admittance_param/damping_zone_twist_error_width (double[6])

This is the dead band width of the TCP speed that determines the convergence of admittance control . Describe in the order of x, y, z, rx, ry, rz .

admittance_param/command_twist_damping_step (double[6])

This is a damper constant to speed up the convergence of admittance control.

13.7.14.6 Reconfigurable Parameters

Same as Parameters .

13.7.15 torobo_controllers/OnlineCartesianAdmittanceController

This controller is almost the same as CartesianAdmittanceController . For more information, please see the following page.

[torobo_controllers/CartesianAdmittanceController](#)

The difference with CartesianAdmittanceController is that when this controller receives the target position, speed, acceleration, and force of TCP , it controls to reach the target state in the shortest time while maintaining the maximum jerk, maximum acceleration, maximum speed, and range of motion. Because of this property, it is suitable for high-frequency commands such as visual servoing.

13.7.15.1 Action Interface

None

13.7.15.2 Subscribed Topics

`cart_command` ([cartesian_trajectory_msgs/CartesianTrajectory](#))

These are the target position, posture, speed, acceleration, and force trajectory of the TCP . However, there must be only one via point. If two or more via points are set, an error will be output and the joint will not move. Note that only link names included in the linear link system to be controlled can be specified in `header.frame_id` . Also, the function to follow the impedance command value specified in this topic has not yet been implemented.

`cart_imp_command` ([cartesian_trajectory_msgs/CartesianImpedanceTrajectory](#))

These are the target position, posture, speed, acceleration, force, and impedance trajectory of the TCP . However, there must be only one via point. If two or more via points are set, an error will be output and the joint will not move. Note that only the link name included in the linear link system to be controlled can be specified in the `header.frame_id` .

`nullspace_command` ([trajectory_msgs/JointTrajectory](#))

Enter the target trajectory for null space joint control. However, there must be only one waypoint. If two or more waypoints are set, an error will be output and the joint will not move. This controller will generate a null space torque to follow the joint trajectory specified here as closely as possible.

13. Software **Specifications**

13.7.15.3 Published Topics

Similar to CartesianAdmittanceController .

13.7.15.4 Services

Similar to CartesianAdmittanceController .

13.7.15.5 Parameters

This is almost the same as CartesianAdmittanceController , but the parameters that only exist in this controller are described below.

range_x (double[2])

The movable range of the TCP in the X -axis direction [m] . Enter the minimum value, then the maximum value.

range_y (double[2])

The movable range of the TCP in the Y-axis direction [m] . Enter the minimum value, then the maximum value.

range_z (double[2])

This is the movable range of the TCP in the Z -axis direction [m] . Enter the minimum value, then the maximum value.

max_translational_jerk (double)

The maximum translational jerk of TCP [m/s^3] .

max_rotational_jerk (double)

The maximum rotational jerk of TCP [rad/s^3] .

default_speed_override (double)

This is the override of the movement. The jerk, acceleration, and speed when moving towards the target point will be the "Maximum value x Override" value. Please enter a value in the range of 0.001 - 1.0 . For safety reasons, it is recommended to set the default value to a small value.

13.7.15.6 Reconfigurable Parameters

Same as Parameters .

13.7.16 omni_drive_controller/OmniDriveController

This controller controls an omnidirectional mobile platform consisting of omni-wheels. It receives speed commands in the x, y, and yaw directions of the mobile platform on a two-dimensional plane as input, calculates the command rotational speed of each wheel, and drives the wheel joints. In order to achieve smooth wheel rotation, the calculated rotational speed command is passed through an acceleration/deceleration trajectory generator and then set as a command to the wheel joints. It also has an odometry (wheel odometry) function that calculates the amount of movement and speed of the mobile platform on a two-dimensional plane from the rotational speed obtained by the encoder of each wheel, and can be used to estimate the self-position of a robot.

13.7.16.1 Action Interface

None

13.7.16.2 Subscribed Topics

`cmd_vel` (`geometry_msgs/Twist`)

This is the target speed on a 2D plane given to the mobile cart. By default, the robot's "base_footprint" coordinate system is the reference coordinate system. The Twist message has six degrees of freedom in 3D space, but this controller only uses the three components linear.x, linear.y, and angular.z, and ignores the other elements. The units of each are [m/s] for translational speed (linear.x, linear.y) and [rad/s] for rotational speed (angular.z).

For safety reasons, if no new speed commands are published to the `cmd_vel` Topic for a certain period of time (0.5 [s]), the speed command to the moving cart will be reset to 0 and the cart will stop.

13.7.16.3 Published Topics

`cmd_vel_display` (`geometry_msgs/TwistStamped`)

This is the current speed command given to the moving cart on a 2D plane. The value subscribed to the `cmd_vel` Topic will be published as is.

`state` (`control_msgs/JointTrajectoryControllerState`)

Information on the speed and acceleration of each wheel.

`odom` (`nav_msgs/Odometry`)

This is odometry information calculated by sequentially integrating the speed information of each wheel.

`/tf` (`tf/tfMessage`)

The same odometry information as above. It is also published to the `/tf` Topic only if the `enable_tf` parameter is set to true.

13.7.16.4 Services

None

13.7.16.5 Parameters

`base_frame_id` (string)

Specify the `frame_id` to be used as the reference coordinate system for the moving cart .

`odom_frame_id` (string)

Specifies the `frame_id` when publishing the self-position obtained by odometry calculation to the `/tf` Topic .

`enable_tf` (bool)

Specify true if you want to publish the self-position obtained by odometry calculation to the `/tf` Topic . If the user performs self-position estimation by combining the wheel odometry information output by this controller with other information such as IMU or LiDAR , specify false to prevent conflict with the information, and do not publish to the `tf` Topic from this controller, or set `odom_frame_id` to a unique name.

`publish_rate` (double)

state The publishing frequency [Hz] to the topic .

`wheel_radius` (double)

The radius of each wheel [m] . The value is already set according to the robot specifications, so there is no need to change it.

`base_radius` (double)

This is the radius value [m] from the center of the mobile platform to the position of each wheel arranged in a circle . The value is already set according to the robot specifications, so there is no need to change it.

`number_of_wheels` (int)

Specify the number of wheels for the omniwheel. The value is already set according to the robot's specifications, so there is no need to change it.

`max_wheel_acceleration` (double)

This is the maximum acceleration [rad/s^2] of each wheel . It is referenced when the acceleration/deceleration trajectory generator generates a trajectory. If a large acceleration is specified, it will take less time to reach the commanded speed, but the entire robot will be more likely to exhibit vibratory behavior during acceleration/deceleration. If a small acceleration is specified, acceleration/deceleration will be smoother and vibrations of the entire robot will be reduced, but the braking distance will be longer. When changing this parameter, it is dangerous to suddenly change the parameter value drastically or give a large speed command, so please adjust the parameter and speed value gradually.

13.7.16.6 Reconfigurable Parameters

driver_params/max_wheel_acceleration (double)

The maximum acceleration of each wheel [rad/s²]. This parameter is linked to max_wheel_acceleration in Parameters.

driver_params/enable_wheel_velocity_smoothing (bool)

Whether or not to generate a trajectory using the wheel acceleration/deceleration trajectory generator.

13.7.17 torobo_controllers/ToroboHandJointTrajectoryController

This controller makes the hand joints follow the target position, speed, and acceleration. When the target position, speed, acceleration, and maximum torque are received, the controller controls the joints to reach the target state in the shortest time possible while maintaining the maximum torque, maximum jerk, maximum acceleration, maximum speed, and range of motion of the joints. The

online_joint_trajectory_controller is used inside the controller, and only the final target point is received. The target position, speed, and acceleration to move toward the target point are generated for each control cycle. Even if the target point changes during movement, the target position, speed, and acceleration can be updated immediately, allowing smooth tracking. The gripping force for each joint can be set by setting the

13.7.17.1 Action Interface

hand_cmd (control_msgs/FollowJointTrajectoryAction)

Enter the target trajectory when using the Action Interface to command. Effort allows you to set the maximum torque [Nm] for each joint. For detailed usage, refer to [Joint Position Command \(Using Ros Actionlib \)](#). However, there must be only **one** waypoint. If two or more waypoints are set, an error will be output and the joint will not move. If the joint trajectory does not reach the final target position or stops midway through movement, a stall will be detected and the Action will end.

13.7.17.2 Subscribed Topics

command (trajectory_msgs/JointTrajectory)

This is the target trajectory. In effort, you can set the maximum torque [Nm] for each joint. However, there must be only **one** waypoint. If two or more waypoints are set, an error will be output and the joint will not move.

13.7.17.3 Published Topics

state (control_msgs/JointTrajectoryControllerState)

The current state of the controller.

13.7.17.4 Services

None

13.7.17.5 Parameters

joints (string[])

A list of the names of the joints to be controlled.

allow_partial_joints_goal (bool)

Whether to accept target joint trajectories for only some joints.

default_max_effort (double)

This is the default value for the command maximum joint torque [Nm] . It is used when the maximum torque is not set in the effort of the target joint trajectory.

goal_tolerance (double)

The tolerance [rad] of the reached position relative to the final target position of the joint trajectory .

stalled_velocity_threshold (double)

The speed range [rad/s] within which the speed is considered to be zero .

stalled_timeout (double)

This is the time [s] from when the speed is considered to be zero until a stall is determined .

stalled_acceleration_threshold (double)

The acceleration threshold [rad/s²] used for stall detection . If the acceleration is greater than this threshold, the stall detection is suppressed.

stalled_command_goal_tolerance (double)

This is the tolerance [rad] of the target trajectory's arrival position relative to the final target position of the joint trajectory . It is used for stall judgment. If the target position trajectory enters this range before the joint trajectory reaches the final target position, a stall judgment is issued.

velocity_lpf_coefficient (double, default: 0.1)

This is the velocity low-pass filter coefficient calculated inside the controller. Please enter a value in the range of 0.0 - 1.0 . The smaller the value, the stronger the low-pass filter will be.

limit_position_tolerance (double, default: 1e-6)

The tolerance of the final target position relative to the joint limits [rad] .

filter_param/default_speed_override/<joint> (double)

This is the override of the operation. The jerk, acceleration, and speed when moving towards the target point will be the "Maximum value x Override" value. Please enter a value in the range of 0.001 - 1.0 .

13.7.17.6 Reconfigurable Parameters

controller_params/allow_partial_joints_goal (bool)

Whether or not to accept target joint trajectories for only some of the joints. This parameter is linked to allow_partial_joints_goal in Parameters.

controller_params/default_max_effort (double)

This is the default value for the maximum command joint torque [Nm] . This parameter is linked to the "default_max_effort" parameter in Parameters .

controller_params/goal_tolerance (double)

The tolerance [rad] of the target position of the joint trajectory relative to the final target position . This parameter is linked to the goal_tolerance parameter in Parameters.

controller_params/stalled_velocity_threshold (double)

The speed range [rad/s] where the speed is considered to be zero . This parameter is linked to the stalled_velocity_threshold in Parameters .

controller_params/stalled_timeout (double)

The time [s] from when the speed is considered to be zero until a stall is determined . This parameter is linked to the stalled_timeout parameter in Parameters.

controller_params/stalled_acceleration_threshold (double)

The acceleration threshold used to determine stall [rad/s²] . This parameter is linked to the stalled_acceleration_threshold parameter in Parameters .

controller_params/stalled_command_goal_tolerance (double)

The tolerance [rad] of the target trajectory position relative to the final target position of the joint trajectory . This parameter is linked to the parameter stalled_command_goal_tolerance .

override_params/<joint>_speed_override (double)

This parameter is linked to filter_param /default_speed_override/<joint> in Parameters .

13.7.18 gpio_controllers/GpioStateController

This is a controller that publishes the state of the general-purpose input/output terminal (GPIO) installed in the robot controller . For details about the hardware of the general-purpose input/output terminal, please refer to [General-purpose output terminal](#) .

13. Software **Specifications**

13.7.18.1 Action Interface

None

13.7.18.2 Subscribed Topics

None

13.7.18.3 Published Topics

analog_io ([std_msgs/Float64MultiArray](#))

Current state of the general-purpose analog I/O port. Each element is in [V] .

digital_io ([std_msgs/UInt8MultiArray](#))

The current state of the general-purpose digital I/O port. Each element can take two values: "0" or " 1 " .

gpio_states ([torobo_sensor_msgs/GpioStates](#))

The current state of all general purpose analog and digital I/O ports. Includes the contents of the analog_io and digital_io topics , plus the name and timestamp value of each port.

13.7.18.4 Services

None

13.7.18.5 Parameters

publish_rate (double)

Publish cycle [Hz] to the "analog_io", "digital_io", and "gpio_states" topics . The maximum cycle that can be set is approximately 160 [Hz] .

13.7.18.6 Reconfigurable Parameters

None

13.7.19 gpio_controllers/GpioController

This is a controller that controls the state of the general-purpose output terminal installed in the robot controller . For details about the hardware of the general-purpose input/output terminal, refer to [General-purpose output terminal](#) .

13.7.19.1 Action Interface

None

13.7.19.2 Subscribed Topics

`analog_io_command` (`std_msgs/Float64MultiArray`)

This is the output voltage value [V] to be commanded to the general-purpose analog output port . Please command it with an array with the same number of elements as the number specified in the `analog_out_ports` parameter.

`digital_io_command` (`std_msgs/UInt8MultiArray`)

This is the output state to be commanded to the general-purpose digital output port. Each element is specified with two values , "0" or " 1. " Please command with an array that has the same number of elements as the number specified in the `digital_out_ports` parameter.

`gpio_command` (`torobo_sensor_msgs/GpioStates`)

This is the output voltage [V] to be commanded to the general-purpose analog output , and the output state ("0" or "1") to be commanded to the general-purpose digital output. For each analog output and digital output, this is used in combination with the port name specified in the `analog_out_ports` or `digital_out_ports` parameter. The command value state is output according to the order of the port name array. The number of port names to be commanded and the number of command value elements must be commanded in an array of the same size.

13.7.19.3 Published Topics

None

13.7.19.4 Services

None

13.7.19.5 Parameters

`analog_out_ports` (`string[]`)

This is the name of the port to be set as the general-purpose analog output terminal. The value that matches the robot specifications has already been set, so there is no need to change it.

`digital_out_ports` (`string[]`)

This is the name of the port to be set as a general-purpose digital output terminal. The value that matches the robot specifications has already been set, so there is no need to change it.

13. Software **Specifications**

13.7.19.6 Reconfigurable Parameters

None

13.7.20 blank

13.8 Load setting function specifications

The control program for this robot contains its own robot model information, such as the robot's link mass, center of gravity position, and inertia matrix. This information is used to solve the equation of motion, and the gravity compensation torque, centrifugal force, Coriolis force, inertia force, and other torques (dynamics torques) for each joint in the robot's current posture are constantly calculated. Using these torques as compensation torques in the control model improves control performance, and comparing the dynamics torque with the value obtained from the torque sensor makes it possible to detect external forces more accurately.

However, when an additional load is applied to the robot's hand, such as when the robot's end effector grasps and carries an object, the difference between the dynamic torque that should be generated at the joint in the robot model calculated above and the torque value that actually acts on the joint becomes large. Therefore, this robot has a function to set additional load information to the hand, etc., and by setting the appropriate load information, it is possible to reduce the difference between the above model and the actual situation.

13.8.1 How to set the load on the hands, etc.

To set the load on the robot's fingers, etc., use the original ROS Services `/torobo/set_load` and `/torobo/unset_load`. For the message specifications, see `torobo_control_msgs / SetLoad.srv` and `torobo_control_msgs / UnsetLoad.srv`.

Links that can have load settings are those named "`~link_load`" in the URDF. Note that for links that are likely to have load settings, such as robot hands, the above settings are configured as the default.

13.8.2 How to check the current load setting

You can check the current load setting by subscribing to the `/torobo/load_inertia` Topic published by `torobo_state_controller/ToroboStateController`. This Topic has a message type of `torobo_control_msgs / LoadInertia`, and [contains](#) a list of links for which load can be set in the "`load_link_names`" field. The array in the "`inertia`" field contains load settings that correspond to the order of the link names in "`load_link_names`".

13.9 TeachingPointManager Specification

A teaching point (TP) is robot posture information that has been prepared in advance. The "home position" can also be considered a type of teaching point. Using teaching points makes it easy to command the robot to perform standard movements. From this point on, teaching points will be abbreviated to TP .

There are two types of TP : "joint angle TP" and "Cartesian TP." Joint angle TP is a TP that consists of angle information for each joint . For example, the home position is classified as a joint angle TP . Cartesian TP is a TP that consists of position and orientation information in the Cartesian (Cartesian coordinate system) . It is used when moving the tip of a robot's tool to a specific position and orientation.

TeachingPointManager provides functions such as registering and acquiring these TPs . Specifically, it provides the following functions in ROS: Provided as a service .

- Registering and deleting TP
- Get TP information by specifying the TP name
- Load TP from a YAML file
- Export TP to a YAML file

When using the functions of TeachingPointManager , it is easy to use the API "teaching_point_utils". teaching_point_utils is a set of functions that wrap the Client of the Ros Service provided by TeachingPointManager .

The following four points will be explained below.

- How to start TeachingPointManager
- Joint angle TP, Cartesian TP specifications
- Teaching_point_utils Features
- Format of the yaml file for recording TP

13.9.1 How to start TeachingPointManager

TeachingPointManager is automatically started when the robot core program is started. Therefore, no special operations are required. No work is required.

13.9.2 Specifications of Joint Angle TP and Cartesian TP

13.9.2.1 Joint angle TP specification

A joint angle TP consists of a TP name and joint angle information.

Joint angle information is sent in the ROS standard message type `sensor_msgs/JointState` The name field is a joint The field contains an array of angle names, and the position field contains an array of joint positions [rad] .

13. Software **Specifications**

13.9.2.2 Cartesian **TP** Specifications

A Cartesian TP consists of the TP name and Cartesian position and orientation information. The Cartesian position and orientation information takes the ROS message type [cartesian_trajectory_msgs/CartesianState](#) that we have defined ourselves . Of the above messages, only frame_id, tcp_frame, and position and orientation information are managed by TeachingPointManager .

13.9.3 **teaching_point_utils** Functions

set_joint_teaching_point(tp_name, joint_state, overwrite=False) Registers the joint angle TP to the ROS Parameter Server .

Parameters

- **tp_name** (str) – name of the TP
- **sensor_msgs/JointState** – Joint angle information consisting of joint name and joint angle [rad]
- **overwrite** (bool) – Whether to overwrite a TP with the same name if one is already registered.

Return value: Whether or not the TP was successfully registered.

Return type: bool

set_cartesian_teaching_point(tp_name, cartesian_state, overwrite=False) Registers a Cartesian TP to the ROS Parameter Server .

Parameters

- **tp_name** (str) – name of the TP
- **cartesian_trajectory_msgs/CartesianState** – Cartesian trajectory information
- **overwrite** (bool) – Whether to overwrite a TP with the same name if one is already registered.

Return value: Whether or not the TP was successfully registered.

Return type: bool

get_teaching_point(tp_name)

Gets TP information. If a non-existent TP name is specified, "None" is returned.

Parameters **tp_name** (str) – name of the TP

Return value TP information

Return type sensor_msgs/JointState or cartesian_trajectory_msgs/CartesianState or None

get_teaching_point_names()

Gets a list of TP names registered in the ROS Parameter Server . If there are no registered TPs , None is returned.

Parameters **tp_name** (str) – name of the TP

Return value: List of registered TP names

Return type list of str or None

delete_teaching_point(tp_name)

Deletes the specified TP from the ROS Parameter Server . If the specified TP does not exist, False is returned.

Parameters **tp_name** (str) – name of the TP

Return value: Whether or not the TP was successfully deleted.

Return type: bool

is_teaching_point_joint_state(tp)

Determines whether TP is a joint angle TP .

Parameter **tp** (sensor_msgs/JointState or cartesian_trajectory_msgs/
CartesianState) – Joint Angle TP or Cartesian TP

Whether the return value TP is a joint angle TP

Return type: bool

is_teaching_point_cartesian_state(tp)

Determines whether the TP is a Cartesian TP .

Parameter **tp** (sensor_msgs/JointState or cartesian_trajectory_msgs/
CartesianState) – Joint Angle TP or Cartesian TP

Returns whether the TP is a Cartesian TP or not.

Return type: bool

load_teaching_point_from_yaml(file_path, overwrite=False)

Registers the TPs described in the yaml in the specified file_path to the ROS Parameter Server . If at least one TP is successfully registered, True is returned.

Parameters

- **tp_name** (str) – name of the TP
- **overwrite** (bool) – Whether to overwrite a TP with the same name if one is already registered.

Return value: Whether or not the TP was successfully registered.

Return type: bool

dump_teaching_point_to_yaml(file_path, overwrite=False)

Outputs the TP registered in the ROS Parameter Server to the specified file_path .

Parameters

- **tp_name** (str) – name of the TP
- **overwrite** (bool) – Whether to overwrite if a yaml file with the same name already exists.

Return value Whether or not the TP was successfully output to a YAML file

13. Software Specifications

Return type: bool

13.9.4 Format of the **yml** file for recording TP

```
#Joint angle TP format joint_tp1:
joint_name: - # TP name. Please avoid using names that overlap with other TPs .
right_arm/joint_1 # Joint angle name.
- right_arm/joint_4 - right_arm/
joint_6 position:
#Joint position [rad] . The order must correspond to the order of the joint angle names.
- 0.1
- 0.2
- 0.3

#Format of joint angle TP # You
can also use brackets [] as follows . joint_tp2:
joint_name:
['left_arm/joint_2', 'left_arm/joint_5', left_arm/joint_7] position: [0.2, 0.4, 0.2]

#Cartesian TP format # TP name. Please
expresses avoid using names that overlap with other TPs . cart_tp1: # Coordinate system that
Point (TCP) is a Cartesian frame_id: base_link tcp_frame: 'right_hand/link_tcp' # Coordinate system of Tool Center
position. position:
- 0.1
- -0.2
- 0.3
quaternion_xyzw: #Cartesian posture.
- 0
- 0
- 0
- 1

#Cartesian TP format # You can also use
brackets [] as follows : cart_tp2: frame_id: 'right_arm/
link_0'
tcp_frame: 'right_hand/link_tip' position: [-0.4,
0.3, 0.3 ] quaternion_xyzw: [0.5, 0.5, -0.5, -0.5]
```

13.10 ToroboHandTeachingMotion Specifications

ToroboHandTeachingMotion allows you to create standard movements for the left and right hands and to control the left and right hands through GUI operations. Standard movements can be sent as position commands.

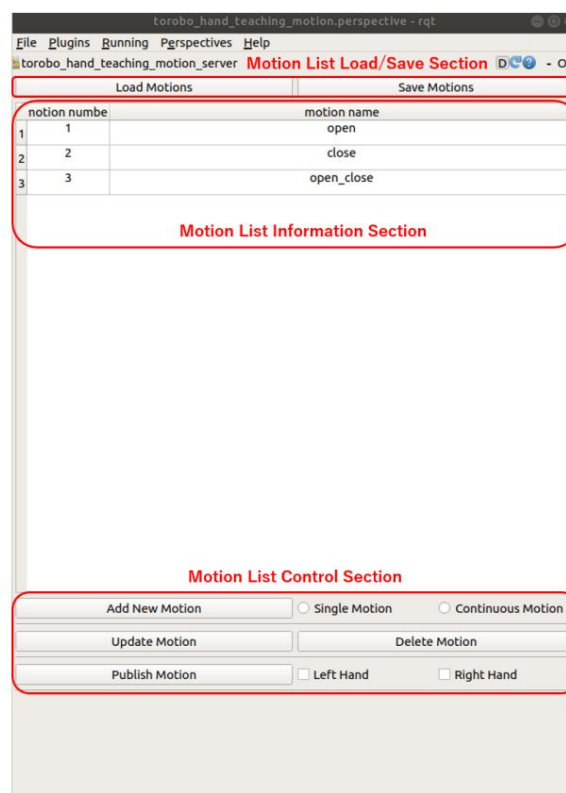
13.10.1 Usage

13.10.1.1 Starting method

To start the ToroboHandTeachingMotion GUI , execute the following command.

```
$ roslaunch torobo_hand_teaching_motion_server torobo_hand_teaching_motion_server.  
ylaunch
```

Then, the following screen should be displayed. However, the contents of the Motion List Information Section may differ depending on the loaded motion list . Please refer to About the [standard motion description](#) yml file for how to write and specify the [yaml](#) file to be loaded as a motion list .



13. Software Specifications

13.10.1.2 GUI Section Configuration

As shown in red in the image above, the torobo_hand_teaching_motion_server GUI consists of three sections .

Motion List Load/Save Section Load and save hand routines

It consists of two buttons for loading and saving hand routines .

Button Name	GUI behavior when pushing
Load Motions	Open the load/save PATH given when executing the above launch file, Reads a yaml file of a list of boilerplate actions .
Save Motions	Open the load/save PATH given when executing the above launch file, Save the current list of boilerplate actions as a yaml file.

Motion List Information Section A list of hand routines that are loaded.

A list of routines for the currently loaded hand will be displayed. The details displayed are as follows:

Name	content
motion number	The action number of the loaded hand routine is displayed.
motion name	The name of the action corresponding to the action number of the loaded hand routine is displayed. can be done.

The details of each action are displayed at the bottom of the window when you select the action number.

There are two types of motion: single motion and continuous motion . Single motion is the motion of the fingers of the hand.

A continuous movement is a movement that moves a joint to a target position. A continuous movement is a movement that executes multiple single movements in sequence.

The following are the actions that can be defined as a single action:

	input	content
Project motion number is required		The operation number of the loaded hand routine is displayed. can be done.
motion name	must	For the action number of the loaded hand routine The operation name is displayed.
Joint Positions	must	The target position of each joint [rad] . Enter the type value.
Joint Efforts	Not required	Effort target value [Nm] for each joint. Please enter a float type number. The maximum value is 1.5 . If the field is left blank, the hand_controller The max_effort is specified.

The following shows the operations that can be defined as continuous operations:

	input	content
Project motion number is	required	The operation number of the loaded hand routine is displayed. can be done.
motion name	must	For the action number of the loaded hand routine The operation name is displayed.
motion numbers are required		A combination of operation numbers that operate consecutively. Please enter an integer value for the operation number. (Example: 1, 2, 1)
motion duration time	must	This is the sleep time between operations when operating continuously . Please enter a float type number in the field . stomach.

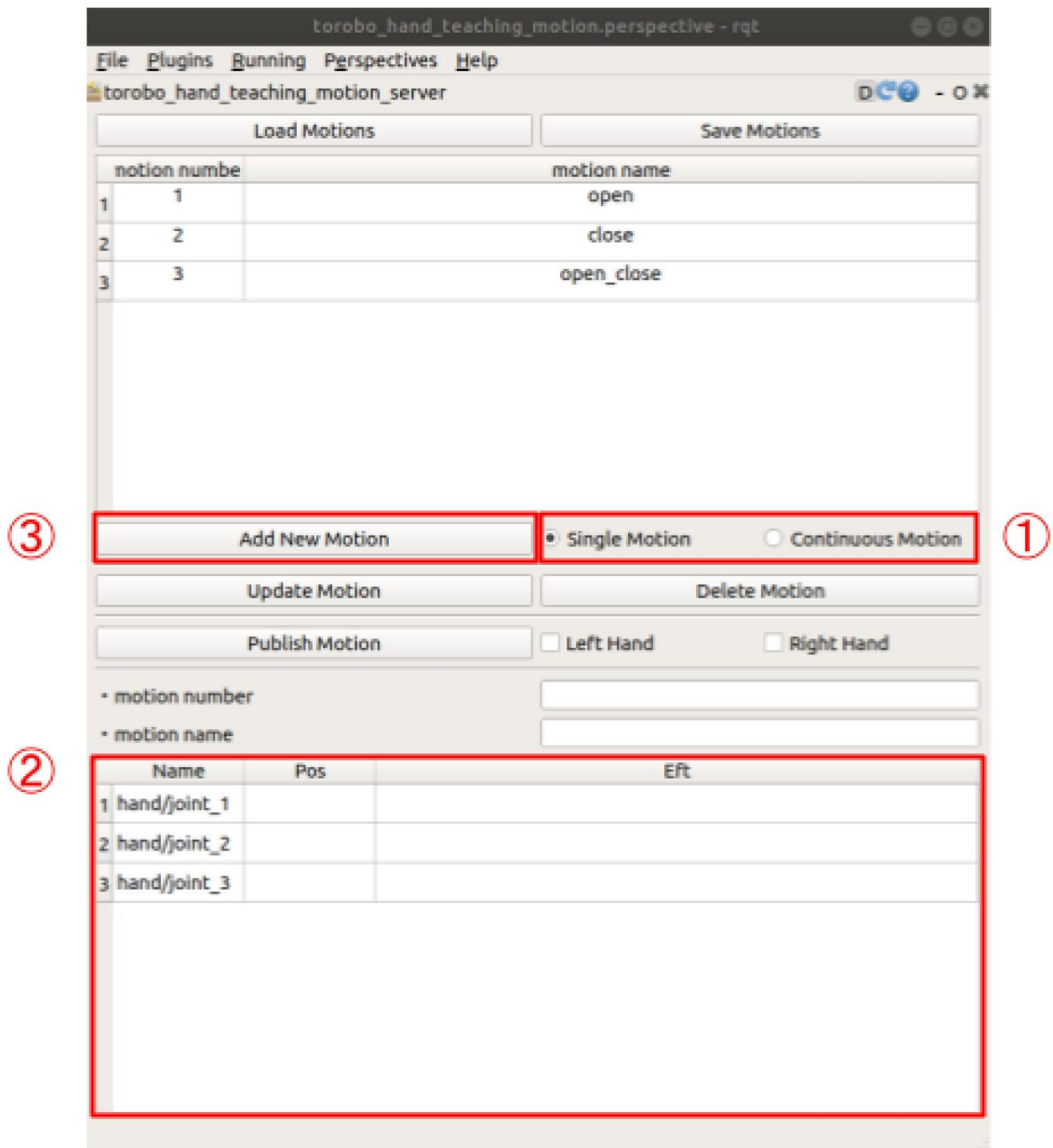
Motion List Control Section Add/update/delete hand routines/issue messages

This section consists of four buttons, each of which is explained below:

Button Name	Push processing
Add New Motion	Add a new routine. To add a routine, click the radio button on the right. Depending on the state of the switch , you can choose between single operation or continuous operation. is.
Update Motion	Update the details of the selected routine.
Delete Motion	Deletes the selected routine.
Publish Motion	The selected standard operation is issued as a position command value message. Depending on the state of the checkbox on the right, only one of the left and right messages will be sent. I will go.

13. Software Specifications

13.10.1.3 Adding a new action



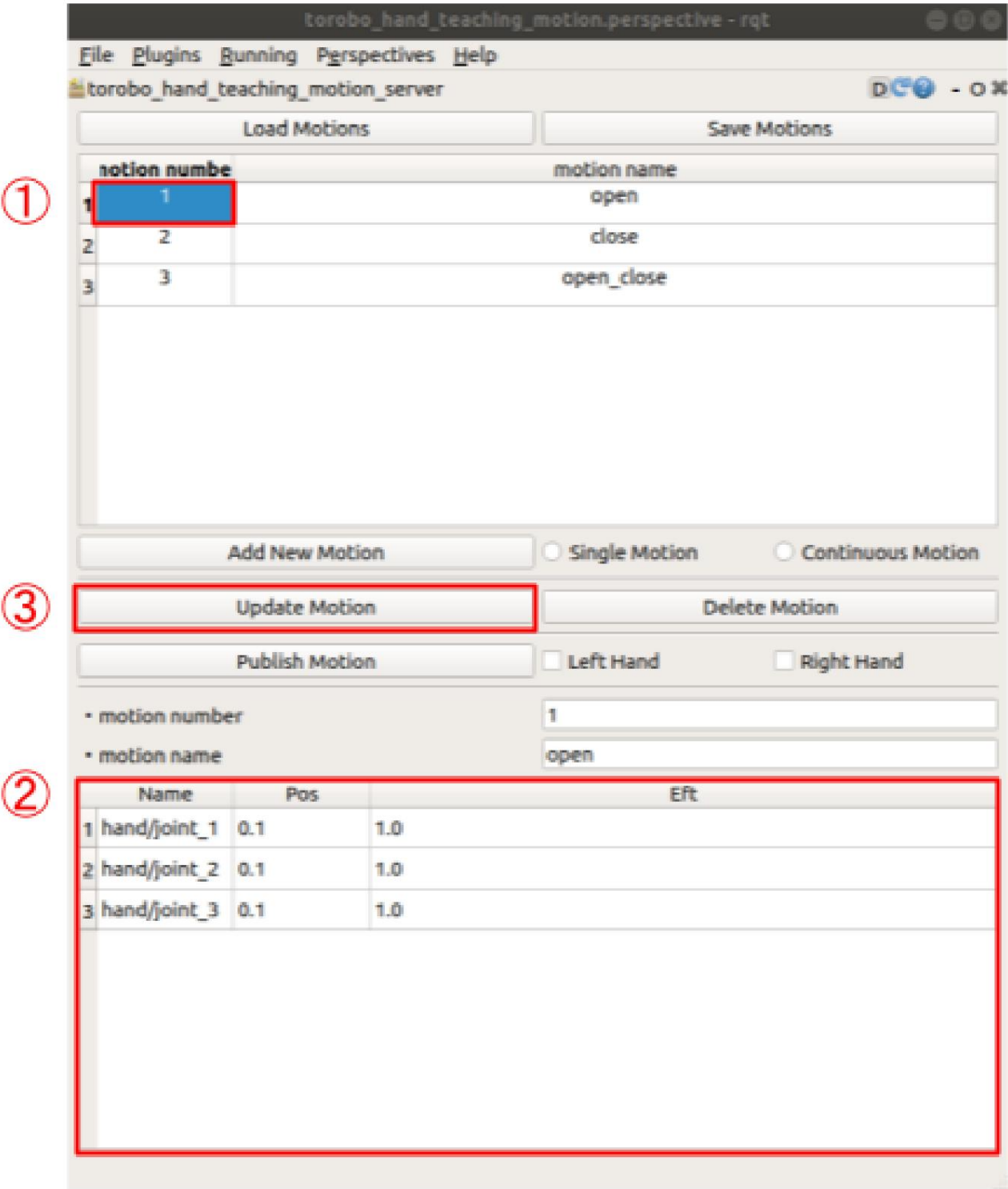
The following describes the procedure for adding a new action.

First, the newly added motion (red circle 1) is Single Motion or Continuous Motion .
Select whether you want to

Once selected, please enter details for each action at the bottom of the window, as shown in red circle 2 .

Finally, press the "Add New Motion" button (red circle 3) . After pressing the button, the motion list information
Make sure the action has been added to the section .

13.10.1.4 Update the operation



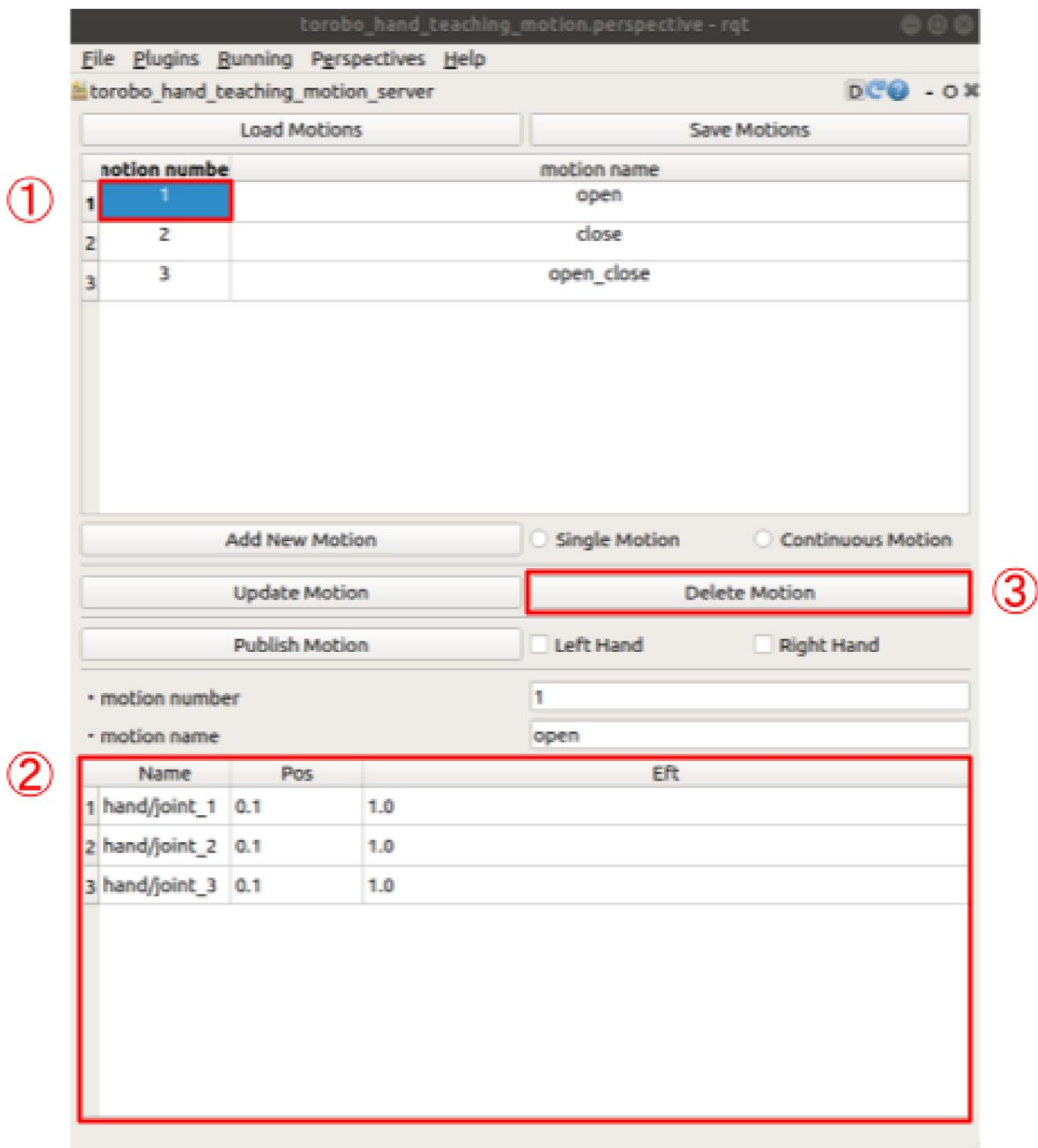
Here are the steps to update the behavior:

First, select the action number of the action you want to update, as shown in the red circle 1 .

When you select it, the details of each action will be displayed at the bottom of the window, as shown in the red circle 2. Edit the items you want to update.

After the update is complete, press the "Update Motion" button in the red circle 3. Pressing the button will update the motion.

13.10.1.5 Delete an action



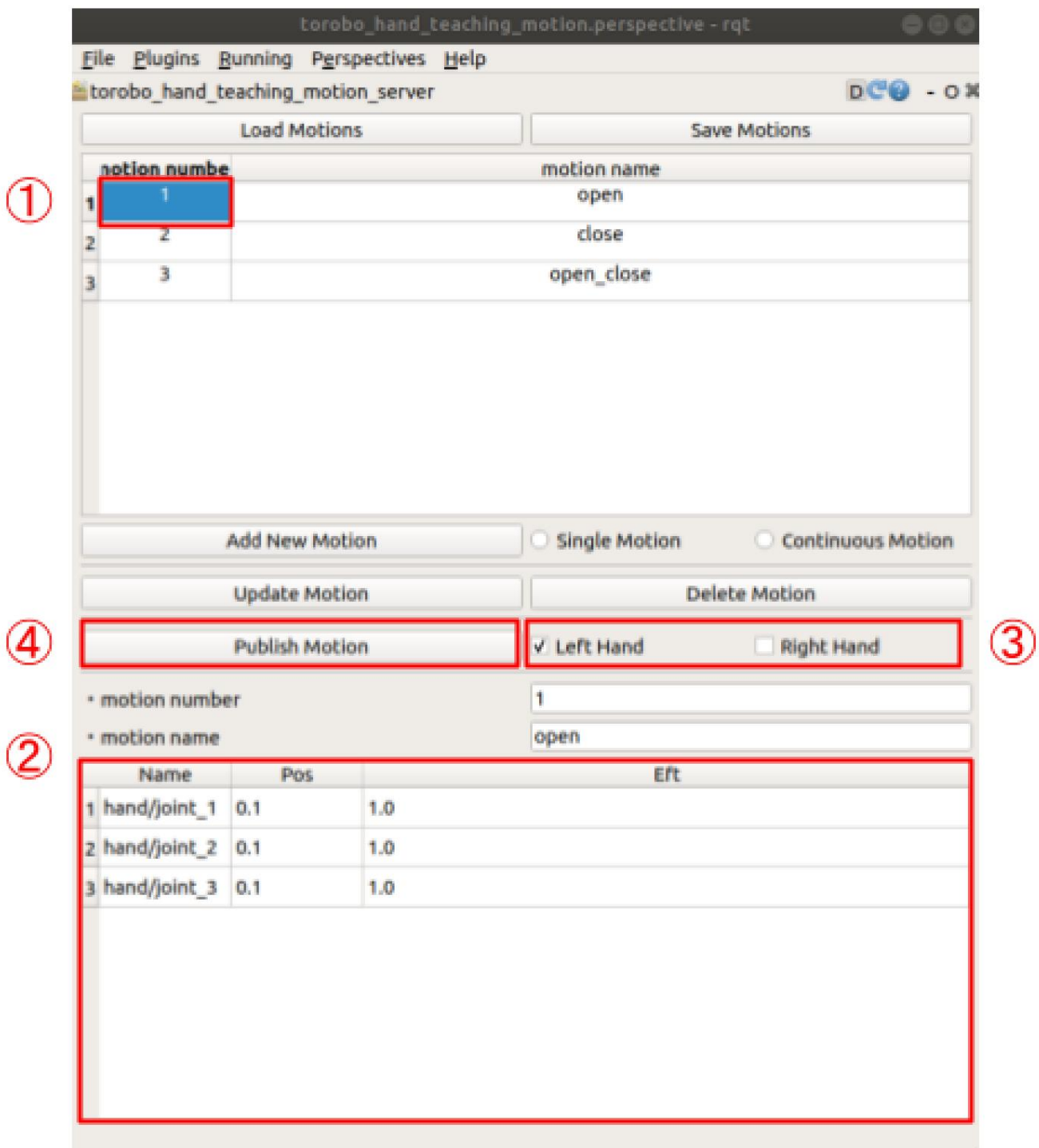
The procedure for deleting an action is as follows:

First, select the action number of the action you want to update, as shown in the red circle 1 .

When you select it, the details of each operation will be displayed at the bottom of the window as shown in the red circle 2. Please confirm the operation you want to delete.

After updating is complete, press the "Delete Motion" button circled in red 3. Pressing the button will delete the selected motion.

13.10.1.6 Issuing Action Messages



This section describes the procedure for issuing an action message.

First, select the action number of the action you want to update, as shown in the red circle 1 .

When you select it, the details of each operation will be displayed at the bottom of the window as shown in the red circle 2 . Please check.

After checking the operation details, select the hand for which you want to issue a message in the red frame of red circle 3. Please note that if both check boxes are not selected/both are selected, the message will not be issued.

13. Software **Specifications**

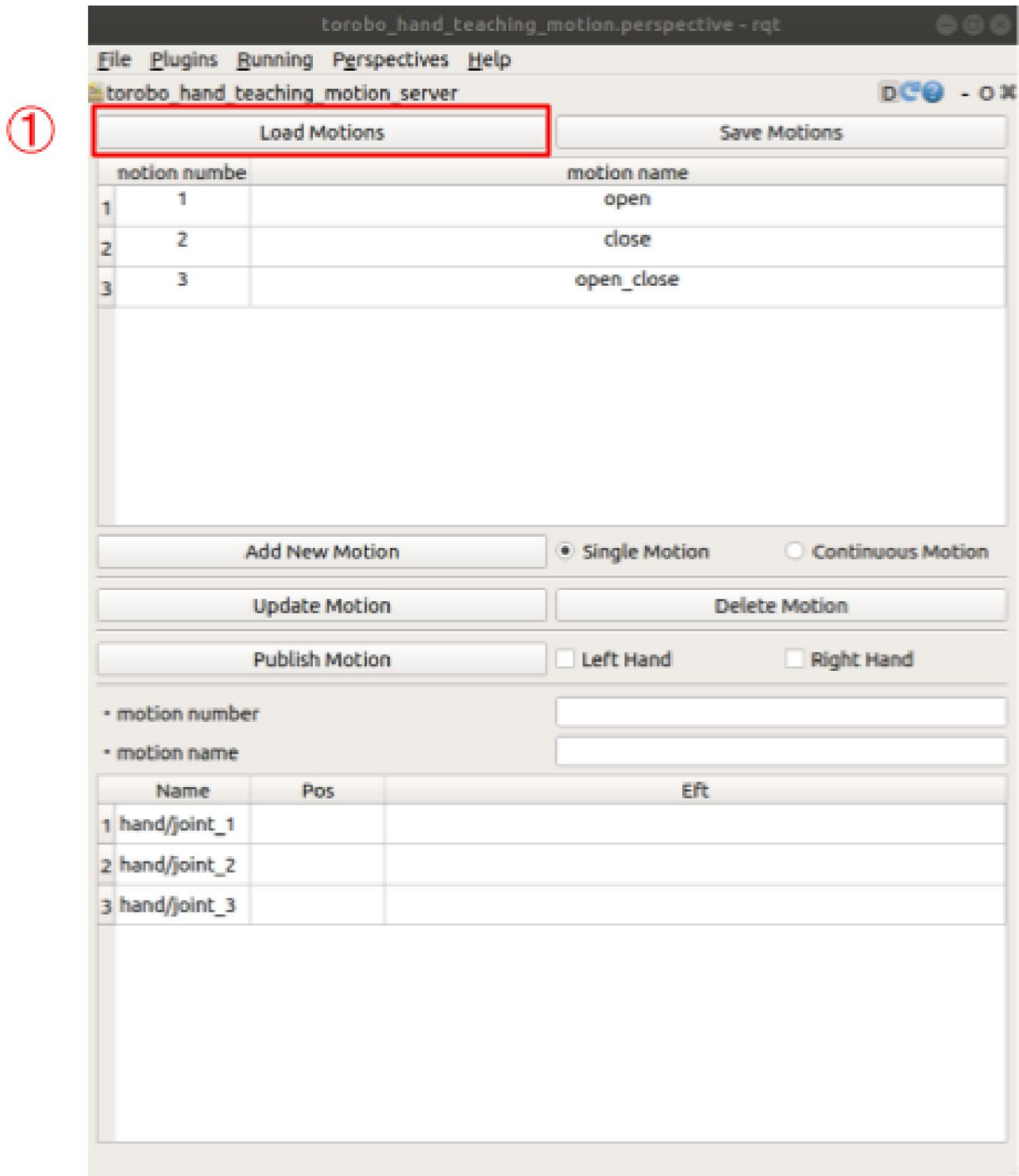
Finally, as shown in the red circle 4 , pressing the “Publish Button” will command the selected action.

13.10.2 About the **YAML** file for standard operation description

ToroboHandTeachingMotion loads the yaml file that describes the routine actions , and assigns the routine actions to the action number. You can load a YAML file in two ways:

13.10.2.1 Loading a **YAML** file containing standard actions from **the GUI**

In this method, you can load a standard motion by selecting "Load Motions" in the GUI and specifying the YAML file to load. It is possible to load actions.



Here are the steps to load a standard motion. Select "Load Motions" as shown in the red circle 1 .

When you select it, a pop-up window will appear where you can select the file.

Please select the route.

13. Software **Specifications**

13.10.2.2 Loading a **yaml** file containing standard actions as **rosparam** from **Launch** etc.

In this method, you can load a standard action by loading a YAML file into rosparam in any way. When ToroboHandTeachingMotion starts the GUI , it queries the rosparam server to see if a standard action list exists. If one exists, it loads the standard action list. As an example, we will explain how to load yaml as rosparam in the launch file .

```
<launch>
<arg name="motion_file_path" default="(YOUR YAML PATH)"/>
  <group ns="torobo"> <!-- To
    load the hand motion list into rosparam at launch, enable the
following -->
    <rosparam command="load" file="$(arg motion_file_path)"/>
    .
    .
    .
  </group> </
launch>
...
```

In the above example, the motion_file_path argument is loaded using the rosparam load command .

13.10.2.3 Directory where sample **YAML** files containing standard actions are placed

There are two types of sample yaml files . Depending on the number of fingers on the robot's hand, Therefore, please use it according to the robot's hand.

A sample yaml file for a hand with four fingers is located in the following PATH :

/opt/ros/\${ROS_DISTRO}/share/torobo_hand_teaching_motion_server/config/ four_fingeres_hand_motion_patterns.yaml

Also, a sample yaml file for a hand with three fingers is located in the following PATH :

/opt/ros/\${ROS_DISTRO}/share/torobo_hand_teaching_motion_server/config/ three_fingeres_hand_motion_patterns.yaml

13.10.2.4 How to write a **YAML** file for standard operation description

The standard action description yaml file lists the standard actions to be loaded into torobo_hand_motion_server .

The format is as follows:

```
motion_list : #Format
  for a single motion (effort values specified) #Specify the motion number at
  the end like "motion_number_motion number". The number must be unique within the yaml file.
Please make sure to do so.
motion_number_1:
  #Action names do not need to be unique.
motion_name: 'motion_name'
```

(Continued on next page)

(Continued from previous page)

```

motion_type: 'single' joint_values:
# Specify joint_values
as decimals up to two decimal places.
joint_name : float_value effort_values:
# It is not necessary to
specify effort_values. When specifying, please write as shown on the left. The value range is the same as joint_ values .

joint_name : float_value

#Format for a single motion ( no effort values specified)
motion_number_2:
motion_name: 'motion_name' motion_type:
'single' joint_values:

joint_name : float_value

#Continuous action format
motion_number_3:
motion_name: 'motion_name' motion_type:
'continuous' #Please specify the motion
number sequence to be operated continuously.
motion_numbers: 'motion_numbers' #Specify the
time interval for the motion to be performed continuously. Specify the value as a float . motion_duration_time:
'motion_duration_time'
...

```

13.10.3 Hand routine action command (using Ros Actionlib)

ToroboHandTeachingMotion provides an Actionlib service that uses a unique msg as a service to move each joint of the hand as a routine action . Users can issue hand routine action commands by calling this service.

Control	ActionService ý	Type of action for control_msg :
mode right_hand_controller	/torobo/ right_hand_teaching_motion_controller /torobo/	torobo_control_msg/ HandTeachingMotion.action
left_hand_controller	left_hand_teaching_motion_controller	torobo_control_msg/ HandTeachingMotion.action

Below, we will explain how to output hand routines using Ros Actionlib from the terminal. First, start the torobo_hand_teaching_motion Action Server . Open the terminal and run the following command.

```

$ cd catkin_ws $ catkin
source $ roslaunch
torobo_hand_teaching_motion_server torobo_hand_teaching_motion_server.
$ launch gui:=false

```

Next, use Ros Actionlib to give commands. There are hands on the left and right, so this time we will give commands to the right hand.

13. Software Specifications

Here is an example of how to send it. When issuing a command to the left hand, replace "right" with "left". When issuing a command from the terminal, publish the hand's standard action number to `/torobo/right_hand_teaching_motion_controller/hand_cmd/goal`. For example,

- For the right hand, perform the Open routine (action number: 1)

To do this, run the following command from the terminal:

```
$ rostopic pub /torobo/right_hand_teaching_motion_controller/hand_cmd/goal torobo_ŷcontrol_msgs/HandTeachingMotionActionGoal
"header:
  seq: 0
  stamp:
    secs: 0
    nsecs: 0
  frame_id: " goal_id:

  stamp:
    secs: 0
    nsecs: 0
  id: "
goal:
  desire_motion_number: 1"
```



Depending on the posture of **the Danger** Robot, it may interfere with the surrounding environment. Make sure there are no objects in the vicinity that may interfere with it, and be ready to press the emergency stop button immediately before issuing a command.



Note: Even if you copy and paste the above command into the terminal, the format may be distorted and it may not be possible to issue the command correctly. In that case, type `rostopic pub /torobo/right_hand_teaching_motion_controller/hand_cmd/goal ŷ` (ŷ: half-width space) in the terminal, then press the Tab key several times. The command after that will be automatically completed.

13.11 Torobo Moveit Commander Specifications

13.11.1 About Torobo Moveit Commander

In MoveIt!, the `move_group` server provides functions such as trajectory planning and execution, and obstacle placement in the surrounding environment. The python client library for this `move_group` server is `moveit_commander`.

When using this `moveit_commander`, the class that wraps the frequently used processes is `ToroboMoveitCommander`. `Torobo Moveit Commander` has the following features:

- An instance of `MoveGroupInterface` is automatically generated for every `move_group`. This allows
This makes it easier to control the `move_group` you specify.

- Cartesian commands for TCP (Tool Center Point) are easy. Maximum speed constraints can be applied, and Cartesian trajectories can be set. The success or failure of road planning is judged automatically.
- The method of `PlanningSceneInterface` , which is an interface to add and remove obstacles to the Planning Scene. Wrap portions and serve.

13.11.2 Sample script

The scripts directory of the `torobo_moveit_commanders` package contains a sample script, `torobo_moveit_commander_sample` , which allows you to plan and execute trajectories using `ToroboMoveitCommander` .

To do this, open a terminal and run the following command:

```
$ roslaunch torobo_moveit_commanders torobo_moveit_commander_sample.launch
```

After the various accessors are executed, the following should occur:

- Move joint to target position
- Move to Cartesian target position
- Place obstacles in the Planning Scene
- Move to joint target position while avoiding obstacles

13.11.3 List of `ToroboMoveitCommander` methods

class `torobo_moveit_commanders.torobo_moveit_commander.ToroboMoveitCommander` Wrapper for utilizing `moveit_commander`.

- Provide a wrapper function for utilizing `MoveGroupCommander` for all `move_groups`.
- Provide a wrapper function for instance of `PlanningSceneInterface`.

add_box_to_scene(**name**: str, **pose_stamped**: PoseStamped, **size**: tuple[float, float, float]) →

None

Add a box to planning scene.

Parameters

- **name** (str) – The name of the box.
- **pose_stamped** (geometry_msgs/PoseStamped) – The pose of the box.
- **size** (tuple[float, float, float]) – The size of the box (x, y, z) [m].

add_cylinder_to_scene(**name**: str, **pose_stamped**: PoseStamped, **height**: float, **radius**: float)

→ None

Add a cylinder to planning scene.

Parameters

13. Software **Specifications**

- **name** (str) – The name of the cylinder.
- **pose_stamped** (geometry_msgs/PoseStamped) – The pose of the cylinder.
- **height** (float) – The height of the cylinder.
- **radius** (float) – The radius of the cylinder.

add_mesh_to_scene(name, pose_stamped, filename, size=(1, 1, 1))

Add a mesh to planning scene.

Parameters

- **name** (str) – The name of the mesh.
- **pose_stamped** (geometry_msgs/PoseStamped) – The pose of the mesh.
- **filename** (str) – The path to the mesh file.
- **size** (tuple[float, float, float]) – Size scaling factor in the (x, y, z) directions.

add_plane_to_scene(name: str, pose_stamped: PoseStamped, normal: list[float, float, float] = [0, 0, 1], offset: float = 0.0) → None Add a sphere to planning scene.

Parameters

- **name** (str) – The name of the plane.
- **pose_stamped** (geometry_msgs/PoseStamped) – The point to be passed through the plane.
- **normal** (list of float) – The normal vector of the plane.
- **offset** (float) – The offset distance from the point to be passed.

add_sphere_to_scene(name: str, pose_stamped: PoseStamped, radius: float) → None
Add a sphere to planning scene.

Parameters

- **name** (str) – The name of the sphere.
- **pose_stamped** (geometry_msgs/PoseStamped) – The pose of the sphere.
- **radius** (float) – The radius of the sphere.

cancel_trajectory_execution(controller_name: str = 'joint_trajectory_controller') → None

Cancel the current trajectory.

Parameters **controller_name** (str) – The name of controller that will be canceled the goal of FollowJointTrajectory Action.

clear_pose_target(move_group_name: str = 'all') → None Clear the pose target of specified move_group.

• **move_group_name** (str) – The name of move_group. To specify the all move_group, set 'all'.

compute_ik(move_group_name, target_pose_stamped, tcp_frame, avoid_collisions=False, timeout=rospy.Duration)
Compute inverse kinematics

Parameters

- **x** (float) – Tooltip's position(meter) and pose(radian)
- **y** (float) – Tooltip's position(meter) and pose(radian)
- **z** (float) – Tooltip's position(meter) and pose(radian)
- **roll** (float) – Tooltip's position(meter) and pose(radian)
- **pitch** (float) – Tooltip's position(meter) and pose(radian)
- **yaw** (float) – Tooltip's position(meter) and pose(radian)

Result of compute_ik (list of joint angles)

Return type JointState

display_planned_path(plan: RobotTrajectory) → None Publish the planned path as moveit_msgs/DisplayTrajectory.

Parameter **plan** (moveit_msgs/RobotTrajectory) – The plan to be published as moveit_msgs/DisplayTrajectory.

execute_trajectory(plan: RobotTrajectory, controller_name: str = 'joint_trajectory_controller') → None Execute the path with specified controller.

Parameters

- **plan** (moveit_msgs/RobotTrajectory) – The robot motion to be executed.
- **controller_name** (str) – The controller name to execute the specified plan.

filter_plan_by_joint_names(plan: RobotTrajectory, joint_names: list[str]) → RobotTrajectory
Extract RobotTrajectory consisting of only the specified joint names.

Parameters

- **plan** (moveit_msgs/RobotTrajectory) – The robot trajectory to be filtered.
- **joint_names** (list of str) – Joint names to be extracted into trajectory.

Returns Trajectory with only the specified joint names.

13. Software **Specifications**

Return type `moveit_msgs/RobotTrajectory`

get_all_joint_names(move_group_name: str) → list
Get the name of all joints of the specified `move_group`.

Parameters **move_group_name** (str) – The name of `move_group`.

Return value Key is `move_group` name. Value is `MoveGroupCommander`.

Return type list of str

get_controller_names()

Get the list of controller names that can be specified for trajectory execution.

Returns The list of available controller names.

Return type list of str

get_current_joint_values(move_group_name: str) → list

Get the current configuration of the specified `move_group` as a list.

Parameters **move_group_name** (str) – The name of `move_group`.

Returns The list of joint configuration [radian or mm].

Return type: list of float

get_current_pose(move_group_name: str, end_effector_link: str = "") → `PoseStamped`

Get the current pose of the end-effector of the group.

Throws an exception if there is not end-effector.

Parameters

- **move_group_name** (str) – The name of `move_group`.
- **end_effector_link** (str) – The name of end effector `_link`.

Returns The pose of the end-effector.

Return type: `geometry_msgs/PoseStamped`

get_end_effector_link(move_group_name: str) → str

Get the name of the link that is considered to be an end-effector of the specified `move_group`.

Return an empty string if there is no end-effector.

Parameters **move_group_name** (str) – The name of `move_group`.

Returns The name of the end-effector link.

Return type str

get_joint_names(move_group_name: str) → list

Get the name of active joints of the specified `move_group`.

Parameters **move_group_name** (str) – The name of move_group.

Returns The list of joint names.

Return type list of str

get_known_object_names(with_type: bool = False) → list

Get the name of known objects in the planning scene.

If with_type is set to true, only return objects that have a known type.

with_type (bool) – Whether to get only objects that have a known type.

Return value The list of object names in the planning scene.

Return type list of str

get_max_acceleration_scaling_factor(move_group_name: str) → float

Get the scaling factor for the maximum joint acceleration.

Parameters **move_group_name** (str) – The name of move_group.

Returns The scaling factor for the maximum joint acceleration of the specified move_group.

Return type: float

get_max_velocity_scaling_factor(move_group_name: str) → float

Get the scaling factor for the maximum joint velocity.

Parameters **move_group_name** (str) – The name of move_group.

Returns The scaling factor for the maximum joint velocity of the specified move_group.

Return type: float

get_move_group_names() → list

Get the names of move_groups.

Returns list of str: The names of move_groups.

get_moveit_commanders() → dict[str, MoveGroupCommander]

Get the instance of MoveGroupCommander Class.

Return value Key is move_group name. Value is MoveGroupCommander.

Return type dict[str, MoveGroupCommander]

get_planning_time(move_group_name: str) → float

Get the amount of time to be used for motion planning.

Parameters **move_group_name** (str) – The name of move_group.

Returns The amount of time to be used for motion planning [seconds].

Return type: float

13. Software Specifications

get_pose_reference_frame(move_group_name: str) → str

Get the reference frame assumed for poses of end-effectors of the specified move_group.

Parameters **move_group_name** (str) – The name of move_group.

Returns The name of the reference frame.

Return type str

plan_joint_value_target(move_group_name: str, target_joint_state: JointState) → Robot-

Trajectory

Plan the motion to the specified goal state of joints.

Parameters

- **move_group_name** (str) – The name of move_group.
- **target_joint_state** (sensor_msgs/JointState) – The goal state of joints.

Returns The planned trajectory.

Return type moveit_msgs/RobotTrajectory

plan_pose_stamped_target(move_group_name: str, target_pose_stamped: PoseStamped, plan_cartesian_path: bool,

tcp_frame: str, eef_step: float = 0.001) → RobotTrajectory

Plan the motion to the specified target pose of the end-effector (Tool Center Point, TCP).

Parameters

- **move_group_name** (str) – The name of move_group.
- **target_pose_stamped** (geometry_msgs/PoseStamped) – The target pose of TCP.
- **plan_cartesian_path** (bool) – When true, compute a sequence of waypoints that make the TCP move in straight line segments to the specified pose. Configurations are computed for every eef_step meters. When false, compute the motion to the specified target pose of TCP. TCP does not move in straight line.
- **tcp_frame** (str) – The name of TCP (Too Center Point). TCP and end-effector are synonymous.
- **eef_step** (float) – Only when the argument paln_cartesian_path is true, this value is used. Configurations for cartesian path are computed for every eef_step meters.

Returns The planned trajectory.

Return type moveit_msgs/RobotTrajectory

remove_world_object(name: Optional[str] = None) → None

Remove an object from planning scene, or all if no name is provided.

Parameters **name** (str) – The name of object to be removed.

set_max_acceleration_scaling_factor(move_group_name: str = 'all', acceleration_scaling_factor: float = 0.1) → None
Set a scaling factor for optionally reducing the maximum joint acceleration.

Allowed values are in (0,1].

Parameters

- **move_group_name** (str) – The name of move_group. To specify the all move_group, set 'all'.
- **velocity_scaling_factor** (float) – A scaling factor for the maximum joint acceleration. Allowed values are in (0,1].

set_max_velocity_scaling_factor(move_group_name: str = 'all', velocity_scaling_factor: float = 0.1) → None
Set a scaling factor for optionally reducing the maximum joint velocity.

Allowed values are in (0,1].

Parameters

- **move_group_name** (str) – The name of move_group. To specify the all move_group, set 'all'.
- **velocity_scaling_factor** (float) – A scaling factor for the maximum joint velocity. Allowed values are in (0,1].

set_ompl_max_wp_distance_in_object_scene(value: float) → None

Specify the value of of maximum_waypoint_distance to be used when an object exists in planning scene.

ToroboMoveItCommander change the parameter /move_group/planning_pipelines/ompl/ompl/maximum_waypoint in ROS parameter server when any object is in planning scene. In order to increase the success probability of path planning when objects are present in the scene, it is desirable to reduce the value of the parameter maximum_waypoint_distance.

Parameter **value** (float) – maximum_waypoint_distance value.

set_planning_time(move_group_name: str = 'all', seconds: float = 2.0) → None

Specify the amount of time to be used for motion planning.

Parameters

- **move_group_name** (str) – The name of move_group. To specify the all move_group, set 'all'.
- **seconds** (float) – The amount of time to be used for motion planning [seconds].

set_retime_algorithm(retime_algorithm: str) → None
Specify the algorithm for time parameterization.

For more information on time parameterization, please see the moveit tutorial.

• **retime_algorithm** (str) – The name of algorithm for time parameter-iterative_time_parameterization, iterative_spline_parameterization or time_optimal_trajectory_generation. The selectable algorithms are:

wait_for_trajectory_completed(controller_name: str = 'joint_trajectory_controller', time-out: float = 0.0) → None

Wait until the trajectory is complete.

Wait for the result of the FollowJointTrajectoryAction of the specified controller.

Parameters **controller_name** (str) – The name of controller that will wait for the result of the action to be returned.

13.12 FlexBE Specifications

FlexBe is a framework for ROS that handles robot state transitions . It allows you to create complex robot behaviors without having to code them manually. For example, you can easily configure state machines with drag and drop, and you can start and monitor execution from the same GUI where you configure the state machine . For more information, see [the official page](#) . Or [the Documentation on the ROS Wiki](#). Please refer to.

13.12.1 Basic Use of FlexBE

For basic usage, [see the Tutorial on the ROS Wiki](#). Please refer to.

13.12.2 How to Start FlexBE

In order to link FlexBE with various ROS nodes provided by our company , FlexBE must be started under the namespace torobo .

You can start FlexBE under the namespace torobo by executing the following command from the terminal .

You can use States and Behaviors defined by users or by our company .

```
$ roslaunch torobo_flexbe_bringup bringup_flexbe_full.launch
```

13.12.3 Custom State

FlexBE provides several general purpose states by default.

- [flexbe_states](#)
- [generic_flexbe_states](#)

However, the processes that can be executed with these are limited. Therefore, we provide several original states developed by our company, Therefore, they are grouped together in packages.

For more information on the processing of each unique State , please see below.

- README of the package that implements your own State
- Docstring for your custom State class
- Sample Behavior using a custom State (described below)

13.12.3.1 torobo_general_states

This is a package that compiles states that perform general-purpose processing . The following states are defined:

- ClearErrorState
- IsInputkeyNoneState
- OutputCurrentPoseState
- OutputFloatState
- OutputPoseStampedState
- OutputPoseState
- OutputStringState
- ServoSwitchState
- TemplateState
- UpdateDynamicReconfParamtState
- SetLoadState
- UnsetLoadState

See the package README for more details .

13.12.3.2 torobo_geometric_states

This is a package that collects states that perform geometric transformations and calculations . The following states are defined:

- DistanceBetweenTwoPoseStampedState
- ExtractPointStFromInputKeyPoseStState
- OffsetPoseStampedState
- OverwriteOrientationOfPoseStampedState
- OutputPoseStampedViaTransformState
- TransformInputKeyPoseStampedState

See the package README for more details .

13. Software **Specifications**

13.12.3.3 torobo_hand_states

This package contains states that change the posture of the hand . The following states are defined:

- HandTeachingMotionState
- BothHandTeachingMotionState

See the package README for more details .

13.12.3.4 torobo_manipulation_states

This is a package that compiles the states that perform basic control and processing of the arm . There are four directories depending on the processing content. It is classified as a cutry.

- Joint directory
 - FollowJointTrajectoryToInputKeyTrajState
 - FollowJointTrajToTeachingPointState
 - OutputOffsettedJointPositionState
 - OutputJointStateState
- Cartesian Directory
 - OutputTCPTargetPoseState
 - PublishCartTrajToInputKeyPoseState
 - PublishCartTrajToTeachingPointState
 - PublishDualCartTrajToTeachingPointState
- moveit directory
 - CreateMoveitCommanderState
 - AddBoxToPlanningSceneState
 - AddCylinderToPlanningSceneState
 - AddMeshToPlanningSceneState
 - RemoveObjectFromPlanningSceneState
 - MoveToInputkeyCartesianPoseState
 - MoveToInputkeyJointPosState
 - MoveToTargetCartesianPoseState
 - MoveToTargetJointPosState
 - MoveToTeachingPointState

- teaching_point_manager directory
 - GetTeachingPointState
 - LoadTeachingPointState
 - OutputCartTPAsPoseStampedState

See the package README for more details .

13.12.4 About Sample Behavior

The Behavior samples we have created are organized by the State packages they use .

13.12.4.1 torobo_general_behavior

This section contains examples of Behavior using the States defined in torobo_general_states .

- example_general_states This is a sample that checks the InputKey and outputs variables of msg type such as PoseStamped and String type as OutputKey . The provided sample executes the following States in order.

1. IsInputkeyNoneState
2. OutputPoseState
3. OutputPoseStampedState
4. OutputCurrentPoseState
5. OutputStringState
6. OutputFloatState

For more information, please check the FlexBE GUI .

- example_servo_state This is a sample that turns the robot's servos ON/OFF . In the provided sample, after executing Clear Error on the robot, the servos of the 5th and 6th axes of the right arm are turned ON , and after 3 seconds, only the servo of the 5th axis of the right arm is turned OFF . For details, please check the FlexBE GUI .

	When the Danger servo is turned on , the brakes on the joints are released and the joints become movable. When turning on the servo , be sure to leave the robot's moving area. Also, be sure to be able to press the emergency stop switch instantly while working .
---	--

- exmaple_set_and_unset_load This is a sample for setting a load on a robot. In the provided sample, a load of 2.0 kg is set on the right arm and released after 5 seconds. For details, see the Load Setting Function Specifications and Please check on the FlexBE GUI .

13. Software **Specifications**

- `example_update_dynamic_reconfigure` This is a sample that changes the parameters corresponding to `dynamic_reconfigure`. In the provided sample, the spring constant of the spring of the target impedance during Cartesian impedance control of the right arm is changed to 1000, and the damper constant of the damper of the target impedance is changed to 40. For details, please check the FlexBE GUI .

13.12.4.2 `torobo_geometric_behaviors`

Here are some examples of Behavior that use the States defined in `torobo_geometric_states` .

- `example_geometric_states` This is a sample that overwrites the posture information of PoseStamped and calculates the distance between two PoseStamped . The provided sample executes the following States in order.

1. `OutputPoseStampedViaTransformState`
2. `OffsetPoseStampedState`
3. `DistanceBetweenTwoPoseStampedState`
4. `OverwriteOrientationOfPoseStampedState`
5. `TransformInputKeyPoseStampedState`
6. `ExtractPointStFromInputKeyPoseStState`

For more information, please check the FlexBE GUI .

13.12.4.3 `torobo_hand_behaviors`

Here are some examples of Behavior using the States defined in `torobo_hand_states` .

- `example_hand_teaching_motion` This is a sample that executes standard hand motion commands. In the provided sample, the left hand moves to motion number 2, then the right hand moves to motion number 1. Finally, the left and right hands move to motion number 1 respectively. For more details, please check the FlexBE GUI .

13.12.4.4 `torobo_manipulation_behaviors`

Here are some examples of Behavior using the States defined in `torobo_manipulation_states` .

- `example_load_and_get_TP` This is a sample that loads a TP from a Yaml file and outputs the joint angle information of an arbitrary joint angle TP name (in the sample , "joint_tp1") as an OutputKey . For details, Please check [the TeachingPointManager specifications](#) and the FlexBE GUI .
- `example_output_cart_tp_as_pose` This is a sample that reads a TP from a Yaml file and outputs the PoseStamped of an arbitrary Cartesian TP name (in the sample, "cart_tp1") as an OutputKey. For details , please check the TeachingPointManager specifications [and](#) the FlexBE GUI .
- `example_follow_joint_traj_to_TP` After reading the TP from the Yaml file , it moves to any joint angle TP in order . (In the sample, "joint_tp1", "joint_tp2"). For details, please check [the specifications of TeachingPointManager](#) and [the original ROS Controller on the FlexBE GUI](#) .

- `example_offset_joint_pos` This is a sample that generates and moves a joint target position offset from the current joint angle. (In this sample, a joint target position offset by -10 degrees for the first axis of the right arm and 10 degrees for the fourth axis of the right arm is generated and moved.) For more information, see [About Our Own ROS Controller](#) and Please check on the FlexBE GUI .
- `example_pub_cart_traj_to_inputkey_pose` This is a sample that generates an arbitrary Cartesian target position as an OutputKey and moves it. (In the sample, the right arm's tool tip is first generated and moved to a target position that is [0.4,-0.25,0.83] in base_link coordinates and quaternion [0,0,0.707,0.707] .) For more information, please check [About Our Own ROS Controller](#) and the FlexBE GUI .
- `example_pub_cart_traj_to_TP` This is a sample that reads a TP from a Yaml file and then moves it to an arbitrary Cartesian TP (in the sample , "cart_tp1"). For details, please check [the specifications of TeachingPointManager](#) and [the original ROS Controller](#) on the FlexBE GUI .
- `example_publish_dual_cart_trajectory` This is a sample that reads the TP from the Yaml file and then moves to the Cartesian TP of the left arm and the right arm ("left_cart_tp" and "right_cart_tp" in the sample). It is possible to control both arms at the same time. For details, please check [the specifications of TeachingPointManager](#) and [the original ROS Controller](#) on the FlexBE GUI .
- `example_add_box_to_planning_scene` This is a sample that uses Torobo MoveIt Commander to generate a trajectory and perform operations . It adds an obstacle rectangular prism (a 0.1m cube in the sample) to the Planning Scene, moves the robot to a desired target position (the first axis of the right arm is 0[rad] in the sample) while avoiding the rectangular prism, and removes the added rectangular prism from the Planning Scene after the movement is complete . For more information, see [About ToroboMoveitCommander](#) and the FlexBE GUI .
- `example_add_cylinder_to_planning_scene` This is a sample that uses Torobo MoveIt Commander to generate a trajectory and perform operations . It adds an obstacle cylinder (in the sample, radius 0.15m , height 0.1m) to the Planning Scene, moves to a desired target position (in the sample, the first axis of the right arm is 0 [rad]) while avoiding the cylinder, and removes the added cylinder from the Planning Scene after the movement is complete . For more information, see [About Torobo Moveit Commander](#) and the FlexBE GUI .
- `example_add_mesh_to_planning_scene` This is a sample that uses Torobo MoveIt Commander to generate a trajectory and perform operations . It adds an obstacle mesh (in the sample, the STL file specified by the parameter "mesh_path") to the Planning Scene, moves to an arbitrary target position (in the sample, the first axis of the right arm is set to 0 [rad]) while avoiding the mesh, and removes the added mesh from the Planning Scene after the movement is complete . [For more information](#), please see [About ToroboMoveitCommander](#) and the FlexBE GUI .
- `example_move_to_inputkey_cartesian_pose` Generates a Cartesian target position as an OutputKey , This is a sample of movement using Torobo MoveIt Commander . (In the sample, the right arm's tool tip is first moved to a target position that is expressed as the base_link coordinate [0.5, -0.25, 0.9] and quaternion [0.5, 0.5, 0.5, 0.5] .) For more information, see [About ToroboMoveitCommander](#) and the FlexBE GUI .
- `example_move_to_target_cartesian_pose` This is a sample of moving to a Cartesian target position using Torobo MoveIt Commander . (In this sample, the palm_link of the right hand is first moved to the target position, which is [0.3, -0.3, 1] in base_link coordinate expression and quaternion [-0.701, 0.0908, 0.0941, 0.700] .) [For more information](#), please check [About ToroboMoveitCommander](#) and FlexBE GUI .

13. Software **Specifications**

- `example_move_to_inputkey_joint_pose` This is a sample that generates the joint target position as an `OutputKey` and moves it using `Torobo MoveIt Commander` . (In this sample, the first axis of the right arm and the second axis of the right arm are moved to `0.4[rad]` and `0[rad]` , respectively .) [For more information](#), please check `ToroboMoveitCommander` and the `FlexBE GUI` .
- `example_move_to_target_joint_pos` This is a sample that generates a joint target position and moves it using `Torobo MoveIt Commander` . (In this sample, the first axis of the right arm and the fourth axis of the right arm are moved to `0.2 [rad]` each .) [For more information](#), please check About `ToroboMoveitCommander` and the `FlexBE GUI` .
- `example_move_to_TP` This is a sample that loads a TP from a Yaml file , moves it to an arbitrary joint angle TP ("joint_tp1" in the sample) using `Torobo MoveIt Commander` , and then moves it to an arbitrary Cartesian TP ("cart_tp1" in the sample). For details, see the `TeachingPointManager specifications` and Please check `Torobo Move it Commander` on the `FlexBE GUI` .

13.12.5 Points to note when creating a State

How to create your own state is explained in [the Tutorials on the ROS Wiki](#). Please refer to the following. Here, special attention is required.

This article describes some important points.

- If an exception occurs in `on_enter()` , it will automatically transition to `execute()` and the Behavior will not stop.
- Even if the following error occurs in `on_execute()` , `on_execute()` will continue in an infinite loop unless return is executed correctly.

1. When an invalid value is returned
2. When the request is not returned
3. When an Excitement Occurs

To avoid an infinite loop of `on_execute()` as shown above , we recommend implementing it according to `TemplateState` in `torobo_generate_states`. Also, if you want to display logs on `FlexBE`, you need to use `Logger` instead of `rospy`. If you create a state according to `TemplateState` mentioned above, you can use `self._logger.info/warn/error` to display logs with the state name, making debugging easier.

14 - Conservation and Management

14.1 How to update the Ubuntu environment

To update all software in the Ubuntu environment on the robot controller PC and the user PC to the latest version, run the following command.

**Note**

In order to update your Ubuntu environment, you will need an internet connection, so please prepare it in advance and read the following steps.

```
$ sudo apt update $ sudo  
apt upgrade --auto-remove
```

When executing the above command, the following error message may be displayed and the update may not be possible.

```
E: Could not get lock /var/lib/dpkg/lock-frontent - open (11: Resource temporarily  
unavailable)  
E: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontent), is  
another process using it?
```

The above error message is most likely to appear immediately after starting up your PC . If you wait a while after starting up and try running the command again, and the same error message still appears, perform the following steps and then run the update command again.

```
$ sudo rm -rf /var/lib/dpkg/lock*
```

If the version of the ROS software installed on the robot controller's PC and the user's PC are different, some of the ROS packages provided by our company may not work properly. In that case, please perform the above updates in both environments, then clean and rebuild the ROS workspace.

14.2 How to update the provided ROS packages

We may distribute update files for the ROS packages we provide . The update files distributed include three update packages for Controller 1 (C1), Controller 2 (C2), and User PC , which are divided into the directories "torobo_ros_dry_airec_c1", "torobo_ros_dry_airec_c2", and torobo_ros , respectively . To apply the update files, update the Ubuntu environment for each PC (see How to Update the [Ubuntu Environment](#)), then update the deb binary and source code as follows.



Note: You will need an internet connection to update the package.
Please prepare in advance and read the following steps.

14.2.1 How to update Controller 1 (C1)

Copy the update file "torobo_ros_dry_airec_c1" to controller 1 as a directory . Start the terminal, change to the destination directory, and then execute the update installation script with the following operations. Administrator privileges are required during execution, so if you are asked to enter a password, enter "torobo" .

```
$ sh install_torobo_ros.sh ~/catkin_ws
```

The above assumes that the workspace for ROS is in "~/catkin_ws" , so if you have created the workspace in a different location, please read appropriately. Executing the above script will install the deb binary and update the source code. If "torobo_ros" already exists in the "~/catkin_ws/src" directory , it will be moved to the "~/catkin_ws/backup_torobo_ros/torobo_ros_%Y%m%d_%H%M" folder and the new "torobo_ros" will be placed in "~/catkin_ws/src/" . After the script has finished, build the updated source code.

```
$ cd ~/catkin_ws $ catkin
config -DCMAKE_BUILD_TYPE=Release $ catkin build $ catkin
source
```

If an error is displayed at this time, follow the steps below to clean the workspace and install any missing external ROS dependent packages, then build the source code again to verify that the error no longer appears.

```
$ cd ~/catkin_ws $ catkin
clean -y $ rosdep update $
rosdep install --from-
paths src --ignore-src -r -y $ catkin build $ catkin source
```

If no errors appear, the update is complete.

14.2.2 How to update Controller 2 (C2)

Copy the update file "torobo_ros_dry_airec_c2" to Controller 2 as a directory . Start the terminal, navigate to the destination directory, and then execute the update installation script with the following operations. Administrator privileges are required during execution, so if you are prompted to enter a password, enter "torobo" .

```
$ sh install_torobo_ros.sh ~/catkin_ws
```

The above assumes that the workspace for ROS is in "~/catkin_ws" , so if you have created the workspace in a different location, please read appropriately. Executing the above script will install the deb binary and update the source code. If "torobo_ros" already exists in the "~/catkin_ws/src" directory , it will be moved to the "~/catkin_ws/backup_torobo_ros/torobo_ros_%Y%m%d_%H%M" folder and the new "torobo_ros" will be placed in "~/catkin_ws/src/" . After the script has finished, build the updated source code.

```
$ cd ~/catkin_ws $ catkin
config -DCMAKE_BUILD_TYPE=Release $ catkin build $ catkin
source
```

If an error is displayed at this time, follow the steps below to clean the workspace and install any missing external ROS dependent packages, then build the source code again to verify that the error no longer appears.

```
$ cd ~/catkin_ws $ catkin
clean -y $ rosdep update $
rosdep install --from-
paths src --ignore-src -r -y $ catkin build $ catkin source
```

If no errors appear, the update is complete.

14.2.3 How to update user PCs

Copy the update file torobo_ros to each directory on the user's PC . Start the terminal, move to the destination directory, and then execute the update installation script with the following operations. Administrator privileges are required during execution, so if you are asked to enter a password, enter the password of a user with administrator privileges.

```
$ sh install_torobo_ros.sh ~/catkin_ws
```

The above assumes that the workspace for ROS is in "~/catkin_ws" , so if you have created the workspace in a different location, please read appropriately. Executing the above script will install the deb binary and update the source code. If "torobo_ros" already exists in the "~/catkin_ws/src" directory , move it to the "~/catkin_ws/backup_torobo_ros/torobo_ros_%Y%m%d_%H%M" folder and then create a new

"torobo_ros" will be placed in "~/catkin_ws/src/" . After the script finishes, build the updated source code.

```
$ cd ~/catkin_ws $ catkin
config -DCMAKE_BUILD_TYPE=Release $ catkin build $ catkin
source
```

If an error is displayed at this time, follow the steps below to clean the workspace and install any missing external ROS dependent packages, then build the source code again to verify that the error no longer appears.

```
$ cd ~/catkin_ws $ catkin
clean -y $ rosdep update $
rosdep install --from-
paths src --ignore-src -r -y $ catkin build $ catkin source
```

If no errors appear, the update is complete.

14.2.4 How to back up source code and check the differences

When updating source code, a backup of the existing "torobo_ros" is saved to "~/catkin_ws/backup_torobo_ros/torobo_ros_%Y%m%d_%H%M" (%Y%m%d_%H%M is the date and time of the update), and a file containing the source code differences is automatically generated in the backup folder. There are three automatically generated files:

- A text file, 'torobo_ros_summary.txt', that displays a list of files that have been changed from the existing 'torobo_ros'
- A patch file, 'torobo_ros_diff.patch', that shows the changes made to the existing 'torobo_ros'
- A patch file showing the changes made by the user since the last time we provided the "torobo_ros" package
torobo_ros_user_diff.patch

The patch file is output in unified format. When viewing the patch file, use Visual Studio
If you open it in a text editor like Code , the changes will be highlighted, which is convenient.

After the update is completed successfully, delete the backup folder if it is no longer needed.

14.2.5 How to check the change log of a deb binary

To check the changelog of the deb binary, copy and extract the "torobo_ros_debs*_changelog.zip" included in the update file to your PC. When you extract "torobo_ros_debs*_changelog.zip", you can view the changelog for each package in text format .

14.3 Managing and deleting logs from the PC in the robot controller

The ROS log in the robot controller 's PC is stored in ~/.ros/log/* . ROS log files are stored for 30 days after the last update, and log files older than 30 days are automatically deleted starting from the oldest file so that the log directory capacity is 5GB or less. Even if the log directory capacity exceeds 5GB , log files that are less than 30 days old will not be deleted.

If the log directory size exceeds 1GB , the following warning will be displayed when ROS starts.

```
WARNING: disk usage in log directory [/home/torobo/.ros/log] is over 1GB.  
It's recommended that you use the 'rosclean' command.
```

If the log files are consuming a large amount of disk space, the user can manually delete the log files.
If you want to manually delete the ROS log files, run the following command:

```
$ rosclean purge
```

14.4 Checking the free disk space of the PC in the robot controller

To check the free disk space, run the following command:

```
$ df -h /  
Filesystem /dev/          Size Used Avail Use% Mounted on  
sda2                234G  30G 193G  14% /
```

"Used" shows the current disk usage, and "Avail" shows the free disk space. If "Avail" falls below about 10GB , problems may occur with the operation of Ubuntu . In this case, delete unnecessary files and log files.

15 - Troubleshooting

This section describes general troubleshooting for the robot.

15.1 An error appears immediately after launching the robot

Immediately after starting the robot, an error may be displayed on the Torobo State Viewer . Check the message displayed in the "Diag" column and make sure that only one of the following error messages is displayed.

- CoreFault
- CycExeFrameMismatch
- DrvNotInit
- DrvFlt
- UnderVolt
- OverEft
- SensInvCrc_Only
- SensInvCrc_MB4

These errors may occur when the power supply or communication is unstable immediately after starting the robot, but once the conditions have stabilized, there will be no problem with the operation of the robot.

Refer to [Checking the Robot Control Mode and Servo Status](#) and click Clear Error on the Torobo State Viewer , then select Reset from the Safety State Reset Button. If the error is cleared and the Safety State transitions to Healthy , recovery from the error has been successful.

15. Troubleshooting

15.2 Unable to communicate with robot

Please check the following items:

- The emergency stop button is released and power is being supplied to the robot
- The robot controller and robot are connected with a LAN cable.

If the emergency stop button is pressed or the robot controller and robot are not connected with a LAN cable, the core program will fail to start. In that case, check that there are no problems with the above items and then run the core program again.

15.3 The core program (**bringup_real.launch**) does not start up properly on the actual robot

If a previously executed core program was not running normally, subsequent core programs may fail to start. Yes. Please execute the following operation after closing all ROS programs.

```
$ ps -A | grep Main
1540 ?          00:03:45 MainThread
```

As shown above, if the 'MainThread' process exists, kill the process with the following command:
So lame.

```
$ pkill -9 MainThread
```

After that, run `bringup_real.launch` again and check that the program starts up normally. If the program still fails to start after performing the above steps, restart the PC in the robot controller and then run `bringup_real.launch` again .

15.4 The core program does not start normally because an error "cannot open shared object file" appears.

When I run the core program, I get the error "cannot open shared object file" like the following example, but it doesn't start normally. It may not rise.

```
/home/torobo/catkin_ws/devel/lib/torobo_collision_detector/torobo_collision_detector_
ynode: error while loading shared libraries: libmoveit_planning_scene_monitor.so.1. y0.8: cannot open shared object file: No such
file or directory
```

In particular, as in the example above, errors may occur if there is an update to a package related to MoveIt!.

In this case, please stop running all ROS programs and then execute the following operations to clean and build the workspace.

```
$ catkin clean -y $ catkin
build $ catkin source
```

Then run the core program again and check that it starts up correctly.

15.5 How to recover from an over position error

If the robot joints are rotated beyond their range of motion while in weight compensation control mode, an OverPosition error will occur. When an OverPosition error occurs, the servo for the corresponding joint cannot be turned ON . Follow the procedure below to return the corresponding joint to its range of motion and recover.

1. Enter the following command in the command box of Torobo State Viewer to release the brakes on the corresponding joints.

```
brake [Joint ID] 0           // brake off the Joint ID
```



When releasing the **Danger** brake, the joint will fall under its own weight. Therefore, when releasing the brake, make sure there is no risk of collision with the robot itself or its surroundings, and release the brake while supporting the robot so that it does not fall. Also, to avoid the danger of the robot falling, release the brake on only one joint at a time.



Depending on the posture of **the Danger Robot**, the robot joints may be subjected to a large weight that cannot be supported by one person. Pressing the emergency stop switch will re-enable the brakes even if they are released, so when releasing the brakes on the joints, make sure you are ready to press the emergency stop switch immediately. Also, if possible, have two or more people support the robot while another person turns the brakes ON and OFF . Please work with the utmost care to avoid accidents.

2. Move the affected joint by hand to bring it back into its range of motion



Note: If you continue to move the robot's joints in a direction beyond their range of motion, the internal wiring may be pulled, causing excessive strain and resulting in failure. When moving a joint with the brake released, always move it in the direction that returns it to its range of motion. If you are unsure which way to move it to return it to its range of motion, move the joint a little and then try steps 3 and 4. If the error recovery fails, go back to step 1 and try moving it in the opposite direction to see if the error can be recovered.

3. Enter the following command to brake the corresponding joint.

```
brake [Joint ID] 1           // brake on the Joint ID
```

4. Clear the error state

15.6 How to recover from an Emergency Stop that was issued due to self-collision detection

The ROS package provided by our company has a self-interference prevention function (`torobo_safety_monitors/SelfCollisionMonitor`). When this function is enabled, if the robot is about to collide with itself, it will issue an Emergency Stop and turn off the servo (this function is enabled by default).

In this case, if left as is, the self-interference state will not be resolved, so the recovery conditions will not be met and it will not be possible to recover from the Emergency Stop state.

To recover from the self-collision state, move the robot to a position where self-collision does not occur by using one of the following methods. You need to move the joints in your foot.

1. Release the brakes on the joints and manually backdrive them
2. Temporarily disable self-interference monitoring and servo-drive the joints

We usually recommend returning to normal operation by using step 1 , but depending on the robot's posture or condition (e.g. when it is necessary to release the brake on the waist of a humanoid robot, and it is dangerous to support the joints manually in case they fall due to their own weight) , it may be safer to temporarily disable the safety monitoring state and servo-drive the joints as in step 2.

Each method is explained below.

15.6.1 Release the brakes on the joints and backdrive them manually

First, enter the following command in the command box of Torobo State Viewer to check the cause of the collision detection. Release the brakes on the joints.

```
brake [Joint ID] 0           // brake off the Joint ID
```

Then, manually move the joint to a position that avoids interference. After moving the joint, be sure to execute the following command. Activate the brake with the

```
brake [Joint ID] 1           // brake on the Joint ID
```

If there are other joints that are causing the collision detection, release the brakes in the same way, move them, and then turn the brakes ON . After the self-collision state is resolved, complete the recovery by following the procedure for [how to return to Healthy from Emergency Stop and Protective Stop](#) .



If you release the **Danger** brake, the joint will fall under its own weight.

When releasing the brakes, make sure there is no risk of collision with the robot itself or its surroundings, and release the brakes while supporting the robot so that it does not fall. Also, release the brakes on only one joint at a time to avoid the danger of the robot falling.



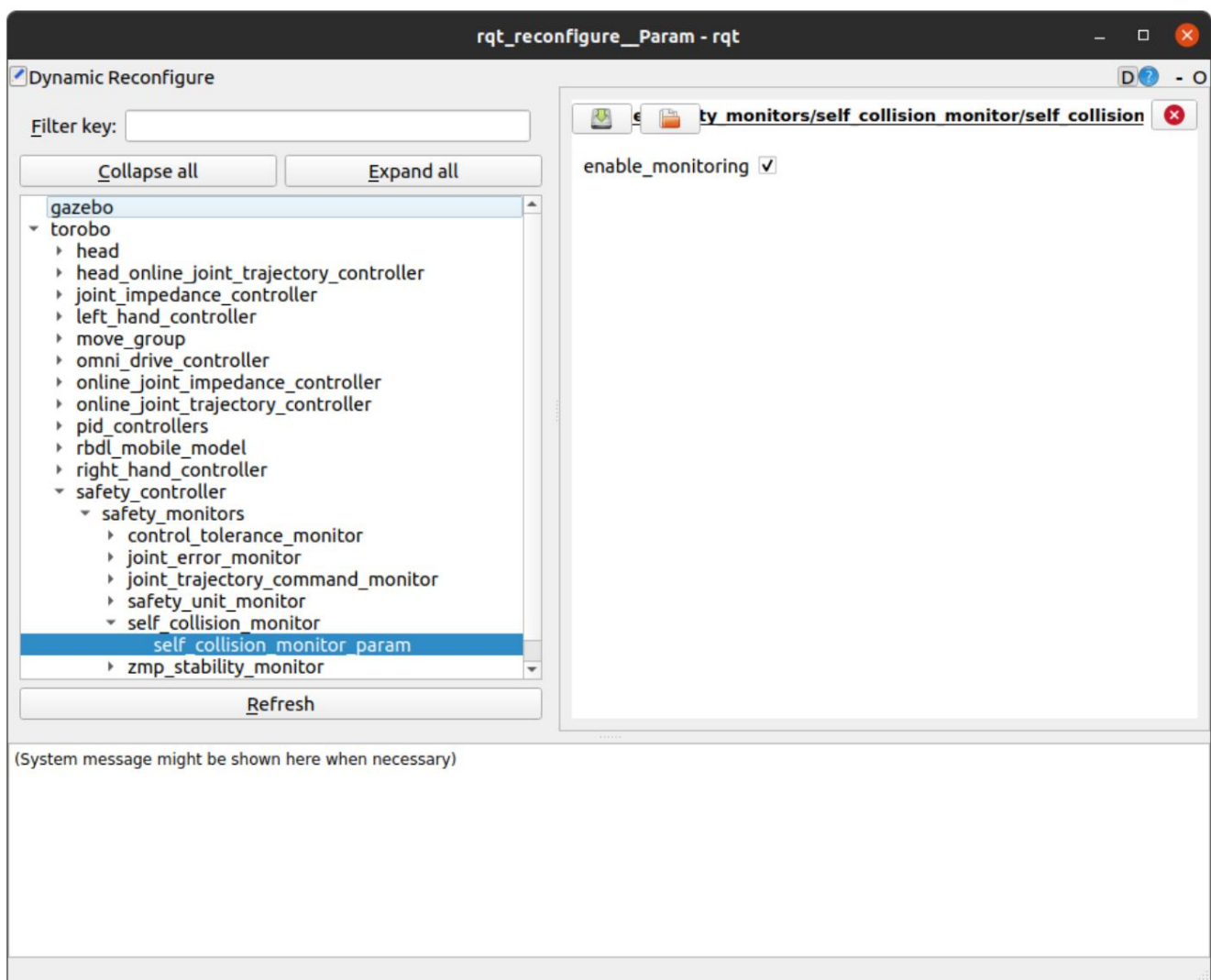
Depending on the posture of **the Danger Robot**, the robot joints may be subjected to a large weight that cannot be supported by one person. Pressing the emergency stop switch will re-enable the brakes even if they are released, so when releasing the brakes on the joints, make sure you are ready to press the emergency stop switch immediately. Also, if possible, have two or more people support the robot while another person turns the brakes ON and OFF . Please work with the utmost care to avoid accidents.

15.6.2 Temporarily disable self-collision monitoring and servo drive joints

First, start `rqt_reconfigure` with the following command .

```
roslaunch rqt_reconfigure rqt_reconfigure
```

From the list on the left side of the displayed GUI , select "torobo" -> "safety_controller" -> "safety_monitors" -> Select ``self\_collision\_monitor" -> ``self_collision_monitor_param".



Click the 'enable_monitoring' check box displayed on the right side of the GUI , and make sure it is unchecked.

This will temporarily disable the self-interference monitoring state, so please recover from the Emergency Stop state by following [the procedure for recovering from Emergency Stop or Protective Stop to Healthy](#) .

Next, use the Torobo State Viewer to turn on the servo of the joint that is causing the collision and issue a command to move the joint to a position that avoids the collision. If you accidentally issue a command to move in a direction that will cause self-collision, the movement may not stop and self-collision may occur, so be careful when issuing the command.

If there are other joints that are causing interference detection, move those joints using the same operation. After the self-collision state is resolved, click the 'enable_monitoring' checkbox displayed on the right side of the dynamic_reconfigure GUI again to return it to a checked state. This will re-enable the self-collision monitoring state and complete the restoration. In this way, if you temporarily disable the safety monitoring state, be sure to re-enable the safety monitoring state after completing the restoration operation.

 Danger: When	servo-driving a robot joint with self-interference monitoring disabled, self-interference may occur due to the issuance of a movement command in the wrong direction. When servo-driving a joint with self-interference monitoring disabled, set the transition time of the movement command to a slow speed, and perform the work in a state where you can immediately press the emergency stop switch if an incorrect operation is performed. In addition, to reduce the possibility of performing an incorrect operation, servo-drive only one joint at a time with self-interference monitoring disabled.
--	--

 Danger: If you	temporarily disable the safety monitoring state, be sure to enable the safety monitoring state again after completing operations such as recovery. If you operate the robot with the safety monitoring state disabled, the robot may run out of control or fall over due to the lack of proper safety monitoring function.
--	---

15.7 When I start a ROS package, **Rviz** or **Gazebo** appears at a certain frequency.

An error occurs at startup

Depending on your PC environment, when you start the torobo_robot ROS package (torobo_bringup) ,
At this temperature , Rviz or Gazebo may encounter an error when starting up.

If you are using a PC environment without a graphics card , please use a PC environment with a graphics card.
Please use it.

For Gazebo , a PC environment with a graphics card (not a built-in type in the CPU, but a dedicated card type) is recommended. In particular, NVIDIA graphics cards are recommended. Please refer to the following website for the recommended PC specifications for Gazebo . http://gazebosim.org/tutorials?tut=guided_b1#Systemrequirements

In addition, if you are using a virtual PC environment, the operation may become unstable. Please prepare a non-virtual PC environment and try starting ROS.

In addition, if you are using a PC environment that does not meet the specifications listed in the hardware chapter ,
Please try to start ROS in a PC environment that satisfies the following .

However , Rviz and Gazebo may still fail to start at a certain frequency .

This is likely due to the implementation of the code, so please contact the ROS or Gazebo developer forums.

15.8 Changes made to URDF are not reflected

If you want to make changes to the robot's URDF , please make the changes on the PC that launches "bringup_xxx.launch" or "torobo_display.launch" . Please be careful as it is easy to make mistakes when sharing "ROS MASTER" between multiple PCs , such as a control PC and a user PC .

In addition, this robot system uses the "xacro" (<http://wiki.ros.org/xacro>) format for URDF files. We use a macro language called ".xacro". When "bringup_xxx.launch" or "torobo_display.launch" is executed, the "xacro" format file is converted to URDF format, but since the conversion takes some time, in order to shorten the startup time, the generated URDF is cached while the ROS "run_id" is the same, and if a cache file exists, the converted URDF is loaded . Therefore, if you want to make changes to the "xacro" format file and load it again, you should quit "ROS_MASTER" (roscore) once to change the "run_id" and then execute "bringup_xxx.launch" etc. again.

15.9 Stop emitting **static_tf** between **map** and **odom** frames

In the ROS package for this robot , the specification is that a static_tf between the map frame and the odom frame is issued by default. However, when using ROS Navigation Stack , if you want to define a tf between the map frame and the odom frame using the self-location estimation result, it will conflict with the static_tf above , and the robot's self-location will not be able to be set properly.

To avoid conflicts, specify static:=false when starting the core program, as in the following example:
Please do so.

List 15.1 Running on a real device

```
$ roslaunch torobo_bringup bringup_real.launch static:=false
```

Listing 15.2 Simulation run

```
$ roslaunch torobo_bringup bringup_sim.launch static:=false
```

15.10 In torque control mode, the joint moves on its own even though no external force is being applied

This phenomenon may occur when an offset occurs in the torque sensor measurement value for some reason. This is because the torque sensor value is used for control in torque control mode. Refer to "Getting the torque sensor offset value" and try the control operation again after getting the torque sensor offset value.

15.11 Acquiring torque sensor offset value

An offset may be added to the torque sensor measurement value due to some cause such as impact force being applied to the robot joint during robot transportation or operation. If an offset is added to the torque sensor measurement value at a joint where no load is applied, it can be dealt with by changing the torque sensor offset using the following procedure.

1. Move the robot to the home position.
2. Turn off the servos on all robot axes.
3. Operate the Torobo State Viewer and switch the controller to "gravity_compensation_controller".
4. The gravity compensation torque value of each joint is displayed in the "RefEff" column, and the torque sensor measurement value of each joint is displayed in the "Eff" column.
Check the names of the joints with the largest difference in the 'Name' column.
5. Run the following command from the terminal. Enter the name of the joint confirmed in step 4 in the [Joint name] section.
So lame.

```
$ rosrn torobo_ethercat_tools effort_calibration.py [Joint name] --store  
yrs:=torobo
```

6. Wait until the following message appears on the terminal where you executed the command.

```
[WARN] [1624968995.359367]: ##### [WARN] [1624968995.360199]: # Effort calibration  
completed! # [WARN] [1624968995.361007]: #####
```

Note



* Do not cut off the power supply to the robot until "Effort calibration completed!" is displayed on the terminal where effort_calibration.py was executed. If the power is cut off during this process, the robot may be damaged.

7. Check that the "RefEff" and "Eff" of the target joint are almost identical values.

15.12 How to return to normal operation when a **Protective Stop** is issued by the ZMP monitoring anti-tip function

The ROS package provided by our company has a function to prevent the robot from falling over by monitoring the ZMP (`torobo_safety_monitors/ZmpStabilityMonitor`). When this function is enabled, if the robot is about to fall over, a Protective Stop or Emergency Stop is issued to stop the robot's operation (this function is enabled by default).

In this case, if left as is, the unstable ZMP state will not be resolved, and the recovery conditions will not be met, making it impossible to recover from the safe stop state.

In order to recover from the unstable ZMP state, use one of the following methods to ensure that the ZMP position is within the support polygon. It is necessary to move the robot's joints into a position where they are in the inner position.

1. Release the brakes on the joints and manually backdrive them
2. Temporarily disable self-interference monitoring and servo-drive the joints

We recommend that you return to normal operation using step 1 if possible . However, in cases where you need to move the hip joint to bring the ZMP position closer to a stable state, you may need to support a very large robot load manually. In these cases, it may be safer to temporarily disable the safety monitoring state and servo-drive the joint as in step 2.

I will.

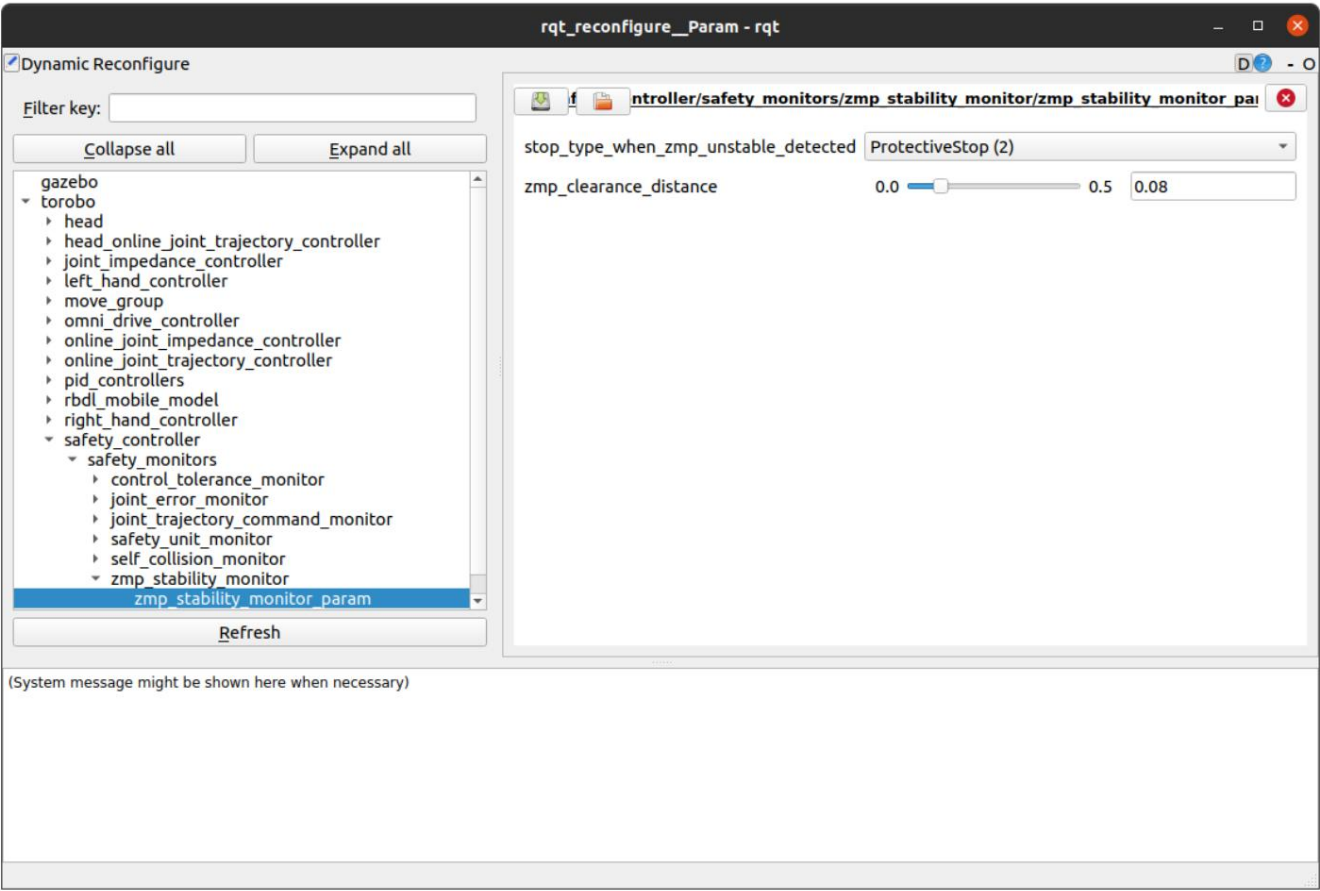
Please refer to method 1 , as it is the same as the recovery method when [an Emergency Stop is issued due to self-collision detection](#) . Method 2 is explained below.

15.12.1 Disable **ZMP** stability monitoring and servo drive joints

First, start `rqt_reconfigure` with the following command .

```
roslaunch rqt_reconfigure rqt_reconfigure
```

From the list on the left side of the displayed GUI , select "torobo" -> "safety_controller" -> "safety_monitors" -> Select "zmp_stability_monitor" -> "zmp_stability_monitor_param".



Click on the combo box next to 'stop_type_when_zmp_unstable_detected' displayed on the right side of the GUI and change it to 'Disabled (0)'. This will temporarily disable the ZMP stability monitoring state, so please return from the Emergency Stop state by following [the procedure for returning to Healthy from Emergency Stop and Protective Stop](#) .

Next, use the Torobo State Viewer to turn on the servo of the joint that is causing the ZMP instability , and issue a command to move the joint to a position that avoids interference. If you accidentally issue a command to move in a direction that makes **the ZMP** unstable, the robot may not stop moving and may fall over, so be careful when issuing the command.

If there are other joints that are causing ZMP instability, move those joints in the same way. After the ZMP instability is resolved, click the 'zmp_unstable_detected' check box displayed on the right side of the dynamic_reconfigure GUI again to return to the 'Emergency Stop (1)' or 'Protective Stop (2)' state. This will re-enable the ZMP stability monitoring state and complete the restoration. In this way, if you temporarily disable the safety monitoring state, be sure to re-enable the safety monitoring state after completing the restoration operation.

Danger When 	disabling ZMP stability monitoring and servo-driving the robot joints, There is a risk that the robot will be transferred if a movement command is issued in the wrong direction. When servo-driving a joint with ZMP stability monitoring disabled, set the transition time of the movement command to low speed and perform the work in a state where the emergency stop switch can be pressed immediately if an incorrect operation is performed. Also, to reduce the possibility of performing an incorrect operation, servo-drive only one joint at a time with self-collision monitoring disabled. Furthermore, if possible, have two or more people support the robot while another person operates the servo. Please work with the utmost care to prevent accidents.
---	--

Danger: If you 	temporarily disable the safety monitoring state, be sure to enable the safety monitoring state again after completing operations such as recovery. If you operate the robot with the safety monitoring state disabled, the robot may run out of control or fall over due to the lack of proper safety monitoring function.
--	---

15.13 The message "**Battery error**" is displayed in the **Error Device of Robot Monitor**

If you charge the battery while the robot or robot controller is running, the message Battery error may appear in the Error Device of Robot Monitor . This is an error that is displayed to prevent overcharging, and occurs when the battery is charged above a certain voltage. If this error occurs, the battery will not charge even if it is connected to a charger. After charging is complete, when the battery voltage drops, the error will automatically disappear and charging will be possible again.

You will be able to do so.

If you turn off the robot before charging, the battery can be charged to nearly its maximum capacity.

15.14 If the problem persists

15.14.1 If the problem persists

Please check the items in the table below and the installed software that can be displayed by executing the following command.

Please send an email to trba-support@robotics.tokyo with the package name and version information. Please contact us at.

```
// Display command for installed package name and version information $
dpkg-query --show -f='${Homepage} ${binary:Package} ${Version} ${Status}\n' | grep -y "https://robotics.tokyo/" | cut -d' ' -f2-
```

15. Troubleshooting

Project	Content example
Action Environment	- Use of the controller PC alone - Use in conjunction with a user PC - If using a user PC , operating environment (PC specs, OS, whether it is a virtual environment, etc.) - Wired LAN or wireless LAN
Error Occurred situation	-Since when -What caused the error to occur? -Frequency of occurrence -Reproducibility
The error occurred near the Never	- Change user PC - Ubuntu Updates -Install or uninstall any software or drivers - Source code changes -Changed workspace -Changed robot placement -Changed robot wiring - Changed the robot configuration - An end effector was attached to or detached from the robot
Error details	-Which joints -Always occurs or occurs under certain conditions -What kind of errors or behaviors occur? - The error message displayed in the diagnostic
Terminal log	- If there is a log message when the error occurred, please include the contents of that message - Attach the ~/ros/log file when the error occurred
Use of self-made software	-Details of the process -Attach the source code if possible

16 -Appendix

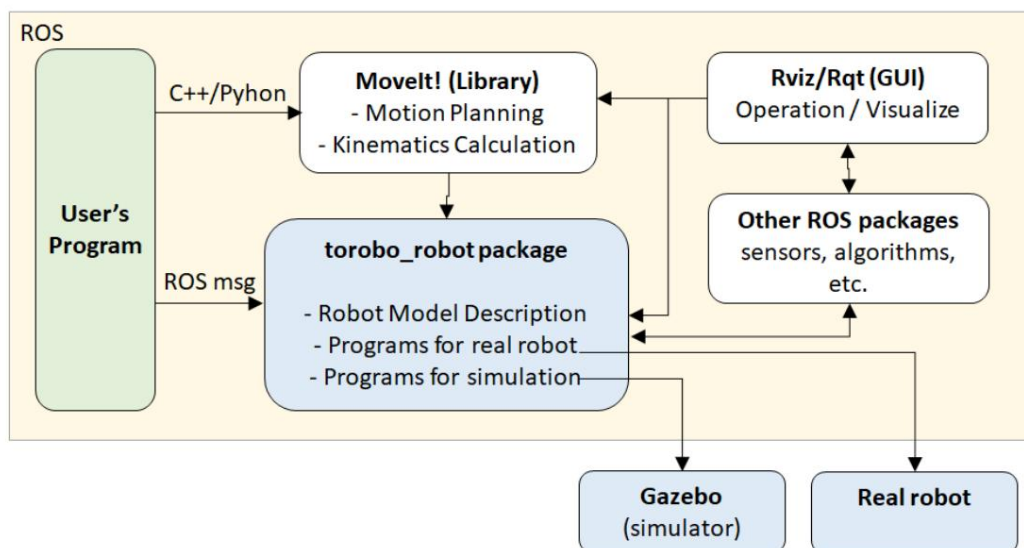
16.1 ROS Overview

ROS is a middleware for robot development called "Robot Operating System."

16.1.1 How ROS works

ROS is provided with various packages, and when ROS is started, each package starts the necessary nodes and communicates with each other. The diagram below shows package communication, but in reality each package is composed of many nodes, and communication occurs between the nodes both within and between packages.

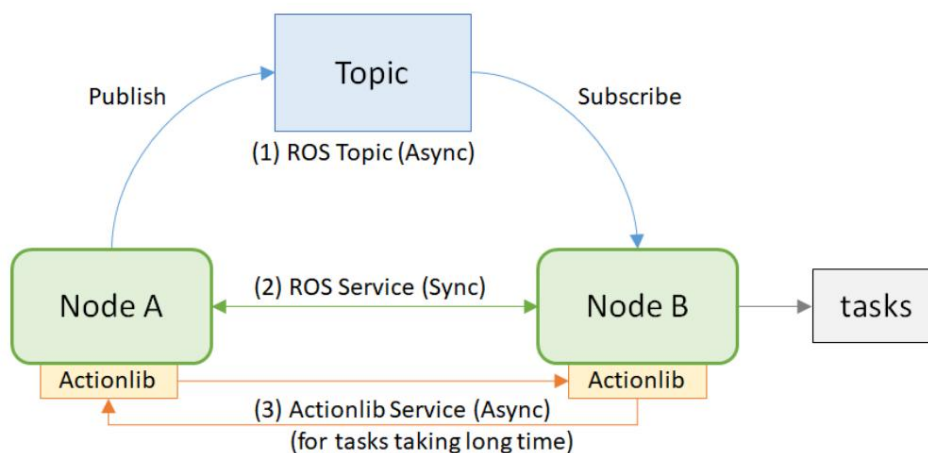
Nodes for GUI tools such as User's Program and Rviz/Rqt, and nodes for sensors, etc. act as triggers, and the nodes communicate with each other to achieve the desired function as a whole.



There are three types of communication methods between ROS nodes .

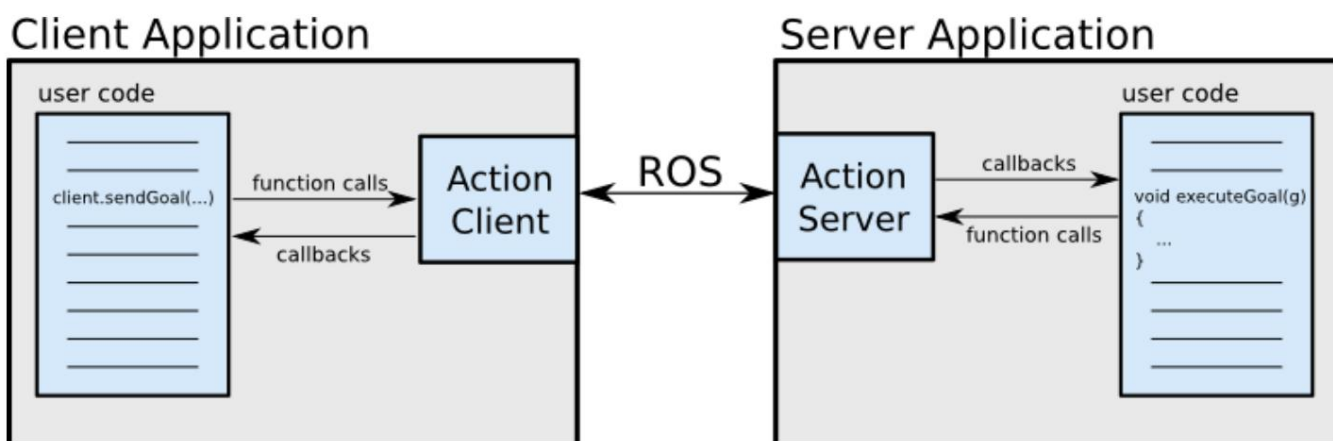
1. ROS Topic Communication (Asynchronous) A Topic is like a bulletin board, where a Publisher posts a ROS Message, which is then read by a Subscriber. Although this is asynchronous communication, data can only be passed in one direction.
2. ROS Service Communication (Synchronous) Unlike topic communication, data can be exchanged in both directions. However, since it is synchronous communication, the client must wait for processing and response from the server.

3. Service communication by Actionlib (asynchronous) The server returns success or failure to the client in response to the calculation results or the request for the actual robot operation. However, the client does not need to wait for the response from the server, and can obtain it at any time. Therefore, it is an asynchronous communication. It is mainly used for processes that take time, such as kinematic calculations and the operation of the actual robot.



16.1.2 Controlling a robot using ROS Actionlib (two-way communication)

Actionlib is a ROS mechanism for asynchronous Server/Client communication. In a Service using Actionlib, The client can move on to the next process without waiting for the server to finish processing, and can get the result of the process (OK/NG) at any time. Actionlib is used for processes that take time (such as robot movement and inverse kinematics calculations).



In the ROS package we provide, we use the ROS standard `control_msgs` (http://wiki.ros.org/control_msgs) as a Service by Actionlib to move the joints of the robot. We provide an Actionlib Service that uses Actionlib. Users can move the joints of a robot by implementing an Actionlib Client to call this service.

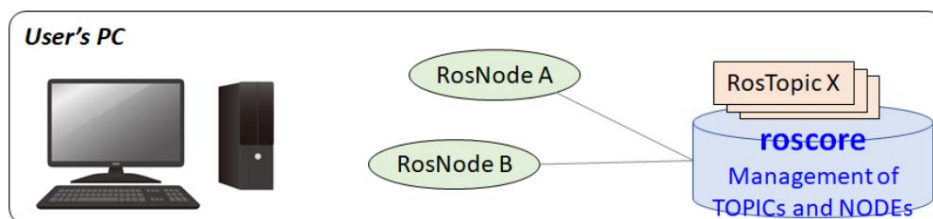
16.1.3 ROS -based distributed system structure

ROS enables a distributed system using multiple PCs .

By using this distributed system mechanism, ROS on the user PC can issue commands to ROS on the control PC. It is possible to do so.

We will explain in detail how the ROS distributed system works.

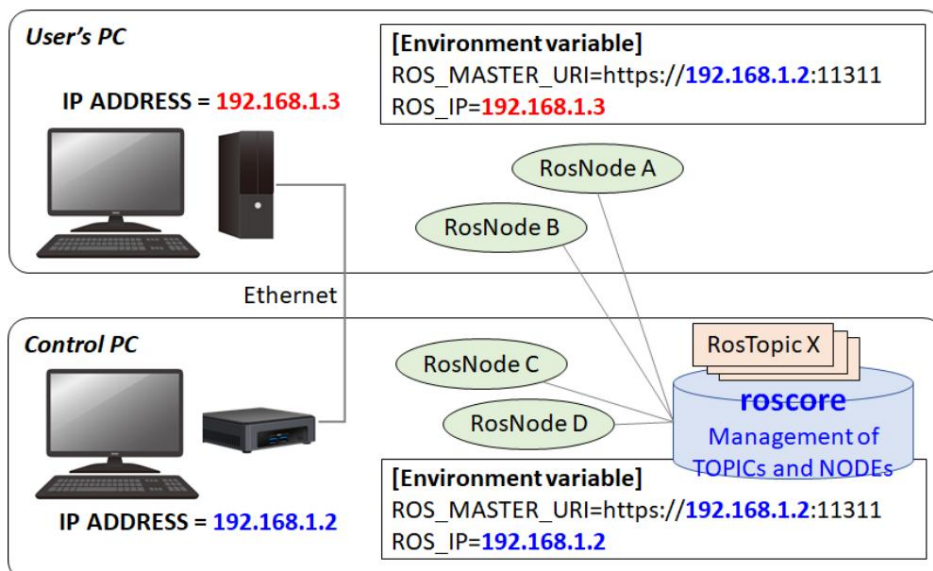
When you start ROS in the terminal (using the roslaunch command or roscore command) , a ROS management program called roscore will start. This roscore manages ROS topics and nodes , enabling communication between nodes.



However, if you specify an IP address other than your own in the ROS_MASTER_URI environment variable , the roscore on the PC with the specified IP address will be referenced without starting the roscore. This mechanism is used to realize a distributed system with multiple PCs .

In a distributed ROS system, only one PC runs a roscore , and that PC 's roscore manages the entire distributed system. You will manage the topics and nodes within the system .

Specifically, as shown in the example below, roscore is started on the control PC , and roscore is not started on the user PC . Refer to the roscore on the control PC .



16.2 Setting up the built-in Wi-Fi router (wireless adapter)

The robot is connected to a small Wi-Fi router, the GL-AR750S-Ext manufactured by GL.iNet . In addition to the router mode, the GL -AR750S-Ext also has a mode called wireless extension mode. In wireless extension mode, by connecting the GL-AR750S-Ext to the network of another wireless router, it is possible to connect devices connected to the GL-AR750S-Ext via wired LAN to the same network.

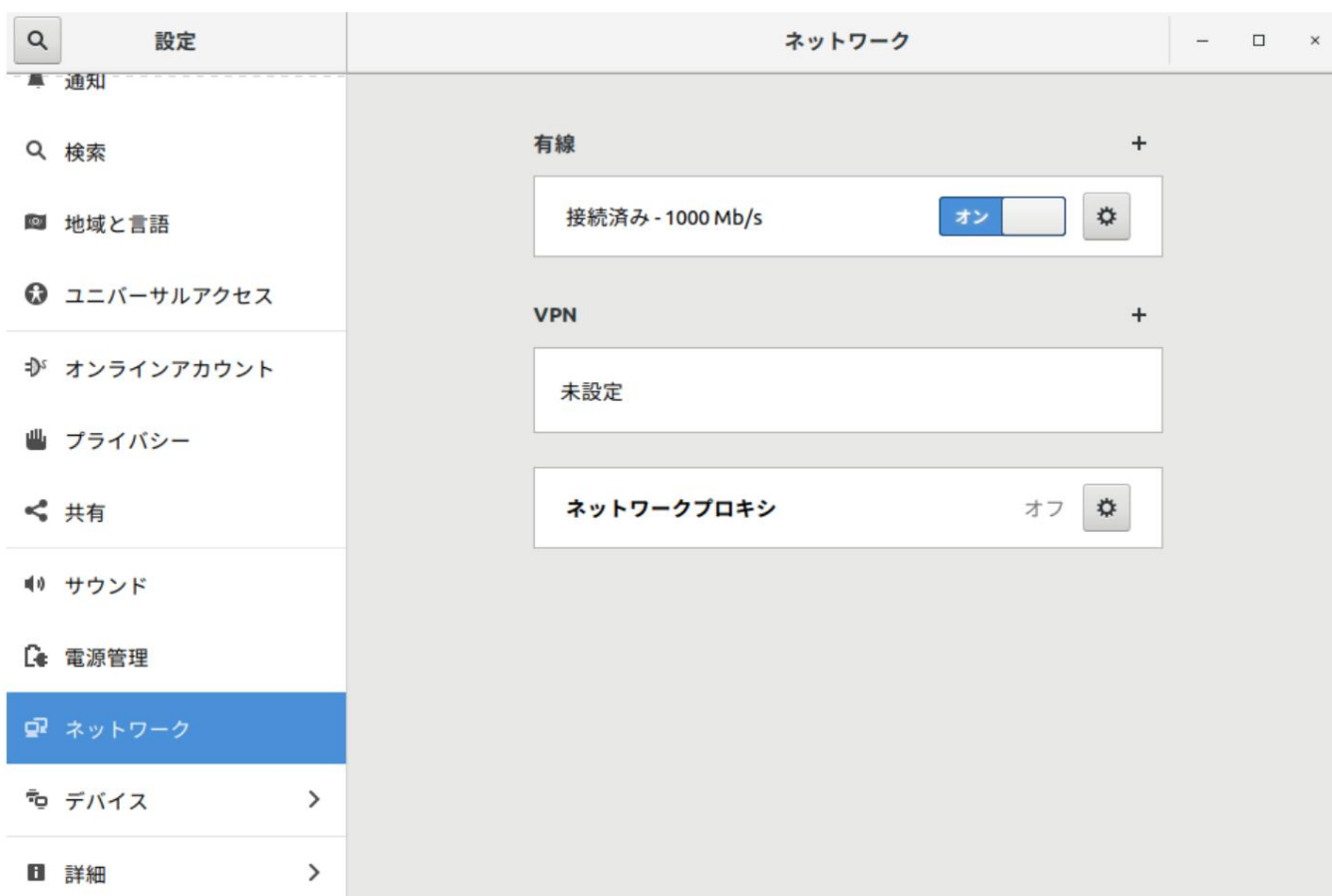
16.2.1 How to connect to the GL-AR750S-Ext setting screen

To connect to the GL-AR750S-Ext setting screen, the PC must be connected to the GL-AR750S-Ext via wired LAN . Connect the LAN cable protruding from the left side of the back of the robot to the LAN port on the top of the back of the robot. This will connect the controller to the GL-AR750S-Ext via wired LAN .

Next, connect a display to the HDMI port on the back of the robot, and connect a mouse and keyboard to the USB ports on Controller 1. Next, open the GNOME Settings app on Controller 1. If you are opening it from a terminal, you can open it with the following command:

```
$ gnome-control-center
```

When you open the network settings menu in the Settings app, you'll see the following screen:



Click the gear icon in the Wired column and open the IPv4 tab. Set the IPv4 method to Manual. Set the address as follows:

address	192.168.8.100
Netmask	255.255.255.0
Gateway	192.168.8.1

Next, turn the "ON/OFF" toggle switch in the Wired section to "OFF" once, then turn it back to "ON" again. This will allow the network settings changes to take effect.

Next, open a browser and access the setting screen at <http://192.168.8.1> . If the connection is successful, you will be logged in. You will be asked for your login password. The password is "torobo" .

Once you have finished configuring the router, change the IPv4 tab in the Network Settings back to Automatic . Please note that the ON/OFF toggle switch must be turned from OFF to ON in order for the settings to take effect .

16.2.2 How to change the network mode of **GL-AR750S-Ext**

GL-AR750S-Ext has network modes such as wireless extension mode and router mode. These can be switched from the "Network Mode" field in the "Other Settings" menu. For details, see the [GL.iNet setup page](#). Please refer to.

When the network mode of GL-AR750S-Ext is switched, the IP assigned to the controller will also change. Please note that in order to enable the change of the assigned IP , you must change the "OFF/ON" toggle switch in the Wired section of the Network Settings menu in the GNOME Settings app from "OFF" to "ON".

16.3 How to change **the IP** address and network connection settings of the controller **PC**

As explained in [the network setup section](#) , a fixed IP address is set for each controller PC when the robot is shipped. If you wish to change the IP address and other settings to suit your network environment, the method is explained below.

16.3.1 Precautions when changing network settings

- If there is an error in the settings, communication over the network between the user PC and the controller PC may not be performed properly. In that case, connect a monitor, keyboard, etc. directly to the controller PC and check the settings.
- For products equipped with a wireless LAN module, a fixed IP address is also set on the wireless LAN module, so we recommend that you change the settings there at the same time (for details on how to set this up, please refer to [Setting up the built-in Wi-Fi router \(wireless adapter\)](#)).

16. Appendix

16.3.2 Preparing for network configuration changes

There are two ways to change the network settings:

- Connect a monitor, keyboard, etc. directly to the controller PC (C1 or C2) and configure the settings on the controller PC .
Change
- Connect to the controller PC via SSH from the user PC and change the settings via a remote connection

Each method is explained below.

16.3.2.1 Changing the settings on the controller PC

Connect a monitor, keyboard, etc. to the HDMI port and USB port on the back of the robot or robot controller (ÿ If there are not enough USB ports, please prepare a separate USB hub). Then, start up the terminal on the controller PC .

16.3.2.2 Changing settings via ssh remote connection

Please refer to [the network setup](#) and connect the user PC and controller PC to the same LAN in advance.
After that, start a terminal on the user PC and connect to the controller PC via ssh .

```
$ ssh torobo@[Control PC's IP Address]
```

16.3.3 Rewriting Netplan configuration

Ubuntu 20.04 uses a utility called Netplan to manage network configuration .

The Netplan configuration file on the robot controller PC is located in the following path:

```
/etc/netplan/99-config.yaml
```

The contents of the above configuration file in the shipped state are shown below.

Listing 16.1 Netplan configuration for controller 1 (C1) in the out-of-the-box state

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eno1:
      optional: true dhcp4:
        false gateway4:
          192.168.1.1 addresses:
            [192.168.1.201/24]
      nameservers:
        addresses: [192.168.1.1]
```

Listing 16.2 Netplan configuration for controller 2 (C2) in the out-of-the-box state

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eno1:
      optional: true dhcp4:
        false gateway4:
          192.168.1.1 addresses:
            [192.168.1.202/24]
      nameservers:
        addresses: [192.168.1.1]
```

In the ethernets section of the configuration file , write the network settings for each NIC . Both controller 1 and controller 2 use a NIC named eno1 for network connections between the controllers, so the settings are written below it. The NIC name can be confirmed with the ifconfig command.

The main configuration items to be described under NIC are shown below. Note that the explanation below assumes that IPv4 is used.

Setting Project	
dhcp4	Description true: Obtain IP address from DHCP server false: Do not use DHCP server
gateway4	Gateway IP address IP address to be set
addresses	to itself addresses: DNS server
nameservers	IP address

Please edit the above items according to your network environment and save it.

16.3.4 Reflecting Netplan settings

The network settings are not yet reflected just because the configuration file has been modified, so you need to run a command to reflect the changes. However, if the changes are reflected immediately, problems such as being unable to connect to the network may occur if there is an error in the settings, so we recommend that you test the settings with the following command before finalizing them.

```
$ sudo netplan try --timeout 60
```

The above command temporarily reflects the settings in Netplan, and then reverts to the original settings after 60 seconds. After executing this command, start another terminal and execute the following command to check whether the IP address has been changed. *It may take some time for the settings to be reflected.

```
$ ip addr
```

(Example output)

(Continued on next page)

16. Appendix

(Continued from previous page)

```

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
   qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00 inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever

    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eno1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
   ydefault qlen 1000
    link/ether c4:00:ad:7d:f0:ee brd ff:ff:ff:ff:ff:ff inet 192.168.1.201/24 brd 192.168.1.255
    scope global eno1
        valid_lft forever preferred_lft forever
    inet6 fe80::44de:3375:2ae3:edb5/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

In the above example output, you can see that the IP address of the NIC named eno1 is 192.168.1.201 .

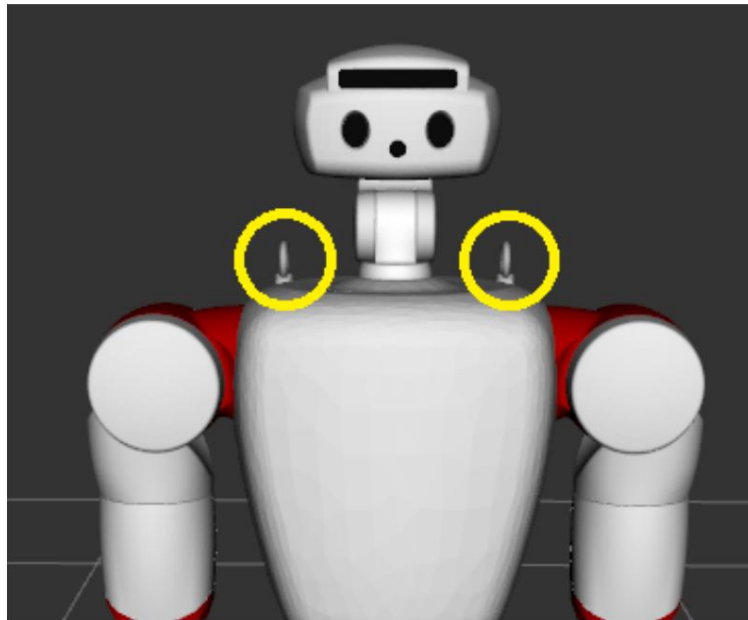
In addition, when using an ssh remote connection, the terminal where the command was executed may not respond after the network setting changes are reflected. In this case, wait a while until the reconnection is successful, or type Enter -> ~ (tilde) -> . (period) in that order, which will send an escape sequence and disconnect the ssh connection.

If the test is successful, use the following command to update the settings.

```
$ sudo netplan apply
```

16.4 Interference mesh of eyebolts for hanging robots

This robot Dry-AIREC can have eyebolts attached to both shoulders to prevent the robot from falling over. This robot software is equipped with a self-collision detection function as explained in [the Self-Collision Detection Function Specifications](#) , and in the default settings at the time of delivery, eyebolts are registered as meshes subject to collision detection as shown in the figure below. This prevents self-collision with the eyebolts due to head movement, but as a result, the movable range of the head joint is partially restricted.



16.4.1 How to change the robot interference mesh to one without eyebolts

If you want to remove the eyebolt from the actual robot and use it to expand the range of motion of the head joint, follow the steps below to Change the bot's interference mesh to one without eyebolts.

1. Open the URDF file located in the following path in an editor.

```
~/catkin_ws/src/torobo_ros/torobo_robot/torobo_resources/urdf/torso/XXX/torso.urdf.
```

yxacro

* The "XXX" part will contain a string or number depending on the robot's configuration.

2. Regarding the description of "mesh" in the link definition of "\${prefix}torso/link_3" from lines 116 to 137 , change the specified parts of "link3_eyebolt.dae" and "link3_eyebolt.stl" to the commented out "link3.dae" and "link3.stl".

List 16.3 Before change (setting a mesh with eyebolts)

(Omitted)

<visual>

<geometry> <!--

<mesh filename="package://torobo_resources/meshes/torso/\${torso_type}/DAE/

ylink3.dae" scale="\${vis_mesh_scale}"/> -->

<mesh filename="package://torobo_resources/meshes/torso/\${torso_type}/DAE/ ylink3_eyebolt.dae" scale="\${vis_mesh_scale}"/> </geometry> </visual>

(Omitted)

<collision>

<geometry> <!--

<mesh filename="package://torobo_resources/meshes/torso/\${torso_type}/DAE/

ylink3.dae" scale="\${vis_mesh_scale}"/> -->

(Continued on next page)

16. Appendix

(Continued from previous page)

```

    <mesh filename="package://torobo_resources/meshes/torso/${torso_type}/DAE/ ylink3_eyebolt.dae" scale="${vis_mesh_scale}"/> </
geometry>

</collision>

```

List 16.4 After modification (setting mesh without eyebolt)

(Omitted)

```

<visual>
  <geometry>
    <mesh filename="package://torobo_resources/meshes/torso/${torso_type}/DAE/
ylink3.dae" scale="${vis_mesh_scale}"/>
    <!-- <mesh filename="package://torobo_resources/meshes/torso/${torso_type}/DAE/ ylink3_eyebolt.dae" scale="${vis_mesh_scale}"/> -->

  </geometry> </
visual>

```

(Omitted)

```

<collision>
  <geometry>
    <mesh filename="package://torobo_resources/meshes/torso/${torso_type}/STL/
ylink3.stl" scale="${col_mesh_scale}"/>
    <!-- <mesh filename="package://torobo_resources/meshes/torso/${torso_type}/STL/ ylink3_eyebolt.stl" scale="${col_mesh_scale}"/> -->

  </geometry>
</collision>

```

3. After launching `bringup_real.launch` etc. , start RViz and check Collision Enabled in RobotModel in the left panel and confirm that the mesh has been changed to one without eyebolts. * The mesh file defined in the "collision" tag will be the target of collision detection.



Note: If you want to attach the eyebolt to the robot again, use the above modified `torso.urdf.xacro`.

If you forget to put it back, the head and eyebolt may interfere with each other, which may damage the robot's exterior or cause the joints to break.

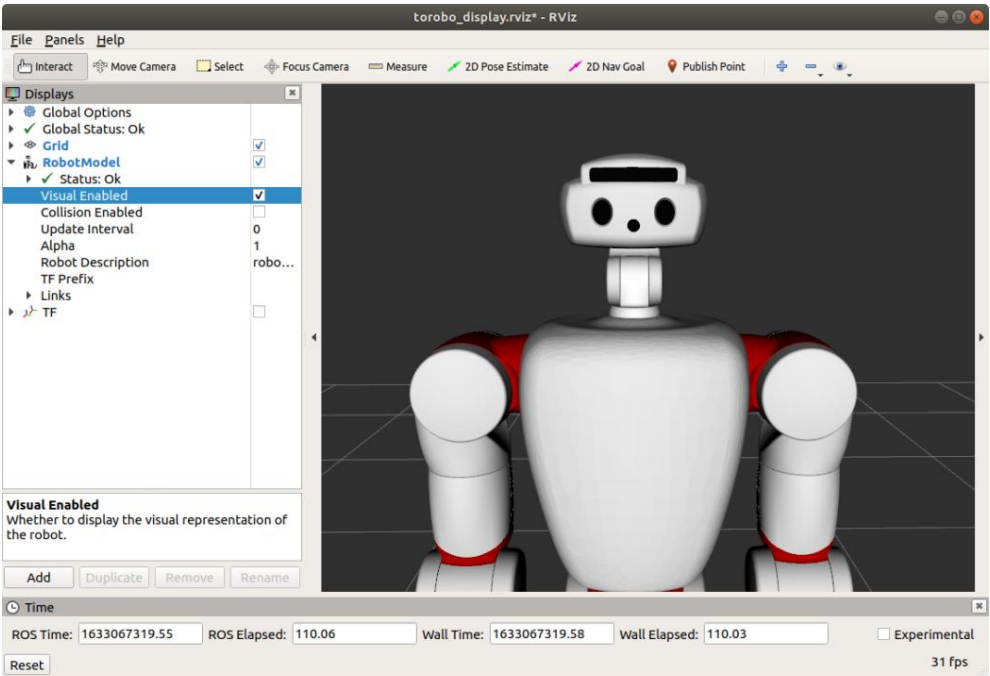


Figure 16.1 Example of Visual Mesh Display

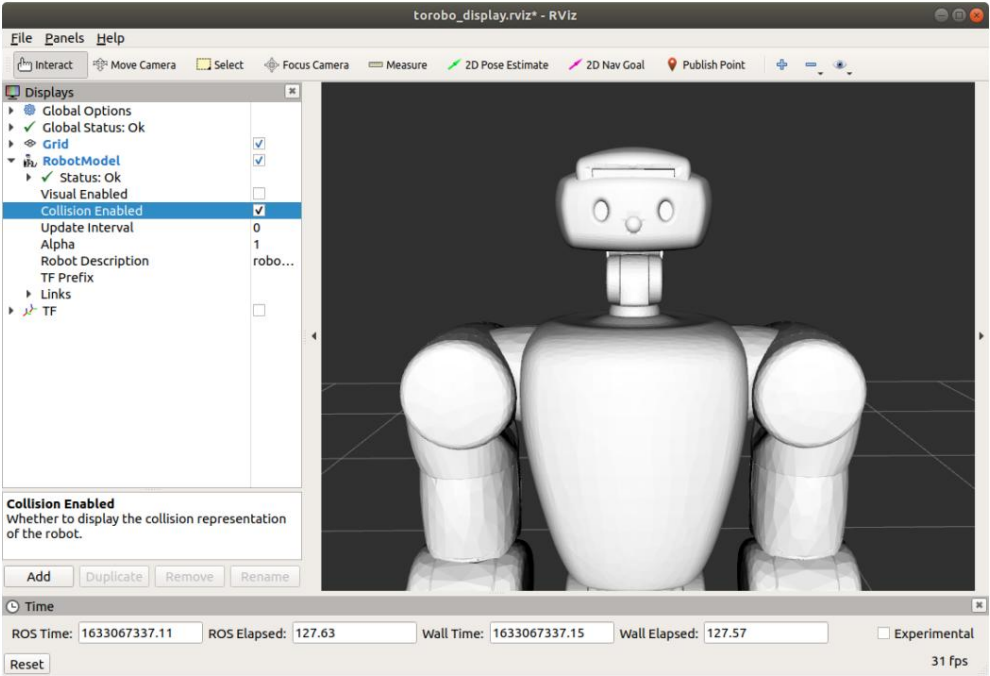


Figure 16.2 Example of collision mesh display

17 – Copyright

- Software (both binary and source code) and manuals accompanying our robots
Copyright belongs to our company.
- These works are protected by copyright law and may not be reproduced in whole or in part by any means without permission.
Therefore, any reproduction, distribution, or publication is strictly prohibited.
- Copying and modification of the source code is permitted only when used in conjunction with our robots (distribution is not permitted).
not available).

18 - Trademarks

- EtherCAT® is a patented technology licensed from Beckhoff Automation GmbH (Germany).

It is a registered trademark.

19 -Revision History

19.1 Release 1.6.0 (2023-03-07)

- Added gpio_command to "Specifications for Original ROS Controller " in "Software Specifications"

19.2 Release 1.5.0 (2022-11-30)

- Corrected the images in "rqt_torobo_joint_trajectory_controller" in the "Tutorial" section to the appropriate ones.
- Added the section "How to change the IP address and network connection settings of the controller PC " to the "Appendix"

19.3 Release 1.4.0 (2022-10-31)

- The entire description has been revised to accommodate Ubuntu 20.04 and ROS Noetic Ninjemys.
- Deleted the text about issuing tf for camera_link in "Starting OMNI shot mini " in "How to use the camera"
- Added a model that supports the "Opening and closing hands" sample script in the "Tutorial" section.
- The sample script for "Tutorial: Position Command Using MoveIt!" has been made compatible with ROS Noetic .
Change to
- The sample script for "Tutorial: Position Command by Teaching Point " has been modified to filter only the joint commands that exist in the controller.
- Added cartesian_trajectory_msgs related items to "Specifications of original ROS messages " in "Software specifications".
- Added Cartesian -related controllers to "Specifications of Original ROS Controller " in "Software Specifications"
- The explanation of "How to update the provided ROS package" in "Maintenance and Management" has been partially revised, and the procedure has been changed to "catkin config
Add the command "-DCMAKE_BUILD_TYPE=Release"
- Item "Temporarily disable self-interference monitoring and servo-drive joints" in "Troubleshooting"
Fix the placement of

19.4 Release 1.3.1 (2022-03-21)

- The launch file specified in "Starting RealSense" in "How to use the camera" was incorrect.
Fixes
- General revisions to the text in the "How to Use the Camera" chapter

19.5 Release 1.3.0 (2022-03-17)

- Fixed an error in some images in the "Interface" section of the "Hardware Overview"
- Updated the contents of "Diag Status List" in "Diag Status Specifications"
- Fixed an error in the "Install provided ROS packages" operation.
- Added the item "Add source command to bashrc " in "Installing provided ROS packages"
- Added "Setting the time synchronization between PCs" to the "Setup" chapter.
- Added explanations on switching controllers to the "Tutorial" chapter.
- Added a corresponding model to the sample script for "Position command using MoveIt!" in the "Tutorial" and added the `set_reference_frame` process.
- An error was found in the "Software Specifications" section of the "Torobo Easy Command Specifications" and was corrected.
- Added explanation to "How to recover from an over position error" in "Troubleshooting".
- Removed the item "The position of the moving platform drifts while running in Gazebo" from "Troubleshooting"
- Added the item "Stop issuing static_tf between map and odom frames" to "Troubleshooting".
- In "Troubleshooting" there is an error "cannot open shared object file" and the core program is not working properly.
Added "Always does not start" item

19.6 Release 1.2.1 (2022-01-07)

- The script arguments were missing in the instructions for "Installing the provided ROS packages" in the "Setup" section.
Add

19.7 Release 1.2.0 (2021-10-14)

- Fixed missing images in tutorial
- Added "JointTrajectoryCommandMonitor" parameters to the "Safety Monitoring Function"
- Added a chapter to the appendix titled "About the interference mesh of eyebolts for hanging robots"
- Changed "How to update the provided ROS packages" in "Maintenance and Management" to use a script.
- Change "User PC Setup" in "Setup" to use script
- Added a command to display installed package names and version information to "Troubleshooting".

19.8 Release 1.1.0 (2021-09-17)

- Fixed some parts of the table layout that were broken
- Added a "Software Specifications" chapter and moved some of the content from the appendix chapter there.
- Changed the expression of the type of "Custom ROS Controller" from name to type name
- Added "Load Setting Function Specifications" to the "Software Specifications" chapter.
- Added "Specifications for center of gravity and ZMP position calculation and evaluation functions" to the "Software specifications" chapter.
- Added "Self-Interference Detection Function Specifications" to the "Software Specifications" chapter.
- Changed the expression of the type of SafetyMonitor in "Safety Monitoring Function" from name to model name
- Added description of ZmpStabilityMonitor and JointTrajectoryCommandMonitor to "Safety Monitoring Function"
- Added description of Reconfigurable Parameters to each type of "Specification of Original ROS Controller " and "Specification of Safety Monitoring Function"
- "Troubleshooting" - " How to return to normal operation when an Emergency Stop is issued due to self-interference detection"
 - Expanded description
- In "Troubleshooting," " When a Protective Stop is issued by the ZMP monitoring fall prevention function
 - Added the section "How to restore the
- Added the section "How to update the provided ROS packages" to "Maintenance and Management".

19.9 Release 1.0.0 (2021-08-31)

- First edition published